



Developer Manual

Interactive LL(1) and SLR(1) tutor

Version: 2.0.0

Date: 2026-10-04

Author: José R. (github.com/jose-rZM)

1 SyntaxTutor	1
1.0.1 Contents	1
1.0.2 Features	2
1.0.3 Getting started	2
1.0.4 How it works	2
1.0.5 Screenshots	3
1.0.6 Documentation	3
1.0.7 Building from source	3
1.0.8 Project structure	4
1.0.9 Background	4
1.0.10 License	4
2 Directory Hierarchy	5
2.1 Directories	5
3 Namespace Index	7
3.1 Namespace List	7
4 Hierarchical Index	9
4.1 Class Hierarchy	9
5 Class Index	11
5.1 Class List	11
6 File Index	13
6.1 File List	13
7 Directory Documentation	15
7.1 app Directory Reference	15
7.2 backend Directory Reference	15
7.3 gui Directory Reference	16
7.4 widgets Directory Reference	16
8 Namespace Documentation	19
8.1 Applcon Namespace Reference	19
8.1.1 Function Documentation	19
8.1.1.1 icon()	19
8.1.2 Variable Documentation	20
8.1.2.1 kSizes	20
8.2 AppLayout Namespace Reference	20
8.2.1 Detailed Description	20
8.2.2 Function Documentation	20
8.2.2.1 keepHeightForWidth()	20
8.3 AppPalette Namespace Reference	21
8.3.1 Function Documentation	21

8.3.1.1 <code>apply()</code>	21
8.3.1.2 <code>linkColor()</code>	22
8.4 AppSettings Namespace Reference	22
8.4.1 Detailed Description	22
8.4.2 Function Documentation	23
8.4.2.1 <code>open()</code>	23
8.4.2.2 <code>registerIdentity()</code>	23
8.5 AppShortcuts Namespace Reference	24
8.5.1 Function Documentation	24
8.5.1.1 <code>primaryModifierName()</code>	24
8.6 AppTextScale Namespace Reference	24
8.6.1 Detailed Description	25
8.6.2 Function Documentation	25
8.6.2.1 <code>clampPercent()</code>	25
8.6.2.2 <code>currentPercent()</code>	26
8.6.2.3 <code>factor()</code>	26
8.6.2.4 <code>legacySettingsKey()</code>	27
8.6.2.5 <code>percentFromLegacyValue()</code>	27
8.6.2.6 <code>settingsKey()</code>	28
8.6.3 Variable Documentation	28
8.6.3.1 <code>kDefaultPercent</code>	28
8.6.3.2 <code>kMaxPercent</code>	28
8.6.3.3 <code>kMinPercent</code>	28
8.6.3.4 <code>kPresetPercents</code>	28
8.7 AppTypography Namespace Reference	29
8.7.1 Detailed Description	30
8.7.2 Enumeration Type Documentation	30
8.7.2.1 <code>Role</code>	30
8.7.3 Function Documentation	30
8.7.3.1 <code>applicationFont()</code>	30
8.7.3.2 <code>basePoints()</code>	31
8.7.3.3 <code>cssSize()</code>	31
8.7.3.4 <code>font()</code>	32
8.7.3.5 <code>formatPoints()</code>	32
8.7.3.6 <code>lengthForText()</code>	33
8.7.3.7 <code>loadStyleSheet()</code>	34
8.7.3.8 <code>points()</code>	34
8.7.3.9 <code>resolveStyleSheet()</code>	35
8.7.3.10 <code>systemFont()</code>	35
8.7.4 Variable Documentation	36
8.7.4.1 <code>kTokens</code>	36
8.8 SyntaxTutor Namespace Reference	36

8.9 SyntaxTutor::Version Namespace Reference	36
8.9.1 Function Documentation	36
8.9.1.1 current()	36
8.9.1.2 raw()	37
9 Class Documentation	39
9.1 AutomatonStateInfo Struct Reference	39
9.1.1 Detailed Description	39
9.1.2 Member Data Documentation	39
9.1.2.1 id	39
9.1.2.2 itemsText	39
9.2 AutomatonTransitionInfo Struct Reference	40
9.2.1 Detailed Description	40
9.2.2 Member Data Documentation	40
9.2.2.1 from	40
9.2.2.2 symbol	40
9.2.2.3 to	40
9.3 AutomatonView Class Reference	41
9.3.1 Detailed Description	42
9.3.2 Constructor & Destructor Documentation	42
9.3.2.1 AutomatonView()	42
9.3.3 Member Function Documentation	43
9.3.3.1 centerOnCurrentState()	43
9.3.3.2 clearStateMarks()	43
9.3.3.3 conflictStates()	43
9.3.3.4 currentStateId()	43
9.3.3.5 fitToView()	43
9.3.3.6 hideStateDetails()	44
9.3.3.7 isFullyRevealed()	44
9.3.3.8 isStateVisible()	44
9.3.3.9 keyPressEvent()	44
9.3.3.10 mousePressEvent()	45
9.3.3.11 mouseReleaseEvent()	45
9.3.3.12 reduceStates()	45
9.3.3.13 revealAll()	45
9.3.3.14 revealState()	45
9.3.3.15 revealTransition()	46
9.3.3.16 setAutomaton()	46
9.3.3.17 setConflictStates()	46
9.3.3.18 setCurrentState()	46
9.3.3.19 setReduceStates()	47
9.3.3.20 shownStateId()	47

9.3.3.21 showStateDetails()	47
9.3.3.22 totalStateCount()	47
9.3.3.23 visibleStateCount()	48
9.3.3.24 visibleTransitionCount()	48
9.3.3.25 wheelEvent()	48
9.3.3.26 zoomIn()	48
9.3.3.27 zoomOut()	49
9.4 AutomatonViewerDialog Class Reference	49
9.4.1 Detailed Description	50
9.4.2 Constructor & Destructor Documentation	50
9.4.2.1 AutomatonViewerDialog()	50
9.4.2.2 ~AutomatonViewerDialog()	51
9.5 CenterAlignDelegate Class Reference	51
9.5.1 Member Function Documentation	51
9.5.1.1 initStyleOption() [1/2]	51
9.5.1.2 initStyleOption() [2/2]	52
9.6 ConversationPdf Class Reference	52
9.6.1 Detailed Description	53
9.6.2 Member Typedef Documentation	53
9.6.2.1 Rule	53
9.6.3 Constructor & Destructor Documentation	53
9.6.3.1 ConversationPdf()	53
9.6.4 Member Function Documentation	54
9.6.4.1 addConversation()	54
9.6.4.2 addDefinitions()	54
9.6.4.3 addGrammar()	54
9.6.4.4 addLines()	55
9.6.4.5 addResult()	55
9.6.4.6 addReview()	55
9.6.4.7 addSection()	55
9.6.4.8 addSubheading()	56
9.6.4.9 addTable()	56
9.6.4.10 html()	56
9.6.4.11 print()	57
9.6.5 Member Data Documentation	57
9.6.5.1 kMaxTableColumns	57
9.7 CustomTextEdit Class Reference	57
9.7.1 Constructor & Destructor Documentation	58
9.7.1.1 CustomTextEdit()	58
9.7.2 Member Function Documentation	59
9.7.2.1 changeEvent()	59
9.7.2.2 keyPressEvent()	59

9.7.2.3 minimumGrowHeight()	59
9.7.2.4 minimumGrowHeightChanged	60
9.7.2.5 newlineShortcutText()	60
9.7.2.6 sendRequested	60
9.7.2.7 setAutoGrowLines()	61
9.7.2.8 showEvent()	61
9.7.3 Property Documentation	61
9.7.3.1 growHeight	61
9.8 ExamRecord Struct Reference	61
9.8.1 Detailed Description	62
9.8.2 Member Data Documentation	62
9.8.2.1 correct	62
9.8.2.2 correctAnswer	62
9.8.2.3 question	62
9.8.2.4 userAnswer	62
9.9 ExamReportDialog Class Reference	63
9.9.1 Detailed Description	63
9.9.2 Constructor & Destructor Documentation	64
9.9.2.1 ExamReportDialog()	64
9.9.3 Member Function Documentation	64
9.9.3.1 exportRequested	64
9.9.3.2 printReport()	65
9.10 ExamSession Class Reference	65
9.10.1 Detailed Description	65
9.10.2 Member Function Documentation	66
9.10.2.1 grade()	66
9.10.2.2 record()	66
9.10.2.3 records()	67
9.10.2.4 right()	67
9.10.2.5 total()	67
9.11 GrammarFactory::FactoryItem Struct Reference	68
9.11.1 Detailed Description	68
9.11.2 Constructor & Destructor Documentation	68
9.11.2.1 FactoryItem()	68
9.11.3 Member Data Documentation	69
9.11.3.1 g_	69
9.11.3.2 st_	69
9.12 Grammar Struct Reference	70
9.12.1 Detailed Description	71
9.12.2 Constructor & Destructor Documentation	71
9.12.2.1 Grammar() [1/2]	71
9.12.2.2 Grammar() [2/2]	72

9.12.3 Member Function Documentation	72
9.12.3.1 AddProduction()	72
9.12.3.2 Debug()	72
9.12.3.3 FilterRulesByConsequent()	73
9.12.3.4 HasEmptyProduction()	73
9.12.3.5 PresentationLess()	74
9.12.3.6 PresentationOrder()	74
9.12.3.7 PresentationRank()	75
9.12.3.8 SetAxiom()	75
9.12.3.9 Split()	76
9.12.4 Member Data Documentation	76
9.12.4.1 axiom_	76
9.12.4.2 g_	76
9.12.4.3 lhs_order_	77
9.12.4.4 st_	77
9.13 GrammarEditorDialog Class Reference	77
9.13.1 Detailed Description	78
9.13.2 Member Enumeration Documentation	78
9.13.2.1 Mode	78
9.13.3 Constructor & Destructor Documentation	79
9.13.3.1 GrammarEditorDialog()	79
9.13.4 Member Function Documentation	79
9.13.4.1 accept()	79
9.13.4.2 grammar()	79
9.14 GrammarFactory Struct Reference	80
9.14.1 Detailed Description	81
9.14.2 Member Function Documentation	81
9.14.2.1 AdjustTerminals()	81
9.14.2.2 CreateLv2Item()	82
9.14.2.3 GenerateNewNonTerminal()	83
9.14.2.4 GenLL1Grammar()	83
9.14.2.5 GenSLR1Grammar()	84
9.14.2.6 HasCycle()	85
9.14.2.7 HasDirectLeftRecursion()	86
9.14.2.8 HasIndirectLeftRecursion()	86
9.14.2.9 HasUnreachableSymbols()	87
9.14.2.10 Init()	87
9.14.2.11 IsInfinite()	87
9.14.2.12 LeftFactorize()	88
9.14.2.13 LongestCommonPrefix()	89
9.14.2.14 Lv1()	90
9.14.2.15 Lv2()	90

9.14.2.16 Lv3()	91
9.14.2.17 Lv4()	92
9.14.2.18 Lv5()	93
9.14.2.19 Lv6()	94
9.14.2.20 Lv7()	95
9.14.2.21 Merge()	96
9.14.2.22 NormalizeNonTerminals()	97
9.14.2.23 NullableSymbols()	98
9.14.2.24 PickOne()	98
9.14.2.25 RemoveLeftRecursion()	99
9.14.2.26 StartsWith()	100
9.14.3 Member Data Documentation	100
9.14.3.1 items	100
9.14.3.2 non_terminal_alphabet_	100
9.14.3.3 terminal_alphabet_	101
9.15 GrammarParseError Struct Reference	101
9.15.1 Detailed Description	101
9.15.2 Member Enumeration Documentation	102
9.15.2.1 Kind	102
9.15.3 Member Data Documentation	102
9.15.3.1 detail	102
9.15.3.2 kind	102
9.15.3.3 line	102
9.16 GrammarParser Struct Reference	102
9.16.1 Detailed Description	103
9.16.2 Member Function Documentation	103
9.16.2.1 IsReservedToken()	103
9.16.2.2 Parse()	104
9.17 GrammarParseResult Struct Reference	104
9.17.1 Detailed Description	105
9.17.2 Member Function Documentation	105
9.17.2.1 Ok()	105
9.17.3 Member Data Documentation	105
9.17.3.1 errors	105
9.17.3.2 grammar	105
9.18 GrammarSyntaxHighlighter Class Reference	106
9.18.1 Detailed Description	106
9.18.2 Constructor & Destructor Documentation	107
9.18.2.1 GrammarSyntaxHighlighter()	107
9.18.3 Member Function Documentation	107
9.18.3.1 highlightBlock()	107
9.18.3.2 setNonTerminals()	107

9.19 GrammarView Class Reference	107
9.19.1 Constructor & Destructor Documentation	108
9.19.1.1 GrammarView()	108
9.19.2 Member Function Documentation	108
9.19.2.1 naturalWidth()	108
9.19.2.2 refresh()	109
9.19.2.3 setRows()	109
9.20 AppLayout::HeightForWidthKeeper Class Reference	109
9.20.1 Detailed Description	110
9.20.2 Constructor & Destructor Documentation	110
9.20.2.1 HeightForWidthKeeper()	110
9.20.3 Member Function Documentation	111
9.20.3.1 eventFilter()	111
9.21 LL1Parser Class Reference	111
9.21.1 Constructor & Destructor Documentation	112
9.21.1.1 LL1Parser() [1/2]	112
9.21.1.2 LL1Parser() [2/2]	113
9.21.2 Member Function Documentation	113
9.21.2.1 ComputeFirstSets()	113
9.21.2.2 ComputeFollowSets()	114
9.21.2.3 CreateLL1Table()	114
9.21.2.4 First()	115
9.21.2.5 Follow()	116
9.21.2.6 PredictionSymbols()	117
9.21.3 Member Data Documentation	117
9.21.3.1 first_sets_	117
9.21.3.2 follow_sets_	118
9.21.3.3 gr_	118
9.21.3.4 ll1_t_	118
9.22 LLTableDialog Class Reference	118
9.22.1 Detailed Description	119
9.22.2 Constructor & Destructor Documentation	119
9.22.2.1 LLTableDialog()	119
9.22.3 Member Function Documentation	120
9.22.3.1 getTableData()	120
9.22.3.2 guidedRequested	120
9.22.3.3 highlightIncorrectCells()	121
9.22.3.4 setGuidedButtonVisible()	121
9.22.3.5 setGuidedModeActive()	121
9.22.3.6 setInitialData()	121
9.22.3.7 submitted	122
9.23 LLTutorWindow Class Reference	122

9.23.1 Detailed Description	125
9.23.2 Constructor & Destructor Documentation	125
9.23.2.1 LLTutorWindow()	125
9.23.2.2 ~LLTutorWindow()	126
9.23.3 Member Function Documentation	127
9.23.3.1 addGrammarMessage()	127
9.23.3.2 addMessage()	127
9.23.3.3 addWidgetMessage()	127
9.23.3.4 animateLabelColor()	127
9.23.3.5 animateLabelPop()	128
9.23.3.6 applyTextScale()	128
9.23.3.7 buildGrammarRows()	128
9.23.3.8 buildTreeNode()	128
9.23.3.9 closeEvent()	129
9.23.3.10 computeSubtreeWidth()	129
9.23.3.11 confirmExitToHome()	129
9.23.3.12 connectGuidedMode()	129
9.23.3.13 drawTree()	130
9.23.3.14 eventFilter()	130
9.23.3.15 examSolutionText()	130
9.23.3.16 exitRequested	131
9.23.3.17 exportConversationToPdf()	131
9.23.3.18 feedback()	131
9.23.3.19 feedbackForA()	132
9.23.3.20 feedbackForA1()	132
9.23.3.21 feedbackForA2()	133
9.23.3.22 feedbackForAPrime()	133
9.23.3.23 feedbackForB()	133
9.23.3.24 feedbackForB1()	133
9.23.3.25 feedbackForB1TreeGraphics()	134
9.23.3.26 feedbackForB1TreeWidget()	134
9.23.3.27 feedbackForB2()	134
9.23.3.28 feedbackForBPrime()	135
9.23.3.29 feedbackForC()	135
9.23.3.30 feedbackForCPrime()	136
9.23.3.31 FormatGrammar()	136
9.23.3.32 generateQuestion()	137
9.23.3.33 handleTableSubmission()	137
9.23.3.34 markLastUserIncorrect()	137
9.23.3.35 postQuestion()	138
9.23.3.36 promptExportFilePath()	138
9.23.3.37 scoreExamTable()	139

9.23.3.38 sessionFinished	140
9.23.3.39 showExamReport()	140
9.23.3.40 showTable()	140
9.23.3.41 showTableForCPrime()	140
9.23.3.42 showTreeGraphics()	141
9.23.3.43 solution()	141
9.23.3.44 solutionForA()	142
9.23.3.45 solutionForA1()	142
9.23.3.46 solutionForA2()	142
9.23.3.47 solutionForB()	142
9.23.3.48 solutionForB1()	143
9.23.3.49 solutionForB2()	143
9.23.3.50 TeachFirstTree()	144
9.23.3.51 TeachFollow()	144
9.23.3.52 TeachLL1Table()	144
9.23.3.53 TeachPredictionSymbols()	144
9.23.3.54 updatePlaceholder()	145
9.23.3.55 updateProgressPanel()	145
9.23.3.56 updateState()	146
9.23.3.57 verifyResponse()	146
9.23.3.58 verifyResponseForA()	146
9.23.3.59 verifyResponseForA1()	147
9.23.3.60 verifyResponseForA2()	147
9.23.3.61 verifyResponseForB()	147
9.23.3.62 verifyResponseForB1()	148
9.23.3.63 verifyResponseForB2()	148
9.23.3.64 verifyResponseForC()	148
9.23.3.65 wrongAnimation()	148
9.23.3.66 wrongUserResponseAnimation()	149
9.24 LLWizard Class Reference	149
9.24.1 Detailed Description	150
9.24.2 Constructor & Destructor Documentation	150
9.24.2.1 LLWizard()	150
9.24.3 Member Function Documentation	150
9.24.3.1 closeButton()	150
9.24.3.2 currentPage()	151
9.24.3.3 isOnLastPage()	151
9.24.3.4 nextButton()	151
9.25 LLWizardPage Class Reference	151
9.25.1 Detailed Description	152
9.25.2 Constructor & Destructor Documentation	152
9.25.2.1 LLWizardPage()	152

9.25.3 Member Function Documentation	153
9.25.3.1 completionChanged	153
9.25.3.2 focusAnswerField()	153
9.25.3.3 isComplete()	153
9.25.3.4 resizeEvent()	153
9.25.3.5 submitRequested	154
9.25.3.6 titleText()	154
9.26 Lr0Item Struct Reference	154
9.26.1 Detailed Description	155
9.26.2 Constructor & Destructor Documentation	155
9.26.2.1 Lr0Item() [1/2]	155
9.26.2.2 Lr0Item() [2/2]	156
9.26.3 Member Function Documentation	156
9.26.3.1 AdvanceDot()	156
9.26.3.2 IsComplete()	156
9.26.3.3 NextToDot()	157
9.26.3.4 operator==()	157
9.26.3.5 PrintItem()	157
9.26.3.6 ToString()	158
9.26.4 Member Data Documentation	158
9.26.4.1 antecedent_	158
9.26.4.2 consequent_	158
9.26.4.3 dot_	158
9.26.4.4 eol_	158
9.26.4.5 epsilon_	159
9.27 MainWindow Class Reference	159
9.27.1 Detailed Description	160
9.27.2 Constructor & Destructor Documentation	160
9.27.2.1 MainWindow()	160
9.27.2.2 ~MainWindow()	161
9.27.3 Member Function Documentation	161
9.27.3.1 setUserLevel()	161
9.27.3.2 thresholdFor()	161
9.27.3.3 userLevel()	162
9.27.3.4 userLevelChanged	162
9.27.3.5 userLevelUp	163
9.27.4 Property Documentation	163
9.27.4.1 userLevel	163
9.28 ConversationPdf::Message Struct Reference	163
9.28.1 Detailed Description	164
9.28.2 Member Data Documentation	164
9.28.2.1 correct	164

9.28.2.2 fromStudent	164
9.28.2.3 text	164
9.29 GrammarView::Row Struct Reference	164
9.29.1 Member Data Documentation	164
9.29.1.1 index	164
9.29.1.2 lhs	164
9.29.1.3 marker	165
9.29.1.4 rhs	165
9.30 SLR1Parser::s_action Struct Reference	165
9.30.1 Member Data Documentation	165
9.30.1.1 action	165
9.30.1.2 item	165
9.31 SLR1Parser Class Reference	166
9.31.1 Detailed Description	167
9.31.2 Member Typedef Documentation	168
9.31.2.1 action_table	168
9.31.2.2 transition_table	168
9.31.3 Member Enumeration Documentation	169
9.31.3.1 Action	169
9.31.4 Constructor & Destructor Documentation	169
9.31.4.1 SLR1Parser() [1/2]	169
9.31.4.2 SLR1Parser() [2/2]	170
9.31.5 Member Function Documentation	170
9.31.5.1 AllItems()	170
9.31.5.2 Closure()	170
9.31.5.3 ClosureUtil()	171
9.31.5.4 ComputeFirstSets()	172
9.31.5.5 ComputeFollowSets()	172
9.31.5.6 Delta()	173
9.31.5.7 First()	174
9.31.5.8 Follow()	175
9.31.5.9 MakeInitialState()	175
9.31.5.10 MakeParser()	176
9.31.5.11 PrintItems()	176
9.31.5.12 SolveLRConflicts()	177
9.31.6 Member Data Documentation	178
9.31.6.1 actions_	178
9.31.6.2 first_sets_	178
9.31.6.3 follow_sets_	178
9.31.6.4 gr_	178
9.31.6.5 states_	178
9.31.6.6 transitions_	178

9.32 SLRTableDialog Class Reference	179
9.32.1 Detailed Description	180
9.32.2 Constructor & Destructor Documentation	180
9.32.2.1 SLRTableDialog()	180
9.32.3 Member Function Documentation	181
9.32.3.1 getTableData()	181
9.32.3.2 guidedRequested	181
9.32.3.3 highlightIncorrectCells()	181
9.32.3.4 highlightInvalidCells()	181
9.32.3.5 setGuidedButtonVisible()	182
9.32.3.6 setGuidedModeActive()	182
9.32.3.7 setInitialData()	182
9.32.3.8 submitted	182
9.33 SLRTutorWindow Class Reference	183
9.33.1 Detailed Description	186
9.33.2 Constructor & Destructor Documentation	186
9.33.2.1 SLRTutorWindow()	186
9.33.2.2 ~SLRTutorWindow()	187
9.33.3 Member Function Documentation	187
9.33.3.1 addGrammarMessage()	187
9.33.3.2 addMessage()	187
9.33.3.3 addUserState()	187
9.33.3.4 addUserTransition()	188
9.33.3.5 addWidgetMessage()	188
9.33.3.6 animateLabelColor()	188
9.33.3.7 animateLabelPop()	188
9.33.3.8 applyTextScale()	189
9.33.3.9 buildGrammarRows()	189
9.33.3.10 closeEvent()	189
9.33.3.11 confirmExitToHome()	189
9.33.3.12 eventFilter()	190
9.33.3.13 examSolutionText()	190
9.33.3.14 exitRequested	190
9.33.3.15 exportConversationToPdf()	190
9.33.3.16 feedback()	191
9.33.3.17 feedbackForA()	191
9.33.3.18 feedbackForA1()	191
9.33.3.19 feedbackForA2()	192
9.33.3.20 feedbackForA3()	192
9.33.3.21 feedbackForA4()	193
9.33.3.22 feedbackForAPrime()	193
9.33.3.23 feedbackForB()	193

9.33.3.24 feedbackForB1()	193
9.33.3.25 feedbackForB2()	193
9.33.3.26 feedbackForBPrime()	194
9.33.3.27 feedbackForC()	194
9.33.3.28 feedbackForCA()	195
9.33.3.29 feedbackForCB()	195
9.33.3.30 feedbackForD()	196
9.33.3.31 feedbackForD1()	196
9.33.3.32 feedbackForD2()	196
9.33.3.33 feedbackForDPrime()	196
9.33.3.34 feedbackForE()	197
9.33.3.35 feedbackForE1()	197
9.33.3.36 feedbackForE2()	197
9.33.3.37 feedbackForF()	197
9.33.3.38 feedbackForFA()	198
9.33.3.39 feedbackForG()	198
9.33.3.40 fillSortedGrammar()	198
9.33.3.41 FormatGrammar()	198
9.33.3.42 generateQuestion()	199
9.33.3.43 initialItemText()	199
9.33.3.44 isReduction()	200
9.33.3.45 launchSLRWizard()	200
9.33.3.46 markLastUserIncorrect()	201
9.33.3.47 openAutomatonViewer()	201
9.33.3.48 postQuestion()	201
9.33.3.49 promptExportFilePath()	202
9.33.3.50 scoreExamTable()	202
9.33.3.51 sessionFinished	203
9.33.3.52 setupAutomatonPanel()	203
9.33.3.53 showExamReport()	203
9.33.3.54 showTable()	204
9.33.3.55 solution()	204
9.33.3.56 solutionCell()	205
9.33.3.57 solutionForA()	206
9.33.3.58 solutionForA1()	206
9.33.3.59 solutionForA2()	206
9.33.3.60 solutionForA3()	207
9.33.3.61 solutionForA4()	207
9.33.3.62 solutionForB()	207
9.33.3.63 solutionForC()	207
9.33.3.64 solutionForCA()	208
9.33.3.65 solutionForCB()	208

9.33.3.66 solutionForD()	208
9.33.3.67 solutionForD1()	208
9.33.3.68 solutionForD2()	209
9.33.3.69 solutionForE()	209
9.33.3.70 solutionForE1()	209
9.33.3.71 solutionForE2()	209
9.33.3.72 solutionForF()	210
9.33.3.73 solutionForFA()	210
9.33.3.74 solutionForG()	210
9.33.3.75 tableColumns()	211
9.33.3.76 TeachClosure()	211
9.33.3.77 TeachClosureStep()	211
9.33.3.78 TeachDeltaFunction()	212
9.33.3.79 updateAutomatonButton()	212
9.33.3.80 updateAutomatonPanel()	212
9.33.3.81 updatePlaceholder()	213
9.33.3.82 updateProgressPanel()	213
9.33.3.83 updateState()	213
9.33.3.84 verifyResponse()	214
9.33.3.85 verifyResponseForA()	214
9.33.3.86 verifyResponseForA1()	214
9.33.3.87 verifyResponseForA2()	215
9.33.3.88 verifyResponseForA3()	215
9.33.3.89 verifyResponseForA4()	215
9.33.3.90 verifyResponseForB()	216
9.33.3.91 verifyResponseForC()	216
9.33.3.92 verifyResponseForCA()	217
9.33.3.93 verifyResponseForCB()	217
9.33.3.94 verifyResponseForD()	217
9.33.3.95 verifyResponseForD1()	218
9.33.3.96 verifyResponseForD2()	218
9.33.3.97 verifyResponseForE()	218
9.33.3.98 verifyResponseForE1()	218
9.33.3.99 verifyResponseForE2()	219
9.33.3.100 verifyResponseForF()	219
9.33.3.101 verifyResponseForFA()	219
9.33.3.102 verifyResponseForG()	219
9.33.3.103 verifyResponseForH()	220
9.33.3.104 wrongAnimation()	220
9.33.3.105 wrongUserResponseAnimation()	220
9.34 SLRWizard Class Reference	221
9.34.1 Detailed Description	221

9.34.2 Constructor & Destructor Documentation	222
9.34.2.1 SLRWizard()	222
9.34.3 Member Function Documentation	222
9.34.3.1 closeButton()	222
9.34.3.2 currentPage()	223
9.34.3.3 isOnLastPage()	223
9.34.3.4 nextButton()	223
9.34.3.5 stdVectorToQVector()	223
9.35 SLRWizardPage Class Reference	224
9.35.1 Detailed Description	225
9.35.2 Constructor & Destructor Documentation	225
9.35.2.1 SLRWizardPage()	225
9.35.3 Member Function Documentation	226
9.35.3.1 completionChanged	226
9.35.3.2 focusAnswerField()	226
9.35.3.3 isComplete()	226
9.35.3.4 resizeEvent()	226
9.35.3.5 submitRequested	226
9.35.3.6 titleText()	226
9.36 state Struct Reference	227
9.36.1 Detailed Description	227
9.36.2 Member Function Documentation	227
9.36.2.1 operator==()	227
9.36.3 Member Data Documentation	227
9.36.3.1 id_	227
9.36.3.2 items_	228
9.37 SymbolTable Struct Reference	228
9.37.1 Detailed Description	229
9.37.2 Member Function Documentation	229
9.37.2.1 ln()	229
9.37.2.2 isMeta()	229
9.37.2.3 isNonTerminal()	230
9.37.2.4 isTerminal()	230
9.37.2.5 isTerminalWthoEol()	231
9.37.2.6 putSymbol()	231
9.37.3 Member Data Documentation	231
9.37.3.1 EOL_	231
9.37.3.2 EPSILON_	232
9.37.3.3 meta_symbols_	232
9.37.3.4 non_terminals_	232
9.37.3.5 st_	232
9.37.3.6 terminals_	232

9.37.3.7 terminals_wtho_eol_	232
9.38 AppTypography::Token Struct Reference	233
9.38.1 Detailed Description	233
9.38.2 Member Data Documentation	233
9.38.2.1 name	233
9.38.2.2 points	233
9.38.2.3 role	233
9.39 LLTutorWindow::TreeNode Struct Reference	233
9.39.1 Detailed Description	234
9.39.2 Member Data Documentation	234
9.39.2.1 children	234
9.39.2.2 label	234
9.40 TutorialManager Class Reference	234
9.40.1 Detailed Description	235
9.40.2 Constructor & Destructor Documentation	236
9.40.2.1 TutorialManager()	236
9.40.3 Member Function Documentation	236
9.40.3.1 addStep()	236
9.40.3.2 clearSteps()	236
9.40.3.3 eventFilter()	236
9.40.3.4 finishLL1()	237
9.40.3.5 finishSLR1()	237
9.40.3.6 hideOverlay()	237
9.40.3.7 ll1Finished	238
9.40.3.8 nextStep	238
9.40.3.9 setRootWindow()	238
9.40.3.10 slr1Finished	239
9.40.3.11 start()	239
9.40.3.12 stepStarted	239
9.40.3.13 tutorialFinished	240
9.41 TutorialStep Struct Reference	240
9.41.1 Detailed Description	240
9.41.2 Member Data Documentation	240
9.41.2.1 htmlText	240
9.41.2.2 target	241
9.42 UniqueQueue< T > Class Template Reference	241
9.42.1 Detailed Description	241
9.42.2 Member Function Documentation	242
9.42.2.1 clear()	242
9.42.2.2 empty()	242
9.42.2.3 front()	242
9.42.2.4 pop()	242

9.42.2.5 push()	243
10 File Documentation	245
10.1 /Users/jose/Documents/cpp/SyntaxTutor/README.md File Reference	245
10.2 app/main.cpp File Reference	245
10.2.1 Function Documentation	246
10.2.1.1 applyAppStyle()	246
10.2.1.2 main()	246
10.3 appversion.h File Reference	247
10.4 appversion.h	248
10.5 backend/grammar.cpp File Reference	248
10.6 backend/grammar.hpp File Reference	248
10.6.1 Typedef Documentation	249
10.6.1.1 production	249
10.7 grammar.hpp	250
10.8 backend/grammar_factory.cpp File Reference	250
10.9 backend/grammar_factory.hpp File Reference	251
10.10 grammar_factory.hpp	252
10.11 backend/grammar_parser.cpp File Reference	253
10.12 backend/grammar_parser.hpp File Reference	254
10.13 grammar_parser.hpp	254
10.14 backend/ll1_parser.cpp File Reference	255
10.15 backend/ll1_parser.hpp File Reference	256
10.16 ll1_parser.hpp	257
10.17 backend/lr0_item.cpp File Reference	257
10.18 backend/lr0_item.hpp File Reference	258
10.19 lr0_item.hpp	259
10.20 backend/slr1_parser.cpp File Reference	259
10.21 backend/slr1_parser.hpp File Reference	260
10.22 slr1_parser.hpp	261
10.23 backend/state.hpp File Reference	262
10.24 state.hpp	262
10.25 backend/symbol_table.cpp File Reference	263
10.26 backend/symbol_table.hpp File Reference	263
10.26.1 Enumeration Type Documentation	264
10.26.1.1 symbol_type	264
10.27 symbol_table.hpp	265
10.28 gui/automatonviewerdialog.cpp File Reference	265
10.29 gui/automatonviewerdialog.h File Reference	266
10.30 automatonviewerdialog.h	267
10.31 gui/conversationpdf.h File Reference	267
10.32 conversationpdf.h	268

10.33	gui/examreportdialog.cpp File Reference	272
10.34	gui/examreportdialog.h File Reference	272
10.35	examreportdialog.h	273
10.36	gui/examsession.h File Reference	274
10.37	examsession.h	274
10.38	gui/grammareditordialog.cpp File Reference	275
10.39	gui/grammareditordialog.h File Reference	276
10.40	grammareditordialog.h	277
10.41	gui/llabledialog.cpp File Reference	278
10.42	gui/llabledialog.h File Reference	278
10.43	llabledialog.h	279
10.44	gui/lltutorwindow.cpp File Reference	280
10.45	gui/lltutorwindow.h File Reference	280
10.45.1	Enumeration Type Documentation	282
10.45.1.1	State	282
10.46	lltutorwindow.h	282
10.47	gui/llwizard.h File Reference	286
10.48	llwizard.h	286
10.49	gui/llwizardpage.h File Reference	289
10.50	llwizardpage.h	290
10.51	gui/mainwindow.cpp File Reference	292
10.52	gui/mainwindow.h File Reference	292
10.53	mainwindow.h	293
10.54	gui/slrtabledialog.cpp File Reference	295
10.55	gui/slrtabledialog.h File Reference	295
10.56	slrtabledialog.h	296
10.57	gui/slrtutorwindow.cpp File Reference	297
10.58	gui/slrtutorwindow.h File Reference	298
10.58.1	Enumeration Type Documentation	299
10.58.1.1	StateSlr	299
10.59	slrtutorwindow.h	300
10.60	gui/slrlizard.h File Reference	304
10.61	slrlizard.h	305
10.62	gui/slrlizardpage.h File Reference	309
10.63	slrlizardpage.h	309
10.64	gui/UniqueQueue.h File Reference	311
10.65	UniqueQueue.h	312
10.66	widgets/appfonts.h File Reference	313
10.66.1	Function Documentation	313
10.66.1.1	appMonospaceFont()	313
10.67	appfonts.h	314
10.68	widgets/appicon.h File Reference	314

10.69 appicon.h	315
10.70 widgets/applayout.h File Reference	316
10.71 applayout.h	317
10.72 widgets/apppalette.h File Reference	318
10.73 apppalette.h	319
10.74 widgets/appsettings.h File Reference	319
10.75 appsettings.h	320
10.76 widgets/appshortcuts.h File Reference	321
10.77 appshortcuts.h	321
10.78 widgets/apptextscale.h File Reference	322
10.79 apptextscale.h	323
10.80 widgets/apptypography.h File Reference	324
10.81 apptypography.h	325
10.82 widgets/automatonview.cpp File Reference	327
10.83 widgets/automatonview.h File Reference	328
10.84 automatonview.h	329
10.85 widgets/customtextedit.cpp File Reference	330
10.86 widgets/customtextedit.h File Reference	331
10.87 customtextedit.h	331
10.88 widgets/grammarview.cpp File Reference	332
10.89 widgets/grammarview.h File Reference	333
10.90 grammarview.h	333
10.91 widgets/tutorialmanager.cpp File Reference	334
10.92 widgets/tutorialmanager.h File Reference	334
10.93 tutorialmanager.h	335

Index**337**

Chapter 1

SyntaxTutor



An interactive tutor for learning LL(1) and SLR(1) parsing.

[(LICENSE)]

[SyntaxTutor](#) helps compiler students understand how LL(1) and SLR(1) parsers are built. The student works through each algorithm one question at a time: FIRST and FOLLOW sets, prediction symbols, LR(0) items, closures, transitions and, finally, the parsing table. Every answer is checked on the spot, and a wrong answer leads to smaller questions that rebuild the reasoning step by step, so the student learns why a result is what it is.

Note

Project status. [SyntaxTutor](#) 2.0 is the final release. The project is complete and no further versions are planned. The source code remains available under the GPLv3 for anyone who wants to study it or build on it.

1.0.1 Contents

- Features
- Getting started
- How it works
- Screenshots
- Documentation
- Building from source
- Project structure
- Background
- License

1.0.2 Features

- **Step-by-step tutors** for LL(1) and SLR(1), with immediate feedback on every answer.
- **Pedagogical branching**: mistakes open smaller questions instead of revealing the result.
- **Your own grammars**: a grammar editor with live validation, or generated grammars at three difficulty levels.
- **Exam mode**: no feedback during the exercise, a 0–10 grade at the end and a question-by-question review.
- **LR(0) automaton** drawn progressively as the student computes states and transitions.
- **Guided table modes** that walk through the LL(1) and SLR(1) tables one cell at a time.
- **PDF export** of the conversation, sets, automaton states, parsing tables and exam reports.
- **Quick references** for the theory of both algorithms.
- **Spanish and English** interface, adjustable text size and gamified progress.

1.0.3 Getting started

Download the build for your system from the [latest release](#). No installation or Internet connection is needed.

System	Download	Requirements
Windows	<code>SyntaxTutor-<version>-windows-x64.zip</code>	Windows 10 or 11, 64-bit
macOS	<code>SyntaxTutor-<version>-macos-arm64.zip</code>	Apple Silicon (M1 or later)
Linux	<code>SyntaxTutor-<version>-x86_64.AppImage</code>	Ubuntu 22.04, Debian 12 or newer

- **Windows**: unzip and run `SyntaxTutor.exe`.
- **macOS**: unzip, move `SyntaxTutor.app` to *Applications* and open it.
- **Linux**: `chmod +x SyntaxTutor-<version>-x86_64.AppImage` and run it. It uses X11, or XWayland on Wayland desktops; start it with `QT_QPA_PLATFORM=wayland` to run natively on Wayland.

Warning

The Windows and macOS builds are not digitally signed, so the system may warn you the first time you open them. The user manual explains how to open them anyway. Only do so with files downloaded from this repository.

Once the application is open, the **Tutorial** button on the home screen gives a short tour of the interface and the first exercises.

1.0.4 How it works

LL(1). The tutor asks for the size of the predictive table and then, rule by rule, for the prediction symbols. A wrong answer is broken down into the FIRST set of the right-hand side and the FOLLOW set of the left-hand side before the question is asked again. The exercise ends with the LL(1) table, where wrong cells are highlighted and a guided mode is available.

SLR(1). The tutor starts from the closure of the initial item and builds the LR(0) collection state by state: number of items, symbols after the dot and every transition $\delta(I, X)$. It then covers the table dimensions, states with complete items, LR(0) conflicts resolved with FOLLOW sets and reduce-only states, and ends with the SLR(1) parsing table.

1.0.5 Screenshots

	
Home. Choose a tutor, the difficulty, your own grammar or exam mode.	Grammar editor. Live validation; only LL(1) or SLR(1) grammars are accepted.
	
LL(1) tutor. A wrong answer leads to questions about FIRST and FOLLOW.	LL(1) table. Wrong cells are highlighted after each submission.
	
SLR(1) tutor. Building the LR(0) collection, transition by transition.	LR(0) automaton. It grows as each state is computed.
	
SLR(1) table. sN, rN, acc and goto entries.	Guided mode. One cell at a time, with a hint for each.
	
LL(1) guided mode. The same help for the predictive table.	Exam report. The grade and a review of every answer, exportable to PDF.

1.0.6 Documentation

- **User manual:** [English](#) · [Español](#)
- **Developer documentation** (Doxygen): [online](#) · [PDF](#)
- **Changes between versions:** CHANGELOG

The developer documentation covers every class and function, with dependency and inheritance graphs. To regenerate it, install [Doxygen](#) and Graphviz and run `doxygen` from the repository root; the output goes to `docs/`.

1.0.7 Building from source

Requirements:

- Qt 6, including `qmake6` and the Qt SVG module
- A C++20 compiler: GCC 11 or newer, a recent Clang, or MSVC 2022
- [GoogleTest](#), only for the backend tests

With GNU make, the repository's `GNUmakefile` builds out of tree:

```
make app      # the application, in build/app
make check   # backend and GUI tests
```

`make check-core` and `make check-ui` run each suite on its own; the same tests run in CI on every push and pull request. To use `qmake` directly, pass `-f Makefile`, because GNU make would otherwise pick up the `GNUmakefile`:

```
qmake6
make -f Makefile
```

1.0.8 Project structure

Path	Contents
src/backend/	Grammar model and parsing engine: FIRST/FOLLOW, LL(1) table, LR(0) items and the SLR(1) table, in plain C++20 with no Qt dependency.
src/gui/	Main window, the LL(1) and SLR(1) tutors with their question flows, table dialogs, guided modes, exam report and PDF export.
src/widgets/	Reusable widgets and helpers: chat input, grammar view, LR(0) automaton view, tutorial overlay, typography and settings.
src/app/	Application entry point.
tests/	GUI tests with fixed grammar fixtures; the backend tests live in <code>src/backend/↔ tests.cpp</code> .
resources/	Icon, logo, fonts and the application style sheet.
translations/	Spanish and English translations.
manual/, docs/	User and developer manuals, and the generated Doxygen output.

1.0.9 Background

[SyntaxTutor](#) 1.x was the Final Degree Project "Interactive Tutorial About Syntax Analyzers" (2025). Version 2 is a later extension and redesign, developed independently of that project: it adds custom grammars, exam mode, the LR(0) automaton, guided table modes and a new interface. See the [CHANGELOG](#) for the full list.

1.0.10 License

[SyntaxTutor](#) is free software, released under the [\[GNU General Public License v3.0\]\(LICENSE\)](#). It is built with [Qt 6](#), also available under the [GPLv3](#), and bundles the JetBrains Mono font under the [SIL Open Font License](#).

Created by [José R.](#). If you use [SyntaxTutor](#) in a course or build on it, a mention of the project is appreciated.

Chapter 2

Directory Hierarchy

2.1 Directories

app	15
main.cpp	245
backend	15
grammar.cpp	248
grammar.hpp	248
grammar_factory.cpp	250
grammar_factory.hpp	251
grammar_parser.cpp	253
grammar_parser.hpp	254
ll1_parser.cpp	255
ll1_parser.hpp	256
lr0_item.cpp	257
lr0_item.hpp	258
slr1_parser.cpp	259
slr1_parser.hpp	260
state.hpp	262
symbol_table.cpp	263
symbol_table.hpp	263
gui	16
automatonviewerdialog.cpp	265
automatonviewerdialog.h	266
conversationpdf.h	267
examreportdialog.cpp	272
examreportdialog.h	272
examsession.h	274
grammareditordialog.cpp	275
grammareditordialog.h	276
lltabledialog.cpp	278
lltabledialog.h	278
lltutorwindow.cpp	280
lltutorwindow.h	280
llwizard.h	286
llwizardpage.h	289
mainwindow.cpp	292
mainwindow.h	292
slrtabledialog.cpp	295
slrtabledialog.h	295

slrtutorwindow.cpp	297
slrtutorwindow.h	298
slrwizard.h	304
slrwizardpage.h	309
UniqueQueue.h	311
widgets	16
appfonts.h	313
appicon.h	314
applayout.h	316
apppalette.h	318
appsettings.h	319
appshortcuts.h	321
apptextscale.h	322
apptypography.h	324
automatonview.cpp	327
automatonview.h	328
customtextedit.cpp	330
customtextedit.h	331
grammarview.cpp	332
grammarview.h	333
tutorialmanager.cpp	334
tutorialmanager.h	334

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all namespaces with brief descriptions:

AppIcon	19
AppLayout	
Layout helpers that keep text from being clipped	20
AppPalette	21
AppSettings	
Where the app keeps its settings	22
AppShortcuts	24
AppTextScale	
The user's text size preference, as a percentage	24
AppTypography	
The application's type scale: every text size on screen, in points	29
SyntaxTutor	36
SyntaxTutor::Version	36

Chapter 4

Hierarchical Index

4.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AutomatonStateInfo	39
AutomatonTransitionInfo	40
ConversationPdf	52
ExamRecord	61
ExamSession	65
GrammarFactory::FactoryItem	68
Grammar	70
GrammarFactory	80
GrammarParseError	101
GrammarParser	102
GrammarParseResult	104
LL1Parser	111
Lr0Item	154
ConversationPdf::Message	163
QDialog	
AutomatonViewerDialog	49
ExamReportDialog	63
GrammarEditorDialog	77
LLTableDialog	118
LLWizard	149
SLRTableDialog	179
SLRWizard	221
QFrame	
GrammarView	107
QGraphicsView	
AutomatonView	41
QMainWindow	
MainWindow	159
QObject	
AppLayout::HeightForWidthKeeper	109
TutorialManager	234
QStyledItemDelegate	
CenterAlignDelegate	51
CenterAlignDelegate	51
QSyntaxHighlighter	

GrammarSyntaxHighlighter	106
QTextEdit	
CustomTextEdit	57
QWidget	
LLTutorWindow	122
LLWizardPage	151
SLRTutorWindow	183
SLRWizardPage	224
GrammarView::Row	164
SLR1Parser::s_action	165
SLR1Parser	166
state	227
SymbolTable	228
AppTypography::Token	233
LLTutorWindow::TreeNode	233
TutorialStep	240
UniqueQueue< T >	241

Chapter 5

Class Index

5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AutomatonStateInfo	
One LR(0) state of the automaton to display	39
AutomatonTransitionInfo	
One delta(I, X) transition of the automaton to display	40
AutomatonView	
Progressive LR(0) automaton visualization	41
AutomatonViewerDialog	
Non-modal floating window that hosts the LR(0) automaton view	49
CenterAlignDelegate	
.	51
ConversationPdf	
Builds the PDF of an LL(1) or SLR(1) session	52
CustomTextEdit	
.	57
ExamRecord	
One graded item of an exam: a question (or table cell), the user's answer, the expected answer, and whether it was correct	61
ExamReportDialog	
End-of-exam report: grade, stats and a question-by-question review	63
ExamSession	
Accumulates graded answers during an exam-mode exercise	65
GrammarFactory::FactoryItem	
Represents an individual grammar item with its associated symbol table	68
Grammar	
Represents a context-free grammar, including its rules, symbol table, and starting symbol	70
GrammarEditorDialog	
Dialog where the user writes their own grammar for an exercise	77
GrammarFactory	
Responsible for creating and managing grammar items and performing checks on grammars . .	80
GrammarParseError	
Describes one problem found while parsing user-written grammar text	101
GrammarParser	
Parses user-written grammar text into a Grammar object	102
GrammarParseResult	
Outcome of parsing user-written grammar text	104
GrammarSyntaxHighlighter	
Lightweight highlighter for user-written grammar text	106

GrammarView	107
AppLayout::HeightForWidthKeeper	
Event filter behind keepHeightForWidth	109
LL1Parser	111
LLTableDialog	
Dialog for filling and submitting an LL(1) parsing table	118
LLTutorWindow	
Main window for the LL(1) interactive tutoring mode in SyntaxTutor	122
LLWizard	
Interactive assistant that guides the student step-by-step through the LL(1) parsing table	149
LLWizardPage	
A single guided step for constructing the LL(1) table	151
Lr0Item	
Represents an LR(0) item used in LR automata construction	154
MainWindow	
Main application window of SyntaxTutor , managing levels, exercises, and UI state	159
ConversationPdf::Message	
One message of the conversation	163
GrammarView::Row	164
SLR1Parser::s_action	165
SLR1Parser	
Implements an SLR(1) parser for context-free grammars	166
SLRTableDialog	
Dialog window for completing and submitting an SLR(1) parsing table	179
SLRTutorWindow	
Main window for the SLR(1) interactive tutoring mode in SyntaxTutor	183
SLRWizard	
Interactive assistant that guides the student step-by-step through the SLR(1) parsing table	221
SLRWizardPage	
A single guided step for constructing the SLR(1) table	224
state	
Represents a state in the LR(0) automaton	227
SymbolTable	
Stores and manages grammar symbols, including their classification and special markers	228
AppTypography::Token	
One entry of the type scale	233
LLTutorWindow::TreeNode	
TreeNode structure used to build derivation trees	233
TutorialManager	
Manages interactive tutorials by highlighting UI elements and guiding the user	234
TutorialStep	
Represents a single step in the tutorial sequence	240
UniqueQueue< T >	
A queue that ensures each element is inserted only once	241

Chapter 6

File Index

6.1 File List

Here is a list of all files with brief descriptions:

appversion.h	247
app/main.cpp	245
backend/grammar.cpp	248
backend/grammar.hpp	248
backend/grammar_factory.cpp	250
backend/grammar_factory.hpp	251
backend/grammar_parser.cpp	253
backend/grammar_parser.hpp	254
backend/ll1_parser.cpp	255
backend/ll1_parser.hpp	256
backend/lr0_item.cpp	257
backend/lr0_item.hpp	258
backend/slr1_parser.cpp	259
backend/slr1_parser.hpp	260
backend/state.hpp	262
backend/symbol_table.cpp	263
backend/symbol_table.hpp	263
gui/automatonviewerdialog.cpp	265
gui/automatonviewerdialog.h	266
gui/conversationpdf.h	267
gui/examreportdialog.cpp	272
gui/examreportdialog.h	272
gui/examsession.h	274
gui/grammareditordialog.cpp	275
gui/grammareditordialog.h	276
gui/lltabledialog.cpp	278
gui/lltabledialog.h	278
gui/lltutorwindow.cpp	280
gui/lltutorwindow.h	280
gui/llwizard.h	286
gui/llwizardpage.h	289
gui/mainwindow.cpp	292
gui/mainwindow.h	292
gui/slrtabledialog.cpp	295
gui/slrtabledialog.h	295

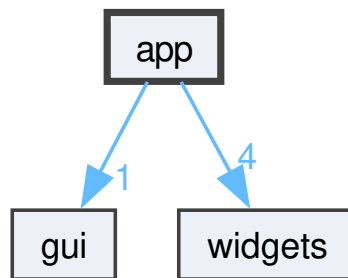
gui/slrtutorwindow.cpp	297
gui/slrtutorwindow.h	298
gui/slrwizard.h	304
gui/slrwizardpage.h	309
gui/UniqueQueue.h	311
widgets/appfonts.h	313
widgets/appicon.h	314
widgets/applayout.h	316
widgets/apppalette.h	318
widgets/appsettings.h	319
widgets/appshortcuts.h	321
widgets/apptextscale.h	322
widgets/apptypography.h	324
widgets/automatonview.cpp	327
widgets/automatonview.h	328
widgets/customtextedit.cpp	330
widgets/customtextedit.h	331
widgets/grammarview.cpp	332
widgets/grammarview.h	333
widgets/tutorialmanager.cpp	334
widgets/tutorialmanager.h	334

Chapter 7

Directory Documentation

7.1 app Directory Reference

Directory dependency graph for app:



Files

- file [main.cpp](#)

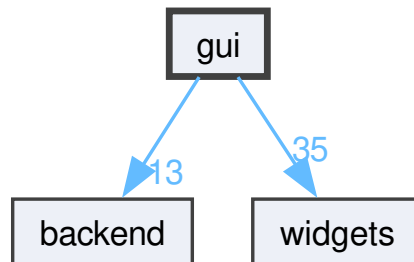
7.2 backend Directory Reference

Files

- file [grammar.cpp](#)
- file [grammar.hpp](#)
- file [grammar_factory.cpp](#)
- file [grammar_factory.hpp](#)
- file [grammar_parser.cpp](#)
- file [grammar_parser.hpp](#)
- file [ll1_parser.cpp](#)
- file [ll1_parser.hpp](#)
- file [lr0_item.cpp](#)
- file [lr0_item.hpp](#)
- file [slr1_parser.cpp](#)
- file [slr1_parser.hpp](#)
- file [state.hpp](#)
- file [symbol_table.cpp](#)
- file [symbol_table.hpp](#)

7.3 gui Directory Reference

Directory dependency graph for gui:



Files

- file [automatonviewerdialog.cpp](#)
- file [automatonviewerdialog.h](#)
- file [conversationpdf.h](#)
- file [examreportdialog.cpp](#)
- file [examreportdialog.h](#)
- file [examsession.h](#)
- file [grammareditordialog.cpp](#)
- file [grammareditordialog.h](#)
- file [ltabledialog.cpp](#)
- file [ltabledialog.h](#)
- file [ltutorwindow.cpp](#)
- file [ltutorwindow.h](#)
- file [llwizard.h](#)
- file [llwizardpage.h](#)
- file [mainwindow.cpp](#)
- file [mainwindow.h](#)
- file [slrtabledialog.cpp](#)
- file [slrtabledialog.h](#)
- file [slrtutorwindow.cpp](#)
- file [slrtutorwindow.h](#)
- file [slrwizard.h](#)
- file [slrwizardpage.h](#)
- file [UniqueQueue.h](#)

7.4 widgets Directory Reference

Files

- file [appfonts.h](#)
- file [appicon.h](#)
- file [applayout.h](#)
- file [apppalette.h](#)
- file [appsettings.h](#)
- file [appshortcuts.h](#)
- file [apptextscale.h](#)

- file [apptypography.h](#)
- file [automatonview.cpp](#)
- file [automatonview.h](#)
- file [customtextedit.cpp](#)
- file [customtextedit.h](#)
- file [grammarview.cpp](#)
- file [grammarview.h](#)
- file [tutorialmanager.cpp](#)
- file [tutorialmanager.h](#)

Chapter 8

Namespace Documentation

8.1 Applcon Namespace Reference

Functions

- `QIcon icon ()`
The application icon, with a drawing for each size.

Variables

- `constexpr int kSizes [] = {16, 24, 32, 48, 64, 128, 256, 512}`
Sizes of the bitmap icon shipped in the resources, in pixels.

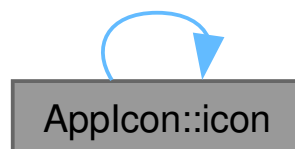
8.1.1 Function Documentation

8.1.1.1 icon()

```
QIcon AppIcon::icon () [inline]
```

The application icon, with a drawing for each size.

Every size is its own bitmap, so a 16 px title bar icon is drawn for 16 px rather than shrunk from a large image. Set once on the application, it is the icon of every window and dialog. Here is the call graph for this function:



Here is the caller graph for this function:



8.1.2 Variable Documentation

8.1.2.1 kSizes

```
int AppIcon::kSizes[] = {16, 24, 32, 48, 64, 128, 256, 512} [constexpr]
```

Sizes of the bitmap icon shipped in the resources, in pixels.

8.2 AppLayout Namespace Reference

Layout helpers that keep text from being clipped.

Classes

- class [HeightForWidthKeeper](#)
Event filter behind [keepHeightForWidth](#).

Functions

- void [keepHeightForWidth](#) (QWidget *widget)
Keeps *widget* at least as tall as its layout needs at its current width.

8.2.1 Detailed Description

Layout helpers that keep text from being clipped.

Qt sizes word-wrapped text by height-for-width, but only some containers honour it. Top-level windows and widgets placed directly in a QMainWindow or a QStackedWidget are given the height their layout guesses for an arbitrary width, so a title that wraps to one more line than the guess is cut in half. The effect depends on font, text size and window width, which is why it only shows on some systems.

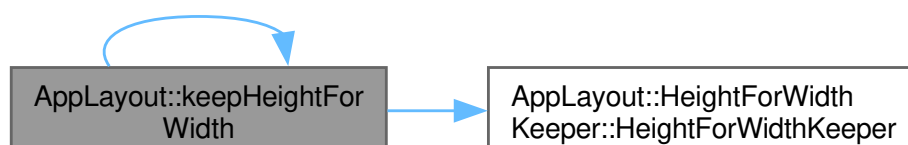
8.2.2 Function Documentation

8.2.2.1 keepHeightForWidth()

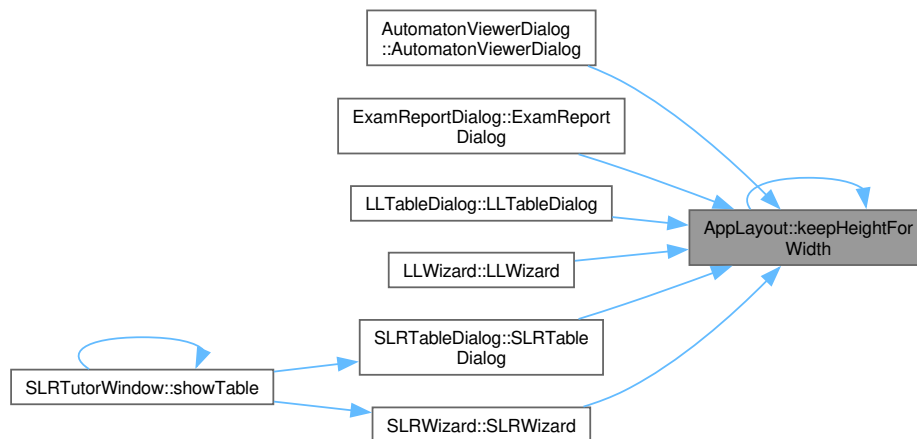
```
void AppLayout::keepHeightForWidth (
    QWidget * widget) [inline]
```

Keeps *widget* at least as tall as its layout needs at its current width.

Re-evaluated whenever the widget is resized or its layout changes, so it follows the window width, the language and the text size. Use it on windows and on containers whose parent ignores height-for-width. Here is the call graph for this function:



Here is the caller graph for this function:



8.3 AppPalette Namespace Reference

Functions

- QColor [linkColor](#) ()
Colour of links in rich text: the teal accent of the theme.
- void [apply](#) ()
Palette entries the theme needs that app.qss cannot set.

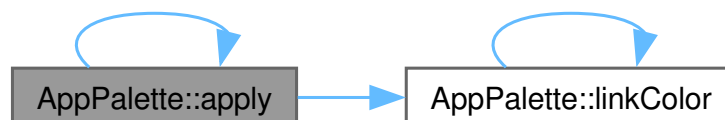
8.3.1 Function Documentation

8.3.1.1 apply()

```
void AppPalette::apply () [inline]
```

Palette entries the theme needs that app.qss cannot set.

Links in rich text (QLabel, QTextBrowser) take their colour from the palette; a style sheet rule on `a` has no effect. The platform's light palette makes them dark blue, barely readable on this dark theme - which is what Linux showed, while macOS hid it with a dark palette of its own. Here is the call graph for this function:



Here is the caller graph for this function:



8.3.1.2 linkColor()

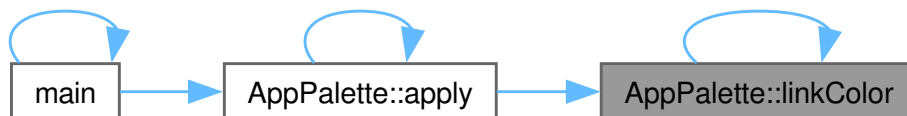
```
QColor AppPalette::linkColor () [inline]
```

Colour of links in rich text: the teal accent of the theme.

Here is the call graph for this function:



Here is the caller graph for this function:



8.4 AppSettings Namespace Reference

Where the app keeps its settings.

Functions

- void [registerIdentity](#) ()
Registers the organisation, domain and application name.
- QSettings [open](#) ()
The app's settings.

8.4.1 Detailed Description

Where the app keeps its settings.

The identity is registered once at startup and every QSettings is built from it, so each platform files the settings its own way:

- macOS: `~/Library/Preferences/me.jram.SyntaxTutor.plist` (the domain)
- Windows: `HKEY_CURRENT_USER\Software\SyntaxTutor\SyntaxTutor`
- Linux: `~/config/SyntaxTutor/SyntaxTutor.conf`

8.4.2 Function Documentation

8.4.2.1 open()

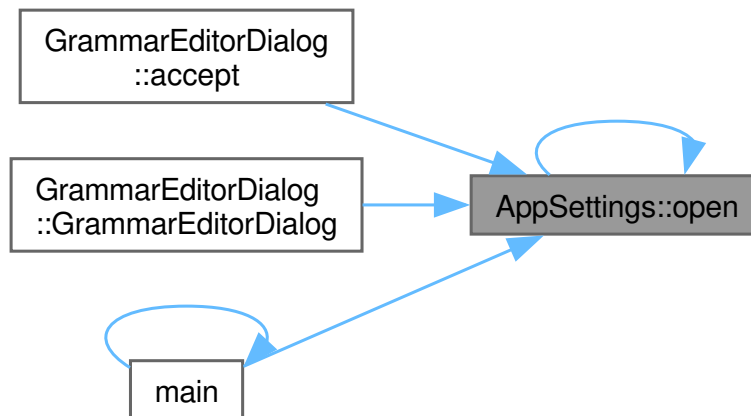
```
QSettings AppSettings::open () [inline]
```

The app's settings.

Built without explicit names so macOS uses the registered domain. Test builds use a store of their own, so running the tests never touches the user's progress. Here is the call graph for this function:



Here is the caller graph for this function:



8.4.2.2 registerIdentity()

```
void AppSettings::registerIdentity () [inline]
```

Registers the organisation, domain and application name.

Here is the call graph for this function:



Here is the caller graph for this function:



8.5 AppShortcuts Namespace Reference

Functions

- QString [primaryModifierName](#) ()
Name of the key behind Qt::ControlModifier on this platform.

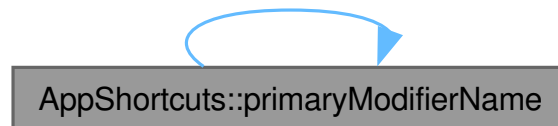
8.5.1 Function Documentation

8.5.1.1 primaryModifierName()

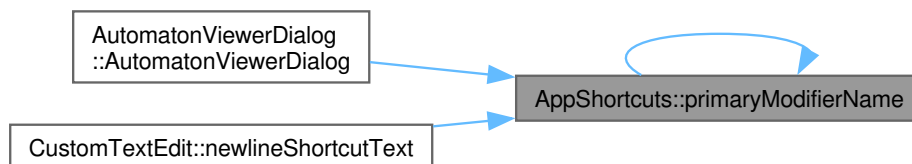
QString AppShortcuts::primaryModifierName () [inline]

Name of the key behind Qt::ControlModifier on this platform.

On macOS Qt maps ControlModifier to the Command key, so a shortcut written for Ctrl is pressed with Cmd there. Use this in any text that names that key, so it matches what the user actually presses. Here is the call graph for this function:



Here is the caller graph for this function:



8.6 AppTextScale Namespace Reference

The user's text size preference, as a percentage.

Functions

- int [clampPercent](#) (int percent)
Keeps a percentage inside the allowed range.
- int & [currentPercent](#) ()
The selected size, in percent.
- double [factor](#) ()
Multiplier currently in effect.
- QString [settingsKey](#) ()
Settings key holding the selected size, in percent.
- QString [legacySettingsKey](#) ()
Key used by the first version of this setting, which named steps.
- int [percentFromLegacyValue](#) (const QString &value)
Percentage matching one of the original named steps.

Variables

- constexpr int `kMinPercent` = 80
Smallest text size the user can pick, in percent.
- constexpr int `kMaxPercent` = 200
Largest text size the user can pick, in percent.
- constexpr int `kDefaultPercent` = 100
Size used until the user picks one, in percent.
- constexpr int `kPresetPercents` [] = {90, 100, 115, 130, 150, 175, 200}
Sizes offered directly in the menu, in percent.

8.6.1 Detailed Description

The user's text size preference, as a percentage.

Only the preference lives here: how big each piece of text is comes from the type scale in [AppTypography](#), which multiplies its point sizes by [factor](#). The platform's own point-to-pixel mapping is left to Qt, so 100% means "as designed" on every system rather than a DPI correction.

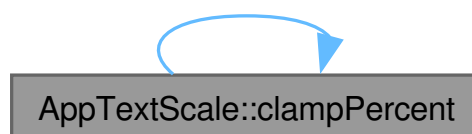
8.6.2 Function Documentation

8.6.2.1 clampPercent()

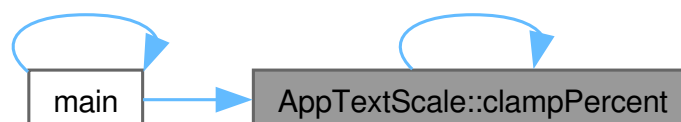
```
int AppTextScale::clampPercent (  
    int percent) [inline]
```

Keeps a percentage inside the allowed range.

Here is the call graph for this function:



Here is the caller graph for this function:

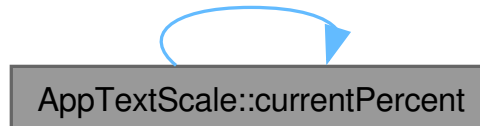


8.6.2.2 currentPercent()

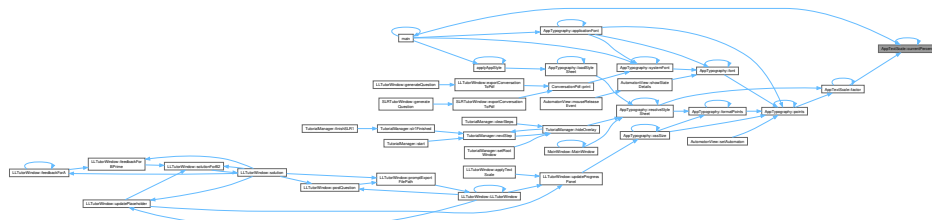
```
int & AppTextScale::currentPercent () [inline]
```

The selected size, in percent.

An inline function with a static local so every translation unit shares one instance without needing a .cpp. Here is the call graph for this function:



Here is the caller graph for this function:



8.6.2.3 factor()

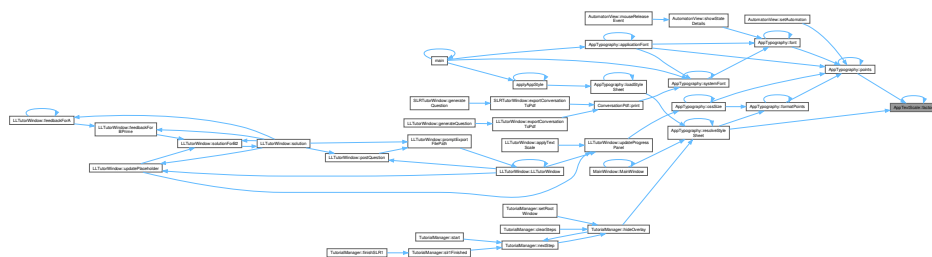
```
double AppTextScale::factor () [inline]
```

Multiplier currently in effect.

Here is the call graph for this function:



Here is the caller graph for this function:

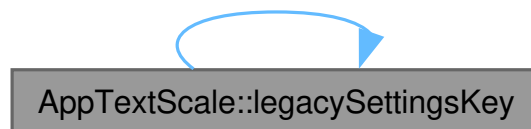


8.6.2.4 legacySettingsKey()

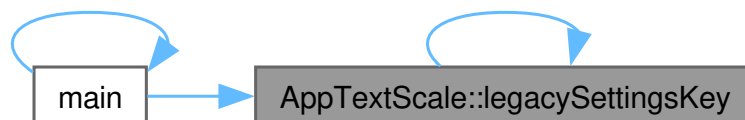
```
QString AppTextScale::legacySettingsKey () [inline]
```

Key used by the first version of this setting, which named steps.

Here is the call graph for this function:



Here is the caller graph for this function:

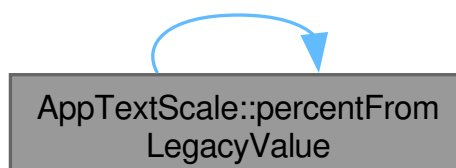


8.6.2.5 percentFromLegacyValue()

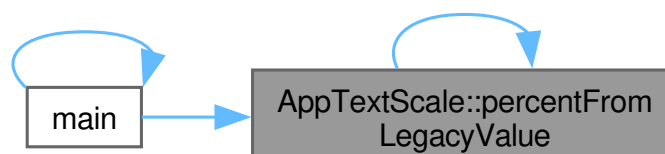
```
int AppTextScale::percentFromLegacyValue (
    const QString & value) [inline]
```

Percentage matching one of the original named steps.

Here is the call graph for this function:



Here is the caller graph for this function:



8.6.2.6 settingsKey()

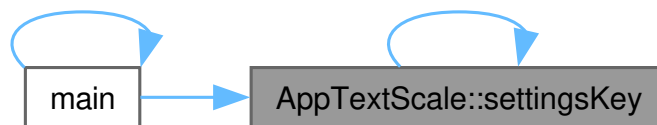
```
QString AppTextScale::settingsKey () [inline]
```

Settings key holding the selected size, in percent.

Here is the call graph for this function:



Here is the caller graph for this function:



8.6.3 Variable Documentation

8.6.3.1 kDefaultPercent

```
int AppTextScale::kDefaultPercent = 100 [constexpr]
```

Size used until the user picks one, in percent.

8.6.3.2 kMaxPercent

```
int AppTextScale::kMaxPercent = 200 [constexpr]
```

Largest text size the user can pick, in percent.

8.6.3.3 kMinPercent

```
int AppTextScale::kMinPercent = 80 [constexpr]
```

Smallest text size the user can pick, in percent.

8.6.3.4 kPresetPercents

```
int AppTextScale::kPresetPercents[] = {90, 100, 115, 130, 150, 175, 200} [constexpr]
```

Sizes offered directly in the menu, in percent.

8.7 AppTypography Namespace Reference

The application's type scale: every text size on screen, in points.

Classes

- struct [Token](#)
One entry of the type scale.

Enumerations

- enum class [Role](#) {
 [Micro](#) , [Caption](#) , [Label](#) , [Body](#) ,
 [BodyLarge](#) , [Reading](#) , [Action](#) , [Headline](#) ,
 [TitleSmall](#) , [Title](#) , [Hero](#) , [Display](#) }
What a piece of text is for, which fixes its size.

Functions

- qreal [basePoints](#) ([Role](#) role)
Size of `role` at a 100% text scale, in points.
- qreal [points](#) ([Role](#) role)
Size of `role` at the current text scale, in points.
- QString [formatPoints](#) (qreal points)
A point size written the way CSS and QSS expect it, e.g. 14.5pt.
- QString [cssSize](#) ([Role](#) role)
Current size of `role` as a CSS length, for rich text and QSS.
- QFont & [systemFont](#) ()
The system font as it was before the app touched it.
- QFont [applicationFont](#) ()
The application font: the system family at the body size.
- QFont [font](#) ([Role](#) role, QFont base=QGuiApplication::font())
base resized to `role`, keeping its family and style.
- QString [resolveStyleSheet](#) (const QString &styleSheet, QStringList *unresolved=nullptr)
Replaces every `$font-<name>` in `styleSheet` with its size.
- QString [loadStyleSheet](#) ()
The application style sheet with its sizes resolved.
- int [lengthForText](#) (int designPx)
A length that has to hold text, grown to how big text renders here.

Variables

- constexpr [Token](#) [kTokens](#) []
The type scale.

8.7.1 Detailed Description

The application's type scale: every text size on screen, in points.

Fonts are sized in points so Qt can map them to each platform's logical DPI; nothing here reads the DPI or the device pixel ratio. The user's text size preference ([AppTextScale](#)) is a plain multiplier on top.

`app.qss` never spells a size out. It names a role, as in `font-size: $font-body;`, and [resolveStyleSheet](#) substitutes the scaled value when the sheet is loaded. C++ code asks for the same roles through `font` or `cssSize`, so both channels share one table.

Paper is a separate domain: the PDF exports keep their own `pt` sizes and never follow the screen preference.

8.7.2 Enumeration Type Documentation

8.7.2.1 Role

```
enum class AppTypography::Role [strong]
```

What a piece of text is for, which fixes its size.

Enumerator

Micro	Automaton edge labels.
Caption	Uppercase captions and the progress panel.
Label	Eyebrows, badges, small buttons and hints.
Body	Default UI text: chat, tables, dialog buttons.
BodyLarge	Primary buttons and subtitles.
Reading	Main reading size: answers, hints, grammar, home options.
Action	The large actions on the home page.
Headline	Transient celebration text.
TitleSmall	Titles of the small information dialogs.
Title	Titles of the large dialogs.
Hero	The home page title.
Display	The exam grade.

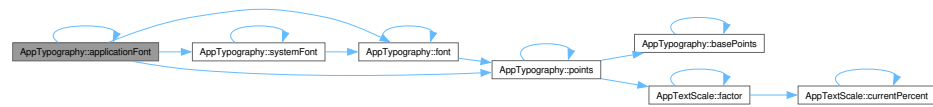
8.7.3 Function Documentation

8.7.3.1 applicationFont()

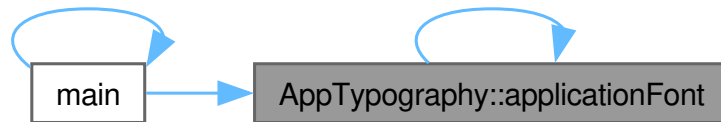
```
QFont AppTypography::applicationFont () [inline]
```

The application font: the system family at the body size.

Here is the call graph for this function:



Here is the caller graph for this function:



8.7.3.2 basePoints()

```

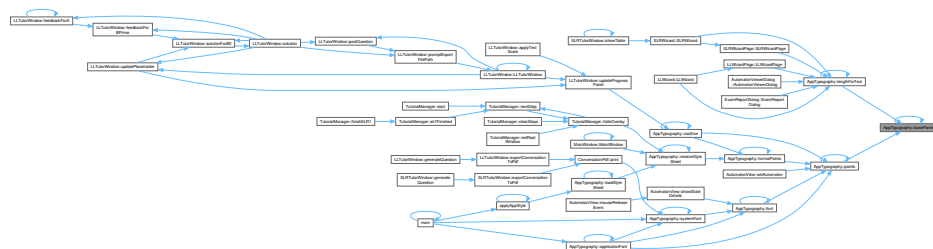
qreal AppTypography::basePoints (
    Role role) [inline]
  
```

Size of `role` at a 100% text scale, in points.

Here is the call graph for this function:



Here is the caller graph for this function:



8.7.3.3 cssSize()

```

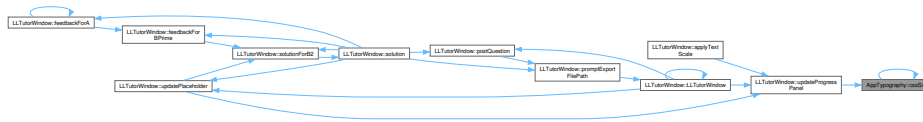
QString AppTypography::cssSize (
    Role role) [inline]
  
```

Current size of `role` as a CSS length, for rich text and QSS.

Here is the call graph for this function:



Here is the caller graph for this function:



8.7.3.4 font()

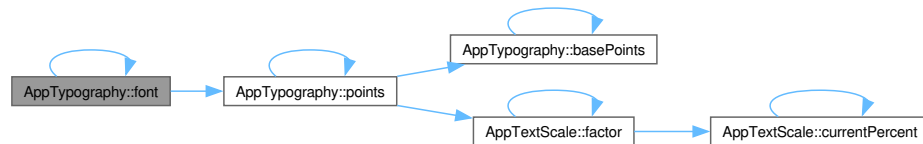
```

QFont AppTypography::font (
    Role role,
    QFont base = QApplication::font()) [inline]

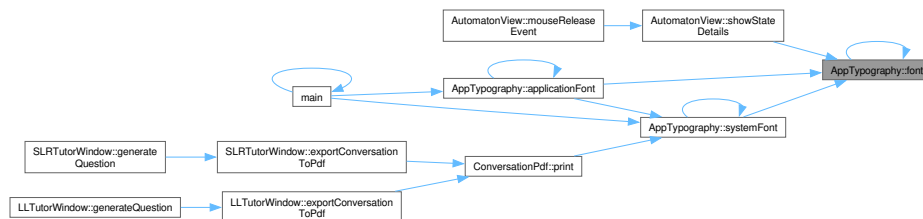
```

base resized to role, keeping its family and style.

Here is the call graph for this function:



Here is the caller graph for this function:



8.7.3.5 formatPoints()

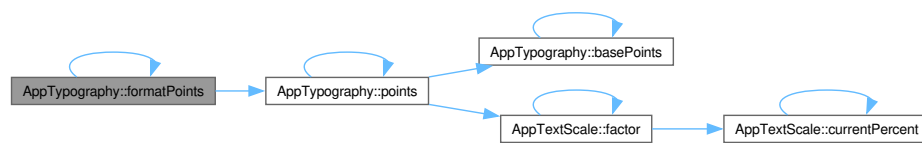
```

QString AppTypography::formatPoints (
    qreal points) [inline]

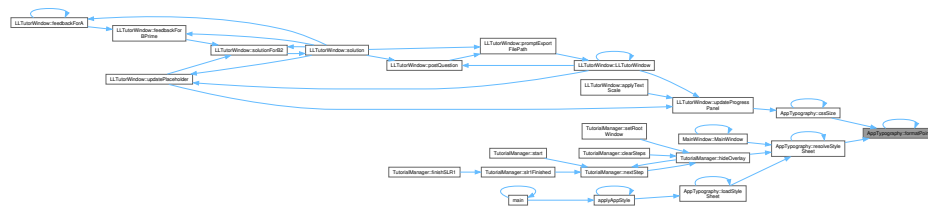
```

A point size written the way CSS and QSS expect it, e.g. 14.5pt.

Here is the call graph for this function:



Here is the caller graph for this function:



8.7.3.6 lengthForText()

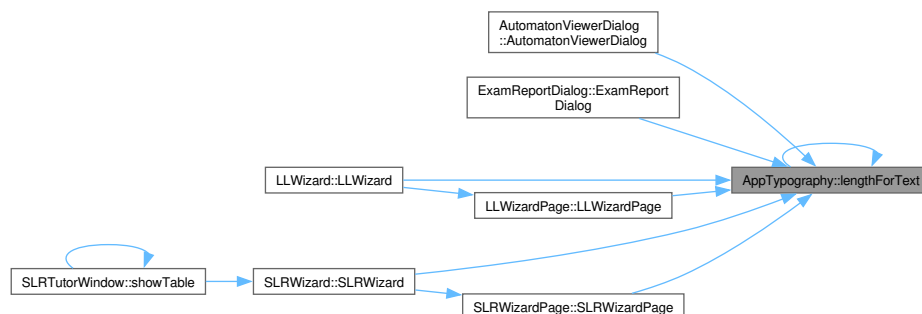
```
int AppTypography::lengthForText (
    int designPx) [inline]
```

A length that has to hold text, grown to how big text renders here.

`designPx` is the length the layout was drawn with, when body text was [Role::Body](#) points - that is, as many pixels - high. It grows in step with the body font as it actually renders, which covers both the user's text scale and the platform's point-to-pixel mapping. Use it for fixed or minimum sizes of things that contain text; leave decoration in plain logical pixels. Here is the call graph for this function:

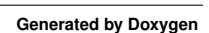


Here is the caller graph for this function:



```
QString AppTypography::loadStyleSheet () [inline]
```

Here is the call graph for this function:

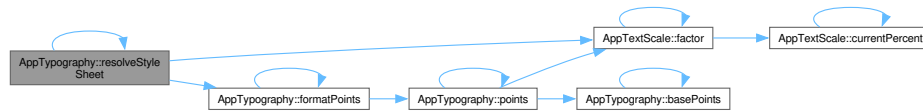


8.7.3.9 resolveStyleSheet()

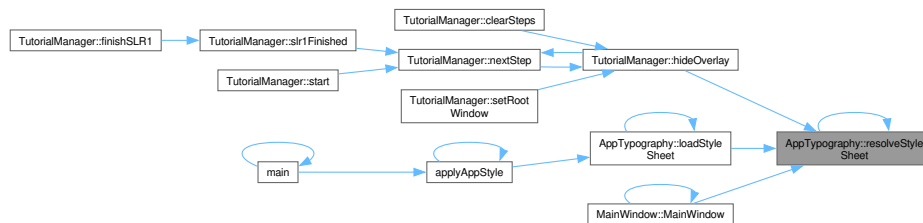
```
QString AppTypography::resolveStyleSheet (
    const QString & styleSheet,
    QStringList * unresolved = nullptr) [inline]
```

Replaces every \$font-<name> in styleSheet with its size.

Names that are not in the type scale are left untouched, so Qt drops the declaration, and are reported through `unresolved` when given. Here is the call graph for this function:



Here is the caller graph for this function:

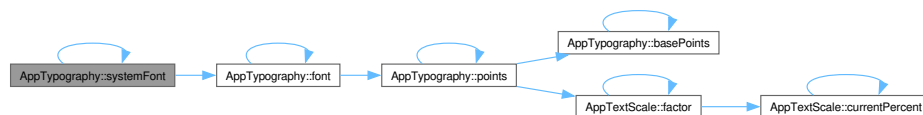


8.7.3.10 systemFont()

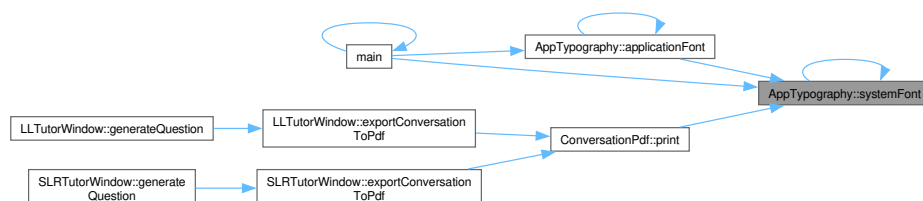
```
QFont & AppTypography::systemFont () [inline]
```

The system font as it was before the app touched it.

Captured once at startup. Only its family and rendering hints are used: the size always comes from the type scale. Here is the call graph for this function:



Here is the caller graph for this function:



8.7.4 Variable Documentation

8.7.4.1 kTokens

`Token` `AppTypography::kTokens[]` `[inline]`, `[constexpr]`

Initial value:

```

= {
    {Role::Micro, QLatin1StringView("micro"), 10.0},
    {Role::Caption, QLatin1StringView("caption"), 11.0},
    {Role::Label, QLatin1StringView("label"), 12.0},
    {Role::Body, QLatin1StringView("body"), 13.0},
    {Role::BodyLarge, QLatin1StringView("body-large"), 14.0},
    {Role::Reading, QLatin1StringView("reading"), 15.0},
    {Role::Action, QLatin1StringView("action"), 16.0},
    {Role::Headline, QLatin1StringView("headline"), 20.0},
    {Role::TitleSmall, QLatin1StringView("title-small"), 24.0},
    {Role::Title, QLatin1StringView("title"), 28.0},
    {Role::Hero, QLatin1StringView("hero"), 30.0},
    {Role::Display, QLatin1StringView("display"), 56.0},
}

```

The type scale.

The values reproduce the original pixel design exactly on macOS, the platform it was drawn on, where one point and one logical pixel coincide. Other platforms get the same physical size instead of the same pixel count.

8.8 SyntaxTutor Namespace Reference

Namespaces

- namespace [Version](#)

8.9 SyntaxTutor::Version Namespace Reference

Functions

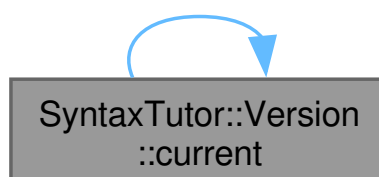
- `QString` [current](#) ()
- `const char *` [raw](#) ()

8.9.1 Function Documentation

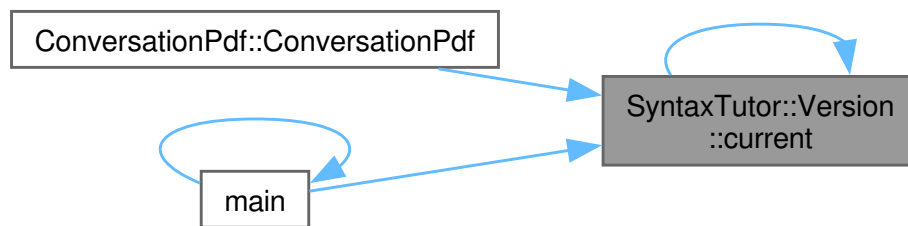
8.9.1.1 current()

`QString` `SyntaxTutor::Version::current` () `[inline]`

Here is the call graph for this function:



Here is the caller graph for this function:



8.9.1.2 raw()

```
const char * SyntaxTutor::Version::raw () [inline]
```

Here is the call graph for this function:



Here is the caller graph for this function:



Chapter 9

Class Documentation

9.1 AutomatonStateInfo Struct Reference

One LR(0) state of the automaton to display.

```
#include <automatonview.h>
```

Public Attributes

- unsigned [id](#)
State id (the N in "IN").
- QString [itemsText](#)
Items of the state, shown when hovering or clicking the node.

9.1.1 Detailed Description

One LR(0) state of the automaton to display.

9.1.2 Member Data Documentation

9.1.2.1 id

```
unsigned AutomatonStateInfo::id
```

[State](#) id (the N in "IN").

9.1.2.2 itemsText

```
QString AutomatonStateInfo::itemsText
```

Items of the state, shown when hovering or clicking the node.

The documentation for this struct was generated from the following file:

- widgets/[automatonview.h](#)

9.2 AutomatonTransitionInfo Struct Reference

One delta(l, X) transition of the automaton to display.

```
#include <automatonview.h>
```

Public Attributes

- unsigned [from](#)
- QString [symbol](#)
- unsigned [to](#)

9.2.1 Detailed Description

One delta(l, X) transition of the automaton to display.

9.2.2 Member Data Documentation

9.2.2.1 from

```
unsigned AutomatonTransitionInfo::from
```

9.2.2.2 symbol

```
QString AutomatonTransitionInfo::symbol
```

9.2.2.3 to

```
unsigned AutomatonTransitionInfo::to
```

The documentation for this struct was generated from the following file:

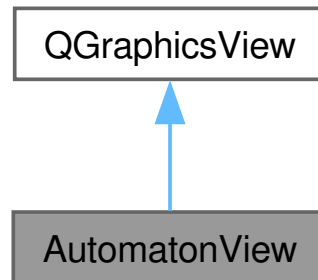
- widgets/[automatonview.h](#)

9.3 AutomatonView Class Reference

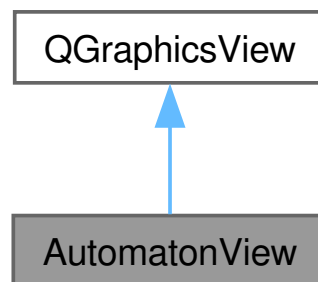
Progressive LR(0) automaton visualization.

```
#include <automatonview.h>
```

Inheritance diagram for AutomatonView:



Collaboration diagram for AutomatonView:



Public Member Functions

- [AutomatonView](#) (QWidget *parent=nullptr)
- void [setAutomaton](#) (const QVector< [AutomatonStateInfo](#) > &states, const QVector< [AutomatonTransitionInfo](#) > &transitions)
Loads the automaton and computes the layout. Everything starts hidden.
- void [revealState](#) (unsigned id)
Makes a state node visible.
- void [revealTransition](#) (unsigned from, unsigned to)
Makes a transition edge visible, together with both endpoint states.
- void [revealAll](#) ()
Reveals every state and transition (consultation mode).
- void [setCurrentState](#) (int id)
Highlights one state as the one being analyzed (-1 for none).
- void [setConflictStates](#) (const QSet< unsigned > &ids)
Marks states that contain an LR(0) conflict.
- void [setReduceStates](#) (const QSet< unsigned > &ids)
Marks states that can perform a reduction.
- void [clearStateMarks](#) ()
Clears conflict and reduce marks.

- void [zoomIn](#) ()
Zooms in one step around the view center.
- void [zoomOut](#) ()
Zooms out one step around the view center.
- void [fitToView](#) ()
Fits all revealed content within the viewport.
- void [centerOnCurrentState](#) ()
Centers (and reveals if needed) the currently inspected state.
- bool [isStateVisible](#) (unsigned id) const
- int [visibleStateCount](#) () const
- int [totalStateCount](#) () const
- int [visibleTransitionCount](#) () const
- bool [isFullyRevealed](#) () const
- int [currentStateId](#) () const
- QSet< unsigned > [conflictStates](#) () const
- QSet< unsigned > [reduceStates](#) () const
- int [shownStateId](#) () const
Id of the state whose items are shown, or -1.
- void [showStateDetails](#) (unsigned id)
*Shows the items of state *id* next to its node.*
- void [hideStateDetails](#) ()
Hides the items shown by a click, if any.

Protected Member Functions

- void [wheelEvent](#) (QWheelEvent *event) override
- void [mousePressEvent](#) (QMouseEvent *event) override
- void [mouseReleaseEvent](#) (QMouseEvent *event) override
- void [keyPressEvent](#) (QKeyEvent *event) override

9.3.1 Detailed Description

Progressive LR(0) automaton visualization.

QGraphicsView-based panel that renders the LR(0) collection as a graph of state nodes (l0, l1, ...) and labelled transition edges. The full automaton is loaded up front with a deterministic BFS-level layout, but every node and edge starts hidden: the SLR tutor reveals them as the student constructs them, so the panel never spoils the construction. Once the collection is complete the tutor calls [revealAll\(\)](#) and the view becomes a stable consultation tool, with optional highlights for the current, conflict and reduce states.

Hovering or clicking a node shows the LR(0) items of that state. The view supports Ctrl+wheel (Cmd+wheel on macOS) zooming and drag scrolling.

9.3.2 Constructor & Destructor Documentation

9.3.2.1 AutomatonView()

```
AutomatonView::AutomatonView (
    QWidget * parent = nullptr) [explicit]
```

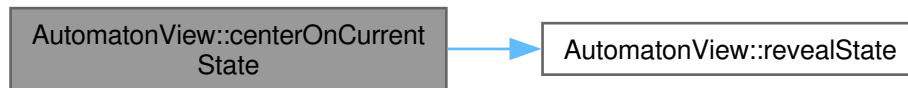

9.3.3 Member Function Documentation

9.3.3.1 centerOnCurrentState()

```
void AutomatonView::centerOnCurrentState ()
```

Centers (and reveals if needed) the currently inspected state.

Here is the call graph for this function:



Here is the caller graph for this function:



9.3.3.2 clearStateMarks()

```
void AutomatonView::clearStateMarks ()
```

Clears conflict and reduce marks.

9.3.3.3 conflictStates()

```
QSet< unsigned > AutomatonView::conflictStates () const [inline]
```

9.3.3.4 currentStateId()

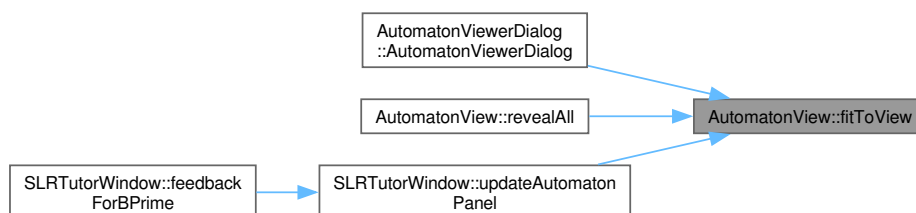
```
int AutomatonView::currentStateId () const [inline]
```

9.3.3.5 fitToView()

```
void AutomatonView::fitToView ()
```

Fits all revealed content within the viewport.

Here is the caller graph for this function:

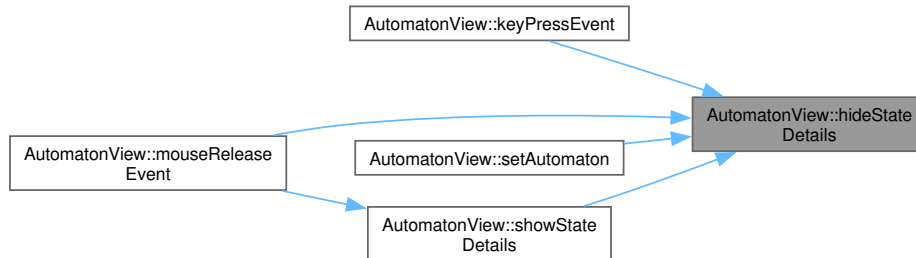


9.3.3.6 hideStateDetails()

```
void AutomatonView::hideStateDetails ()
```

Hides the items shown by a click, if any.

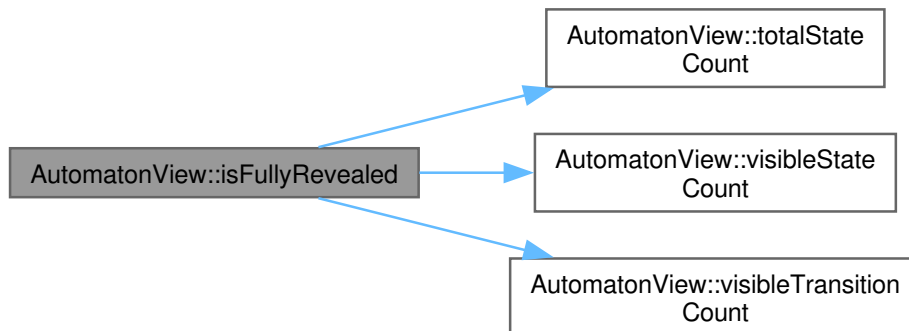
Here is the caller graph for this function:



9.3.3.7 isFullyRevealed()

```
bool AutomatonView::isFullyRevealed () const
```

Here is the call graph for this function:



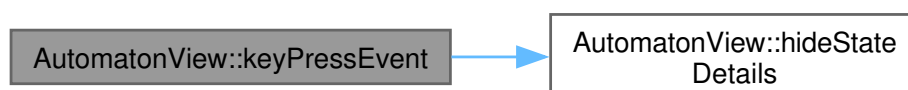
9.3.3.8 isStateVisible()

```
bool AutomatonView::isStateVisible (
    unsigned id) const
```

9.3.3.9 keyPressEvent()

```
void AutomatonView::keyPressEvent (
    QKeyEvent * event) [override], [protected]
```

Here is the call graph for this function:



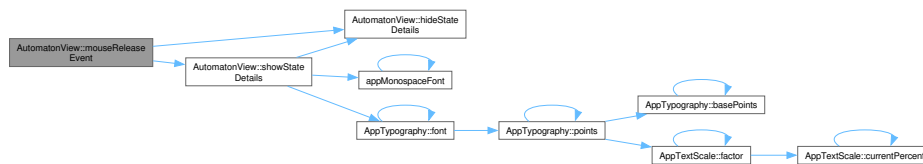
9.3.3.10 mousePressEvent()

```
void AutomatonView::mousePressEvent (
    QMouseEvent * event) [override], [protected]
```

9.3.3.11 mouseReleaseEvent()

```
void AutomatonView::mouseReleaseEvent (
    QMouseEvent * event) [override], [protected]
```

Here is the call graph for this function:

**9.3.3.12 reduceStates()**

```
QSet< unsigned > AutomatonView::reduceStates () const [inline]
```

9.3.3.13 revealAll()

```
void AutomatonView::revealAll ()
```

Reveals every state and transition (consultation mode).

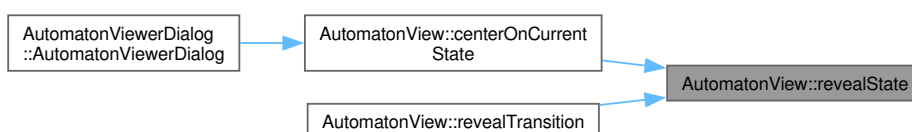
Here is the call graph for this function:

**9.3.3.14 revealState()**

```
void AutomatonView::revealState (
    unsigned id)
```

Makes a state node visible.

Here is the caller graph for this function:



9.3.3.15 revealTransition()

```
void AutomatonView::revealTransition (
    unsigned from,
    unsigned to)
```

Makes a transition edge visible, together with both endpoint states.

Here is the call graph for this function:



9.3.3.16 setAutomaton()

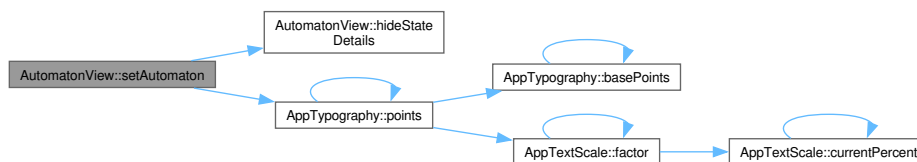
```
void AutomatonView::setAutomaton (
    const QVector< AutomatonStateInfo > & states,
    const QVector< AutomatonTransitionInfo > & transitions)
```

Loads the automaton and computes the layout. Everything starts hidden.

Parameters

<i>states</i>	All LR(0) states.
<i>transitions</i>	All delta transitions.

Here is the call graph for this function:



9.3.3.17 setConflictStates()

```
void AutomatonView::setConflictStates (
    const QSet< unsigned > & ids)
```

Marks states that contain an LR(0) conflict.

9.3.3.18 setCurrentState()

```
void AutomatonView::setCurrentState (
    int id)
```

Highlights one state as the one being analyzed (-1 for none).

9.3.3.19 setReduceStates()

```
void AutomatonView::setReduceStates (
    const QSet< unsigned > & ids)
```

Marks states that can perform a reduction.

9.3.3.20 shownStateId()

```
int AutomatonView::shownStateId () const [inline]
```

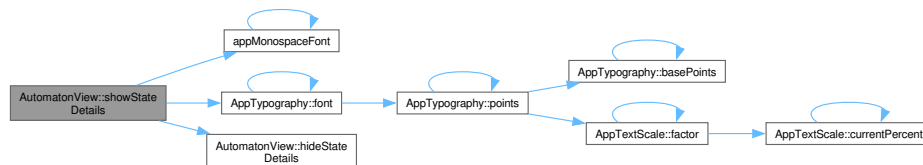
Id of the state whose items are shown, or -1.

9.3.3.21 showStateDetails()

```
void AutomatonView::showStateDetails (
    unsigned id)
```

Shows the items of state *id* next to its node.

Here is the call graph for this function:

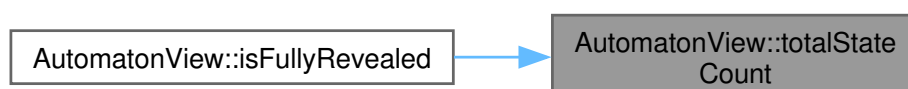


Here is the caller graph for this function:

**9.3.3.22 totalStateCount()**

```
int AutomatonView::totalStateCount () const [inline]
```

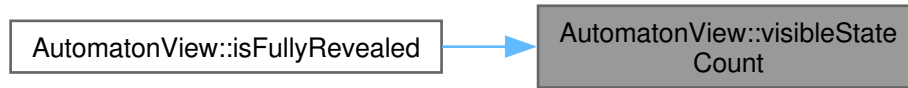
Here is the caller graph for this function:



9.3.3.23 visibleStateCount()

```
int AutomatonView::visibleStateCount () const
```

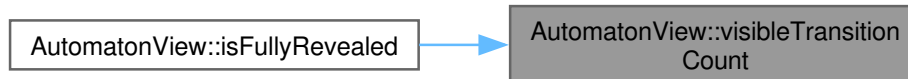
Here is the caller graph for this function:



9.3.3.24 visibleTransitionCount()

```
int AutomatonView::visibleTransitionCount () const
```

Here is the caller graph for this function:



9.3.3.25 wheelEvent()

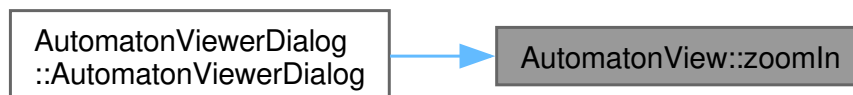
```
void AutomatonView::wheelEvent (
    QWheelEvent * event) [override], [protected]
```

9.3.3.26 zoomIn()

```
void AutomatonView::zoomIn ()
```

Zooms in one step around the view center.

Here is the caller graph for this function:

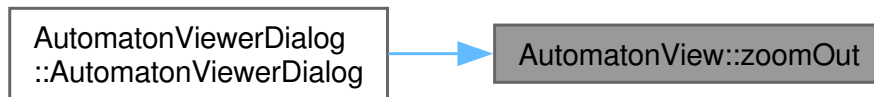


9.3.3.27 zoomOut()

```
void AutomatonView::zoomOut ()
```

Zooms out one step around the view center.

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

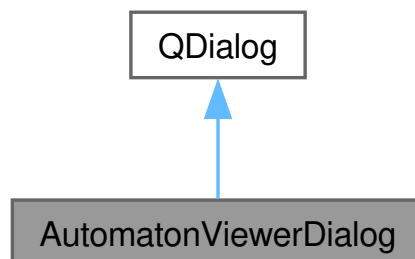
- [widgets/automatonview.h](#)
- [widgets/automatonview.cpp](#)

9.4 AutomatonViewerDialog Class Reference

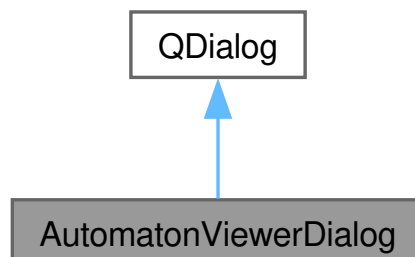
Non-modal floating window that hosts the LR(0) automaton view.

```
#include <automatonviewerdialog.h>
```

Inheritance diagram for AutomatonViewerDialog:



Collaboration diagram for AutomatonViewerDialog:



Public Member Functions

- [AutomatonViewerDialog](#) ([AutomatonView](#) *view, QWidget *parent=nullptr)
Builds the viewer around an existing automaton view.
- [~AutomatonViewerDialog](#) () override

9.4.1 Detailed Description

Non-modal floating window that hosts the LR(0) automaton view.

The dialog is a thin shell: it embeds an externally-owned [AutomatonView](#) (so its progressive reveal state is preserved across open/close cycles) and adds a toolbar with zoom in/out, fit-to-view and center-current actions. It never owns the view: on destruction it reparents the view out so the tutor can keep reusing it. Being non-modal, the student can keep it open while continuing the exercise.

9.4.2 Constructor & Destructor Documentation

9.4.2.1 AutomatonViewerDialog()

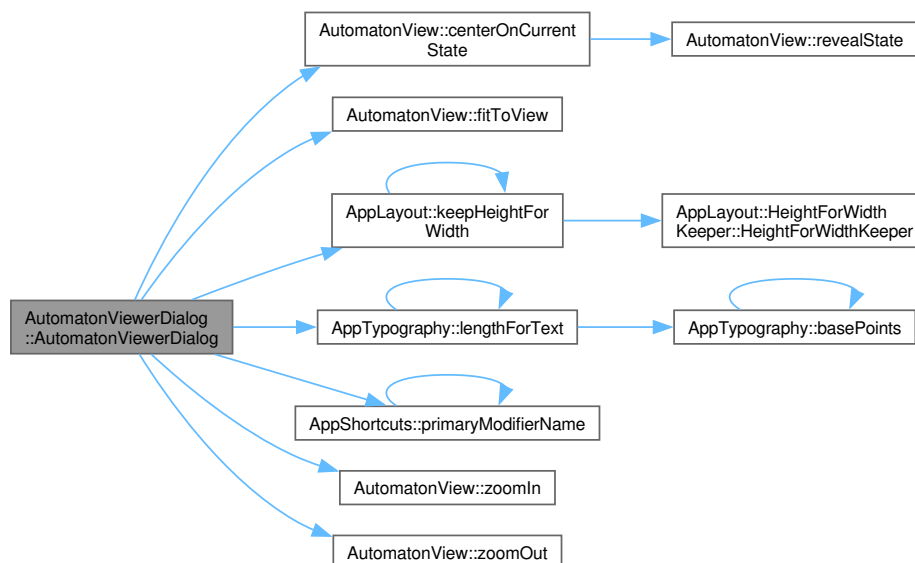
```
AutomatonViewerDialog::AutomatonViewerDialog (
    AutomatonView * view,
    QWidget * parent = nullptr) [explicit]
```

Builds the viewer around an existing automaton view.

Parameters

<i>view</i>	The automaton view to embed; ownership is not taken.
<i>parent</i>	Parent widget.

Here is the call graph for this function:



9.4.2.2 ~AutomatonViewerDialog()

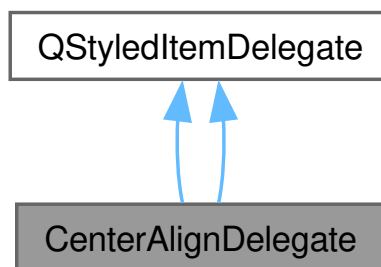
```
AutomatonViewerDialog::~AutomatonViewerDialog () [override]
```

The documentation for this class was generated from the following files:

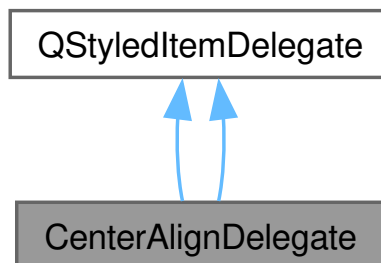
- [gui/automatonviewerdialog.h](#)
- [gui/automatonviewerdialog.cpp](#)

9.5 CenterAlignDelegate Class Reference

Inheritance diagram for CenterAlignDelegate:



Collaboration diagram for CenterAlignDelegate:



Public Member Functions

- void [initStyleOption](#) (QStyleOptionViewItem *opt, const QModelIndex &idx) const override
- void [initStyleOption](#) (QStyleOptionViewItem *opt, const QModelIndex &idx) const override

9.5.1 Member Function Documentation

9.5.1.1 initStyleOption() [1/2]

```
void CenterAlignDelegate::initStyleOption (
    QStyleOptionViewItem * opt,
    const QModelIndex & idx) const [inline], [override]
```

9.5.1.2 initStyleOption() [2/2]

```
void CenterAlignDelegate::initStyleOption (
    QStyleOptionViewItem * opt,
    const QModelIndex & idx) const [inline], [override]
```

The documentation for this class was generated from the following files:

- [gui/lltabledialog.cpp](#)
- [gui/sltabledialog.cpp](#)

9.6 ConversationPdf Class Reference

Builds the PDF of an LL(1) or SLR(1) session.

```
#include <conversationpdf.h>
```

Classes

- struct [Message](#)
One message of the conversation.

Public Types

- using [Rule](#) = QPair<QString, QVector<QString>>
A grammar rule: antecedent and consequent symbols.

Public Member Functions

- [ConversationPdf](#) (const QString &exercise)
Starts a document for the exercise named `exercise`.
- void [addGrammar](#) (const QVector< [Rule](#) > &rules, bool numbered)
The grammar, one rule per line; numbered when `numbered`.
- void [addConversation](#) (const QVector< [Message](#) > &messages)
The tutor's questions and the student's answers, in order.
- void [addResult](#) (const QString &grade, bool passed, const QStringList &lines)
The headline of an exam: the 0-10 grade beside its summary.
- void [addReview](#) (const QVector< [ExamRecord](#) > &records)
Every graded item of an exam, with both answers.
- void [addSection](#) (const QString &title, bool newPage=false)
A heading for the next block; `newPage` starts a new page.
- void [addDefinitions](#) (const QVector< QPair< QString, QString > > &rows)
Name/value pairs, such as `FIRST(A) = { a, b }`.
- void [addSubheading](#) (const QString &text)
A small heading inside a section, such as a state's name.
- void [addLines](#) (const QStringList &lines)
Lines in a monospaced block, such as the items of a state.
- void [addTable](#) (const QStringList &headers, const QVector< QStringList > &rows)
A table with a row header column.
- QString [html](#) () const
The document as HTML, for inspection.
- void [print](#) (const QString &filePath) const
Prints the document to `filePath` as an A4 PDF.

Static Public Attributes

- static constexpr int `kMaxTableColumns` = 10
Data columns per block before a wide table is split.

9.6.1 Detailed Description

Builds the PDF of an LL(1) or SLR(1) session.

Both tutors describe their exercise - grammar, conversation, sets, table, or the graded review of an exam - and this class lays it out the same way. It only uses the subset of HTML and CSS that QTextDocument supports when printing (tables, backgrounds, fonts), and splits tables too wide for an A4 page into blocks of columns, so grammars typed by the user, however large, stay readable.

9.6.2 Member Typedef Documentation

9.6.2.1 Rule

```
using ConversationPdf::Rule = QPair<QString, QVector<QString>>
```

A grammar rule: antecedent and consequent symbols.

9.6.3 Constructor & Destructor Documentation

9.6.3.1 ConversationPdf()

```
ConversationPdf::ConversationPdf (  
    const QString & exercise) [inline], [explicit]
```

Starts a document for the exercise named `exercise`.

Here is the call graph for this function:



9.6.4 Member Function Documentation

9.6.4.1 addConversation()

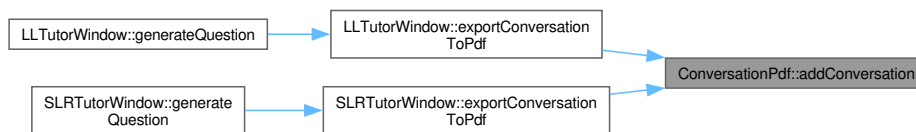
```
void ConversationPdf::addConversation (
    const QVector< Message > & messages) [inline]
```

The tutor's questions and the student's answers, in order.

Here is the call graph for this function:



Here is the caller graph for this function:



9.6.4.2 addDefinitions()

```
void ConversationPdf::addDefinitions (
    const QVector< QPair< QString, QString > > & rows) [inline]
```

Name/value pairs, such as `FIRST(A) = { a, b }`.

Here is the caller graph for this function:



9.6.4.3 addGrammar()

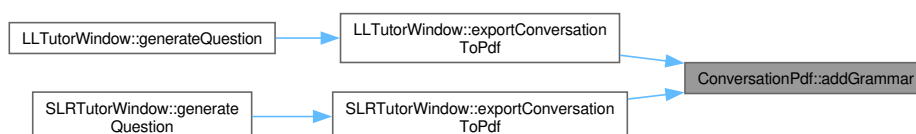
```
void ConversationPdf::addGrammar (
    const QVector< Rule > & rules,
    bool numbered) [inline]
```

The grammar, one rule per line; numbered when numbered.

Here is the call graph for this function:



Here is the caller graph for this function:



9.6.4.4 addLines()

```
void ConversationPdf::addLines (
    const QStringList & lines) [inline]
```

Lines in a monospaced block, such as the items of a state.

Here is the caller graph for this function:



9.6.4.5 addResult()

```
void ConversationPdf::addResult (
    const QString & grade,
    bool passed,
    const QStringList & lines) [inline]
```

The headline of an exam: the 0-10 grade beside its summary.

`grade` is already formatted; `lines` go under it, in order, and the grade and the last line take the pass or fail colour.

9.6.4.6 addReview()

```
void ConversationPdf::addReview (
    const QVector< ExamRecord > & records) [inline]
```

Every graded item of an exam, with both answers.

Here is the call graph for this function:

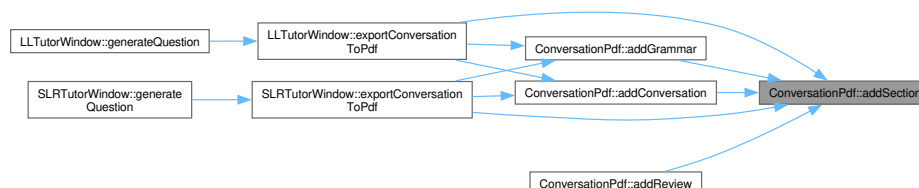


9.6.4.7 addSection()

```
void ConversationPdf::addSection (
    const QString & title,
    bool newPage = false) [inline]
```

A heading for the next block; `newPage` starts a new page.

Here is the caller graph for this function:



9.6.4.8 addSubheading()

```
void ConversationPdf::addSubheading (
    const QString & text) [inline]
```

A small heading inside a section, such as a state's name.

Here is the caller graph for this function:

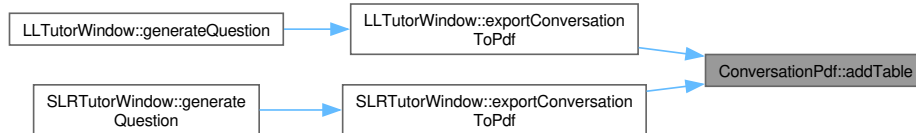


9.6.4.9 addTable()

```
void ConversationPdf::addTable (
    const QStringList & headers,
    const QVector< QStringList > & rows) [inline]
```

A table with a row header column.

`headers` starts with the corner cell; each row of `rows` starts with its row header. Tables wider than `kMaxTableColumns` data columns are printed as consecutive blocks, each repeating the row headers, and every block repeats its header row on each page it spans. Here is the caller graph for this function:



9.6.4.10 html()

```
QString ConversationPdf::html () const [inline]
```

The document as HTML, for inspection.

Here is the caller graph for this function:

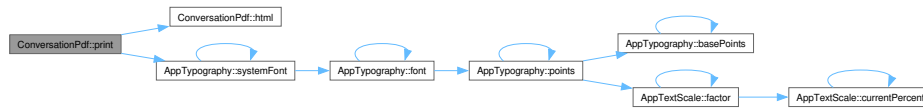


9.6.4.11 print()

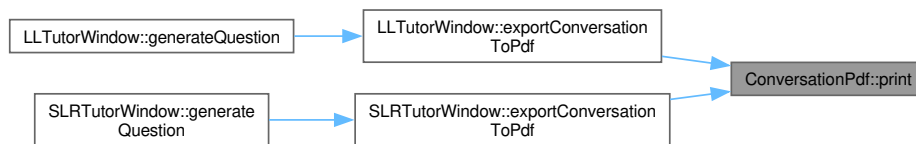
```
void ConversationPdf::print (
    const QString & filePath) const [inline]
```

Prints the document to `filePath` as an A4 PDF.

Here is the call graph for this function:



Here is the caller graph for this function:



9.6.5 Member Data Documentation

9.6.5.1 kMaxTableColumns

```
int ConversationPdf::kMaxTableColumns = 10 [static], [constexpr]
```

Data columns per block before a wide table is split.

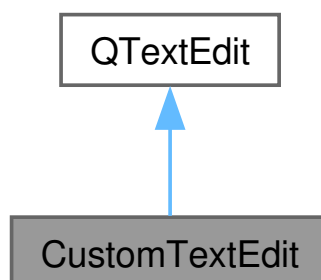
The documentation for this class was generated from the following file:

- [gui/conversationpdf.h](#)

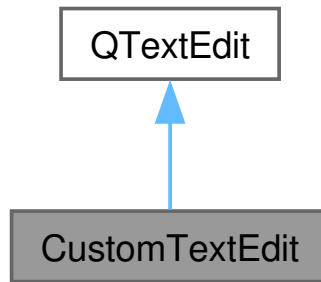
9.7 CustomTextEdit Class Reference

```
#include <customtextedit.h>
```

Inheritance diagram for CustomTextEdit:



Collaboration diagram for CustomTextEdit:



Signals

- void [sendRequested](#) ()
- void [minimumGrowHeightChanged](#) (int height)
The height for the minimum number of lines has changed.

Public Member Functions

- [CustomTextEdit](#) (QWidget *parent=nullptr)
- void [setAutoGrowLines](#) (int minLines, int maxLines)
Makes the height follow the content, animated, between `minLines` and `maxLines` lines of text. Content beyond that scrolls.
- int [minimumGrowHeight](#) () const
Height of the box when it holds its minimum number of lines.

Static Public Member Functions

- static QString [newlineShortcutText](#) ()
User-visible name of the newline shortcut for the current platform.

Protected Member Functions

- void [showEvent](#) (QShowEvent *event) override
- void [changeEvent](#) (QEvent *event) override
- void [keyPressEvent](#) (QKeyEvent *event) override

Properties

- int [growHeight](#)

9.7.1 Constructor & Destructor Documentation

9.7.1.1 CustomTextEdit()

```

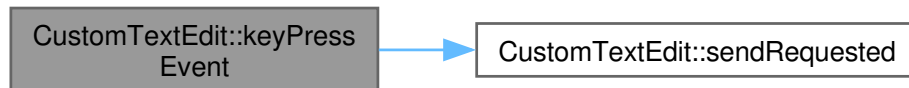
CustomTextEdit::CustomTextEdit (
    QWidget * parent = nullptr) [explicit]
  
```


9.7.2.1 changeEvent()

Here is the caller graph for this function:



Here is the call graph for this function:



```

graph LR
    A[LLTwinWindows-frontend] --> D[LLTwinWindows-solutionForRZ]
    B[LLTwinWindows-frontendForOffice] --> D
    D --> C[LLTwinWindows-solution]
    C --> E[LLTwinWindows-postQuestion]
    E --> F[LLTwinWindows-generateOpenPathway]
    F --> G[LLTwinWindows-LLTwinWindows]
    G --> H[Custom7-articles-minimumSize-keepOnChange]
    H --> I[Custom7-articles-keepOnEvent]
  
```

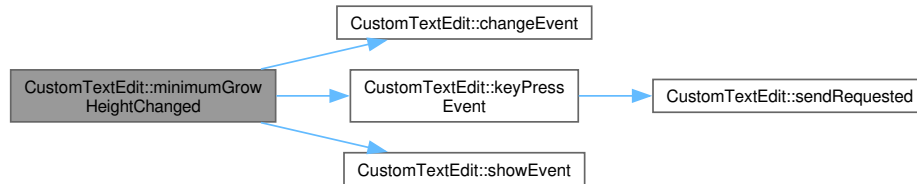
Height of the box when it holds its minimum number of lines.

9.7.2.4 minimumGrowHeightChanged

```
void CustomTextEdit::minimumGrowHeightChanged (
    int height) [signal]
```

The height for the minimum number of lines has changed.

Here is the call graph for this function:



Here is the caller graph for this function:



9.7.2.5 newlineShortcutText()

```
QString CustomTextEdit::newlineShortcutText () [static]
```

User-visible name of the newline shortcut for the current platform.

Here is the call graph for this function:



9.7.2.6 sendRequested

```
void CustomTextEdit::sendRequested () [signal]
```

Here is the caller graph for this function:



9.8.1 Detailed Description

One graded item of an exam: a question (or table cell), the user's answer, the expected answer, and whether it was correct.

9.8.2 Member Data Documentation

9.8.2.1 correct

```
bool ExamRecord::correct
```

Whether the student's answer was correct.

9.8.2.2 correctAnswer

```
QString ExamRecord::correctAnswer
```

Expected answer.

9.8.2.3 question

```
QString ExamRecord::question
```

Question text as it was shown to the student.

9.8.2.4 userAnswer

```
QString ExamRecord::userAnswer
```

Answer the student gave.

The documentation for this struct was generated from the following file:

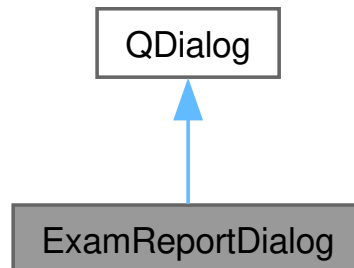
- [gui/examsession.h](#)

9.9 ExamReportDialog Class Reference

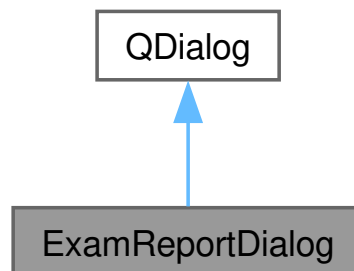
End-of-exam report: grade, stats and a question-by-question review.

```
#include <examreportdialog.h>
```

Inheritance diagram for ExamReportDialog:



Collaboration diagram for ExamReportDialog:



Signals

- void [exportRequested](#) ()
Emitted when the user asks to export the report to PDF.

Public Member Functions

- [ExamReportDialog](#) (const [ExamSession](#) &session, const QString &examTitle, const QVector< [ConversationPdf::Rule](#) > &grammar, bool numberedGrammar, QWidget *parent=nullptr)
Builds the report for a finished exam session.
- void [printReport](#) (const QString &filePath) const
Prints the full report to filePath as a PDF.

9.9.1 Detailed Description

End-of-exam report: grade, stats and a question-by-question review.

Shown when an exam-mode exercise finishes. The headline is the 0-10 grade; below it a review lists every graded item with the student's answer and the expected one, like a corrected exam sheet. The full report can be exported to PDF through the owning tutor window.

9.9.2 Constructor & Destructor Documentation

9.9.2.1 ExamReportDialog()

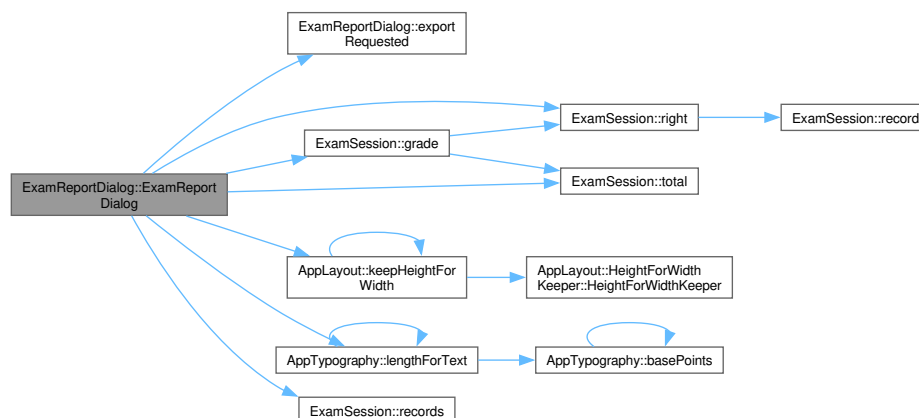
```
ExamReportDialog::ExamReportDialog (
    const ExamSession & session,
    const QString & examTitle,
    const QVector< ConversationPdf::Rule > & grammar,
    bool numberedGrammar,
    QWidget * parent = nullptr)
```

Builds the report for a finished exam session.

Parameters

<i>session</i>	Graded exam data.
<i>examTitle</i>	Exercise name shown in the header (e.g. "Examen LL(1)").
<i>grammar</i>	Grammar of the exam, printed before the review.
<i>numberedGrammar</i>	Whether the printed rules carry their numbers.
<i>parent</i>	Parent widget.

Here is the call graph for this function:



9.9.3 Member Function Documentation

9.9.3.1 exportRequested

```
void ExamReportDialog::exportRequested () [signal]
```

Emitted when the user asks to export the report to PDF.

Here is the caller graph for this function:



9.9.3.2 printReport()

```
void ExamReportDialog::printReport (
    const QString & filePath) const [inline]
```

Prints the full report to `filePath` as a PDF.

The documentation for this class was generated from the following files:

- [gui/examreportdialog.h](#)
- [gui/examreportdialog.cpp](#)

9.10 ExamSession Class Reference

Accumulates graded answers during an exam-mode exercise.

```
#include <examsession.h>
```

Public Member Functions

- void [record](#) (const QString &question, const QString &userAnswer, const QString &correctAnswer, bool correct)
Records one graded item.
- int [total](#) () const
Total number of graded items.
- int [right](#) () const
Number of correct items.
- double [grade](#) () const
Grade on the Spanish 0-10 scale.
- const QVector< [ExamRecord](#) > & [records](#) () const
All graded items in exam order.

9.10.1 Detailed Description

Accumulates graded answers during an exam-mode exercise.

In exam mode the tutors give no feedback: every answer is recorded here with its real correctness while the exercise advances along the correct path. At the end the session produces the stats and the 0-10 grade shown in the exam report.

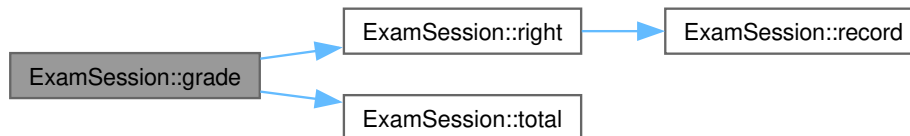
9.10.2 Member Function Documentation

9.10.2.1 grade()

```
double ExamSession::grade () const [inline]
```

Grade on the Spanish 0-10 scale.

An empty exam grades 0. Here is the call graph for this function:



Here is the caller graph for this function:



9.10.2.2 record()

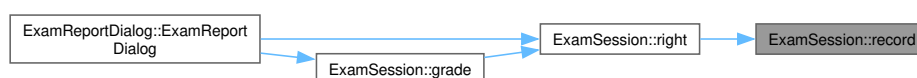
```
void ExamSession::record (
    const QString & question,
    const QString & userAnswer,
    const QString & correctAnswer,
    bool correct) [inline]
```

Records one graded item.

Parameters

<i>question</i>	Question text shown to the student.
<i>userAnswer</i>	Answer the student gave.
<i>correctAnswer</i>	Expected answer.
<i>correct</i>	Whether the answer was correct.

Here is the caller graph for this function:



9.10.2.3 records()

```
const QVector< ExamRecord > & ExamSession::records () const [inline]
```

All graded items in exam order.

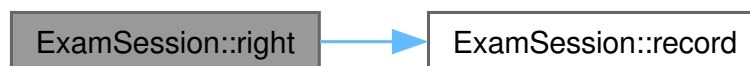
Here is the caller graph for this function:

**9.10.2.4 right()**

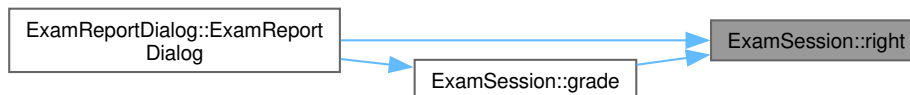
```
int ExamSession::right () const [inline]
```

Number of correct items.

Here is the call graph for this function:



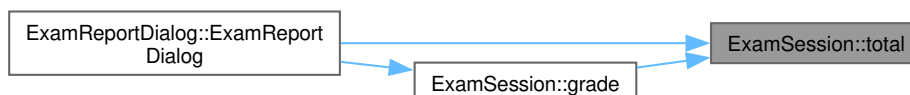
Here is the caller graph for this function:

**9.10.2.5 total()**

```
int ExamSession::total () const [inline]
```

Total number of graded items.

Here is the caller graph for this function:



The documentation for this class was generated from the following file:

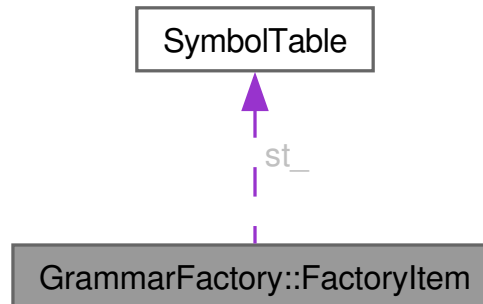
- [gui/examsession.h](#)

9.11 GrammarFactory::FactoryItem Struct Reference

Represents an individual grammar item with its associated symbol table.

```
#include <grammar_factory.hpp>
```

Collaboration diagram for GrammarFactory::FactoryItem:



Public Member Functions

- [FactoryItem](#) (const std::unordered_map< std::string, std::vector< [production](#) > > &grammar)
Constructor that initializes a [FactoryItem](#) with the provided grammar.

Public Attributes

- std::unordered_map< std::string, std::vector< [production](#) > > [g_](#)
Stores the grammar rules where each key is a non-terminal symbol and each value is a vector of production rules.
- [SymbolTable](#) [st_](#)
Symbol table associated with this grammar item.

9.11.1 Detailed Description

Represents an individual grammar item with its associated symbol table.

9.11.2 Constructor & Destructor Documentation

9.11.2.1 FactoryItem()

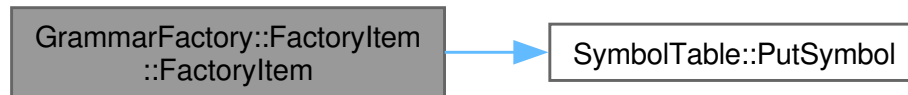
```
GrammarFactory::FactoryItem::FactoryItem (
    const std::unordered_map< std::string, std::vector< production > > & grammar)
[explicit]
```

Constructor that initializes a [FactoryItem](#) with the provided grammar.

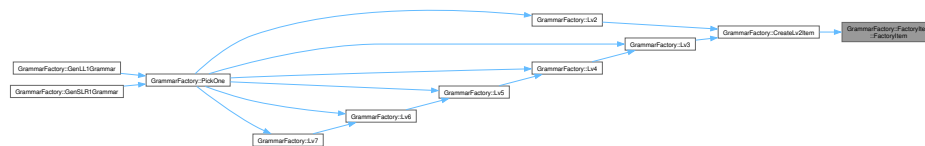
Parameters

<i>grammar</i>	The grammar to initialize the FactoryItem with.
----------------	---

Here is the call graph for this function:



Here is the caller graph for this function:

**9.11.3 Member Data Documentation****9.11.3.1 g_**

```
std::unordered_map<std::string, std::vector<production> > GrammarFactory::FactoryItem::g_
```

Stores the grammar rules where each key is a non-terminal symbol and each value is a vector of production rules.

9.11.3.2 st_

```
SymbolTable GrammarFactory::FactoryItem::st_
```

Symbol table associated with this grammar item.

The documentation for this struct was generated from the following files:

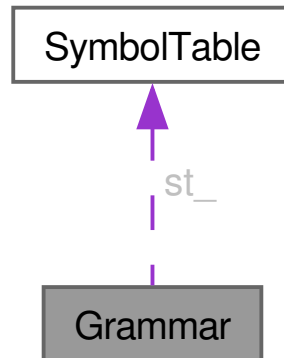
- [backend/grammar_factory.hpp](#)
- [backend/grammar_factory.cpp](#)

9.12 Grammar Struct Reference

Represents a context-free grammar, including its rules, symbol table, and starting symbol.

```
#include <grammar.hpp>
```

Collaboration diagram for Grammar:



Public Member Functions

- [Grammar](#) ()
- [Grammar](#) (const std::unordered_map< std::string, std::vector< [production](#) > > &grammar)
- int [PresentationRank](#) (const std::string &antecedent) const
Ordering rank of an antecedent for display and question order.
- bool [PresentationLess](#) (const std::string &lhs, const std::string &rhs) const
Orders two antecedents for presentation.
- std::vector< std::string > [PresentationOrder](#) () const
Antecedents other than the axiom, in presentation order.
- void [SetAxiom](#) (const std::string &axiom)
Sets the axiom (entry point) of the grammar.
- bool [HasEmptyProduction](#) (const std::string &antecedent) const
Checks if a given antecedent has an empty production.
- std::vector< std::pair< const std::string, [production](#) > > [FilterRulesByConsequent](#) (const std::string &arg) const
Filters grammar rules that contain a specific token in their consequent.
- void [Debug](#) () const
Prints the current grammar structure to standard output.
- void [AddProduction](#) (const std::string &antecedent, const std::vector< std::string > &consequent)
Adds a production rule to the grammar and updates the symbol table.
- std::vector< std::string > [Split](#) (const std::string &s)
Splits a string into grammar symbols using the current symbol table.

Public Attributes

- std::unordered_map< std::string, std::vector< [production](#) > > [g_](#)
Stores the grammar rules with each antecedent mapped to a list of productions.
- std::vector< std::string > [lhs_order_](#)
Antecedents in the order the grammar was written.
- std::string [axiom_](#)
The axiom or entry point of the grammar.
- [SymbolTable](#) [st_](#)
Symbol table of the grammar.

9.12.1 Detailed Description

Represents a context-free grammar, including its rules, symbol table, and starting symbol.

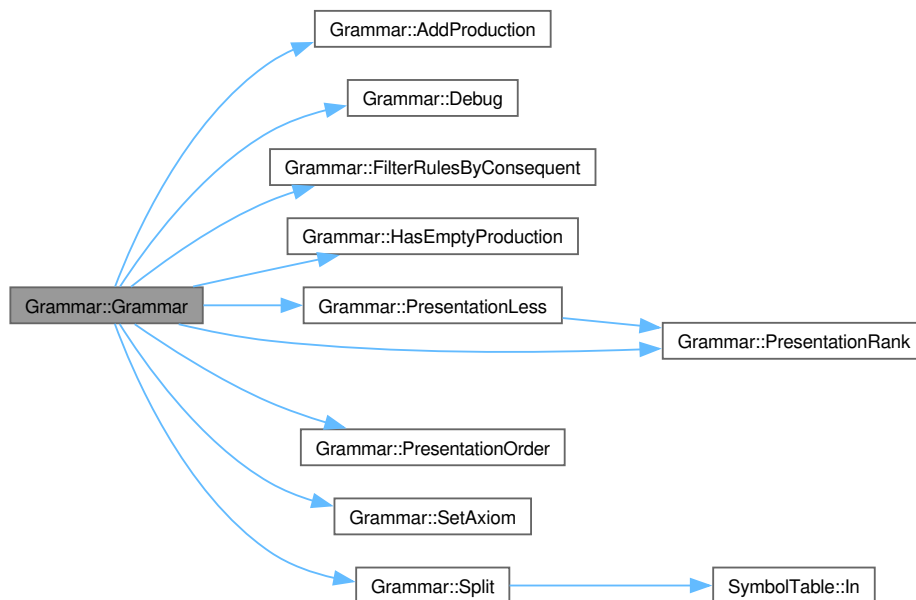
This structure encapsulates all components required to define and manipulate a grammar, including production rules, the associated symbol table, and metadata such as the start symbol. It supports construction, transformation, and analysis of grammars.

9.12.2 Constructor & Destructor Documentation

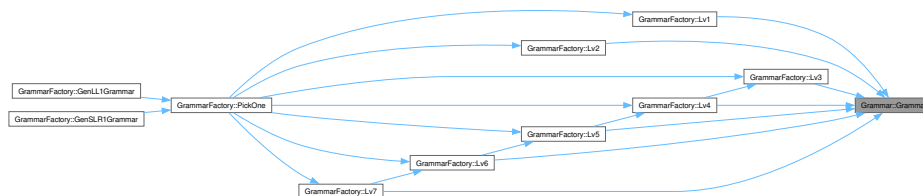
9.12.2.1 Grammar() [1/2]

```
Grammar::Grammar () [default]
```

Here is the call graph for this function:



Here is the caller graph for this function:



9.12.2.2 Grammar() [2/2]

```
Grammar::Grammar (
    const std::unordered_map< std::string, std::vector< production > > & grammar)
[explicit]
```

Here is the call graph for this function:



9.12.3 Member Function Documentation

9.12.3.1 AddProduction()

```
void Grammar::AddProduction (
    const std::string & antecedent,
    const std::vector< std::string > & consequent)
```

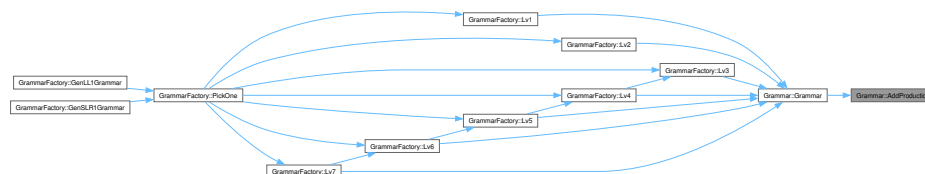
Adds a production rule to the grammar and updates the symbol table.

This function inserts a new production of the form $A \rightarrow \alpha$ into the grammar, where *antecedent* is the non-terminal A and *consequent* is the sequence α . It also updates the internal symbol table to reflect any new symbols introduced.

Parameters

<i>antecedent</i>	The left-hand side non-terminal of the production.
<i>consequent</i>	The right-hand side sequence of grammar symbols.

Here is the caller graph for this function:



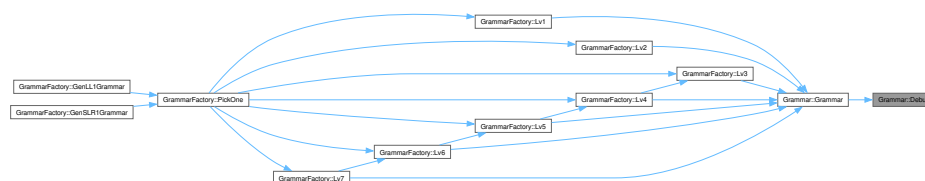
9.12.3.2 Debug()

```
void Grammar::Debug () const
```

Prints the current grammar structure to standard output.

This function provides a debug view of the grammar by printing out all rules, the axiom, and other relevant details.

Here is the caller graph for this function:



9.12.3.3 FilterRulesByConsequent()

```
std::vector< std::pair< const std::string, production > > Grammar::FilterRulesByConsequent (
    const std::string & arg) const
```

Filters grammar rules that contain a specific token in their consequent.

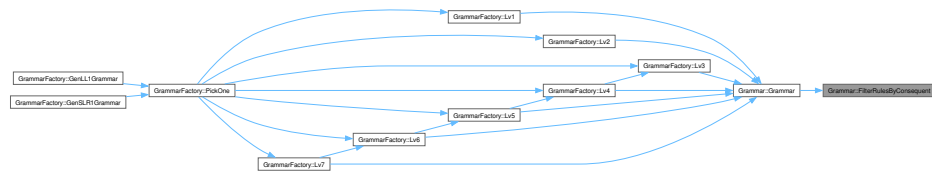
Parameters

<i>arg</i>	The token to search for within the consequents of the rules.
------------	--

Returns

std::vector of pairs where each pair contains an antecedent and its respective production that includes the specified token.

Searches for rules in which the specified token is part of the consequent and returns those rules. Here is the caller graph for this function:



9.12.3.4 HasEmptyProduction()

```
bool Grammar::HasEmptyProduction (
    const std::string & antecedent) const
```

Checks if a given antecedent has an empty production.

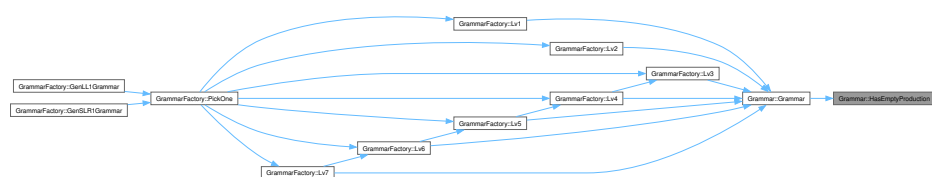
Parameters

<i>antecedent</i>	The left-hand side (LHS) symbol to check.
-------------------	---

Returns

true if there exists an empty production for the antecedent, otherwise false.

An empty production is represented as `<antecedent> -> ;`, indicating that the antecedent can produce an empty string. Here is the caller graph for this function:



9.12.3.5 PresentationLess()

```
bool Grammar::PresentationLess (
    const std::string & lhs,
    const std::string & rhs) const
```

Orders two antecedents for presentation.

Compares [PresentationRank](#) and falls back to lexicographic order on a tie.

Parameters

<i>lhs</i>	First antecedent.
<i>rhs</i>	Second antecedent.

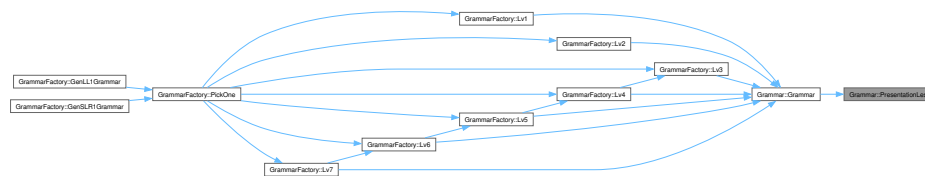
Returns

true if *lhs* should be shown before *rhs*.

Here is the call graph for this function:



Here is the caller graph for this function:

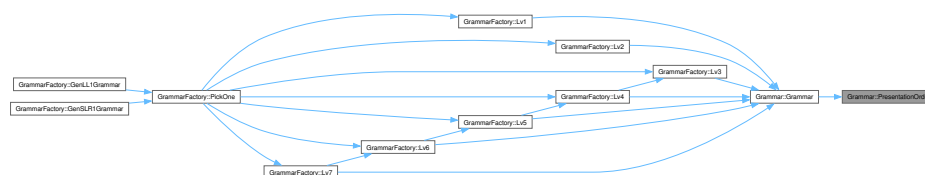


9.12.3.6 PresentationOrder()

```
std::vector< std::string > Grammar::PresentationOrder () const
```

Antecedents other than the axiom, in presentation order.

Here is the caller graph for this function:



9.12.3.7 PresentationRank()

```
int Grammar::PresentationRank (
    const std::string & antecedent) const
```

Ordering rank of an antecedent for display and question order.

The axiom always ranks first, because the tutors show its rule at the top. A user-written grammar then follows [lhs_order_](#); an antecedent with no recorded position ranks last so the caller's lexicographic tie-break decides, which is what every generated grammar gets.

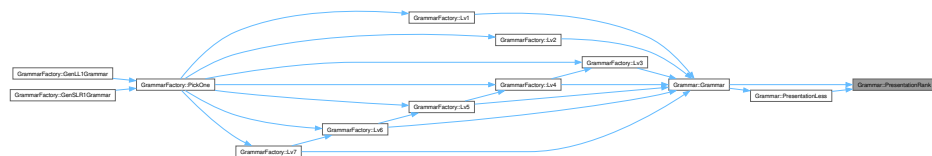
Parameters

<i>antecedent</i>	Left-hand side to rank.
-------------------	-------------------------

Returns

Rank, lowest first.

Here is the caller graph for this function:



9.12.3.8 SetAxiom()

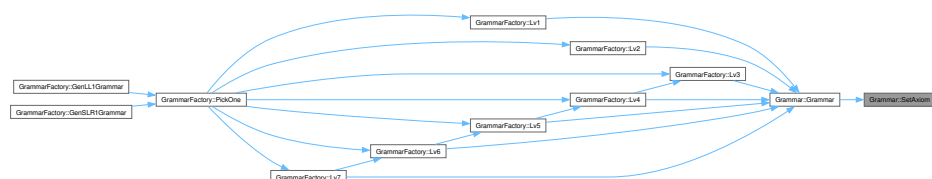
```
void Grammar::SetAxiom (
    const std::string & axiom)
```

Sets the axiom (entry point) of the grammar.

Parameters

<i>axiom</i>	The entry point or start symbol of the grammar.
--------------	---

Defines the starting point for the grammar, which is used in parsing algorithms and must be a non-terminal symbol present in the grammar. Here is the caller graph for this function:



9.12.3.9 Split()

```
std::vector< std::string > Grammar::Split (
    const std::string & s)
```

Splits a string into grammar symbols using the current symbol table.

This function tokenizes the input string *s* into a sequence of grammar symbols based on the known entries in the symbol table. It uses a greedy approach, matching the longest valid symbol at each step.

Parameters

<i>s</i>	The input string to split.
----------	----------------------------

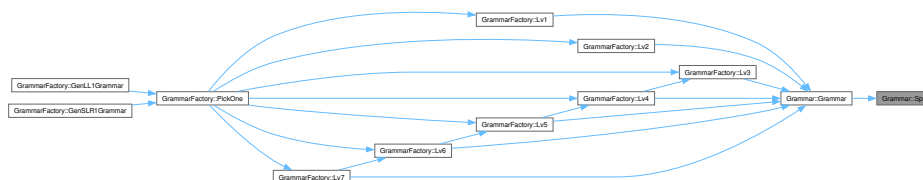
Returns

A vector of grammar symbols extracted from the string.

Here is the call graph for this function:



Here is the caller graph for this function:



9.12.4 Member Data Documentation

9.12.4.1 axiom_

```
std::string Grammar::axiom_
```

The axiom or entry point of the grammar.

9.12.4.2 g_

```
std::unordered_map<std::string, std::vector<production> > Grammar::g_
```

Stores the grammar rules with each antecedent mapped to a list of productions.

9.12.4.3 lhs_order_

```
std::vector<std::string> Grammar::lhs_order_
```

Antecedents in the order the grammar was written.

Only filled in for a grammar the user typed, where the order carries intent and the student expects to see their own rules back in the same sequence. A generated grammar leaves this empty, and presentation then falls back to lexicographic order.

9.12.4.4 st_

```
SymbolTable Grammar::st_
```

Symbol table of the grammar.

The documentation for this struct was generated from the following files:

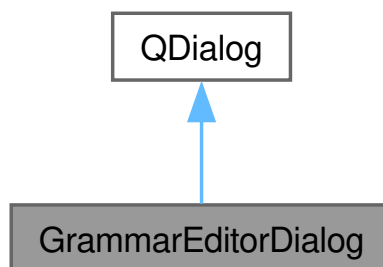
- [backend/grammar.hpp](#)
- [backend/grammar.cpp](#)

9.13 GrammarEditorDialog Class Reference

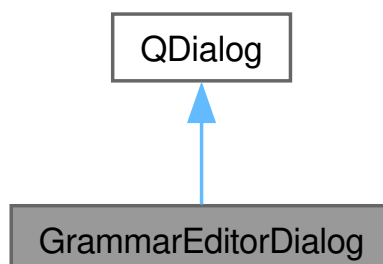
Dialog where the user writes their own grammar for an exercise.

```
#include <grammareditordialog.h>
```

Inheritance diagram for GrammarEditorDialog:



Collaboration diagram for GrammarEditorDialog:



Public Types

- enum class `Mode` { `LL1` , `SLR1` }

Exercise the grammar is being written for.

Public Member Functions

- `GrammarEditorDialog` (`Mode` mode, `QWidget *parent=nullptr`)

Constructs the editor for the given exercise kind.

- const `Grammar` & `grammar` () const

Returns the validated, augmented grammar.

Protected Member Functions

- void `accept` () override

9.13.1 Detailed Description

Dialog where the user writes their own grammar for an exercise.

The user types rules in the `LHS -> SYMBOL . . .` format. The text is parsed and validated live: format errors are reported with line numbers, the detected axiom / non-terminals / terminals are summarized, and the grammar is checked to be LL(1) or SLR(1) depending on the exercise the dialog was opened for. The last accepted grammar is persisted through `QSettings` so it can be practiced again.

9.13.2 Member Enumeration Documentation

9.13.2.1 Mode

```
enum class GrammarEditorDialog::Mode [strong]
```

Exercise the grammar is being written for.

Enumerator

LL1	
SLR1	

9.13.3 Constructor & Destructor Documentation

9.13.3.1 GrammarEditorDialog()

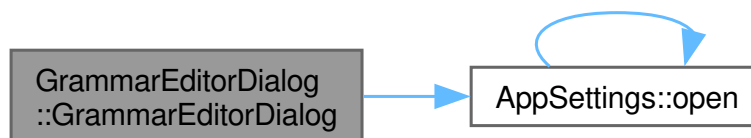
```
GrammarEditorDialog::GrammarEditorDialog (
    Mode mode,
    QWidget * parent = nullptr) [explicit]
```

Constructs the editor for the given exercise kind.

Parameters

<i>mode</i>	Exercise the grammar must be valid for.
<i>parent</i>	Parent widget.

Here is the call graph for this function:

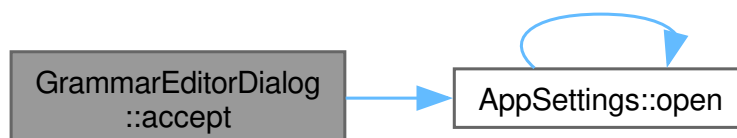


9.13.4 Member Function Documentation

9.13.4.1 accept()

```
void GrammarEditorDialog::accept () [override], [protected]
```

Here is the call graph for this function:



9.13.4.2 grammar()

```
const Grammar & GrammarEditorDialog::grammar () const [inline]
```

Returns the validated, augmented grammar.

Only meaningful after the dialog was accepted.

The documentation for this class was generated from the following files:

- [gui/grammareditordialog.h](#)
- [gui/grammareditordialog.cpp](#)

9.14 GrammarFactory Struct Reference

Responsible for creating and managing grammar items and performing checks on grammars.

```
#include <grammar_factory.hpp>
```

Classes

- struct [FactoryItem](#)
Represents an individual grammar item with its associated symbol table.

Public Member Functions

- void [Init](#) ()
Initializes the [GrammarFactory](#) and populates the items vector with initial grammar items.
- [Grammar PickOne](#) (int level)
Picks a random grammar based on the specified difficulty level (1, 2, or 3).
- [Grammar GenLL1Grammar](#) (int level)
Generates a LL(1) random grammar based on the specified difficulty level.
- [Grammar GenSLR1Grammar](#) (int level)
Generates a SLR(1) random grammar based on the specified difficulty level.
- [Grammar Lv1](#) ()
Generates a Level 1 grammar.
- [Grammar Lv2](#) ()
Generates a Level 2 grammar by combining Level 1 items.
- [Grammar Lv3](#) ()
Generates a Level 3 grammar by combining a Level 2 item and a Level 1 item.
- [Grammar Lv4](#) ()
Generates a Level 4 grammar by combining Level 3 and Level 1 items.
- [Grammar Lv5](#) ()
Generates a Level 5 grammar by combining Level 4 and Level 1 items.
- [Grammar Lv6](#) ()
Generates a Level 6 grammar by combining Level 5 and Level 1 items.
- [Grammar Lv7](#) ()
Generates a Level 7 grammar by combining Level 6 and Level 1 items.
- [FactoryItem CreateLv2Item](#) ()
Creates a Level 2 grammar item for use in grammar generation.
- bool [HasUnreachableSymbols](#) ([Grammar](#) &grammar) const
Checks if a grammar contains unreachable symbols (non-terminals that cannot be derived from the start symbol).
- bool [IsInfinite](#) ([Grammar](#) &grammar) const
Checks if a grammar is infinite, meaning there are non-terminal symbols that can never derive a terminal string. This happens when a production leads to an infinite recursion or an endless derivation without reaching terminal symbols. For example, a production like:
- bool [HasDirectLeftRecursion](#) (const [Grammar](#) &grammar) const
Checks if a grammar contains direct left recursion (a non-terminal can produce itself on the left side of a production in one step).
- bool [HasIndirectLeftRecursion](#) (const [Grammar](#) &grammar) const
Checks if a grammar contains indirect left recursion.
- bool [HasCycle](#) (const std::unordered_map< std::string, std::unordered_set< std::string > > &graph) const
Checks if directed graph has a cycle using topological sort.

- `std::unordered_set< std::string > NullableSymbols (const Grammar &grammar) const`
Find nullable symbols in a grammar.
- `void RemoveLeftRecursion (Grammar &grammar)`
Removes direct left recursion in a grammar. A grammar has direct left recursion when one of its productions is.
- `void LeftFactorize (Grammar &grammar)`
Performs left factorization. A grammar can be left factorized if it has productions with the same prefix for one non-terminal. For example:
- `std::vector< std::string > LongestCommonPrefix (const std::vector< production > &productions)`
Finds the longest common prefix among a set of productions.
- `bool StartsWith (const production &prod, const std::vector< std::string > &prefix)`
Checks if a production starts with a given prefix.
- `std::string GenerateNewNonTerminal (Grammar &grammar, const std::string &base)`
Generates a new non-terminal symbol that is unique in the grammar.
- `void NormalizeNonTerminals (FactoryItem &item, const std::string &nt) const`
Replaces all non-terminal symbols in a grammar item with a single target non-terminal.
- `void AdjustTerminals (FactoryItem &base, const FactoryItem &cmb, const std::string &target_nt) const`
Adjusts the terminal symbols between two grammar items.
- `std::unordered_map< std::string, std::vector< production > > Merge (const FactoryItem &base, const FactoryItem &cmb) const`
Merges the grammar rules of two grammar items into a single grammar.

Public Attributes

- `std::vector< FactoryItem > items`
A vector of FactoryItem objects representing different level 1 grammar items created by the Init method.
- `std::vector< std::string > terminal_alphabet_`
A vector of terminal symbols (alphabet) used in the grammar.
- `std::vector< std::string > non_terminal_alphabet_`
A vector of non-terminal symbols (alphabet) used in the grammar.

9.14.1 Detailed Description

Responsible for creating and managing grammar items and performing checks on grammars.

9.14.2 Member Function Documentation

9.14.2.1 AdjustTerminals()

```
void GrammarFactory::AdjustTerminals (
    FactoryItem & base,
    const FactoryItem & cmb,
    const std::string & target_nt) const
```

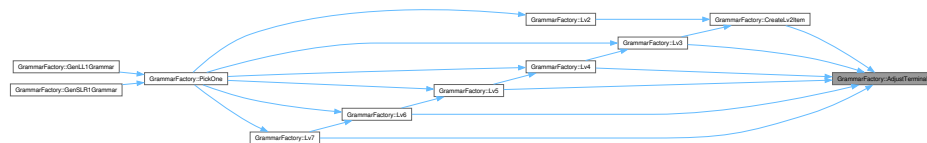
Adjusts the terminal symbols between two grammar items.

This function modifies the terminal symbols of a base grammar item so that they do not conflict with those of the item being combined. It also renames terminals to ensure consistency and inserts the target non-terminal where appropriate.

Parameters

<i>base</i>	The base grammar item to adjust.
<i>cmb</i>	The grammar item being combined with the base.
<i>target</i> ↔ <i>_nt</i>	The target non-terminal symbol used for replacement.

Here is the caller graph for this function:

**9.14.2.2 CreateLv2Item()**

```
GrammarFactory::FactoryItem GrammarFactory::CreateLv2Item ()
```

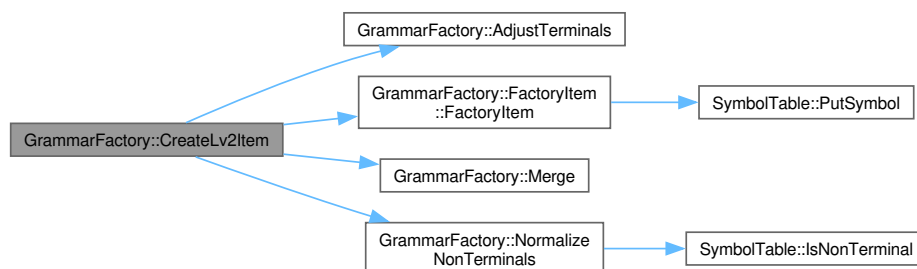
Creates a Level 2 grammar item for use in grammar generation.

This function generates a Level 2 grammar item, which can be used as a building block for creating more complex grammars.

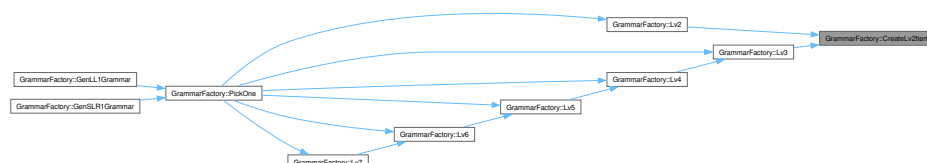
Returns

A [FactoryItem](#) representing a Level 2 grammar.

Here is the call graph for this function:



Here is the caller graph for this function:



9.14.2.3 GenerateNewNonTerminal()

```
std::string GrammarFactory::GenerateNewNonTerminal (
    Grammar & grammar,
    const std::string & base)
```

Generates a new non-terminal symbol that is unique in the grammar.

This function creates a new non-terminal symbol by appending a prime symbol (') to the base name until the resulting symbol is not already present in the grammar's symbol table. It is used during left factorization to introduce new non-terminals for factored productions.

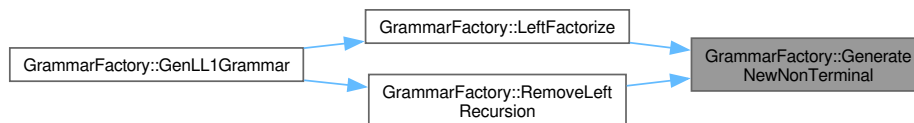
Parameters

<i>grammar</i>	The grammar in which the new non-terminal will be added.
<i>base</i>	The base name for the new non-terminal.

Returns

A unique non-terminal symbol derived from the base name.

Here is the caller graph for this function:



9.14.2.4 GenLL1Grammar()

```
Grammar GrammarFactory::GenLL1Grammar (
    int level)
```

Generates a LL(1) random grammar based on the specified difficulty level.

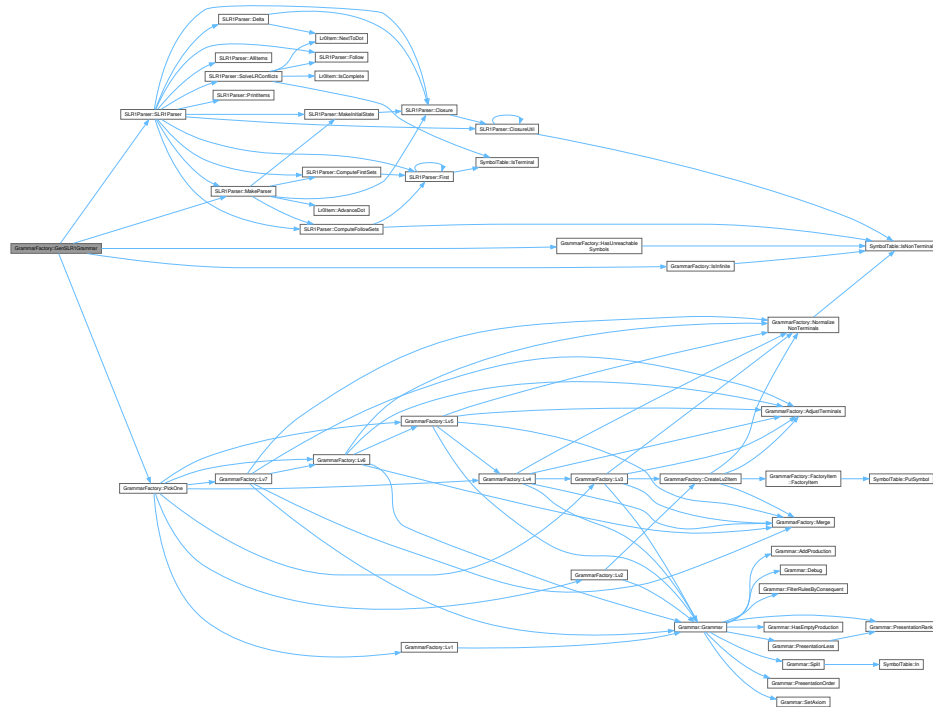
Parameters

<i>level</i>	The difficulty level (1, 2, or 3)
--------------	-----------------------------------

Returns

A random SLR(1) grammar.

Here is the call graph for this function:

**9.14.2.6 HasCycle()**

```
bool GrammarFactory::HasCycle (
    const std::unordered_map< std::string, std::unordered_set< std::string > > &
    graph) const
```

Checks if directed graph has a cycle using topological sort.

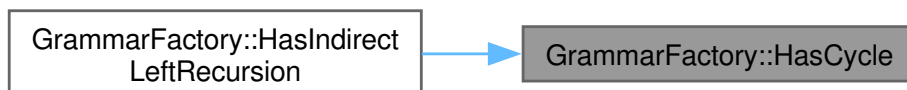
Parameters

<i>graph</i>	The directed graph.
--------------	---------------------

Returns

true if grammar has cycle.

Here is the caller graph for this function:



9.14.2.7 HasDirectLeftRecursion()

```
bool GrammarFactory::HasDirectLeftRecursion (
    const Grammar & grammar) const
```

Checks if a grammar contains direct left recursion (a non-terminal can produce itself on the left side of a production in one step).

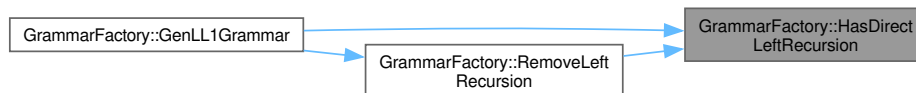
Parameters

<i>grammar</i>	The grammar to check.
----------------	-----------------------

Returns

true if there is direct left recursion, false otherwise.

Here is the caller graph for this function:



9.14.2.8 HasIndirectLeftRecursion()

```
bool GrammarFactory::HasIndirectLeftRecursion (
    const Grammar & grammar) const
```

Checks if a grammar contains indirect left recursion.

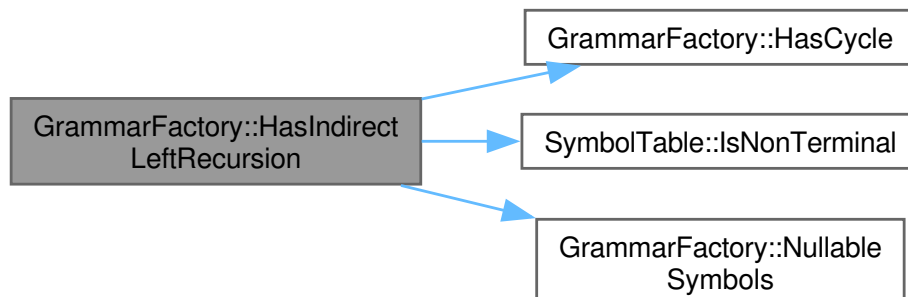
Parameters

<i>grammar</i>	The grammar to check.
----------------	-----------------------

Returns

true if there is direct left recursion, false otherwise.

Here is the call graph for this function:



9.14.2.9 HasUnreachableSymbols()

```
bool GrammarFactory::HasUnreachableSymbols (
    Grammar & grammar) const
```

Checks if a grammar contains unreachable symbols (non-terminals that cannot be derived from the start symbol).

Parameters

<i>grammar</i>	The grammar to check.
----------------	-----------------------

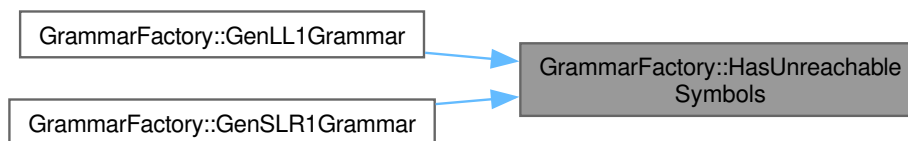
Returns

true if there are unreachable symbols, false otherwise.

Here is the call graph for this function:



Here is the caller graph for this function:



9.14.2.10 Init()

```
void GrammarFactory::Init ()
```

Initializes the [GrammarFactory](#) and populates the items vector with initial grammar items.

9.14.2.11 IsInfinite()

```
bool GrammarFactory::IsInfinite (
    Grammar & grammar) const
```

Checks if a grammar is infinite, meaning there are non-terminal symbols that can never derive a terminal string. This happens when a production leads to an infinite recursion or an endless derivation without reaching terminal symbols. For example, a production like:

```
S -> A
A -> a A | B
B -> c B
```

could lead to an infinite derivation of non-terminals.

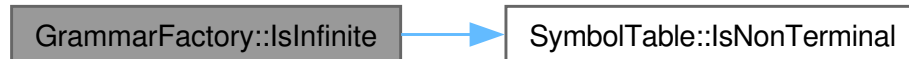
Parameters

<i>grammar</i>	The grammar to check.
----------------	-----------------------

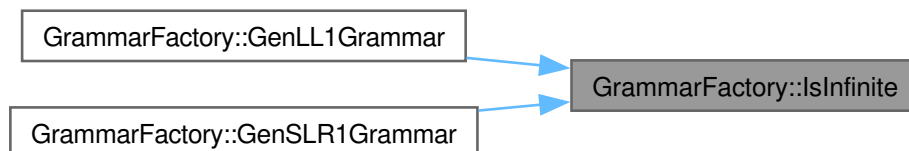
Returns

true if the grammar has infinite derivations, false otherwise.

Here is the call graph for this function:



Here is the caller graph for this function:

**9.14.2.12 LeftFactorize()**

```
void GrammarFactory::LeftFactorize (
    Grammar & grammar)
```

Performs left factorization. A grammar can be left factorized if it has productions with the same prefix for one non-terminal. For example:

```
A -> a x | a y
```

could be left factorized because it has "a" as the common prefix. The left factorization is done by adding a new non-terminal symbol that contains the uncommon part, and by unifying the common prefix in one production. So:

```
A -> a x | a y
```

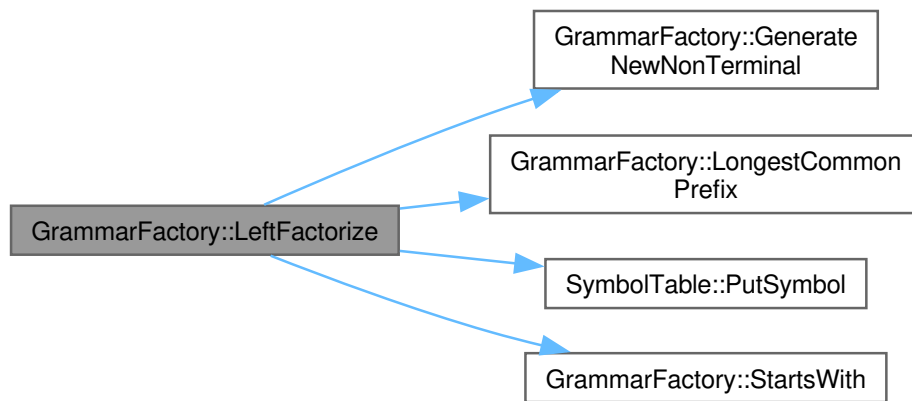
would become:

```
A -> a A'
A' -> x | y
```

Parameters

<i>grammar</i>	The grammar to be left factorized.
----------------	------------------------------------

Here is the call graph for this function:



Here is the caller graph for this function:



9.14.2.13 LongestCommonPrefix()

```
std::vector< std::string > GrammarFactory::LongestCommonPrefix (
    const std::vector< production > & productions)
```

Finds the longest common prefix among a set of productions.

This function computes the longest sequence of symbols that is common to the beginning of all productions in the given vector. It is used during left factorization to identify common prefixes that can be factored out.

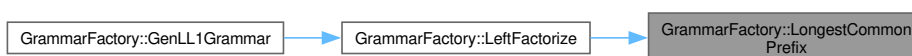
Parameters

<i>productions</i>	A vector of productions to analyze.
--------------------	-------------------------------------

Returns

A vector of strings representing the longest common prefix. If no common prefix exists, an empty vector is returned.

Here is the caller graph for this function:



9.14.2.14 Lv1()

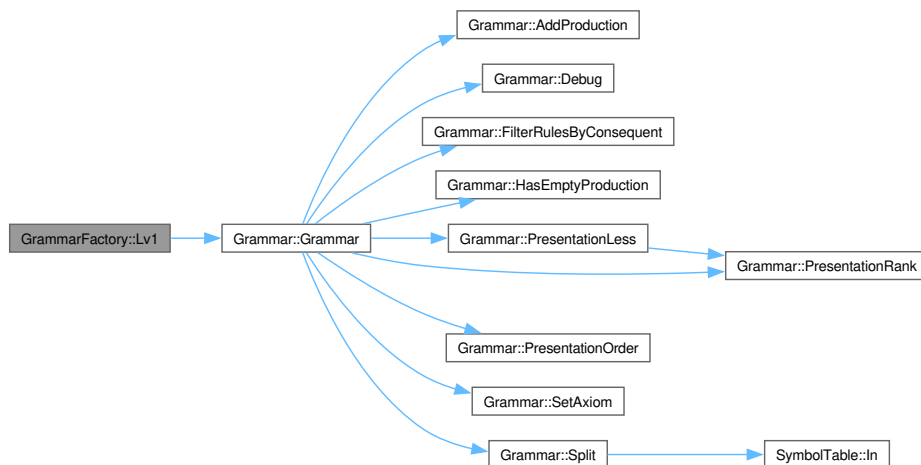
```
Grammar GrammarFactory::Lv1 ()
```

Generates a Level 1 grammar.

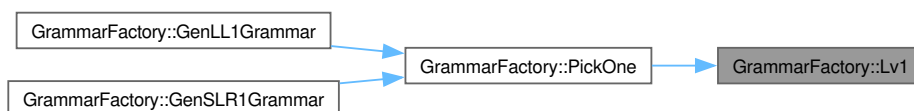
Returns

A Level 1 grammar.

Here is the call graph for this function:



Here is the caller graph for this function:



9.14.2.15 Lv2()

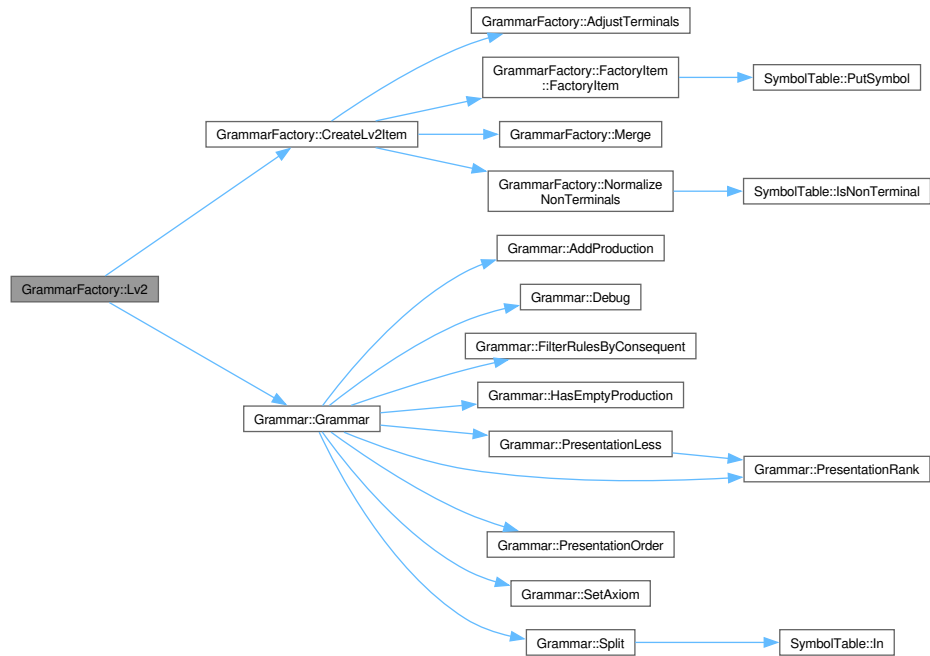
```
Grammar GrammarFactory::Lv2 ()
```

Generates a Level 2 grammar by combining Level 1 items.

Returns

A Level 2 grammar.

Here is the call graph for this function:



Here is the caller graph for this function:



9.14.2.16 Lv3()

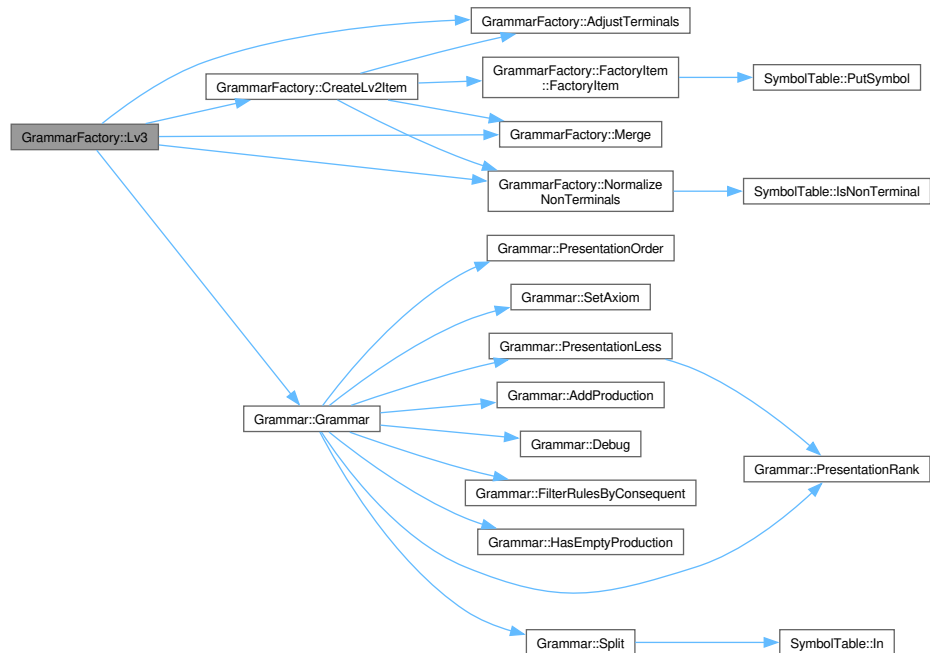
```
Grammar GrammarFactory::Lv3 ()
```

Generates a Level 3 grammar by combining a Level 2 item and a Level 1 item.

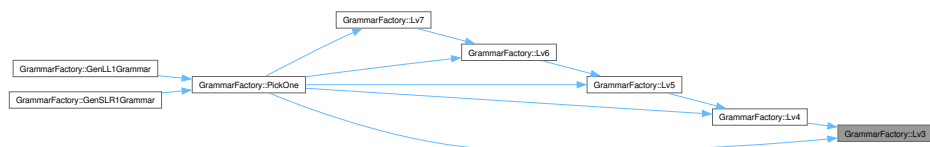
Returns

A Level 3 grammar.

Here is the call graph for this function:



Here is the caller graph for this function:

**9.14.2.17 Lv4()**

```
Grammar GrammarFactory::Lv4 ()
```

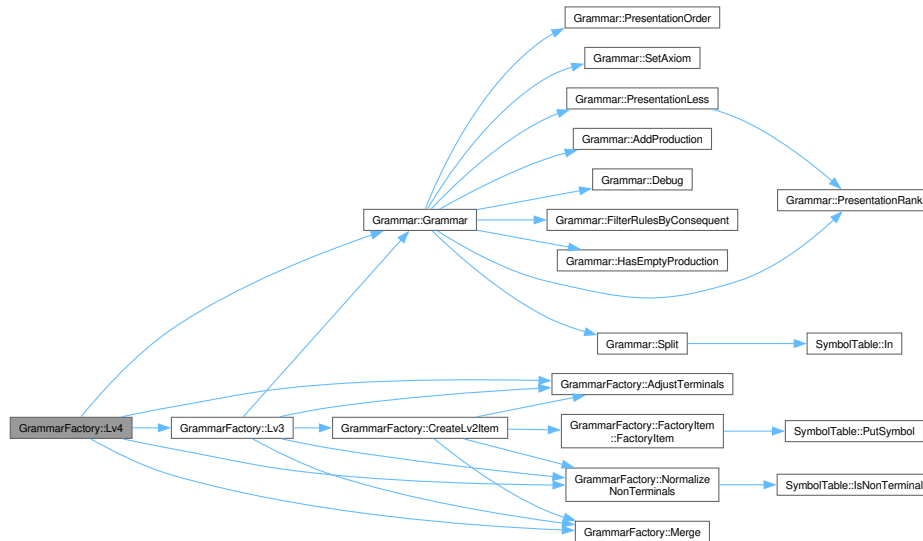
Generates a Level 4 grammar by combining Level 3 and Level 1 items.

This function creates a more complex grammar by combining elements from Level 3 and Level 1 grammars. It is used to generate grammars with increased complexity for testing or parsing purposes.

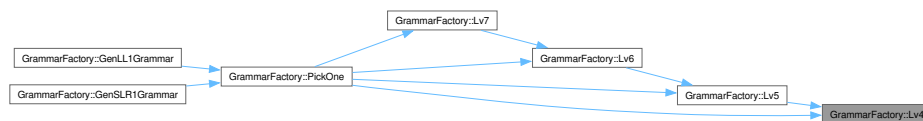
Returns

A Level 4 grammar.

Here is the call graph for this function:



Here is the caller graph for this function:



9.14.2.18 Lv5()

```
Grammar GrammarFactory::Lv5 ()
```

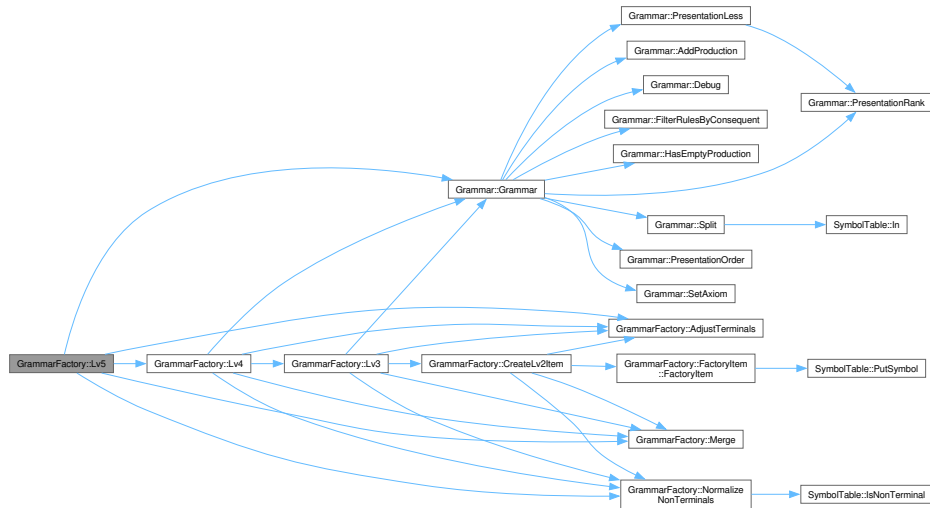
Generates a Level 5 grammar by combining Level 4 and Level 1 items.

This function creates a more advanced grammar by combining elements from Level 4 and Level 1 grammars. It is used to generate grammars with higher complexity for testing or parsing purposes.

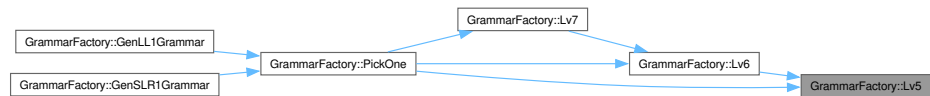
Returns

A Level 5 grammar.

Here is the call graph for this function:



Here is the caller graph for this function:

**9.14.2.19 Lv6()**

```
Grammar GrammarFactory::Lv6 ()
```

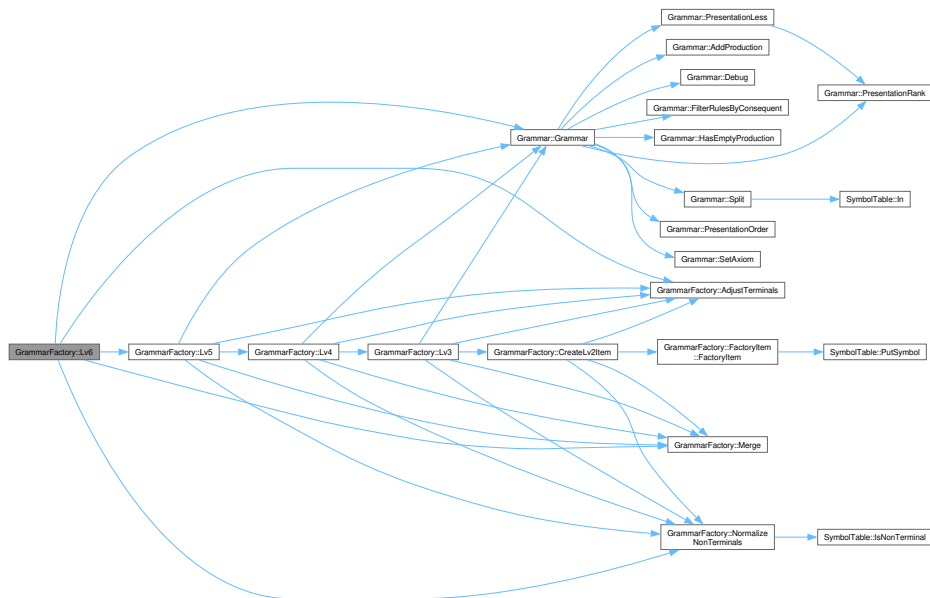
Generates a Level 6 grammar by combining Level 5 and Level 1 items.

This function creates a highly complex grammar by combining elements from Level 5 and Level 1 grammars. It is used to generate grammars with advanced structures for testing or parsing purposes.

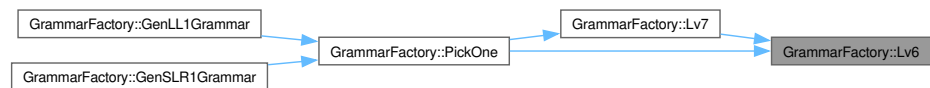
Returns

A Level 6 grammar.

Here is the call graph for this function:



Here is the caller graph for this function:

**9.14.2.20 Lv7()**

```
Grammar GrammarFactory::Lv7 ()
```

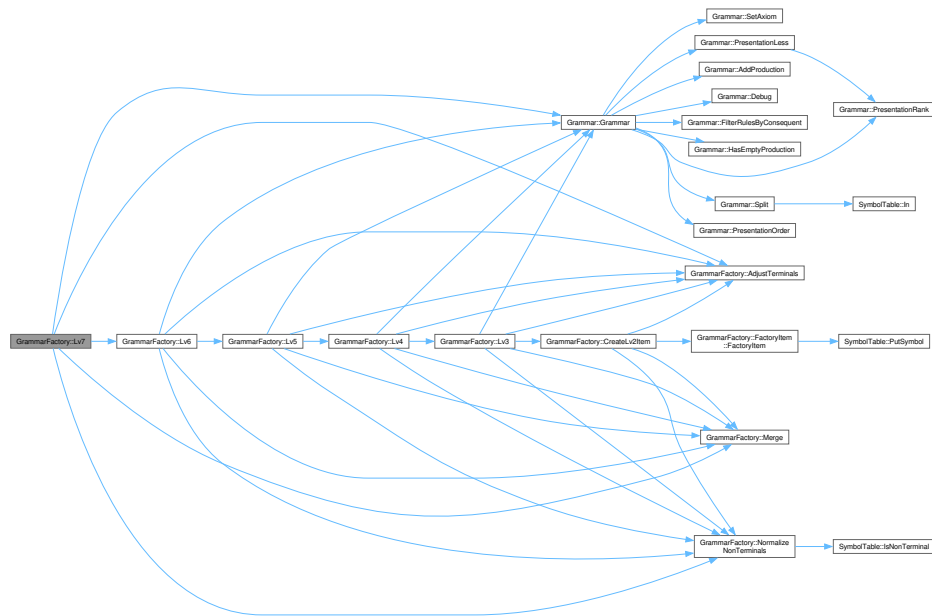
Generates a Level 7 grammar by combining Level 6 and Level 1 items.

This function creates a very complex grammar by combining elements from Level 6 and Level 1 grammars. It is used to generate grammars with highly advanced structures for testing or parsing purposes.

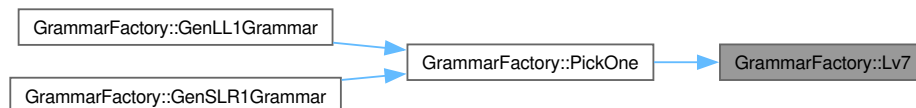
Returns

A Level 7 grammar.

Here is the call graph for this function:



Here is the caller graph for this function:

**9.14.2.21 Merge()**

```
std::unordered_map< std::string, std::vector< production > > GrammarFactory::Merge (
    const FactoryItem & base,
    const FactoryItem & cmb) const
```

Merges the grammar rules of two grammar items into a single grammar.

This function performs a raw combination of the production rules from both grammar items, resulting in a single grammar map that contains all productions.

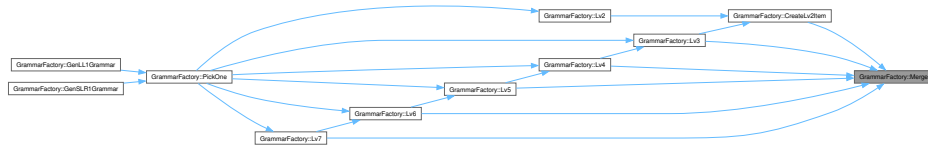
Parameters

<i>base</i>	The first grammar item.
<i>cmb</i>	The second grammar item.

Returns

A merged grammar map containing all production rules from both inputs.

Here is the caller graph for this function:

**9.14.2.22 NormalizeNonTerminals()**

```
void GrammarFactory::NormalizeNonTerminals (
    FactoryItem & item,
    const std::string & nt) const
```

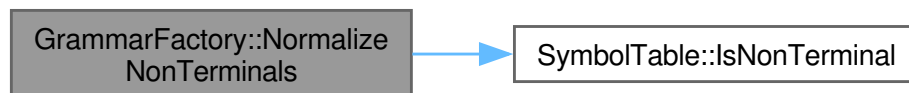
Replaces all non-terminal symbols in a grammar item with a single target non-terminal.

This function is used during grammar combination to normalize the non-terminal symbols in a given [FactoryItem](#), so that they are consistent and compatible with another item.

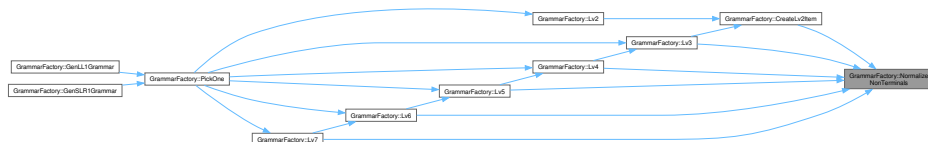
Parameters

<i>item</i>	The grammar item whose non-terminals will be renamed.
<i>nt</i>	The new non-terminal symbol that will replace all existing ones.

Here is the call graph for this function:



Here is the caller graph for this function:



9.14.2.23 NullableSymbols()

```
std::unordered_set< std::string > GrammarFactory::NullableSymbols (
    const Grammar & grammar) const
```

Find nullable symbols in a grammar.

Parameters

<i>grammar</i>	The grammar to check.
----------------	-----------------------

Returns

set of nullable symbols.

Here is the caller graph for this function:



9.14.2.24 PickOne()

```
Grammar GrammarFactory::PickOne (
    int level)
```

Picks a random grammar based on the specified difficulty level (1, 2, or 3).

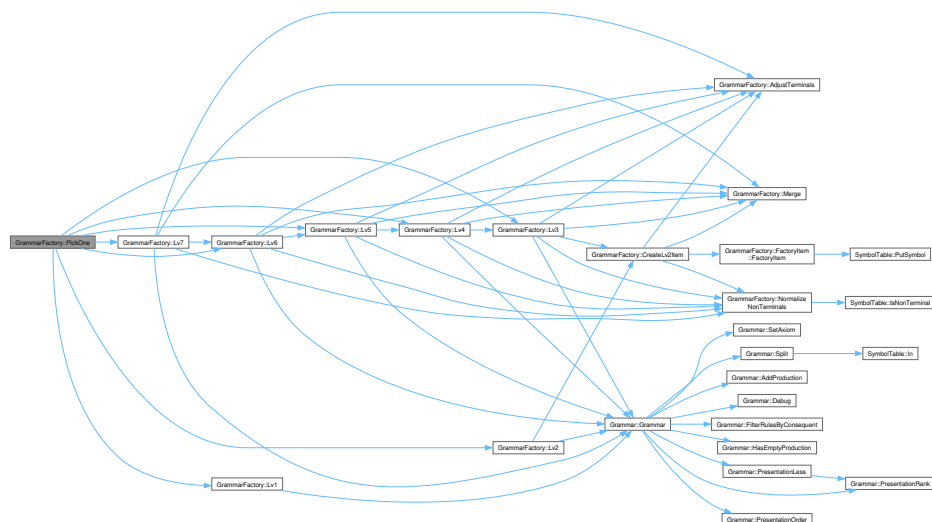
Parameters

<i>level</i>	The difficulty level (1, 2, or 3).
--------------	------------------------------------

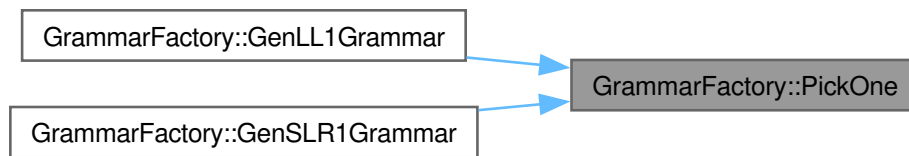
Returns

A randomly picked grammar.

Here is the call graph for this function:



Here is the caller graph for this function:



9.14.2.25 RemoveLeftRecursion()

```
void GrammarFactory::RemoveLeftRecursion (
    Grammar & grammar)
```

Removes direct left recursion in a grammar. A grammar has direct left recursion when one of its productions is.

$A \rightarrow A a$

where A is a non-terminal symbol and "a" the rest of the production. The procedure removes direct left recursion by adding a new non-terminal. So, if the productions with left recursion are:

$A \rightarrow A a \mid b$

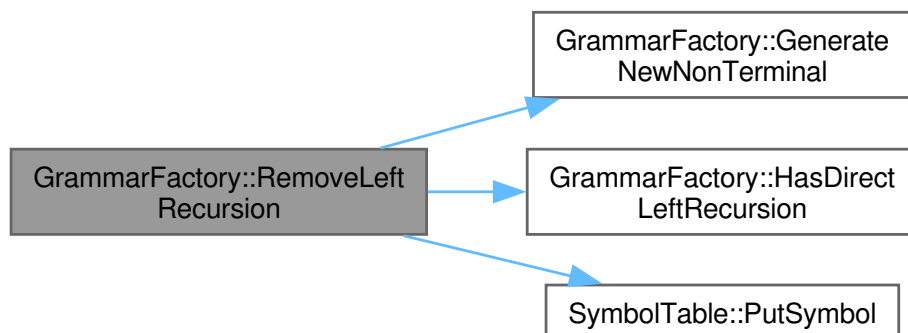
the result would be:

$A \rightarrow b A'$
 $A' \rightarrow a A' \mid \text{epsilon}$

Parameters

<i>grammar</i>	The grammar to remove left recursion.
----------------	---------------------------------------

Here is the call graph for this function:



Here is the caller graph for this function:



9.14.2.26 StartsWith()

```
bool GrammarFactory::StartsWith (
    const production & prod,
    const std::vector< std::string > & prefix)
```

Checks if a production starts with a given prefix.

This function determines whether the symbols in a production match the provided prefix sequence at the beginning. It is used during left factorization to identify productions that share a common prefix.

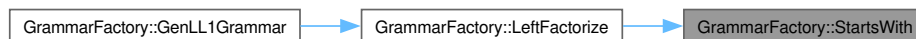
Parameters

<i>prod</i>	The production to check.
<i>prefix</i>	The sequence of symbols to compare against the beginning of the production.

Returns

`true` if the production starts with the prefix, `false` otherwise.

Here is the caller graph for this function:



9.14.3 Member Data Documentation

9.14.3.1 items

```
std::vector<FactoryItem> GrammarFactory::items
```

A vector of [FactoryItem](#) objects representing different level 1 grammar items created by the Init method.

9.14.3.2 non_terminal_alphabet_

```
std::vector<std::string> GrammarFactory::non_terminal_alphabet_
```

Initial value:

```
{ "A", "B", "C", "D",
                                     "E", "F", "G" }
```

A vector of non-terminal symbols (alphabet) used in the grammar.

9.14.3.3 terminal_alphabet_

```
std::vector<std::string> GrammarFactory::terminal_alphabet_
```

Initial value:

```
{ "a", "b", "c", "d", "e", "f",  
                                     "g", "h", "i", "j", "k", "l" }
```

A vector of terminal symbols (alphabet) used in the grammar.

The documentation for this struct was generated from the following files:

- [backend/grammar_factory.hpp](#)
- [backend/grammar_factory.cpp](#)

9.15 GrammarParseError Struct Reference

Describes one problem found while parsing user-written grammar text.

```
#include <grammar_parser.hpp>
```

Public Types

- enum class [Kind](#) {
[EmptyGrammar](#) , [MissingArrow](#) , [MissingEndDot](#) , [EmptyLeftHandSide](#) ,
[MultipleLeftHandSide](#) , [ExtraArrow](#) , [ReservedSymbol](#) }
Machine-readable category of a grammar text error.

Public Attributes

- [Kind kind](#)
Category of the error.
- int [line](#)
1-based line in the source text where the rule starts.
- std::string [detail](#)
Offending token or rule fragment, for diagnostics.

9.15.1 Detailed Description

Describes one problem found while parsing user-written grammar text.

Errors carry a machine-readable kind, the 1-based line where the offending rule starts, and the offending fragment so the GUI can build a localized, user-friendly message.

9.15.2 Member Enumeration Documentation

9.15.2.1 Kind

```
enum class GrammarParseError::Kind [strong]
```

Machine-readable category of a grammar text error.

Enumerator

EmptyGrammar	The text contains no rules at all.
MissingArrow	A rule has no ">" separator.
MissingEndDot	Text after the last "." looks like an unterminated rule.
EmptyLeftHandSide	A rule has no symbol before ">".
MultipleLeftHandSide	A rule has more than one symbol before ">".
ExtraArrow	A rule body contains another ">", usually a missing dot.
ReservedSymbol	A symbol uses a reserved character or reserved word.

9.15.3 Member Data Documentation

9.15.3.1 detail

```
std::string GrammarParseError::detail
```

Offending token or rule fragment, for diagnostics.

9.15.3.2 kind

```
Kind GrammarParseError::kind
```

Category of the error.

9.15.3.3 line

```
int GrammarParseError::line
```

1-based line in the source text where the rule starts.

The documentation for this struct was generated from the following file:

- [backend/grammar_parser.hpp](#)

9.16 GrammarParser Struct Reference

Parses user-written grammar text into a [Grammar](#) object.

```
#include <grammar_parser.hpp>
```

Static Public Member Functions

- static [GrammarParseResult Parse](#) (const std::string &text)
Parses grammar text into an augmented [Grammar](#).
- static bool [IsReservedToken](#) (const std::string &token)
Checks whether a token cannot be used as a grammar symbol.

9.16.1 Detailed Description

Parses user-written grammar text into a [Grammar](#) object.

Input format, kept deliberately flexible:

- Each rule is LHS -> SYMBOL SYMBOL and ends with a dot. Rules may span several lines and several rules may share one line.
- Symbols are separated by whitespace; any whitespace-free string is a valid symbol, so multi-character tokens such as `id` or `function` are supported. `A b` is two symbols while `Ab` is one.
- "→" is accepted as an alias of "->", with or without surrounding spaces.
- Alternatives can be grouped with |: `A -> a | b .`
- An empty right-hand side (`A -> .` or an empty | branch) denotes an epsilon production.
- Non-terminals are exactly the symbols that appear on a left-hand side; every other symbol is a terminal. The first rule defines the axiom.

9.16.2 Member Function Documentation

9.16.2.1 IsReservedToken()

```
bool GrammarParser::IsReservedToken (
    const std::string & token) [static]
```

Checks whether a token cannot be used as a grammar symbol.

Reserved tokens are the meta symbols (\$, EPSILON, ϵ , $\cdot$) and any token containing characters used by the tutors as separators or markers (dot, comma, colon, semicolon, pipe or an embedded arrow).

Parameters

<i>token</i>	Token to check.
--------------	-----------------

Returns

true if the token is reserved and must be rejected.

9.16.2.2 Parse()

```
GrammarParseResult GrammarParser::Parse (  
    const std::string & text) [static]
```

Parses grammar text into an augmented [Grammar](#).

Parameters

<i>text</i>	User-written grammar description.
-------------	-----------------------------------

Returns

Parse result with either a ready-to-use grammar or the list of errors found.

The documentation for this struct was generated from the following files:

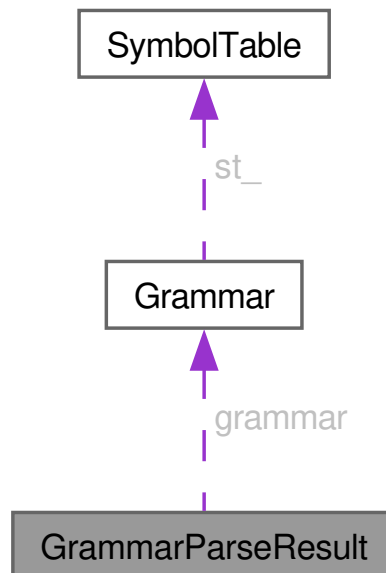
- [backend/grammar_parser.hpp](#)
- [backend/grammar_parser.cpp](#)

9.17 GrammarParseResult Struct Reference

Outcome of parsing user-written grammar text.

```
#include <grammar_parser.hpp>
```

Collaboration diagram for GrammarParseResult:



Public Member Functions

- bool [Ok](#) () const
True when parsing succeeded and [grammar](#) is usable.

Public Attributes

- [Grammar grammar](#)
Resulting grammar. Only meaningful when [errors](#) is empty.
- `std::vector< GrammarParseError > errors`
All problems found in the text, in source order.

9.17.1 Detailed Description

Outcome of parsing user-written grammar text.

When [errors](#) is empty, [grammar](#) holds a fully-built grammar: symbols are classified (non-terminals are exactly the left-hand sides), the axiom is the left-hand side of the first rule, and the grammar has been augmented with a fresh axiom rule `X -> <axiom> $` following the app-wide convention.

9.17.2 Member Function Documentation

9.17.2.1 `Ok()`

```
bool GrammarParseResult::Ok () const [inline]
```

True when parsing succeeded and [grammar](#) is usable.

9.17.3 Member Data Documentation

9.17.3.1 `errors`

```
std::vector<GrammarParseError> GrammarParseResult::errors
```

All problems found in the text, in source order.

9.17.3.2 `grammar`

```
Grammar GrammarParseResult::grammar
```

Resulting grammar. Only meaningful when [errors](#) is empty.

The documentation for this struct was generated from the following file:

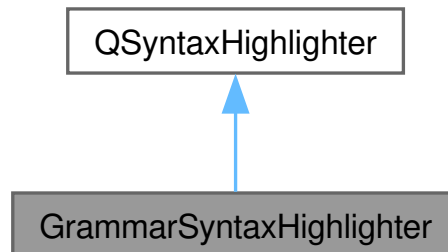
- [backend/grammar_parser.hpp](#)

9.18 GrammarSyntaxHighlighter Class Reference

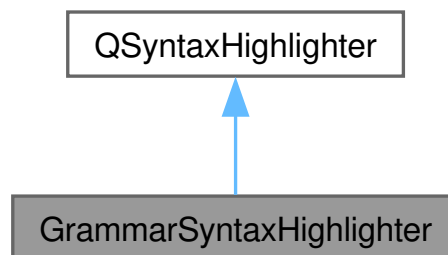
Lightweight highlighter for user-written grammar text.

```
#include <grammareditordialog.h>
```

Inheritance diagram for GrammarSyntaxHighlighter:



Collaboration diagram for GrammarSyntaxHighlighter:



Public Member Functions

- [GrammarSyntaxHighlighter](#) (QTextDocument *parent)
- void [setNonTerminals](#) (const QSet< QString > &nonTerminals)
Updates the set of symbols rendered as non-terminals.

Protected Member Functions

- void [highlightBlock](#) (const QString &text) override

9.18.1 Detailed Description

Lightweight highlighter for user-written grammar text.

Colors the structural pieces of a grammar (arrows, rule dots and alternative pipes) and renders the currently-detected non-terminals in the accent color, so the user gets immediate feedback on how their text is being interpreted.

9.18.2 Constructor & Destructor Documentation

9.18.2.1 GrammarSyntaxHighlighter()

```
GrammarSyntaxHighlighter::GrammarSyntaxHighlighter (
    QTextDocument * parent) [explicit]
```

9.18.3 Member Function Documentation

9.18.3.1 highlightBlock()

```
void GrammarSyntaxHighlighter::highlightBlock (
    const QString & text) [override], [protected]
```

9.18.3.2 setNonTerminals()

```
void GrammarSyntaxHighlighter::setNonTerminals (
    const QSet< QString > & nonTerminals)
```

Updates the set of symbols rendered as non-terminals.

Triggers a rehighlight only when the set actually changes.

Parameters

<i>nonTerminals</i>	Symbols currently classified as non-terminals.
---------------------	--

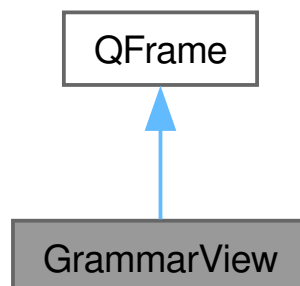
The documentation for this class was generated from the following files:

- [gui/grammareditordialog.h](#)
- [gui/grammareditordialog.cpp](#)

9.19 GrammarView Class Reference

```
#include <grammarview.h>
```

Inheritance diagram for GrammarView:

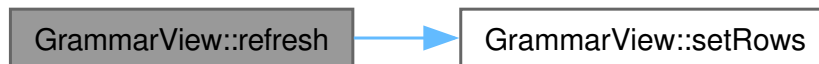


9.19.2.2 refresh()

```
void GrammarView::refresh ()
```

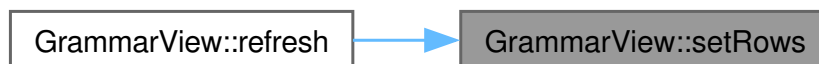
Rebuilds the rows so a new text size takes effect.

Here is the call graph for this function:

**9.19.2.3 setRows()**

```
void GrammarView::setRows (
    const QVector< Row > & rows)
```

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

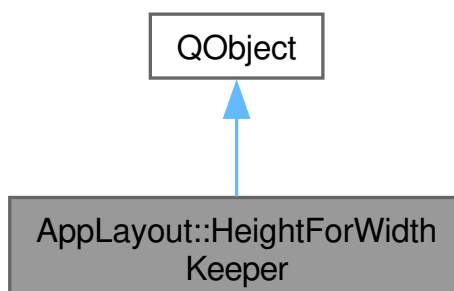
- [widgets/grammarview.h](#)
- [widgets/grammarview.cpp](#)

9.20 AppLayout::HeightForWidthKeeper Class Reference

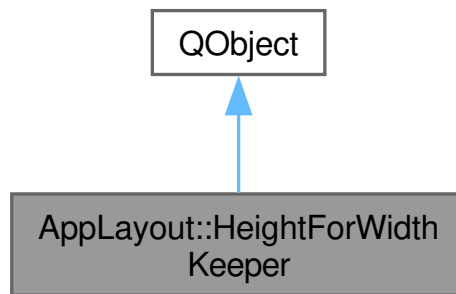
Event filter behind [keepHeightForWidth](#).

```
#include <applayout.h>
```

Inheritance diagram for `AppLayout::HeightForWidthKeeper`:



Collaboration diagram for `AppLayout::HeightForWidthKeeper`:



Public Member Functions

- [HeightForWidthKeeper](#) (`QWidget *widget`)

Protected Member Functions

- bool [eventFilter](#) (`QObject *watched`, `QEvent *event`) override

9.20.1 Detailed Description

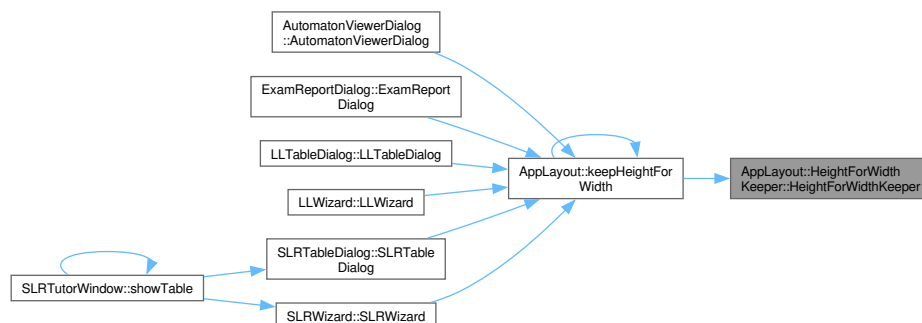
Event filter behind [keepHeightForWidth](#).

9.20.2 Constructor & Destructor Documentation

9.20.2.1 HeightForWidthKeeper()

```
AppLayout::HeightForWidthKeeper::HeightForWidthKeeper (
    QWidget * widget) [inline], [explicit]
```

Here is the caller graph for this function:



9.20.3 Member Function Documentation

9.20.3.1 eventFilter()

```
bool AppLayout::HeightForWidthKeeper::eventFilter (
    QObject * watched,
    QEvent * event) [inline], [override], [protected]
```

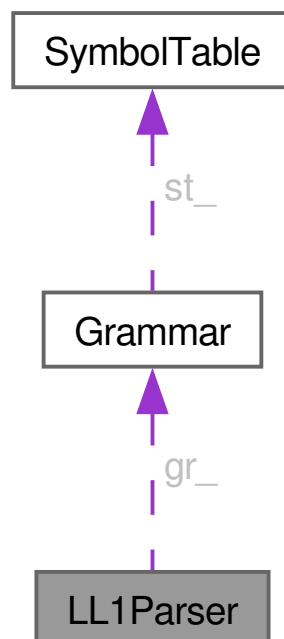
The documentation for this class was generated from the following file:

- widgets/aplayout.h

9.21 LL1Parser Class Reference

```
#include <ll1_parser.hpp>
```

Collaboration diagram for LL1Parser:



Public Member Functions

- [LL1Parser](#) ()=default
- [LL1Parser](#) ([Grammar](#) gr)
Constructs an [LL1Parser](#) with a grammar object and an input file.
- bool [CreateLL1Table](#) ()
Creates the LL(1) parsing table for the grammar.
- void [First](#) (std::span< const std::string > rule, std::unordered_set< std::string > &result)
Calculates the FIRST set for a given production rule in a grammar.
- void [ComputeFirstSets](#) ()
Computes the FIRST sets for all non-terminal symbols in the grammar.

9.21.1.2 LL1Parser() [2/2]

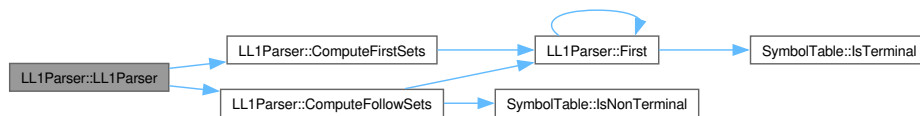
```
LL1Parser::LL1Parser (
    Grammar gr) [explicit]
```

Constructs an [LL1Parser](#) with a grammar object and an input file.

Parameters

<i>gr</i>	Grammar object to parse with
-----------	--

Here is the call graph for this function:



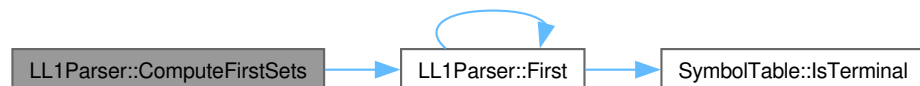
9.21.2 Member Function Documentation

9.21.2.1 ComputeFirstSets()

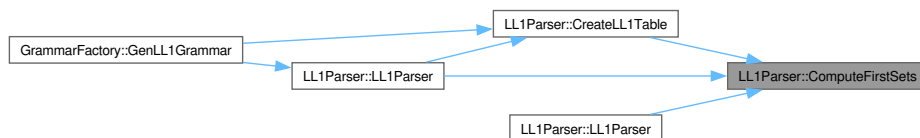
```
void LL1Parser::ComputeFirstSets ()
```

Computes the FIRST sets for all non-terminal symbols in the grammar.

This function calculates the FIRST set for each non-terminal symbol in the grammar by iteratively applying a least fixed-point algorithm. This approach ensures that the FIRST sets are fully populated by repeatedly expanding and updating the sets until no further changes occur (i.e., a fixed-point is reached). Here is the call graph for this function:



Here is the caller graph for this function:



9.21.2.2 ComputeFollowSets()

```
void LL1Parser::ComputeFollowSets ()
```

Computes the FOLLOW sets for all non-terminal symbols in the grammar. The FOLLOW set of a non-terminal symbol A contains all terminal symbols that can appear immediately after A in any sentential form derived from the grammar's start symbol. Additionally, if A can be the last symbol in a derivation, the end-of-input marker ($\$$) is included in its FOLLOW set. This function computes the FOLLOW sets using the following rules:

1. Initialize $FOLLOW(S) = \{ \$ \}$, where S is the start symbol.
2. For each production rule of the form $A \rightarrow \alpha B \beta$:
 - Add $FIRST(\beta) \setminus \{ \epsilon \}$ to $FOLLOW(B)$.
 - If $\epsilon \in FIRST(\beta)$, add $FOLLOW(A)$ to $FOLLOW(B)$.
3. Repeat step 2 until no changes occur in any FOLLOW set. The computed FOLLOW sets are cached in the `follow_sets_` member variable for later use by the parser.

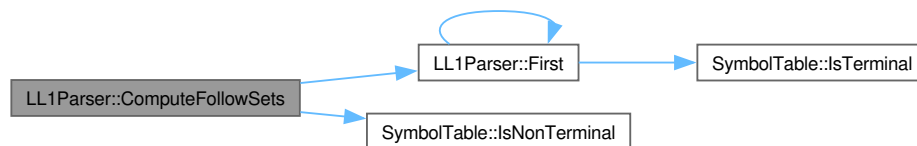
Note

This function assumes that the FIRST sets for all symbols have already been computed and are available in the `first_sets_` member variable.

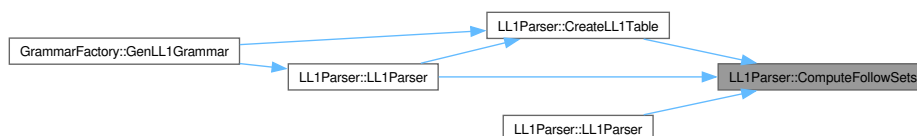
See also

[First](#)
[follow_sets_](#)

Here is the call graph for this function:



Here is the caller graph for this function:



9.21.2.3 CreateLL1Table()

```
bool LL1Parser::CreateLL1Table ()
```

Creates the LL(1) parsing table for the grammar.

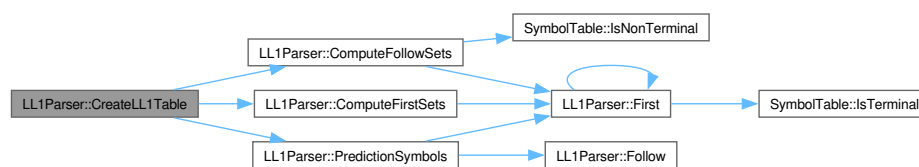
This function constructs the LL(1) parsing table by iterating over each production in the grammar and determining the appropriate cells for each non-terminal and director symbol (prediction symbol) combination. If the grammar is LL(1) compatible, each cell will contain at most one production, indicating no conflicts. If conflicts are found, the function will return `false`, signaling that the grammar is not LL(1).

- For each production rule $A \rightarrow \alpha$, the function calculates the prediction symbols using the `PredictionSymbols` function.
- It then fills the parsing table at the cell corresponding to the non-terminal A and each prediction symbol in the set.
- If a cell already contains a production, this indicates a conflict, meaning the grammar is not LL(1).

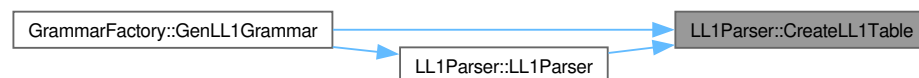
Returns

`true` if the table is created successfully, indicating the grammar is LL(1) compatible; `false` if any conflicts are detected, showing that the grammar does not meet LL(1) requirements.

Here is the call graph for this function:



Here is the caller graph for this function:



9.21.2.4 First()

```

void LL1Parser::First (
    std::span< const std::string > rule,
    std::unordered_set< std::string > & result)
  
```

Calculates the FIRST set for a given production rule in a grammar.

The FIRST set of a production rule contains all terminal symbols that can appear at the beginning of any string derived from that rule. If the rule can derive the empty string (epsilon), epsilon is included in the FIRST set.

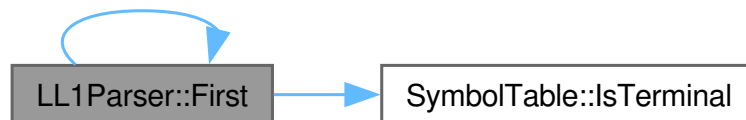
This function computes the FIRST set by examining each symbol in the production rule:

- If a terminal symbol is encountered, it is added directly to the FIRST set, as it is the starting symbol of some derivation.
- If a non-terminal symbol is encountered, its FIRST set is recursively computed and added to the result, excluding epsilon unless it is followed by another symbol that could also lead to epsilon.
- If the entire rule could derive epsilon (i.e., each symbol in the rule can derive epsilon), then epsilon is added to the FIRST set.

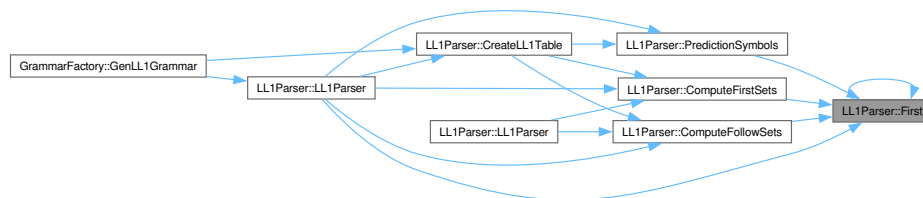
Parameters

<i>rule</i>	A span of strings representing the production rule for which to compute the FIRST set. Each string in the span is a symbol (either terminal or non-terminal).
<i>result</i>	A reference to an unordered set of strings where the computed FIRST set will be stored. The set will contain all terminal symbols that can start derivations of the rule, and possibly epsilon if the rule can derive an empty string.

Here is the call graph for this function:



Here is the caller graph for this function:

**9.21.2.5 Follow()**

```
std::unordered_set< std::string > LL1Parser::Follow (
    const std::string & arg)
```

Computes the FOLLOW set for a given non-terminal symbol in the grammar.

The FOLLOW set for a non-terminal symbol includes all symbols that can appear immediately to the right of that symbol in any derivation, as well as any end-of-input markers if the symbol can appear at the end of derivations. FOLLOW sets are used in LL(1) parsing table construction to determine possible continuations after a non-terminal.

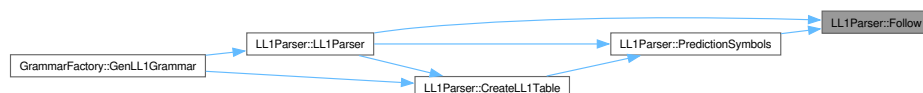
Parameters

<i>arg</i>	Non-terminal symbol for which to compute the FOLLOW set.
------------	--

Returns

An unordered set of strings containing symbols that form the FOLLOW set for `arg`.

Here is the caller graph for this function:



9.21.2.6 PredictionSymbols()

```
std::unordered_set< std::string > LL1Parser::PredictionSymbols (
    const std::string & antecedent,
    const std::vector< std::string > & consequent)
```

Computes the prediction symbols for a given production rule.

- The prediction symbols for a rule determine the set of input symbols that can trigger this rule in the parsing table. This function calculates the prediction symbols based on the FIRST set of the consequent and, if epsilon (the empty symbol) is in the FIRST set, also includes the FOLLOW set of the antecedent.
- - If the FIRST set of the consequent does not contain epsilon, the prediction symbols are simply the FIRST symbols of the consequent.

If the FIRST set of the consequent contains epsilon, the prediction symbols are computed as $FIRST(consequent) \setminus \{\epsilon\} \cup FOLLOW(antecedent)$.

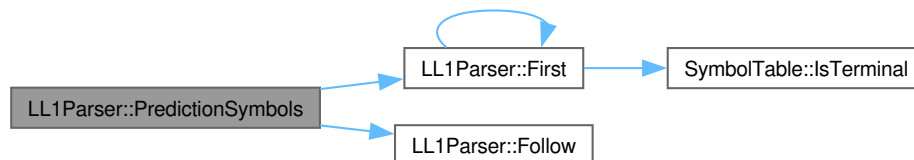
Parameters

<i>antecedent</i>	The left-hand side non-terminal symbol of the rule.
<i>consequent</i>	A vector of symbols on the right-hand side of the rule (production body).

Returns

An unordered set of strings containing the prediction symbols for the specified rule.

Here is the call graph for this function:



Here is the caller graph for this function:



9.21.3 Member Data Documentation

9.21.3.1 first_sets_

```
std::unordered_map<std::string, std::unordered_set<std::string> > LL1Parser::first_sets_
```

FIRST sets for each non-terminal in the grammar.

9.21.3.2 follow_sets_

```
std::unordered_map<std::string, std::unordered_set<std::string> > LL1Parser::follow_sets_
```

FOLLOW sets for each non-terminal in the grammar.

9.21.3.3 gr_

```
Grammar LL1Parser::gr_
```

Grammar object associated with this parser.

9.21.3.4 ll1_t_

```
ll1_table LL1Parser::ll1_t_
```

The LL(1) parsing table, mapping non-terminals and terminals to productions.

The documentation for this class was generated from the following files:

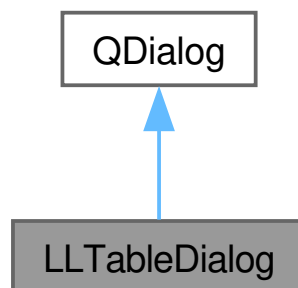
- [backend/ll1_parser.hpp](#)
- [backend/ll1_parser.cpp](#)

9.22 LLTableDialog Class Reference

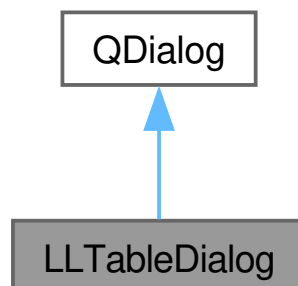
Dialog for filling and submitting an LL(1) parsing table.

```
#include <lltabledialog.h>
```

Inheritance diagram for LLTableDialog:



Collaboration diagram for LLTableDialog:



Signals

- void [submitted](#) (const QVector< QVector< QString > > &data)
Signal emitted when the user submits the table.
- void [guidedRequested](#) (const QVector< QVector< QString > > &data)
Signal emitted when the user asks for the guided mode.

Public Member Functions

- [LLTableDialog](#) (const QStringList &rowHeaders, const QStringList &colHeaders, QWidget *parent, QVector< QVector< QString > > *initialData=nullptr)
Constructs the LL(1) table dialog with given headers and optional initial data.
- QVector< QVector< QString > > [getTableData](#) () const
Returns the contents of the table filled by the user.
- void [setInitialData](#) (const QVector< QVector< QString > > &data)
Pre-fills the table with existing user data.
- void [highlightIncorrectCells](#) (const QList< QPair< int, int > > &coords)
Highlights cells that are incorrect based on provided coordinates.
- void [setGuidedModeActive](#) (bool active)
Enables or disables the dialog buttons while the guided mode dialog is open.
- void [setGuidedButtonVisible](#) (bool visible)
Shows or hides the guided mode button.

9.22.1 Detailed Description

Dialog for filling and submitting an LL(1) parsing table.

This class represents a dialog window that displays a table for users to complete the LL(1) parsing matrix. It provides functionality to initialize the table with data, retrieve the user's input, and highlight incorrect answers.

9.22.2 Constructor & Destructor Documentation

9.22.2.1 LLTableDialog()

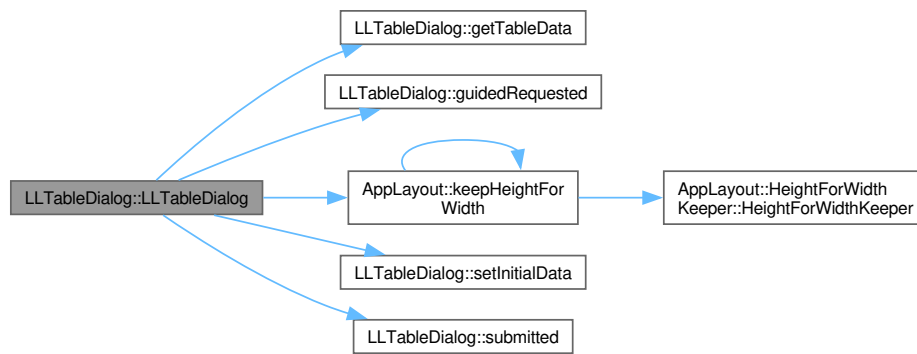
```
LLTableDialog::LLTableDialog (
    const QStringList & rowHeaders,
    const QStringList & colHeaders,
    QWidget * parent,
    QVector< QVector< QString > > * initialData = nullptr)
```

Constructs the LL(1) table dialog with given headers and optional initial data.

Parameters

<i>rowHeaders</i>	Row labels (non-terminal symbols).
<i>colHeaders</i>	Column labels (terminal symbols).
<i>parent</i>	Parent widget.
<i>initialData</i>	Optional initial table data to pre-fill cells.

Here is the call graph for this function:



9.22.3 Member Function Documentation

9.22.3.1 getTableData()

```
QVector< QVector< QString > > LLTableDialog::getTableData () const
```

Returns the contents of the table filled by the user.

Returns

A 2D vector representing the LL(1) table.

Here is the caller graph for this function:



9.22.3.2 guidedRequested

```
void LLTableDialog::guidedRequested (
    const QVector< QVector< QString > > & data) [signal]
```

Signal emitted when the user asks for the guided mode.

Parameters

<i>data</i>	The current table data, restored when the wizard closes.
-------------	--

Here is the caller graph for this function:



9.22.3.3 highlightIncorrectCells()

```
void LLTableDialog::highlightIncorrectCells (
    const QList< QPair< int, int > > & coords)
```

Highlights cells that are incorrect based on provided coordinates.

Parameters

<i>coords</i>	A list of (row, column) pairs to highlight as incorrect.
---------------	--

9.22.3.4 setGuidedButtonVisible()

```
void LLTableDialog::setGuidedButtonVisible (
    bool visible)
```

Shows or hides the guided mode button.

Exam mode hides it: the guided walkthrough would reveal the answers.

Parameters

<i>visible</i>	Whether the button should be visible.
----------------	---------------------------------------

9.22.3.5 setGuidedModeActive()

```
void LLTableDialog::setGuidedModeActive (
    bool active)
```

Enables or disables the dialog buttons while the guided mode dialog is open.

Parameters

<i>active</i>	Whether the guided mode is currently active.
---------------	--

9.22.3.6 setInitialData()

```
void LLTableDialog::setInitialData (
    const QVector< QVector< QString > > & data)
```

Pre-fills the table with existing user data.

This is used to populate the table with a previous (possibly incorrect) answer when retrying a task or providing feedback.

Parameters

<i>data</i>	A 2D vector of strings representing the initial cell values.
-------------	--

Here is the caller graph for this function:

**9.22.3.7 submitted**

```
void LLTableDialog::submitted (
    const QVector< QVector< QString > > & data) [signal]
```

Signal emitted when the user submits the table.

Parameters

<i>data</i>	The filled table data submitted by the user.
-------------	--

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

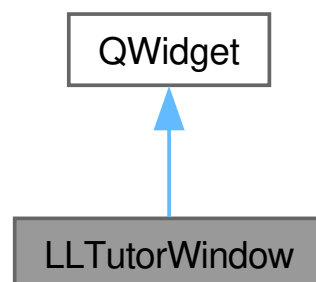
- [gui/lltabledialog.h](#)
- [gui/lltabledialog.cpp](#)

9.23 LLTutorWindow Class Reference

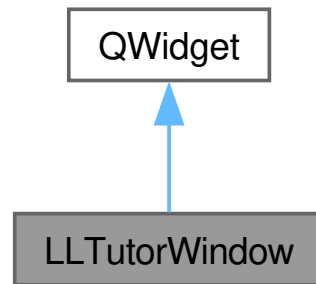
Main window for the LL(1) interactive tutoring mode in [SyntaxTutor](#).

```
#include <lltutorwindow.h>
```

Inheritance diagram for LLTutorWindow:



Collaboration diagram for LLTutorWindow:



Classes

- struct [TreeNode](#)
[TreeNode](#) structure used to build derivation trees.

Signals

- void [sessionFinished](#) (int cntRight, int cntWrong)
- void [exitRequested](#) (bool applyResults, int cntRight, int cntWrong)

Public Member Functions

- void [applyTextScale](#) ()
Re-applies the current text size to this tutor.
- [LLTutorWindow](#) (const [Grammar](#) &grammar, [TutorialManager](#) *tm=nullptr, QWidget *parent=nullptr, bool examMode=false)
Constructs the LL(1) tutor window with a given grammar.
- [~LLTutorWindow](#) ()
- QString [generateQuestion](#) ()
Generates a question for the current state of the tutor.
- void [updateState](#) (bool isCorrect)
Updates the tutor state after verifying user response.
- QString [FormatGrammar](#) (const [Grammar](#) &grammar)
Formats a grammar for display in the chat interface.
- QVector< [GrammarView::Row](#) > [buildGrammarRows](#) (const [Grammar](#) &grammar) const
- void [addMessage](#) (const QString &text, bool isUser)
- void [addGrammarMessage](#) ()
< Add text message to chat
- void [addWidgetMessage](#) (QWidget *widget)
- void [exportConversationToPdf](#) (const QString &filePath)
< Add widget (e.g., table, tree)
- void [showTable](#) ()
< Export chat to PDF
- void [showTableForCPrime](#) ()
Display the full LL(1) table in C' ex.
- void [updateProgressPanel](#) ()
- void [animateLabelPop](#) (QLabel *label)
- void [animateLabelColor](#) (QLabel *label, const QColor &flashColor)

- void [wrongAnimation](#) ()
Visual shake/flash for incorrect answer.
- void [wrongUserResponseAnimation](#) ()
Animation specific to user chat input.
- void [markLastUserIncorrect](#) ()
Marks last message as incorrect.
- void [TeachFirstTree](#) (const std::vector< std::string > &symbols, std::unordered_set< std::string > &first_set, int depth, std::unordered_set< std::string > &processing, QTreeWidgetItem *parent)
- std::unique_ptr< [TreeNode](#) > [buildTreeNode](#) (const std::vector< std::string > &symbols, std::unordered_set< std::string > &first_set, int depth, std::vector< std::pair< std::string, std::vector< std::string > > > &active_derivations)
- int [computeSubtreeWidth](#) (const std::unique_ptr< [TreeNode](#) > &node, int hSpacing)
- void [drawTree](#) (const std::unique_ptr< [TreeNode](#) > &root, QGraphicsScene *scene, QPointF pos, int hSpacing, int vSpacing)
- void [showTreeGraphics](#) (std::unique_ptr< [TreeNode](#) > root)
- bool [verifyResponse](#) (const QString &userResponse)
- bool [verifyResponseForA](#) (const QString &userResponse)
- bool [verifyResponseForA1](#) (const QString &userResponse)
- bool [verifyResponseForA2](#) (const QString &userResponse)
- bool [verifyResponseForB](#) (const QString &userResponse)
- bool [verifyResponseForB1](#) (const QString &userResponse)
- bool [verifyResponseForB2](#) (const QString &userResponse)
- bool [verifyResponseForC](#) ()
- QString [solution](#) (const std::string &state)
- QStringList [solutionForA](#) ()
- QString [solutionForA1](#) ()
- QString [solutionForA2](#) ()
- QSet< QString > [solutionForB](#) ()
- QSet< QString > [solutionForB1](#) ()
- QSet< QString > [solutionForB2](#) ()
- QString [feedback](#) ()
- QString [feedbackForA](#) ()
- QString [feedbackForA1](#) ()
- QString [feedbackForA2](#) ()
- QString [feedbackForAPrime](#) ()
- QString [feedbackForB](#) ()
- QString [feedbackForB1](#) ()
- QString [feedbackForB2](#) ()
- QString [feedbackForBPrime](#) ()
- QString [feedbackForC](#) ()
- QString [feedbackForCPrime](#) ()
- void [feedbackForB1TreeWidget](#) ()
- void [feedbackForB1TreeGraphics](#) ()
- QString [TeachFollow](#) (const QString &nt)
- QString [TeachPredictionSymbols](#) (const QString &ant, const [production](#) &conseq)
- QString [TeachLL1Table](#) ()
- void [handleTableSubmission](#) (const QVector< QVector< QString > > &raw, const QStringList &colHeaders)
- void [connectGuidedMode](#) ([LLTableDialog](#) *dialog, const QStringList &colHeaders)
Hooks the table dialog's guided mode button to the LL wizard.
- void [updatePlaceholder](#) ()
- bool [confirmExitToHome](#) ()
- QString [promptExportFilePath](#) () const
- void [postQuestion](#) ()
Shows and remembers the next question.

- QString [examSolutionText](#) ()
Expected answer for the current state.
- void [scoreExamTable](#) ()
Grades the LL table cell by cell.
- void [showExamReport](#) ()
Opens the end-of-exam report dialog.

Protected Member Functions

- void [closeEvent](#) (QCloseEvent *event) override
- bool [eventFilter](#) (QObject *obj, QEvent *event) override

9.23.1 Detailed Description

Main window for the LL(1) interactive tutoring mode in [SyntaxTutor](#).

This class guides students through the construction and analysis of LL(1) parsing tables. It uses a finite-state sequence to present progressively more complex tasks, verifies user responses, provides corrective feedback, and supports visualizations like derivation trees.

The tutor is designed to teach the student *how* the LL(1) table is built, not just test it — including interactive tasks, animated feedback, and hints.

Key features include:

- Interactive question flow based on grammar analysis.
- Derivation tree generation ([TeachFirst](#)).
- Step-by-step verification of FIRST, FOLLOW, prediction symbols, and table entries.
- Exportable conversation log for grading or review.

9.23.2 Constructor & Destructor Documentation

9.23.2.1 LLTutorWindow()

```
LLTutorWindow::LLTutorWindow (
    const Grammar & grammar,
    TutorialManager * tm = nullptr,
    QWidget * parent = nullptr,
    bool examMode = false) [explicit]
```

Constructs the LL(1) tutor window with a given grammar.

Parameters

<i>grammar</i>	The grammar to use during the session.
<i>tm</i>	Optional pointer to the tutorial manager (for help overlays).

9.23.3 Member Function Documentation

9.23.3.1 addGrammarMessage()

```
void LLTutorWindow::addGrammarMessage ()
```

< Add text message to chat

Here is the caller graph for this function:



9.23.3.2 addMessage()

```
void LLTutorWindow::addMessage (
    const QString & text,
    bool isUser)
```

Here is the caller graph for this function:



9.23.3.3 addWidgetMessage()

```
void LLTutorWindow::addWidgetMessage (
    QWidget * widget)
```

Here is the caller graph for this function:



9.23.3.4 animateLabelColor()

```
void LLTutorWindow::animateLabelColor (
    QLabel * label,
    const QColor & flashColor)
```

9.23.3.5 animateLabelPop()

```
void LLTutorWindow::animateLabelPop (
    QLabel * label)
```

9.23.3.6 applyTextScale()

```
void LLTutorWindow::applyTextScale ()
```

Re-applies the current text size to this tutor.

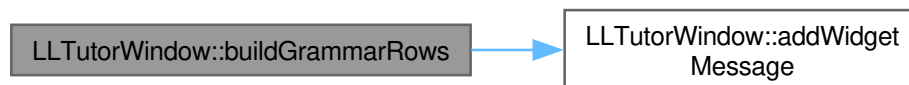
The chat bubbles size themselves from font metrics and the grammar panel from the card's width, so both have to be recomputed when the user picks a different text size. Here is the call graph for this function:



9.23.3.7 buildGrammarRows()

```
QVector< GrammarView::Row > LLTutorWindow::buildGrammarRows (
    const Grammar & grammar) const
```

Here is the call graph for this function:



Here is the caller graph for this function:



9.23.3.8 buildTreeNode()

```
std::unique_ptr< LLTutorWindow::TreeNode > LLTutorWindow::buildTreeNode (
    const std::vector< std::string > & symbols,
    std::unordered_set< std::string > & first_set,
    int depth,
    std::vector< std::pair< std::string, std::vector< std::string > > > & active_↔
    derivations)
```

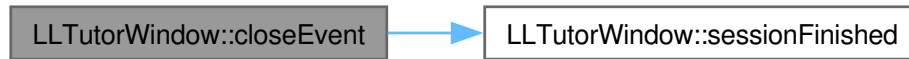
Here is the caller graph for this function:



9.23.3.9 closeEvent()

```
void LLTutorWindow::closeEvent (
    QCloseEvent * event) [inline], [override], [protected]
```

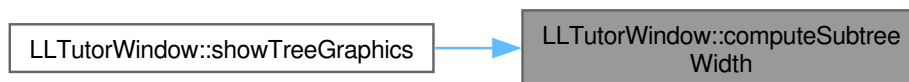
Here is the call graph for this function:



9.23.3.10 computeSubtreeWidth()

```
int LLTutorWindow::computeSubtreeWidth (
    const std::unique_ptr< TreeNode > & node,
    int hSpacing)
```

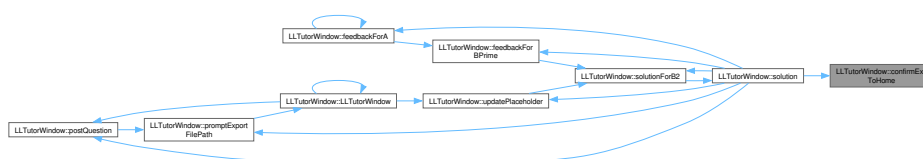
Here is the caller graph for this function:



9.23.3.11 confirmExitToHome()

```
bool LLTutorWindow::confirmExitToHome ()
```

Here is the caller graph for this function:



9.23.3.12 connectGuidedMode()

```
void LLTutorWindow::connectGuidedMode (
    LLTableDialog * dialog,
    const QStringList & colHeaders)
```

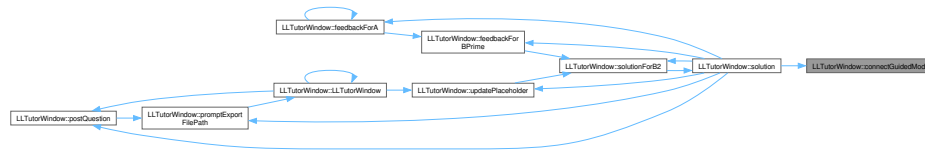
Hooks the table dialog's guided mode button to the LL wizard.

Opens the step-by-step LL(1) table assistant when requested, freezing the table dialog while it is active and restoring the user's snapshot when it closes.

Parameters

<i>dialog</i>	Table dialog to connect.
<i>colHeaders</i>	Terminal symbols in the dialog's column order.

Here is the caller graph for this function:

**9.23.3.13 drawTree()**

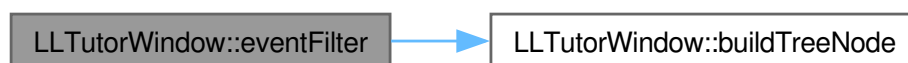
```
void LLTutorWindow::drawTree (
    const std::unique_ptr< TreeNode > & root,
    QGraphicsScene * scene,
    QPointF pos,
    int hSpacing,
    int vSpacing)
```

Here is the caller graph for this function:

**9.23.3.14 eventFilter()**

```
bool LLTutorWindow::eventFilter (
    QObject * obj,
    QEvent * event) [override], [protected]
```

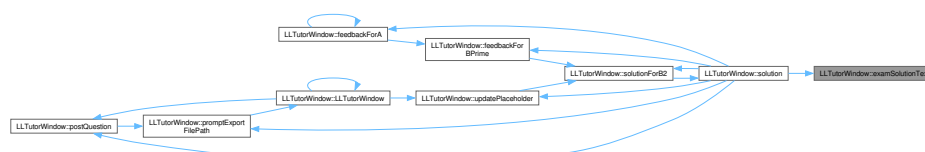
Here is the call graph for this function:

**9.23.3.15 examSolutionText()**

```
QString LLTutorWindow::examSolutionText ()
```

Expected answer for the current state.

Here is the caller graph for this function:



9.23.3.16 exitRequested

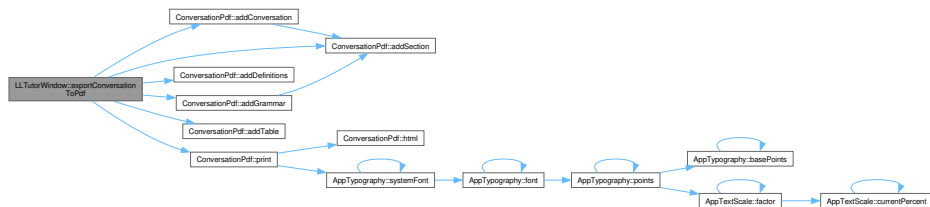
```
void LLTutorWindow::exitRequested (
    bool applyResults,
    int cntRight,
    int cntWrong) [signal]
```

9.23.3.17 exportConversationToPdf()

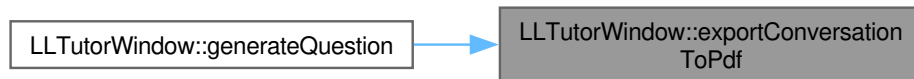
```
void LLTutorWindow::exportConversationToPdf (
    const QString & filePath)
```

< Add widget (e.g., table, tree)

Here is the call graph for this function:



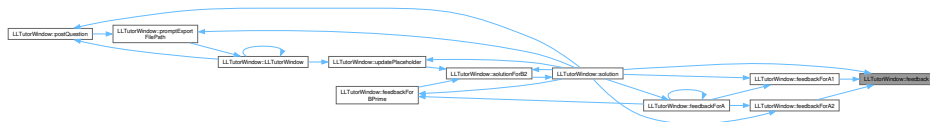
Here is the caller graph for this function:



9.23.3.18 feedback()

```
QString LLTutorWindow::feedback ()
```

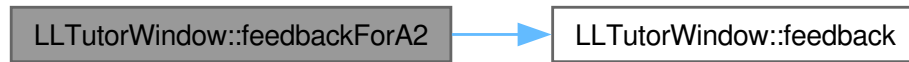
Here is the caller graph for this function:



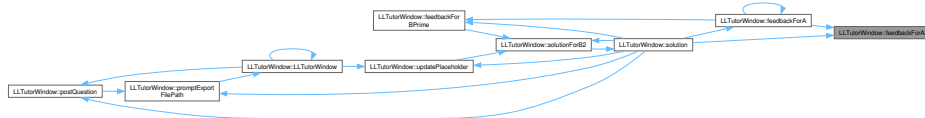
9.23.3.21 feedbackForA2()

QString LLTutorWindow::feedbackForA2 ()

Here is the call graph for this function:



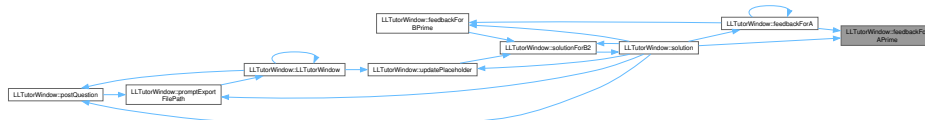
Here is the caller graph for this function:



9.23.3.22 feedbackForAPrime()

QString LLTutorWindow::feedbackForAPrime ()

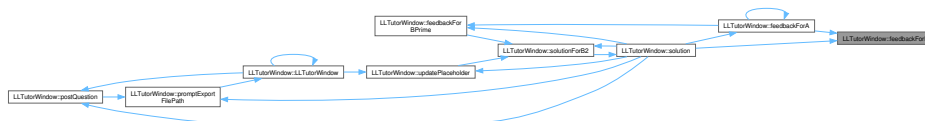
Here is the caller graph for this function:



9.23.3.23 feedbackForB()

QString LLTutorWindow::feedbackForB ()

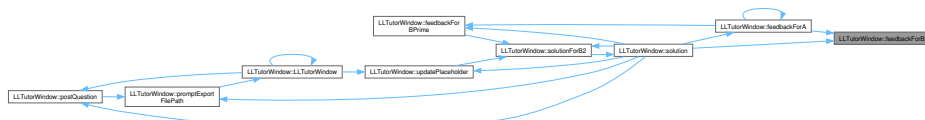
Here is the caller graph for this function:



9.23.3.24 feedbackForB1()

QString LLTutorWindow::feedbackForB1 ()

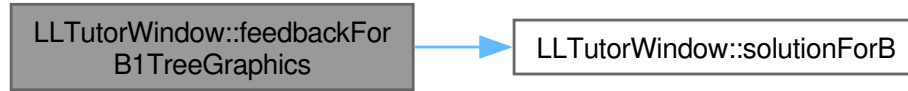
Here is the caller graph for this function:



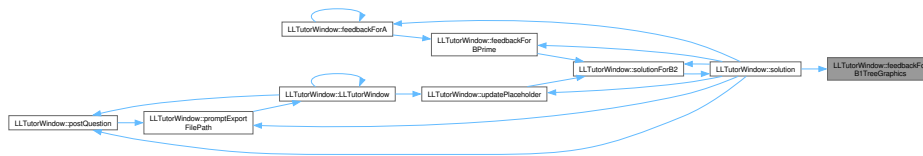
9.23.3.25 feedbackForB1TreeGraphics()

```
void LLTutorWindow::feedbackForB1TreeGraphics ()
```

Here is the call graph for this function:



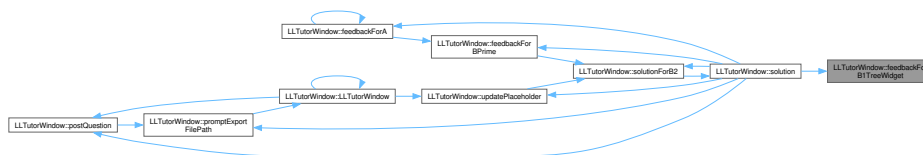
Here is the caller graph for this function:



9.23.3.26 feedbackForB1TreeWidget()

```
void LLTutorWindow::feedbackForB1TreeWidget ()
```

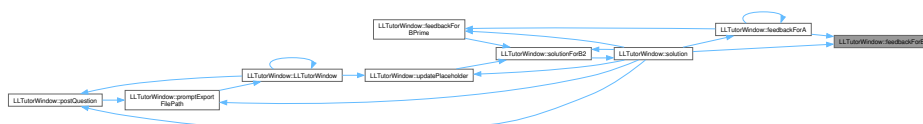
Here is the caller graph for this function:



9.23.3.27 feedbackForB2()

```
QString LLTutorWindow::feedbackForB2 ()
```

Here is the caller graph for this function:



```
QString LLTutorWindow::feedbackForBPrime ()
```

[illegible]

```

graph LR
    A[LLTutorWindow::poseQuestion] --> B[LLTutorWindow::promptExportFilePath]
    B --> C[LLTutorWindow::LLTutorWindow]
    C --> D[LLTutorWindow::updatePuzzleHolder]
    D --> E[LLTutorWindow::solutionForEq2]
    E --> F[LLTutorWindow::solution]
    F --> G[LLTutorWindow::feedbackForA]
    G --> H[LLTutorWindow::feedbackForBPhone]
    H --> A
    H --> B
    H --> C
    H --> D
    H --> E
    H --> F
    H --> G
    H --> H
  
```

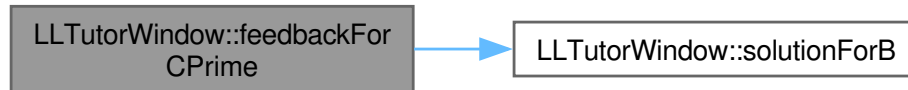
```
QString LLTutorWindow::feedbackForC ()
```

```
LLTutorWindow::feedbackForC → LLTutorWindow::TeachPrediction Symbols
```

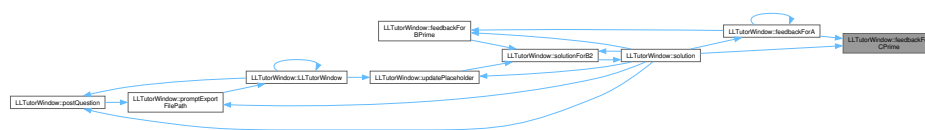
9.23.3.30 feedbackForCPrime()

```
QString LLTutorWindow::feedbackForCPrime ()
```

Here is the call graph for this function:



Here is the caller graph for this function:



9.23.3.31 FormatGrammar()

```
QString LLTutorWindow::FormatGrammar (
    const Grammar & grammar)
```

Formats a grammar for display in the chat interface.

Parameters

<i>grammar</i>	The grammar to format.
----------------	------------------------

Returns

A QString representation.

Here is the caller graph for this function:



9.23.3.32 generateQuestion()

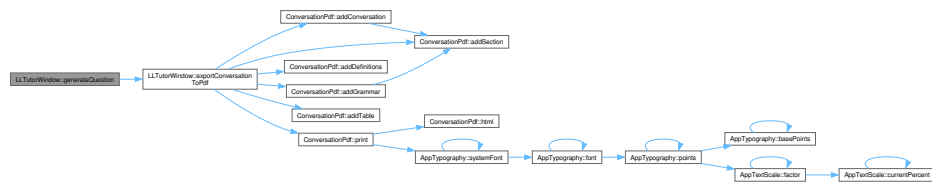
```
QString LLTutorWindow::generateQuestion ()
```

Generates a question for the current state of the tutor.

Returns

A formatted question string.

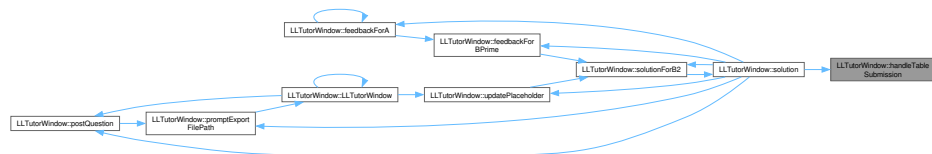
Here is the call graph for this function:



9.23.3.33 handleTableSubmission()

```
void LLTutorWindow::handleTableSubmission (
    const QVector< QVector< QString > > & raw,
    const QStringList & colHeaders)
```

Here is the caller graph for this function:



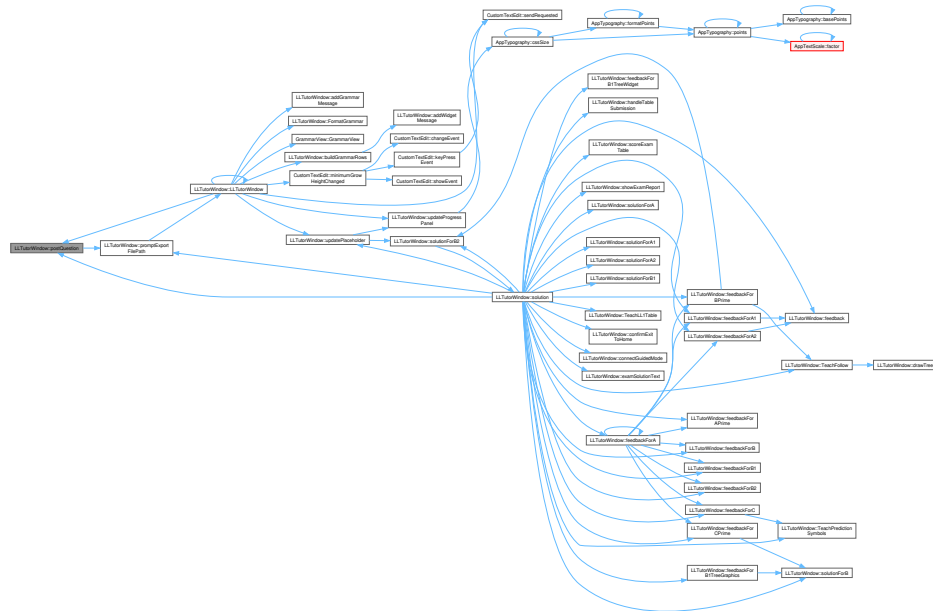
9.23.3.34 markLastUserIncorrect()

```
void LLTutorWindow::markLastUserIncorrect ()
```

Marks last message as incorrect.

```
void LLTutorWindow::postQuestion ()
```

Here is the call graph for this function:

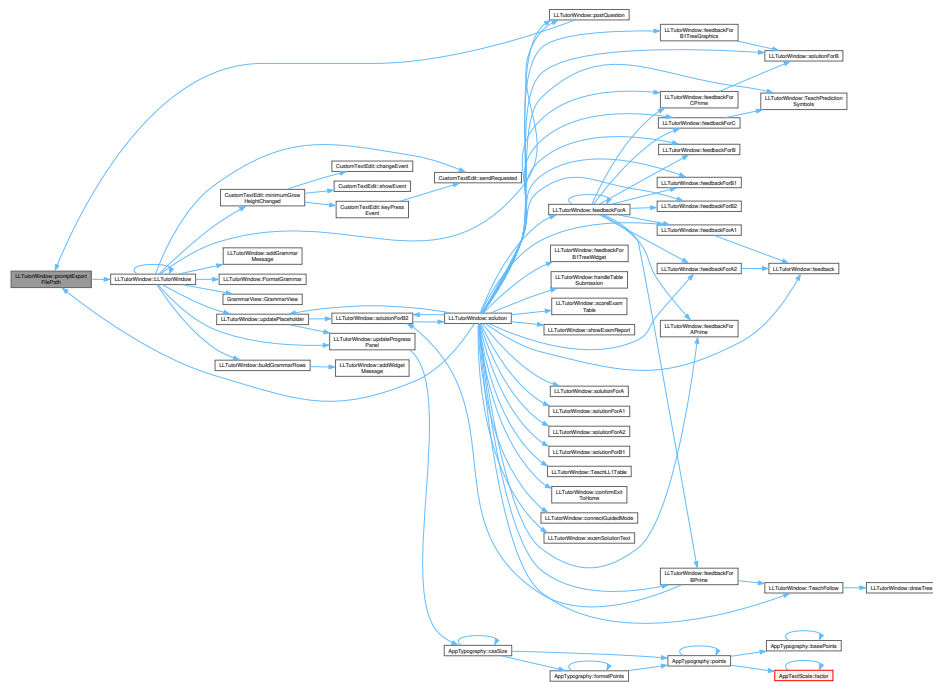


```
graph LR
    A[LLTutorWindow.feedbackForA] --> B[LLTutorWindow.feedbackForBPrime]
```

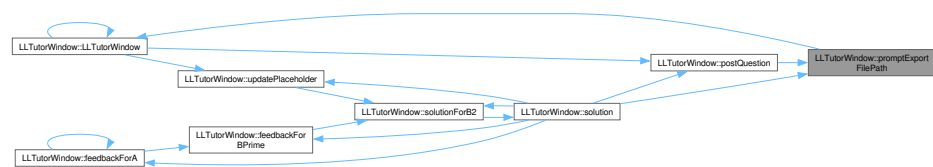


```
QString LLTutorWindow::promptExportFilePath () const
```


Here is the call graph for this function:



Here is the caller graph for this function:

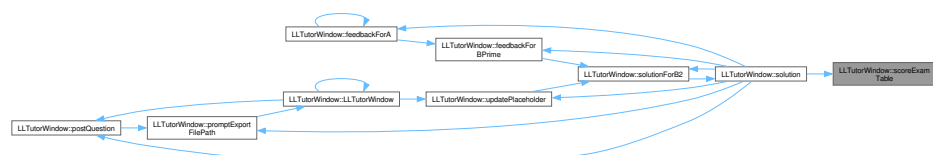


9.23.3.37 scoreExamTable()

```
void LLTutorWindow::scoreExamTable ()
```

Grades the LL table cell by cell.

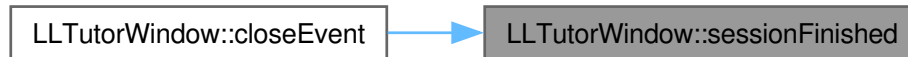
Here is the caller graph for this function:



9.23.3.38 sessionFinished

```
void LLTutorWindow::sessionFinished (
    int cntRight,
    int cntWrong) [signal]
```

Here is the caller graph for this function:

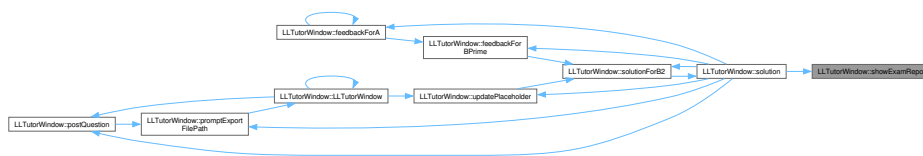


9.23.3.39 showExamReport()

```
void LLTutorWindow::showExamReport ()
```

Opens the end-of-exam report dialog.

Here is the caller graph for this function:



9.23.3.40 showTable()

```
void LLTutorWindow::showTable ()
```

< Export chat to PDF

Display the full LL(1) table in C ex. Here is the caller graph for this function:

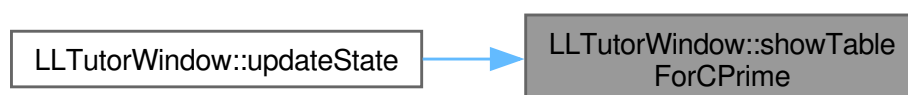


9.23.3.41 showTableForCPrime()

```
void LLTutorWindow::showTableForCPrime ()
```

Display the full LL(1) table in C' ex.

Here is the caller graph for this function:



```
void LLTutorWindow::showTreeGraphics (
    std::unique_ptr< TreeNode > root)
```

```
LLTutorWindow::showTreeGraphics → LLTutorWindow::computeSubtreeWidth
```

```
QString LLTutorWindow::solution (
    const std::string & state)
```

[illegible]

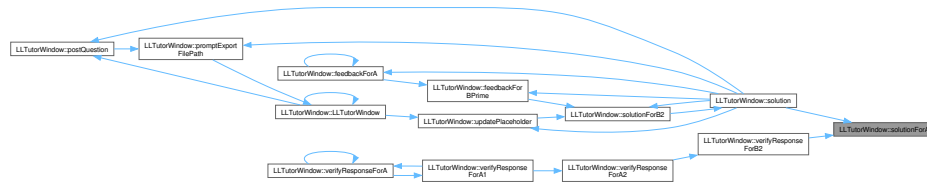
```

graph LR
    A[LLTutorWindow::postQuestion] --> B[LLTutorWindow::promptExportFilePath]
    B --> C[LLTutorWindow::LLTutorWindow]
    C --> D[LLTutorWindow::updatePlaceholder]
    D --> E[LLTutorWindow::solutionForBPrime]
    E --> F[LLTutorWindow::solutionForB2]
    F --> G[LLTutorWindow::solution]
    G --> A
    G --> B
    G --> C
    G --> D
    G --> E
    G --> F
    E --> E
    F --> F
    G --> G
  
```

9.23.3.44 solutionForA()

```
QStringList LLTutorWindow::solutionForA ()
```

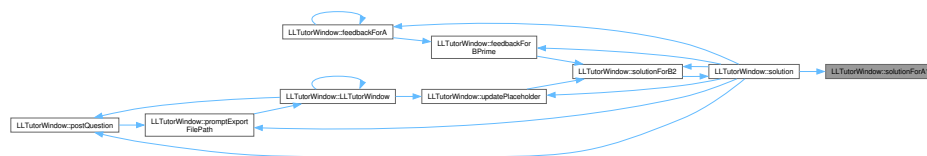
Here is the caller graph for this function:



9.23.3.45 solutionForA1()

```
QString LLTutorWindow::solutionForA1 ()
```

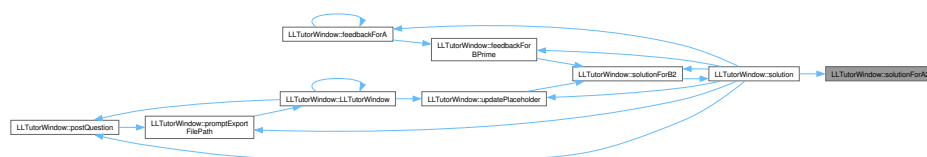
Here is the caller graph for this function:



9.23.3.46 solutionForA2()

```
QString LLTutorWindow::solutionForA2 ()
```

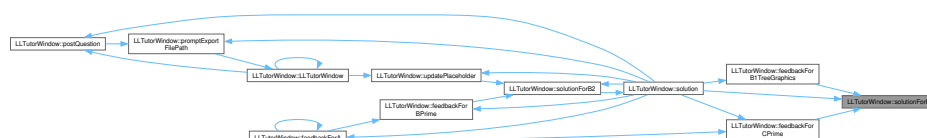
Here is the caller graph for this function:



9.23.3.47 solutionForB()

```
QSet< QString > LLTutorWindow::solutionForB ()
```

Here is the caller graph for this function:



```
QSet< QString > LLTutorWindow::solutionForB1 ( )
```

```
QSet< QString > LLTutorWindow::solutionForB2 ( )
```

[illegible]

9.23.3.50 TeachFirstTree()

```
void LLTutorWindow::TeachFirstTree (
    const std::vector< std::string > & symbols,
    std::unordered_set< std::string > & first_set,
    int depth,
    std::unordered_set< std::string > & processing,
    QTreeWidgetItem * parent)
```

9.23.3.51 TeachFollow()

```
QString LLTutorWindow::TeachFollow (
    const QString & nt)
```

Here is the call graph for this function:



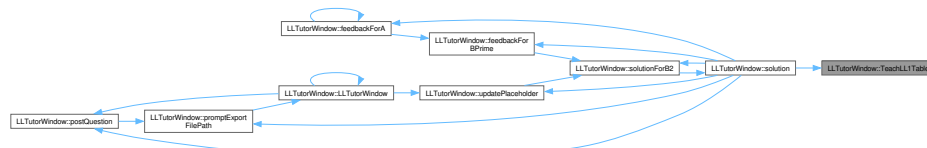
Here is the caller graph for this function:



9.23.3.52 TeachLL1Table()

```
QString LLTutorWindow::TeachLL1Table ()
```

Here is the caller graph for this function:



9.23.3.53 TeachPredictionSymbols()

```
QString LLTutorWindow::TeachPredictionSymbols (
    const QString & ant,
    const production & conseq)
```

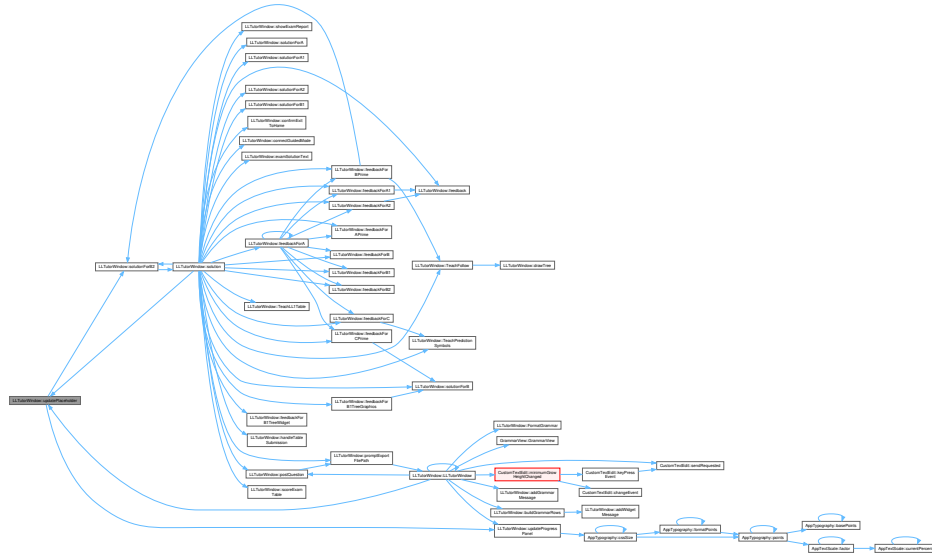
Here is the caller graph for this function:



9.23.3.54 updatePlaceholder()

```
void LLTutorWindow::updatePlaceholder ()
```

Here is the call graph for this function:



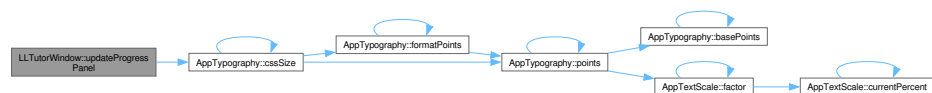
Here is the caller graph for this function:



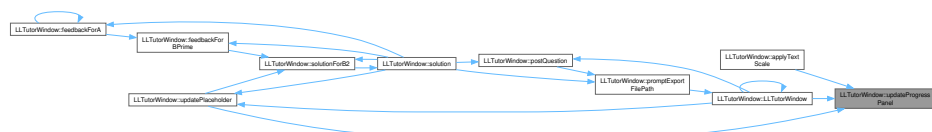
9.23.3.55 updateProgressPanel()

```
void LLTutorWindow::updateProgressPanel ()
```

Here is the call graph for this function:



Here is the caller graph for this function:



9.23.3.56 updateState()

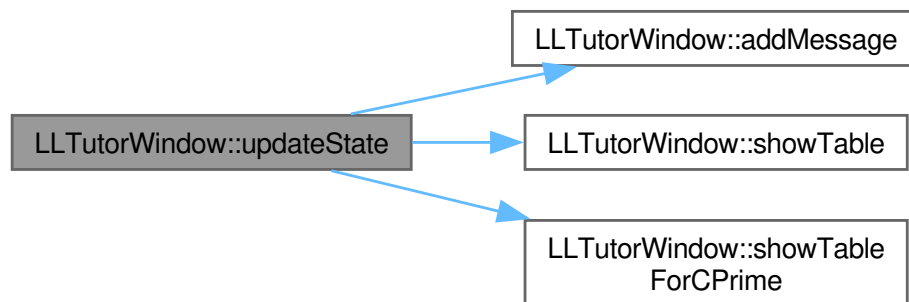
```
void LLTutorWindow::updateState (
    bool isCorrect)
```

Updates the tutor state after verifying user response.

Parameters

<i>isCorrect</i>	Whether the user answered correctly.
------------------	--------------------------------------

Here is the call graph for this function:



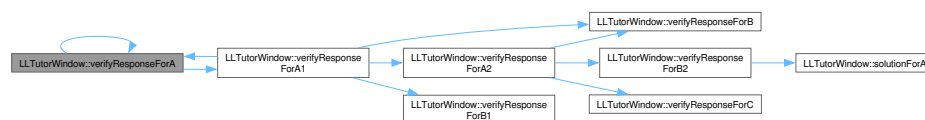
9.23.3.57 verifyResponse()

```
bool LLTutorWindow::verifyResponse (
    const QString & userResponse)
```

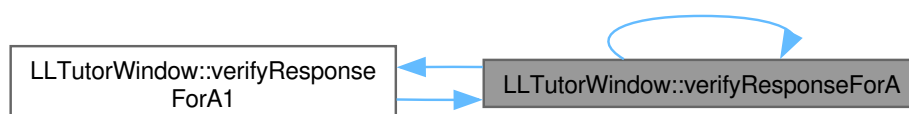
9.23.3.58 verifyResponseForA()

```
bool LLTutorWindow::verifyResponseForA (
    const QString & userResponse)
```

Here is the call graph for this function:



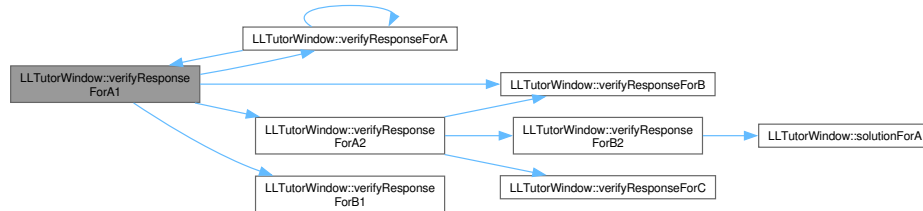
Here is the caller graph for this function:



9.23.3.59 verifyResponseForA1()

```
bool LLTutorWindow::verifyResponseForA1 (
    const QString & userResponse)
```

Here is the call graph for this function:



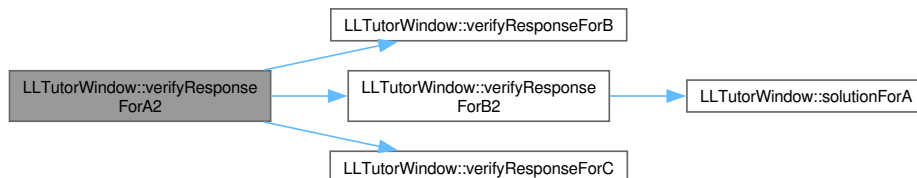
Here is the caller graph for this function:



9.23.3.60 verifyResponseForA2()

```
bool LLTutorWindow::verifyResponseForA2 (
    const QString & userResponse)
```

Here is the call graph for this function:



Here is the caller graph for this function:



9.23.3.61 verifyResponseForB()

```
bool LLTutorWindow::verifyResponseForB (
    const QString & userResponse)
```

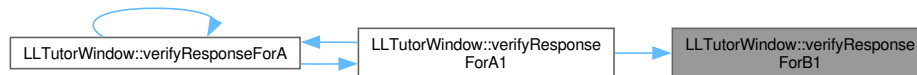
Here is the caller graph for this function:



9.23.3.62 verifyResponseForB1()

```
bool LLTutorWindow::verifyResponseForB1 (
    const QString & userResponse)
```

Here is the caller graph for this function:



9.23.3.63 verifyResponseForB2()

```
bool LLTutorWindow::verifyResponseForB2 (
    const QString & userResponse)
```

Here is the call graph for this function:



Here is the caller graph for this function:



9.23.3.64 verifyResponseForC()

```
bool LLTutorWindow::verifyResponseForC ()
```

Here is the caller graph for this function:



9.23.3.65 wrongAnimation()

```
void LLTutorWindow::wrongAnimation ()
```

Visual shake/flash for incorrect answer.

9.23.3.66 wrongUserResponseAnimation()

```
void LLTutorWindow::wrongUserResponseAnimation ()
```

Animation specific to user chat input.

The documentation for this class was generated from the following files:

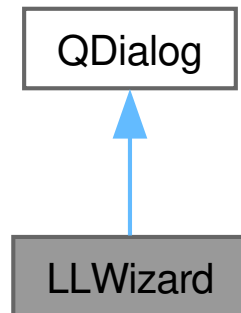
- [gui/lltutorwindow.h](#)
- [gui/lltutorwindow.cpp](#)

9.24 LLWizard Class Reference

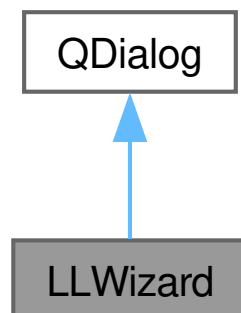
Interactive assistant that guides the student step-by-step through the LL(1) parsing table.

```
#include <llwizard.h>
```

Inheritance diagram for LLWizard:



Collaboration diagram for LLWizard:



Public Member Functions

- [LLWizard](#) ([LL1Parser](#) &parser, const QStringList &rowHeaders, const QStringList &colHeaders, QWidget *parent=nullptr)
Constructs the LL(1) guided dialog from the computed table.
- [LLWizardPage](#) * [currentPage](#) () const
- QPushButton * [nextButton](#) () const
- QPushButton * [closeButton](#) () const
- bool [isOnLastPage](#) () const

9.24.1 Detailed Description

Interactive assistant that guides the student step-by-step through the LL(1) parsing table.

This dialog presents one non-empty cell of the LL(1) table at a time, asking the user to deduce the production that belongs in it from the prediction symbols (símbolos directores): the cell (A, b) holds the production $A \rightarrow \alpha$ with $b \in SD(A \rightarrow \alpha)$. It mirrors the SLR(1) guided dialog and intentionally avoids the native wizard widget to keep a consistent look across platforms.

9.24.2 Constructor & Destructor Documentation

9.24.2.1 LLWizard()

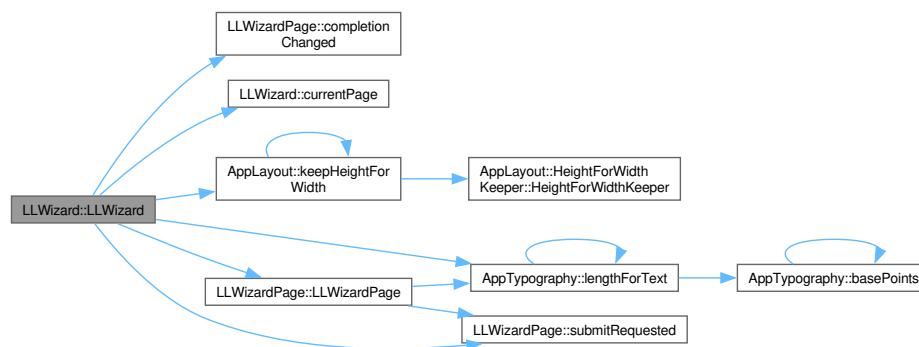
```
LLWizard::LLWizard (
    LL1Parser & parser,
    const QStringList & rowHeaders,
    const QStringList & colHeaders,
    QWidget * parent = nullptr) [inline]
```

Constructs the LL(1) guided dialog from the computed table.

Parameters

<i>parser</i>	The LL(1) parser instance holding the reference table.
<i>rowHeaders</i>	Non-terminal symbols in display order.
<i>colHeaders</i>	Terminal symbols in display order.
<i>parent</i>	Parent widget.

Here is the call graph for this function:



9.24.3 Member Function Documentation

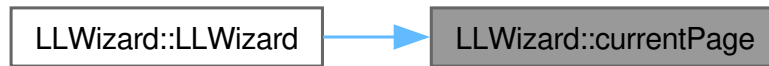
9.24.3.1 closeButton()

```
QPushButton * LLWizard::closeButton () const [inline]
```

9.24.3.2 currentPage()

```
LLWizardPage * LLWizard::currentPage () const [inline]
```

Here is the caller graph for this function:



9.24.3.3 isOnLastPage()

```
bool LLWizard::isOnLastPage () const [inline]
```

9.24.3.4 nextButton()

```
QPushButton * LLWizard::nextButton () const [inline]
```

The documentation for this class was generated from the following file:

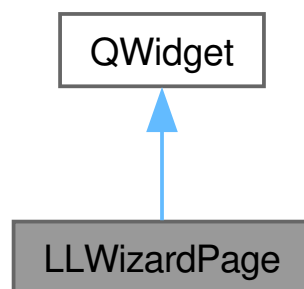
- [gui/llwizard.h](#)

9.25 LLWizardPage Class Reference

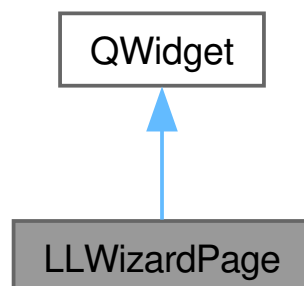
A single guided step for constructing the LL(1) table.

```
#include <llwizardpage.h>
```

Inheritance diagram for LLWizardPage:



Collaboration diagram for LLWizardPage:



Signals

- void [completionChanged](#) (bool complete)
- void [submitRequested](#) ()

Public Member Functions

- [LLWizardPage](#) (const QString &nonTerminal, const QString &symbol, const QString &explanation, const QString &expected, QWidget *parent=nullptr)
Constructs a page for a specific cell in the LL(1) table.
- QString [titleText](#) () const
- bool [isComplete](#) () const
- void [focusAnswerField](#) ()

Protected Member Functions

- void [resizeEvent](#) (QResizeEvent *event) override

9.25.1 Detailed Description

A single guided step for constructing the LL(1) table.

This widget presents one (non-terminal, terminal) cell at a time with a compact explanation card and an answer card. The expected answer is the production that belongs in the cell; the input is validated ignoring whitespace so "a B", "a B" and "aB" are all accepted. It emits a signal whenever the completion state changes.

9.25.2 Constructor & Destructor Documentation

9.25.2.1 LLWizardPage()

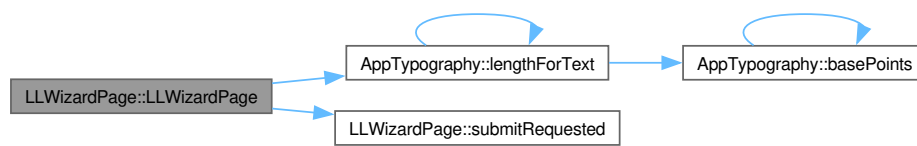
```
LLWizardPage::LLWizardPage (
    const QString & nonTerminal,
    const QString & symbol,
    const QString & explanation,
    const QString & expected,
    QWidget * parent = nullptr) [inline]
```

Constructs a page for a specific cell in the LL(1) table.

Parameters

<i>nonTerminal</i>	The non-terminal symbol (row header).
<i>symbol</i>	The terminal symbol (column header).
<i>explanation</i>	A pedagogical explanation shown to the user.
<i>expected</i>	The expected production, space-separated (e.g. "a B" or "EPSILON").
<i>parent</i>	The parent widget.

Here is the call graph for this function:



Here is the caller graph for this function:



9.25.3 Member Function Documentation

9.25.3.1 completionChanged

```
void LLWizardPage::completionChanged (
    bool complete) [signal]
```

Here is the caller graph for this function:



9.25.3.2 focusAnswerField()

```
void LLWizardPage::focusAnswerField () [inline]
```

9.25.3.3 isComplete()

```
bool LLWizardPage::isComplete () const [inline]
```

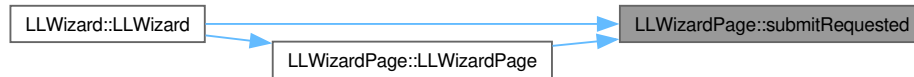
9.25.3.4 resizeEvent()

```
void LLWizardPage::resizeEvent (
    QResizeEvent * event) [inline], [override], [protected]
```

9.25.3.5 submitRequested

```
void LLWizardPage::submitRequested () [signal]
```

Here is the caller graph for this function:



9.25.3.6 titleText()

```
QString LLWizardPage::titleText () const [inline]
```

The documentation for this class was generated from the following file:

- [gui/llwizardpage.h](#)

9.26 Lr0Item Struct Reference

Represents an LR(0) item used in LR automata construction.

```
#include <lr0_item.hpp>
```

Public Member Functions

- [Lr0Item](#) (std::string antecedent, std::vector< std::string > consequent, std::string epsilon, std::string eol)
Constructs an LR(0) item with the dot at position 0.
- [Lr0Item](#) (std::string antecedent, std::vector< std::string > consequent, unsigned int dot, std::string epsilon, std::string eol)
Constructs an LR(0) item with a custom dot position.
- std::string [NextToDot](#) () const
Returns the symbol immediately after the dot, or empty if the dot is at the end.
- void [PrintItem](#) () const
Prints the LR(0) item to the standard output in a human-readable format.
- std::string [ToString](#) () const
Converts the item to a string representation, including the dot position.
- void [AdvanceDot](#) ()
Advances the dot one position to the right.
- bool [IsComplete](#) () const
Checks whether the dot has reached the end of the production.
- bool [operator==](#) (const [Lr0Item](#) &other) const
Equality operator for comparing two LR(0) items.

Public Attributes

- `std::string antecedent_`
The non-terminal on the left-hand side of the production.
- `std::vector< std::string > consequent_`
The sequence of symbols on the right-hand side of the production.
- `std::string epsilon_`
The symbol representing the empty string (ϵ).
- `std::string eol_`
The symbol representing end-of-line or end-of-input (\$).
- `unsigned int dot_ = 0`
The position of the dot (·) in the production.

9.26.1 Detailed Description

Represents an LR(0) item used in LR automata construction.

An LR(0) item has a production of the form $A \rightarrow \alpha \bullet \beta$, where the dot indicates the current parsing position.

This structure tracks the antecedent (left-hand side), consequent (right-hand side), the dot position, and special symbols like EPSILON and end-of-line (\$).

9.26.2 Constructor & Destructor Documentation

9.26.2.1 Lr0Item() [1/2]

```
Lr0Item::Lr0Item (
    std::string antecedent,
    std::vector< std::string > consequent,
    std::string epsilon,
    std::string eol)
```

Constructs an LR(0) item with the dot at position 0.

Parameters

<i>antecedent</i>	The left-hand side non-terminal.
<i>consequent</i>	The right-hand side of the production.
<i>epsilon</i>	The EPSILON symbol.
<i>eol</i>	The end-of-line symbol.

Here is the caller graph for this function:



9.26.2.2 Lr0Item() [2/2]

```
Lr0Item::Lr0Item (
    std::string antecedent,
    std::vector< std::string > consequent,
    unsigned int dot,
    std::string epsilon,
    std::string eol)
```

Constructs an LR(0) item with a custom dot position.

Parameters

<i>antecedent</i>	The left-hand side non-terminal.
<i>consequent</i>	The right-hand side of the production.
<i>dot</i>	The position of the dot.
<i>epsilon</i>	The EPSILON symbol.
<i>eol</i>	The end-of-line symbol.

9.26.3 Member Function Documentation

9.26.3.1 AdvanceDot()

```
void Lr0Item::AdvanceDot ()
```

Advances the dot one position to the right.

Here is the caller graph for this function:



9.26.3.2 IsComplete()

```
bool Lr0Item::IsComplete () const
```

Checks whether the dot has reached the end of the production.

Returns

true if the item is complete; false otherwise.

Here is the caller graph for this function:



9.26.3.3 NextToDot()

```
std::string Lr0Item::NextToDot () const
```

Returns the symbol immediately after the dot, or empty if the dot is at the end.

Returns

The symbol after the dot, or an empty string.

Here is the caller graph for this function:



9.26.3.4 operator==()

```
bool Lr0Item::operator== (
    const Lr0Item & other) const
```

Equality operator for comparing two LR(0) items.

Parameters

<i>other</i>	The item to compare with.
--------------	---------------------------

Returns

true if both items are equal; false otherwise.

Here is the call graph for this function:



9.26.3.5 PrintItem()

```
void Lr0Item::PrintItem () const
```

Prints the LR(0) item to the standard output in a human-readable format.

9.26.3.6 ToString()

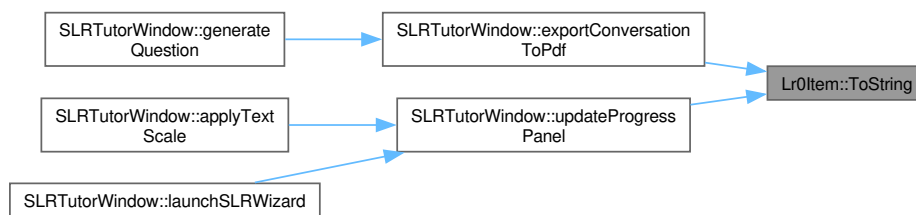
```
std::string Lr0Item::ToString () const
```

Converts the item to a string representation, including the dot position.

Returns

A string representation of the item.

Here is the caller graph for this function:



9.26.4 Member Data Documentation

9.26.4.1 antecedent_

```
std::string Lr0Item::antecedent_
```

The non-terminal on the left-hand side of the production.

9.26.4.2 consequent_

```
std::vector<std::string> Lr0Item::consequent_
```

The sequence of symbols on the right-hand side of the production.

9.26.4.3 dot_

```
unsigned int Lr0Item::dot_ = 0
```

The position of the dot (·) in the production.

9.26.4.4 eol_

```
std::string Lr0Item::eol_
```

The symbol representing end-of-line or end-of-input (\$).

9.26.4.5 epsilon_

```
std::string Lr0Item::epsilon_
```

The symbol representing the empty string (ϵ).

The documentation for this struct was generated from the following files:

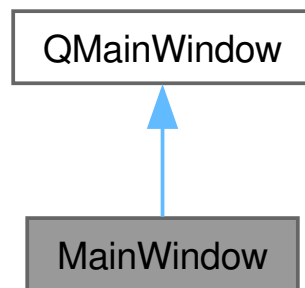
- [backend/lr0_item.hpp](#)
- [backend/lr0_item.cpp](#)

9.27 MainWindow Class Reference

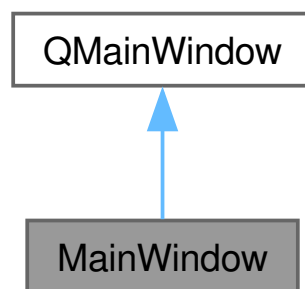
Main application window of [SyntaxTutor](#), managing levels, exercises, and UI state.

```
#include <mainwindow.h>
```

Inheritance diagram for MainWindow:



Collaboration diagram for MainWindow:



Signals

- void [userLevelChanged](#) (unsigned lvl)
Emitted when the user's level changes.
- void [userLevelUp](#) (unsigned newLevel)
Emitted when the user levels up.

Public Member Functions

- [MainWindow](#) (QWidget *parent=nullptr)
Constructs the main window.
- [~MainWindow](#) ()
Destructor.
- unsigned [thresholdFor](#) (unsigned level)
Returns the required score threshold to unlock a level.
- unsigned [userLevel](#) () const
Returns the current user level.
- void [setUserLevel](#) (unsigned lvl)
Sets the user level, clamping it to the allowed maximum.

Properties

- unsigned [userLevel](#)

9.27.1 Detailed Description

Main application window of [SyntaxTutor](#), managing levels, exercises, and UI state.

This class serves as the central hub of the application. It handles level selection, navigation to LL(1) and SLR(1) exercises, tutorial management, settings persistence, and emits signals for user progress. It also includes UI logic for dynamic behavior like unlocking levels and changing language.

9.27.2 Constructor & Destructor Documentation

9.27.2.1 MainWindow()

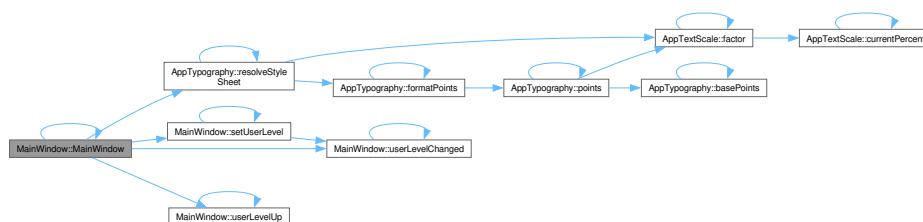
```
MainWindow::MainWindow (
    QWidget * parent = nullptr)
```

Constructs the main window.

Parameters

<i>parent</i>	Parent widget.
---------------	----------------

Here is the call graph for this function:



Here is the caller graph for this function:



9.27.2.2 ~MainWindow()

```
MainWindow::~MainWindow ()
```

Destructor.

9.27.3 Member Function Documentation

9.27.3.1 setUserLevel()

```
void MainWindow::setUserLevel (
    unsigned lvl) [inline]
```

Sets the user level, clamping it to the allowed maximum.

Parameters

<i>lvl</i>	New level to assign.
------------	----------------------

Here is the call graph for this function:



Here is the caller graph for this function:



9.27.3.2 thresholdFor()

```
unsigned MainWindow::thresholdFor (
    unsigned level) [inline]
```

Returns the required score threshold to unlock a level.

Parameters

<i>level</i>	The level number.
--------------	-------------------

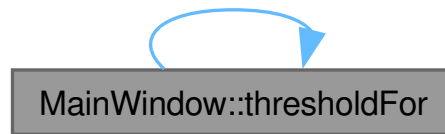
Returns

The score needed to unlock the given level.

Here is the call graph for this function:



Here is the caller graph for this function:

**9.27.3.3 userLevel()**

```
unsigned MainWindow::userLevel () const [inline]
```

Returns the current user level.

9.27.3.4 userLevelChanged

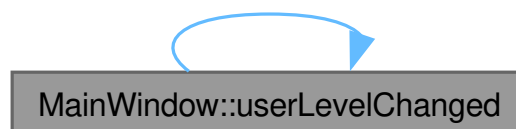
```
void MainWindow::userLevelChanged (
    unsigned lvl) [signal]
```

Emitted when the user's level changes.

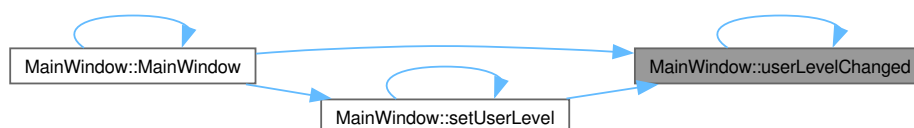
Parameters

<i>lvl</i>	New user level.
------------	-----------------

Here is the call graph for this function:



Here is the caller graph for this function:



9.27.3.5 userLevelUp

```
void MainWindow::userLevelUp (
    unsigned newLevel) [signal]
```

Emitted when the user levels up.

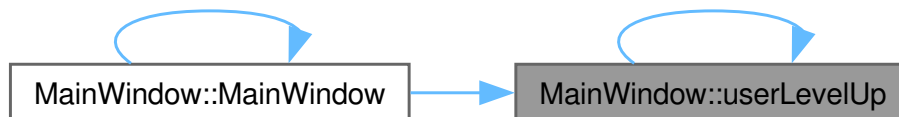
Parameters

<i>newLevel</i>	The new level achieved.
-----------------	-------------------------

Here is the call graph for this function:



Here is the caller graph for this function:



9.27.4 Property Documentation

9.27.4.1 userLevel

```
unsigned MainWindow::userLevel [read], [write]
```

The documentation for this class was generated from the following files:

- [gui/mainwindow.h](#)
- [gui/mainwindow.cpp](#)

9.28 ConversationPdf::Message Struct Reference

One message of the conversation.

```
#include <conversationpdf.h>
```

Public Attributes

- QString [text](#)
- bool [fromStudent](#) = false
- bool [correct](#) = true

9.28.1 Detailed Description

One message of the conversation.

9.28.2 Member Data Documentation

9.28.2.1 correct

```
bool ConversationPdf::Message::correct = true
```

9.28.2.2 fromStudent

```
bool ConversationPdf::Message::fromStudent = false
```

9.28.2.3 text

```
QString ConversationPdf::Message::text
```

The documentation for this struct was generated from the following file:

- [gui/conversationpdf.h](#)

9.29 GrammarView::Row Struct Reference

```
#include <grammarview.h>
```

Public Attributes

- QString [index](#)
- QString [lhs](#)
- QString [marker](#)
- QString [rhs](#)

9.29.1 Member Data Documentation

9.29.1.1 index

```
QString GrammarView::Row::index
```

9.29.1.2 lhs

```
QString GrammarView::Row::lhs
```

9.29.1.3 marker

```
QString GrammarView::Row::marker
```

9.29.1.4 rhs

```
QString GrammarView::Row::rhs
```

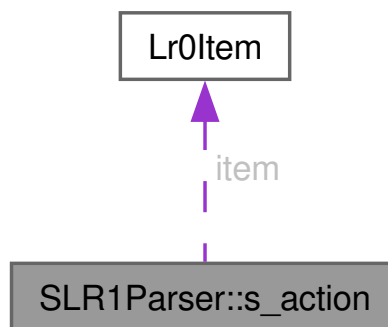
The documentation for this struct was generated from the following file:

- [widgets/grammarview.h](#)

9.30 SLR1Parser::s_action Struct Reference

```
#include <slr1_parser.hpp>
```

Collaboration diagram for SLR1Parser::s_action:



Public Attributes

- const [Lr0Item](#) * [item](#)
- [Action](#) [action](#)

9.30.1 Member Data Documentation

9.30.1.1 action

```
Action SLR1Parser::s_action::action
```

9.30.1.2 item

```
const Lr0Item* SLR1Parser::s_action::item
```

The documentation for this struct was generated from the following file:

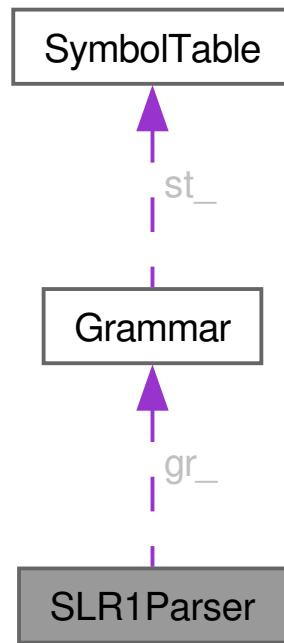
- [backend/slr1_parser.hpp](#)

9.31 SLR1Parser Class Reference

Implements an SLR(1) parser for context-free grammars.

```
#include <slr1_parser.hpp>
```

Collaboration diagram for SLR1Parser:



Classes

- struct [s_action](#)

Public Types

- enum class [Action](#) { [Shift](#) , [Reduce](#) , [Accept](#) , [Empty](#) }
Represents the possible actions in the SLR(1) parsing table.
- using [action_table](#)
Represents the action table for the SLR(1) parser.
- using [transition_table](#)
Represents the transition table for the SLR(1) parser.

Public Member Functions

- [SLR1Parser](#) ()=default
- [SLR1Parser](#) ([Grammar](#) gr)
- `std::unordered_set< Lr0Item > AllItems () const`
Retrieves all LR(0) items in the grammar. This function returns a set of all LR(0) items derived from the grammar's productions. Each LR(0) item represents a production with a marker indicating the current position in the production (e.g., $A \rightarrow \alpha \bullet \beta$).
- void [Closure](#) (std::unordered_set< [Lr0Item](#) > &items)

- Computes the closure of a set of LR(0) items.*
- void [ClosureUtil](#) (std::unordered_set< [Lr0Item](#) > &items, unsigned int size, std::unordered_set< std::string > &visited)
 - Helper function for computing the closure of LR(0) items.*
- std::unordered_set< [Lr0Item](#) > [Delta](#) (const std::unordered_set< [Lr0Item](#) > &items, const std::string &str)
 - Computes the GOTO transition (δ) for a given set of LR(0) items and a symbol. This function is equivalent to the $\delta(I, X)$ function in LR parsing, where it computes the set of items reached from a state I via symbol X .*
- bool [SolveLRConflicts](#) (const [state](#) &st)
 - Resolves LR conflicts in a given state.*
- void [First](#) (std::span< const std::string > rule, std::unordered_set< std::string > &result)
 - Calculates the FIRST set for a given production rule in a grammar.*
- void [ComputeFirstSets](#) ()
 - Computes the FIRST sets for all non-terminal symbols in the grammar.*
- void [ComputeFollowSets](#) ()
 - Computes the FOLLOW sets for all non-terminal symbols in the grammar. The FOLLOW set of a non-terminal symbol A contains all terminal symbols that can appear immediately after A in any sentential form derived from the grammar's start symbol. Additionally, if A can be the last symbol in a derivation, the end-of-input marker ($\$$) is included in its FOLLOW set. This function computes the FOLLOW sets using the following rules:*
- std::unordered_set< std::string > [Follow](#) (const std::string &arg)
 - Computes the FOLLOW set for a given non-terminal symbol in the grammar.*
- void [MakeInitialState](#) ()
 - Creates the initial state of the parser's state machine.*
- bool [MakeParser](#) ()
 - Constructs the SLR(1) parsing tables (action and transition tables).*
- std::string [PrintItems](#) (const std::unordered_set< [Lr0Item](#) > &items) const
 - Returns a string representation of a set of LR(0) items.*

Public Attributes

- [Grammar](#) [gr_](#)
 - The grammar being processed by the parser.*
- std::unordered_map< std::string, std::unordered_set< std::string > > [first_sets_](#)
 - Cached FIRST sets for all symbols in the grammar.*
- std::unordered_map< std::string, std::unordered_set< std::string > > [follow_sets_](#)
 - Cached FOLLOW sets for all non-terminal symbols in the grammar.*
- [action_table](#) [actions_](#)
 - The action table used by the parser to determine shift/reduce actions.*
- [transition_table](#) [transitions_](#)
 - The transition table used by the parser to determine state transitions.*
- std::unordered_set< [state](#) > [states_](#)
 - The set of states in the parser's state machine.*

9.31.1 Detailed Description

Implements an SLR(1) parser for context-free grammars.

This class builds an SLR(1) parsing table and LR(0) automaton from a given grammar. It provides methods for computing closure sets, GOTO transitions, constructing states, and performing syntax analysis using the generated table.

9.31.2 Member Typedef Documentation

9.31.2.1 `action_table`

using `SLR1Parser::action_table`

Initial value:

```
std::map<unsigned int, std::map<std::string, SLR1Parser::s_action>>
```

Represents the action table for the SLR(1) parser.

The action table is a map that associates each state and input symbol with a specific action (`Shift`, `Reduce`, `Accept`, or `Empty`). It is used to determine the parser's behavior during the parsing process.

The table is structured as:

- Outer map: Keys are state IDs (unsigned int).
- Inner map: Keys are input symbols (`std::string`), and values are `s_action` structs representing the action to take.

9.31.2.2 `transition_table`

using `SLR1Parser::transition_table`

Initial value:

```
std::map<unsigned int, std::map<std::string, unsigned int>>
```

Represents the transition table for the SLR(1) parser.

The transition table is a map that associates each state and symbol with the next state to transition to. It is used to guide the parser's state transitions during the parsing process.

The table is structured as:

- Outer map: Keys are state IDs (unsigned int).
- Inner map: Keys are symbols (`std::string`), and values are the next state IDs (unsigned int).

9.31.3 Member Enumeration Documentation

9.31.3.1 Action

```
enum class SLR1Parser::Action [strong]
```

Represents the possible actions in the SLR(1) parsing table.

This enumeration defines the types of actions that can be taken by the parser during the parsing process:

- **Shift**: Shift the input symbol onto the stack and transition to a new state.
- **Reduce**: Reduce a production rule and pop symbols from the stack.
- **Accept**: Accept the input as a valid string in the grammar.
- **Empty**: No action is defined for the current state and input symbol.

Enumerator

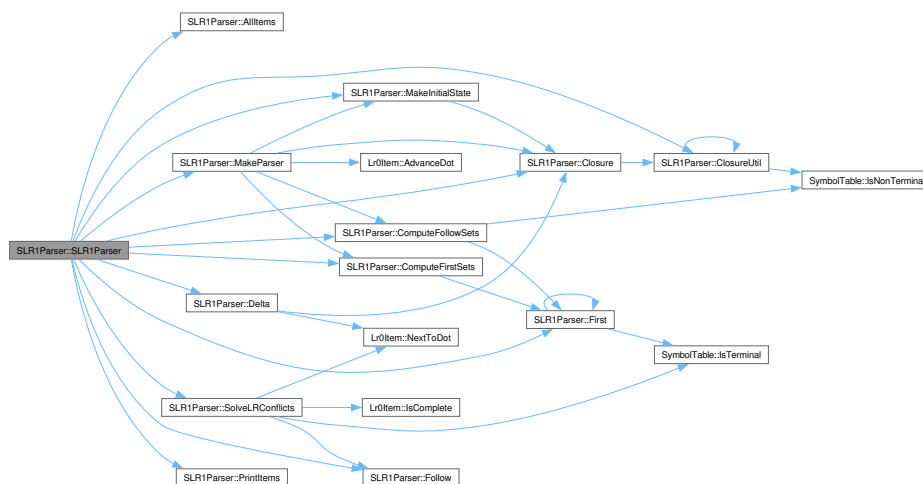
Shift	
Reduce	
Accept	
Empty	

9.31.4 Constructor & Destructor Documentation

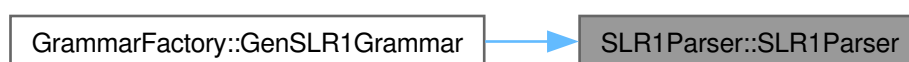
9.31.4.1 SLR1Parser() [1/2]

```
SLR1Parser::SLR1Parser () [default]
```

Here is the call graph for this function:



Here is the caller graph for this function:



9.31.4.2 SLR1Parser() [2/2]

```
SLR1Parser::SLR1Parser (
    Grammar gr) [explicit]
```

Here is the call graph for this function:



9.31.5 Member Function Documentation

9.31.5.1 AllItems()

```
std::unordered_set< Lr0Item > SLR1Parser::AllItems () const
```

Retrieves all LR(0) items in the grammar. This function returns a set of all LR(0) items derived from the grammar's productions. Each LR(0) item represents a production with a marker indicating the current position in the production (e.g., $A \rightarrow \alpha \bullet \beta$).

Returns

A set of all LR(0) items in the grammar.

Here is the caller graph for this function:



9.31.5.2 Closure()

```
void SLR1Parser::Closure (
    std::unordered_set< Lr0Item > & items)
```

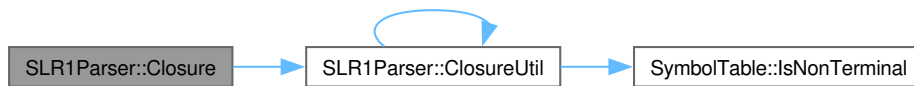
Computes the closure of a set of LR(0) items.

This function computes the closure of a given set of LR(0) items by adding all items that can be derived from the current items using the grammar's productions. The closure operation ensures that all possible derivations are considered when constructing the parser's states.

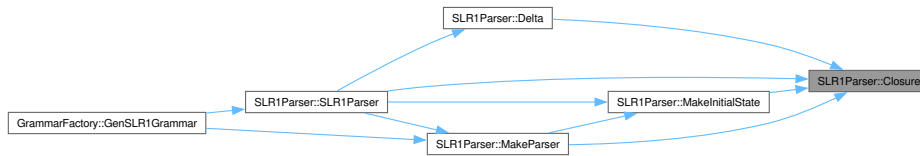
Parameters

<i>items</i>	The set of LR(0) items for which to compute the closure.
--------------	--

Here is the call graph for this function:



Here is the caller graph for this function:



9.31.5.3 ClosureUtil()

```

void SLR1Parser::ClosureUtil (
    std::unordered_set< Lr0Item > & items,
    unsigned int size,
    std::unordered_set< std::string > & visited)
  
```

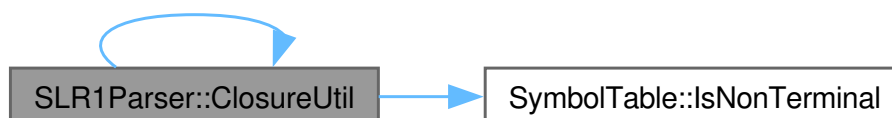
Helper function for computing the closure of LR(0) items.

This function recursively computes the closure of a set of LR(0) items by adding items derived from non-terminal symbols. It avoids redundant work by tracking visited non-terminals and stopping when no new items are added.

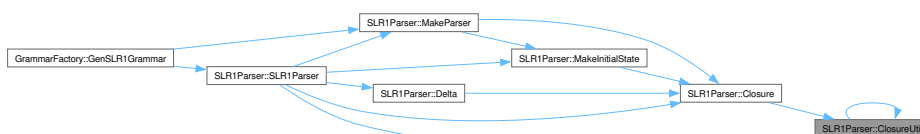
Parameters

<i>items</i>	The set of LR(0) items being processed.
<i>size</i>	The size of the items set at the start of the current iteration.
<i>visited</i>	A set of non-terminals that have already been processed.

Here is the call graph for this function:



Here is the caller graph for this function:

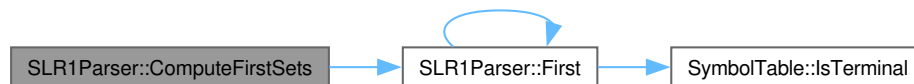


9.31.5.4 ComputeFirstSets()

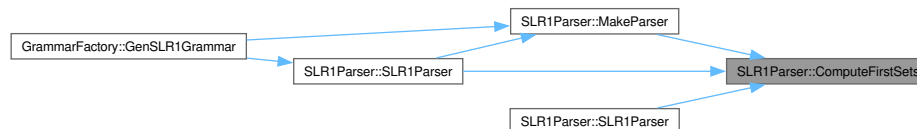
```
void SLR1Parser::ComputeFirstSets ()
```

Computes the FIRST sets for all non-terminal symbols in the grammar.

This function calculates the FIRST set for each non-terminal symbol in the grammar by iteratively applying a least fixed-point algorithm. This approach ensures that the FIRST sets are fully populated by repeatedly expanding and updating the sets until no further changes occur (i.e., a fixed-point is reached). Here is the call graph for this function:



Here is the caller graph for this function:



9.31.5.5 ComputeFollowSets()

```
void SLR1Parser::ComputeFollowSets ()
```

Computes the FOLLOW sets for all non-terminal symbols in the grammar. The FOLLOW set of a non-terminal symbol A contains all terminal symbols that can appear immediately after A in any sentential form derived from the grammar's start symbol. Additionally, if A can be the last symbol in a derivation, the end-of-input marker ($\$$) is included in its FOLLOW set. This function computes the FOLLOW sets using the following rules:

1. Initialize $FOLLOW(S) = \{ \$ \}$, where S is the start symbol.
2. For each production rule of the form $A \rightarrow \alpha B \beta$:
 - Add $FIRST(\beta) \setminus \{ \epsilon \}$ to $FOLLOW(B)$.
 - If $\epsilon \in FIRST(\beta)$, add $FOLLOW(A)$ to $FOLLOW(B)$.
3. Repeat step 2 until no changes occur in any FOLLOW set. The computed FOLLOW sets are cached in the `follow_sets_` member variable for later use by the parser.

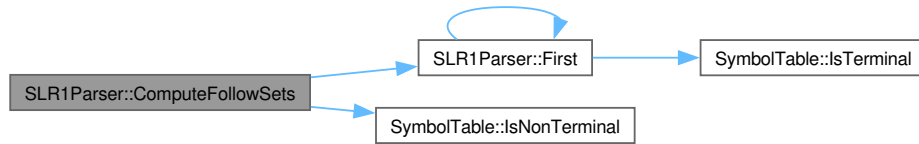
Note

This function assumes that the FIRST sets for all symbols have already been computed and are available in the `first_sets_` member variable.

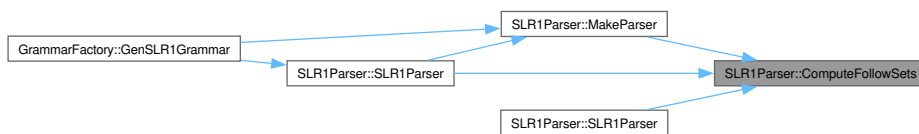
See also

[First](#)
[follow_sets_](#)

Here is the call graph for this function:



Here is the caller graph for this function:



9.31.5.6 Delta()

```

std::unordered_set< Lr0Item > SLR1Parser::Delta (
    const std::unordered_set< Lr0Item > & items,
    const std::string & str)
  
```

Computes the GOTO transition (δ) for a given set of LR(0) items and a symbol. This function is equivalent to the $\delta(I, X)$ function in LR parsing, where it computes the set of items reached from a state I via symbol X .

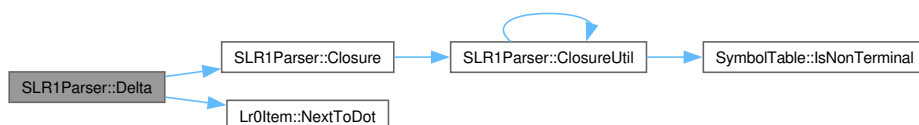
Parameters

<i>items</i>	The current set of LR(0) items (state).
<i>str</i>	The grammar symbol used for the transition.

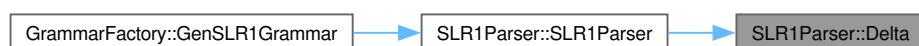
Returns

The resulting item set after the GOTO transition.

Here is the call graph for this function:



Here is the caller graph for this function:



9.31.5.7 First()

```
void SLR1Parser::First (
    std::span< const std::string > rule,
    std::unordered_set< std::string > & result)
```

Calculates the FIRST set for a given production rule in a grammar.

The FIRST set of a production rule contains all terminal symbols that can appear at the beginning of any string derived from that rule. If the rule can derive the empty string (epsilon), epsilon is included in the FIRST set.

This function computes the FIRST set by examining each symbol in the production rule:

- If a terminal symbol is encountered, it is added directly to the FIRST set, as it is the starting symbol of some derivation.
- If a non-terminal symbol is encountered, its FIRST set is recursively computed and added to the result, excluding epsilon unless it is followed by another symbol that could also lead to epsilon.
- If the entire rule could derive epsilon (i.e., each symbol in the rule can derive epsilon), then epsilon is added to the FIRST set.

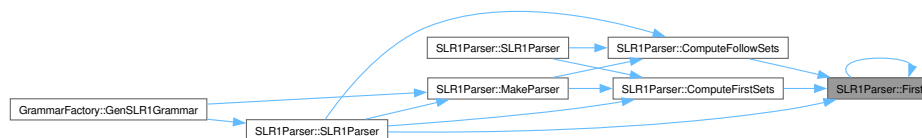
Parameters

<i>rule</i>	A span of strings representing the production rule for which to compute the FIRST set. Each string in the span is a symbol (either terminal or non-terminal).
<i>result</i>	A reference to an unordered set of strings where the computed FIRST set will be stored. The set will contain all terminal symbols that can start derivations of the rule, and possibly epsilon if the rule can derive an empty string.

Here is the call graph for this function:



Here is the caller graph for this function:



9.31.5.8 Follow()

```
std::unordered_set< std::string > SLR1Parser::Follow (
    const std::string & arg)
```

Computes the FOLLOW set for a given non-terminal symbol in the grammar.

The FOLLOW set for a non-terminal symbol includes all symbols that can appear immediately to the right of that symbol in any derivation, as well as any end-of-input markers if the symbol can appear at the end of derivations. FOLLOW sets are used in LL(1) parsing table construction to determine possible continuations after a non-terminal.

Parameters

<i>arg</i>	Non-terminal symbol for which to compute the FOLLOW set.
------------	--

Returns

An unordered set of strings containing symbols that form the FOLLOW set for *arg*.

Here is the caller graph for this function:



9.31.5.9 MakeInitialState()

```
void SLR1Parser::MakeInitialState ()
```

Creates the initial state of the parser's state machine.

This function initializes the starting state of the parser by computing the closure of the initial set of LR(0) items derived from the grammar's start symbol. The initial state is added to the `states_` set, and its transitions are prepared for further processing in the parser construction.

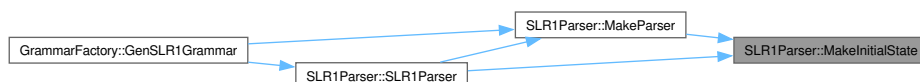
See also

[states_](#)
[transitions_](#)

Here is the call graph for this function:



Here is the caller graph for this function:



9.31.5.10 MakeParser()

```
bool SLR1Parser::MakeParser ()
```

Constructs the SLR(1) parsing tables (action and transition tables).

This function builds the SLR(1) parsing tables by computing the canonical collection of LR(0) items, generating the action and transition tables, and resolving conflicts (if any). It returns `true` if the grammar is SLR(1) and the tables are successfully constructed, or `false` if a conflict is detected that cannot be resolved.

Returns

`true` if the parsing tables are successfully constructed, `false` if the grammar is not SLR(1) or a conflict is encountered.

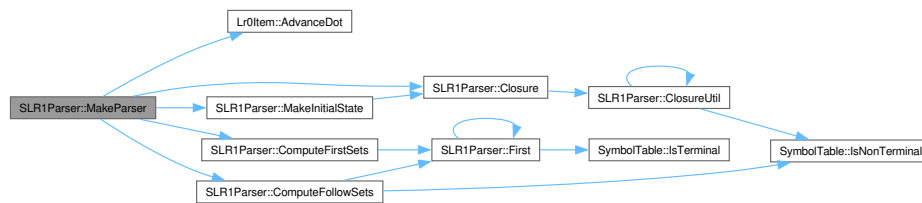
See also

[actions_](#)

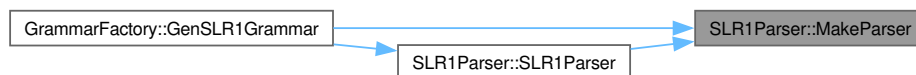
[transitions_](#)

[states_](#)

Here is the call graph for this function:



Here is the caller graph for this function:



9.31.5.11 PrintItems()

```
std::string SLR1Parser::PrintItems (
    const std::unordered_set< Lr0Item > & items) const
```

Returns a string representation of a set of LR(0) items.

This function converts a set of LR(0) items into a human-readable string, including dot positions, to help visualize parser states.

Parameters

<i>items</i>	The set of LR(0) items to print.
--------------	----------------------------------

Returns

A formatted string representation of the items.

Here is the caller graph for this function:

**9.31.5.12 SolveLRConflicts()**

```
bool SLR1Parser::SolveLRConflicts (
    const state & st)
```

Resolves LR conflicts in a given state.

This function attempts to resolve shift/reduce or reduce/reduce conflicts in a given state using SLR(1) parsing rules. It checks the FOLLOW sets of non-terminals to determine the correct action and updates the action table accordingly.

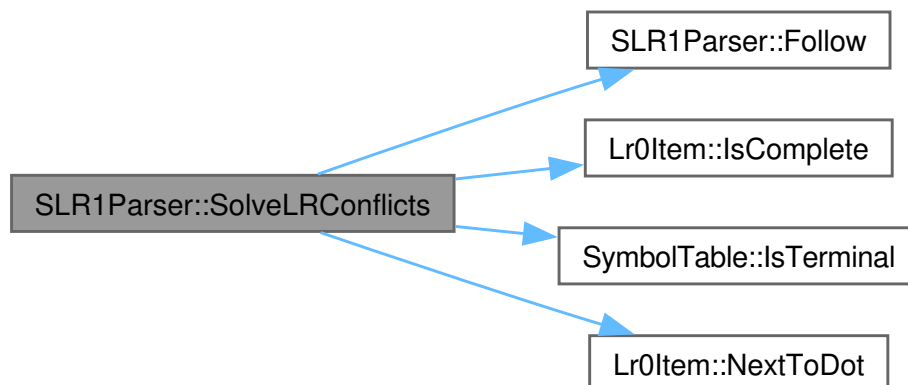
Parameters

<i>st</i>	The state in which to resolve conflicts.
-----------	--

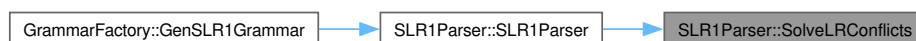
Returns

`true` if all conflicts are resolved, `false` if an unresolvable conflict is detected.

Here is the call graph for this function:



Here is the caller graph for this function:



9.31.6 Member Data Documentation

9.31.6.1 actions_

`action_table` SLR1Parser::actions_

The action table used by the parser to determine shift/reduce actions.

9.31.6.2 first_sets_

`std::unordered_map<std::string, std::unordered_set<std::string> >` SLR1Parser::first_sets_

Cached FIRST sets for all symbols in the grammar.

9.31.6.3 follow_sets_

`std::unordered_map<std::string, std::unordered_set<std::string> >` SLR1Parser::follow_sets_

Cached FOLLOW sets for all non-terminal symbols in the grammar.

9.31.6.4 gr_

`Grammar` SLR1Parser::gr_

The grammar being processed by the parser.

9.31.6.5 states_

`std::unordered_set<state>` SLR1Parser::states_

The set of states in the parser's state machine.

9.31.6.6 transitions_

`transition_table` SLR1Parser::transitions_

The transition table used by the parser to determine state transitions.

The documentation for this class was generated from the following files:

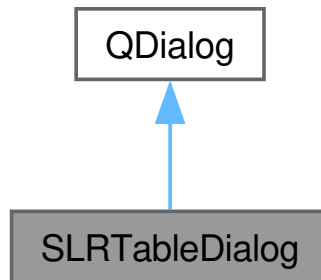
- [backend/slr1_parser.hpp](#)
- [backend/slr1_parser.cpp](#)

9.32 SLRTableDialog Class Reference

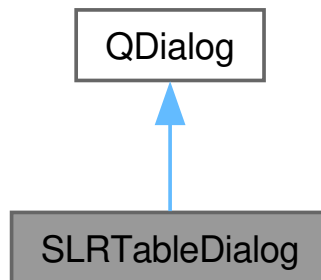
Dialog window for completing and submitting an SLR(1) parsing table.

```
#include <slrtabledialog.h>
```

Inheritance diagram for SLRTableDialog:



Collaboration diagram for SLRTableDialog:



Signals

- void [submitted](#) (const QVector< QVector< QString > > &data)
- void [guidedRequested](#) (const QVector< QVector< QString > > &data)

Public Member Functions

- [SLRTableDialog](#) (int rowCount, int colCount, const QStringList &colHeaders, QWidget *parent=nullptr, QVector< QVector< QString > > *initialData=nullptr)
Constructs the SLR(1) table dialog.
- QVector< QVector< QString > > [getTableData](#) () const
Retrieves the content of the table after user interaction.
- void [setInitialData](#) (const QVector< QVector< QString > > &data)
Fills the table with existing data.
- void [setGuidedModeActive](#) (bool active)
- void [setGuidedButtonVisible](#) (bool visible)
Shows or hides the guided mode button.
- void [highlightIncorrectCells](#) (const QList< QPair< int, int > > &coords)
- void [highlightInvalidCells](#) (const QList< QPair< int, int > > &coords)

9.32.1 Detailed Description

Dialog window for completing and submitting an SLR(1) parsing table.

This class displays a table-based UI for students to fill in the ACTION and GOTO parts of the SLR(1) parsing table. It supports initializing the table with data, retrieving user input, and integrating with correction logic in tutorial or challenge mode.

9.32.2 Constructor & Destructor Documentation

9.32.2.1 SLRTableDialog()

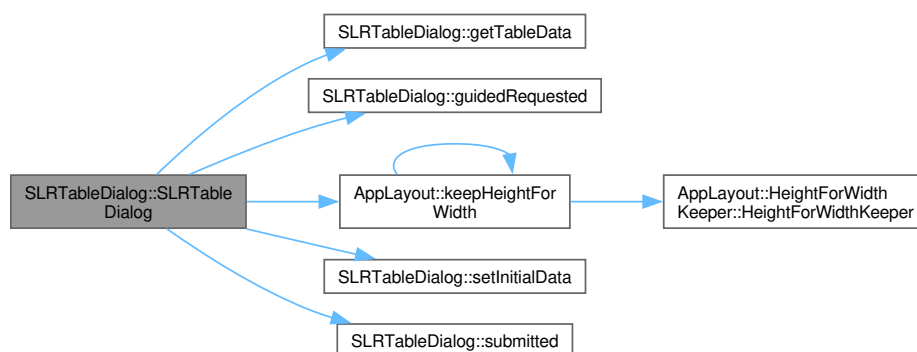
```
SLRTableDialog::SLRTableDialog (
    int rowCount,
    int colCount,
    const QStringList & colHeaders,
    QWidget * parent = nullptr,
    QVector< QVector< QString > > * initialData = nullptr)
```

Constructs the SLR(1) table dialog.

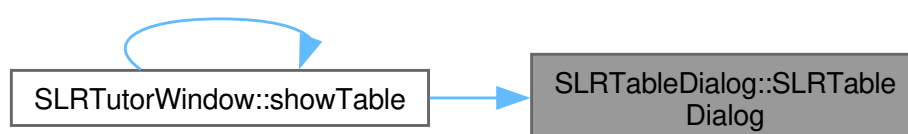
Parameters

<i>rowCount</i>	Number of rows (usually equal to number of LR(0) states).
<i>colCount</i>	Number of columns (symbols = terminals + non-terminals).
<i>colHeaders</i>	Header labels for the columns.
<i>parent</i>	Parent widget.
<i>initialData</i>	Optional initial data to pre-fill the table.

Here is the call graph for this function:



Here is the caller graph for this function:



9.32.3 Member Function Documentation

9.32.3.1 getTableData()

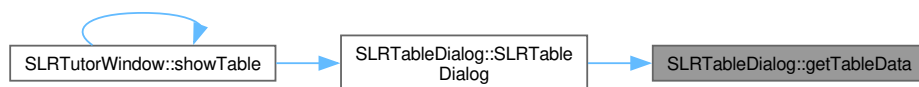
```
QVector< QVector< QString > > SLRTableDialog::getTableData () const
```

Retrieves the content of the table after user interaction.

Returns

A 2D vector representing the current table values.

Here is the caller graph for this function:



9.32.3.2 guidedRequested

```
void SLRTableDialog::guidedRequested (
    const QVector< QVector< QString > > & data) [signal]
```

Here is the caller graph for this function:



9.32.3.3 highlightIncorrectCells()

```
void SLRTableDialog::highlightIncorrectCells (
    const QList< QPair< int, int > > & coords)
```

9.32.3.4 highlightInvalidCells()

```
void SLRTableDialog::highlightInvalidCells (
    const QList< QPair< int, int > > & coords)
```

9.32.3.5 setGuidedButtonVisible()

```
void SLRTableDialog::setGuidedButtonVisible (
    bool visible)
```

Shows or hides the guided mode button.

Exam mode hides it: the guided walkthrough would reveal the answers.

Parameters

<i>visible</i>	Whether the button should be visible.
----------------	---------------------------------------

9.32.3.6 setGuidedModeActive()

```
void SLRTableDialog::setGuidedModeActive (
    bool active)
```

9.32.3.7 setInitialData()

```
void SLRTableDialog::setInitialData (
    const QVector< QVector< QString > > & data)
```

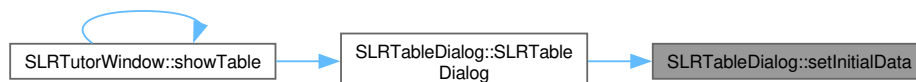
Fills the table with existing data.

This method is used to show a previous user submission (e.g., during retries or feedback).

Parameters

<i>data</i>	2D vector containing the table data to display.
-------------	---

Here is the caller graph for this function:



9.32.3.8 submitted

```
void SLRTableDialog::submitted (
    const QVector< QVector< QString > > & data) [signal]
```

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

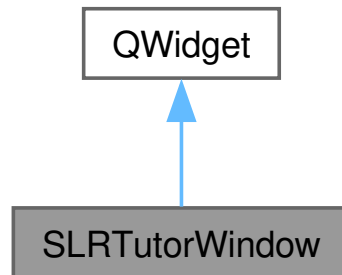
- [gui/slrtabledialog.h](#)
- [gui/slrtabledialog.cpp](#)

9.33 SLRTutorWindow Class Reference

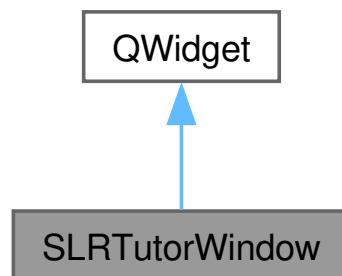
Main window for the SLR(1) interactive tutoring mode in [SyntaxTutor](#).

```
#include <slrtutorwindow.h>
```

Inheritance diagram for SLRTutorWindow:



Collaboration diagram for SLRTutorWindow:



Signals

- void [sessionFinished](#) (int cntRight, int cntWrong)
- void [exitRequested](#) (bool applyResults, int cntRight, int cntWrong)

Public Member Functions

- void [applyTextScale](#) ()
Re-applies the current text size to this tutor.
- [SLRTutorWindow](#) (const [Grammar](#) &g, [TutorialManager](#) *tm=nullptr, QWidget *parent=nullptr, bool examMode=false)
Constructs the SLR(1) tutor window with a given grammar.
- [~SLRTutorWindow](#) ()
- QString [generateQuestion](#) ()
Generates a new question for the current tutor state.
- void [updateState](#) (bool isCorrect)
Updates tutor state based on whether the last answer was correct.
- QString [FormatGrammar](#) (const [Grammar](#) &grammar)
- QVector< [GrammarView::Row](#) > [buildGrammarRows](#) (const [Grammar](#) &grammar) const
< Utility for displaying grammar

- void [fillSortedGrammar](#) ()
- void [addMessage](#) (const QString &text, bool isUser)
 - < Prepares grammar in display-friendly format
- void [addGrammarMessage](#) ()
 - < Add message to chat
- void [addWidgetMessage](#) (QWidget *widget)
- void [exportConversationToPdf](#) (const QString &filePath)
- void [showTable](#) ()
 - < Export full interaction
- void [launchSLRWizard](#) ()
 - < Render SLR(1) table
- void [updateProgressPanel](#) ()
- void [addUserState](#) (unsigned id)
 - < Refresh visual progress
- void [addUserTransition](#) (unsigned fromId, const std::string &symbol, unsigned told)
 - < Register a user-created state
- void [animateLabelPop](#) (QLabel *label)
- void [animateLabelColor](#) (QLabel *label, const QColor &flashColor)
- void [wrongAnimation](#) ()
- void [wrongUserResponseAnimation](#) ()
- void [markLastUserIncorrect](#) ()
- bool [verifyResponse](#) (const QString &userResponse)
- bool [verifyResponseForA](#) (const QString &userResponse)
- bool [verifyResponseForA1](#) (const QString &userResponse)
- bool [verifyResponseForA2](#) (const QString &userResponse)
- bool [verifyResponseForA3](#) (const QString &userResponse)
- bool [verifyResponseForA4](#) (const QString &userResponse)
- bool [verifyResponseForB](#) (const QString &userResponse)
- bool [verifyResponseForC](#) (const QString &userResponse)
- bool [verifyResponseForCA](#) (const QString &userResponse)
- bool [verifyResponseForCB](#) (const QString &userResponse)
- bool [verifyResponseForD](#) (const QString &userResponse)
- bool [verifyResponseForD1](#) (const QString &userResponse)
- bool [verifyResponseForD2](#) (const QString &userResponse)
- bool [verifyResponseForE](#) (const QString &userResponse)
- bool [verifyResponseForE1](#) (const QString &userResponse)
- bool [verifyResponseForE2](#) (const QString &userResponse)
- bool [verifyResponseForF](#) (const QString &userResponse)
- bool [verifyResponseForFA](#) (const QString &userResponse)
- bool [verifyResponseForG](#) (const QString &userResponse)
- bool [verifyResponseForH](#) ()
- QString [solution](#) (const std::string &state)
- std::unordered_set< [Lr0Item](#) > [solutionForA](#) ()
- QString [solutionForA1](#) ()
- QString [solutionForA2](#) ()
- std::vector< std::pair< std::string, std::vector< std::string > > > [solutionForA3](#) ()
- std::unordered_set< [Lr0Item](#) > [solutionForA4](#) ()
- unsigned [solutionForB](#) ()
- unsigned [solutionForC](#) ()
- QStringList [solutionForCA](#) ()
- std::unordered_set< [Lr0Item](#) > [solutionForCB](#) ()
- QStringList [solutionForD](#) ()
- QString [solutionForD1](#) ()
- QString [solutionForD2](#) ()

- std::ptrdiff_t [solutionForE](#) ()
- QSet< unsigned > [solutionForE1](#) ()
- QMap< unsigned, unsigned > [solutionForE2](#) ()
- QSet< unsigned > [solutionForF](#) ()
- QSet< QString > [solutionForFA](#) ()
- bool [isReduction](#) (const [Lr0Item](#) &item) const
Whether item reduces: complete, and not the axiom's.
- QStringList [tableColumns](#) ()
Columns of the SLR(1) table: terminals, \$, non-terminals.
- QString [solutionCell](#) (unsigned stateId, const QString &symbol)
The correct content of a table cell, as the student writes it (sN, rN, acc, or a state number for a non-terminal).
- QString [initialItemText](#) () const
The grammar's initial item, e.g. "S -> · A \$".
- QSet< QString > [solutionForG](#) ()
- QString [feedback](#) ()
- QString [feedbackForA](#) ()
- QString [feedbackForA1](#) ()
- QString [feedbackForA2](#) ()
- QString [feedbackForA3](#) ()
- QString [feedbackForA4](#) ()
- QString [feedbackForAPrime](#) ()
- QString [feedbackForB](#) ()
- QString [feedbackForB1](#) ()
- QString [feedbackForB2](#) ()
- QString [feedbackForBPrime](#) ()
- QString [feedbackForC](#) ()
- QString [feedbackForCA](#) ()
- QString [feedbackForCB](#) ()
- QString [feedbackForD](#) ()
- QString [feedbackForD1](#) ()
- QString [feedbackForD2](#) ()
- QString [feedbackForDPrime](#) ()
- QString [feedbackForE](#) ()
- QString [feedbackForE1](#) ()
- QString [feedbackForE2](#) ()
- QString [feedbackForF](#) ()
- QString [feedbackForFA](#) ()
- QString [feedbackForG](#) ()
- QString [TeachDeltaFunction](#) (const std::unordered_set< [Lr0Item](#) > &items, const QString &symbol)
- void [TeachClosureStep](#) (std::unordered_set< [Lr0Item](#) > &items, unsigned int size, std::unordered_set< std::string > &visited, int depth, QString &output)
- QString [TeachClosure](#) (const std::unordered_set< [Lr0Item](#) > &initialItems)
- void [updatePlaceholder](#) ()
- bool [confirmExitToHome](#) ()
- QString [promptExportFilePath](#) () const
- void [setupAutomatonPanel](#) ()
Builds the (hidden) automaton view and prepares the button.
- void [updateAutomatonPanel](#) ()
Syncs the automaton view with the current tutor state.
- void [openAutomatonViewer](#) ()
Opens the floating automaton viewer, or raises it if already open.
- void [updateAutomatonButton](#) ()
Enables the viewer button once at least one state (I0) exists.

- void [postQuestion](#) ()
Shows and remembers the next question.
- QString [examSolutionText](#) ()
Expected answer for the current state.
- void [scoreExamTable](#) (const QStringList &colHeaders)
Grades the SLR table per cell.
- void [showExamReport](#) ()
Opens the end-of-exam report.

Protected Member Functions

- void [closeEvent](#) (QCloseEvent *event) override
- bool [eventFilter](#) (QObject *obj, QEvent *event) override

9.33.1 Detailed Description

Main window for the SLR(1) interactive tutoring mode in [SyntaxTutor](#).

This class implements an interactive, step-by-step tutorial to teach students how to construct SLR(1) parsing tables, including closure, GOTO, automaton construction, FOLLOW sets, and the final table.

It supports animated feedback, pedagogical guidance, error correction, and export of the tutoring session.

The tutor follows a finite-state flow ([StateSlr](#)) to structure learning, with corrective explanations and automatic evaluation at each step.

9.33.2 Constructor & Destructor Documentation

9.33.2.1 SLRTutorWindow()

```
SLRTutorWindow::SLRTutorWindow (
    const Grammar & g,
    TutorialManager * tm = nullptr,
    QWidget * parent = nullptr,
    bool examMode = false) [explicit]
```

Constructs the SLR(1) tutor window with a given grammar.

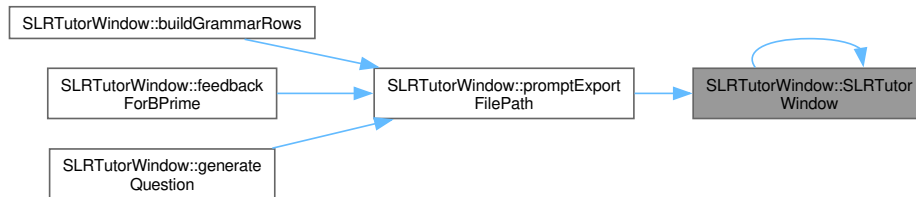
Parameters

<i>g</i>	The grammar used for the session.
<i>tm</i>	Optional pointer to the tutorial manager (for guided tour).
<i>parent</i>	Parent widget.
<i>examMode</i>	When true, all feedback is suppressed: answers are recorded silently, error sub-questions never trigger, and a graded report is shown at the end.

Here is the call graph for this function:



Here is the caller graph for this function:



9.33.2.2 ~SLRTutorWindow()

```
SLRTutorWindow::~~SLRTutorWindow ()
```

9.33.3 Member Function Documentation

9.33.3.1 addGrammarMessage()

```
void SLRTutorWindow::addGrammarMessage ()
```

< Add message to chat

9.33.3.2 addMessage()

```
void SLRTutorWindow::addMessage (
    const QString & text,
    bool isUser)
```

< Prepares grammar in display-friendly format

9.33.3.3 addUserState()

```
void SLRTutorWindow::addUserState (
    unsigned id)
```

< Refresh visual progress

Here is the caller graph for this function:



9.33.3.4 addUserTransition()

```
void SLRTutorWindow::addUserTransition (
    unsigned fromId,
    const std::string & symbol,
    unsigned toId)
```

< Register a user-created state

Here is the caller graph for this function:



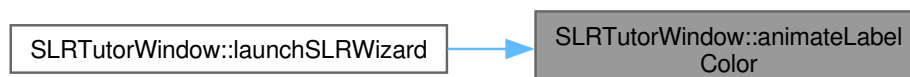
9.33.3.5 addWidgetMessage()

```
void SLRTutorWindow::addWidgetMessage (
    QWidget * widget)
```

9.33.3.6 animateLabelColor()

```
void SLRTutorWindow::animateLabelColor (
    QLabel * label,
    const QColor & flashColor)
```

Here is the caller graph for this function:



9.33.3.7 animateLabelPop()

```
void SLRTutorWindow::animateLabelPop (
    QLabel * label)
```

Here is the caller graph for this function:

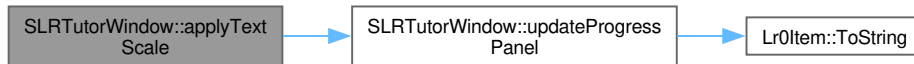


9.33.3.8 applyTextScale()

```
void SLRTutorWindow::applyTextScale ()
```

Re-applies the current text size to this tutor.

The chat bubbles size themselves from font metrics and the grammar panel from the card's width, so both have to be recomputed when the user picks a different text size. Here is the call graph for this function:

**9.33.3.9 buildGrammarRows()**

```
QVector< GrammarView::Row > SLRTutorWindow::buildGrammarRows (
    const Grammar & grammar) const
```

< Utility for displaying grammar

Here is the call graph for this function:

**9.33.3.10 closeEvent()**

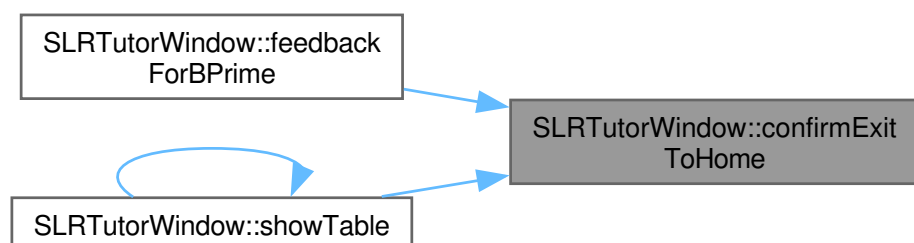
```
void SLRTutorWindow::closeEvent (
    QCloseEvent * event) [inline], [override], [protected]
```

Here is the call graph for this function:

**9.33.3.11 confirmExitToHome()**

```
bool SLRTutorWindow::confirmExitToHome ()
```

Here is the caller graph for this function:



9.33.3.12 eventFilter()

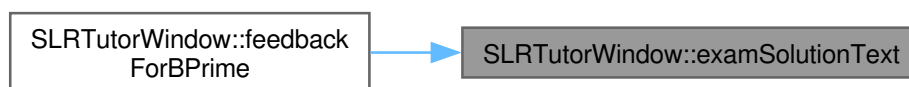
```
bool SLRTutorWindow::eventFilter (
    QObject * obj,
    QEvent * event) [override], [protected]
```

9.33.3.13 examSolutionText()

```
QString SLRTutorWindow::examSolutionText ()
```

Expected answer for the current state.

Here is the caller graph for this function:



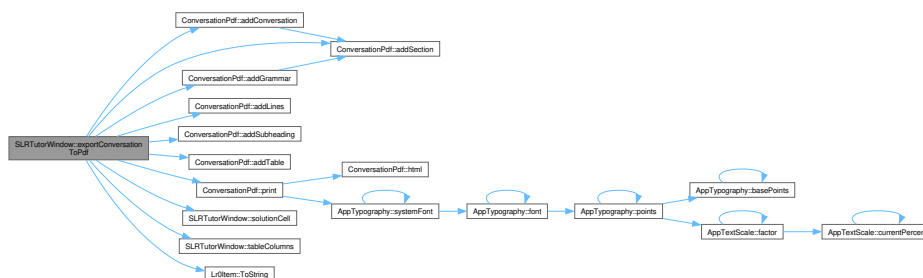
9.33.3.14 exitRequested

```
void SLRTutorWindow::exitRequested (
    bool applyResults,
    int cntRight,
    int cntWrong) [signal]
```

9.33.3.15 exportConversationToPdf()

```
void SLRTutorWindow::exportConversationToPdf (
    const QString & filePath)
```

Here is the call graph for this function:



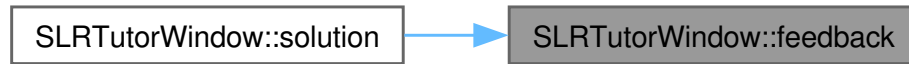
Here is the caller graph for this function:



9.33.3.16 feedback()

```
QString SLRTutorWindow::feedback ()
```

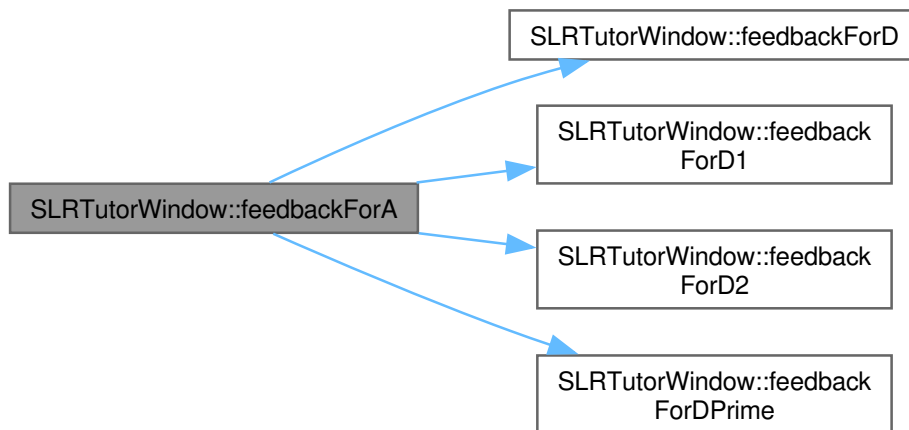
Here is the caller graph for this function:



9.33.3.17 feedbackForA()

```
QString SLRTutorWindow::feedbackForA ()
```

Here is the call graph for this function:



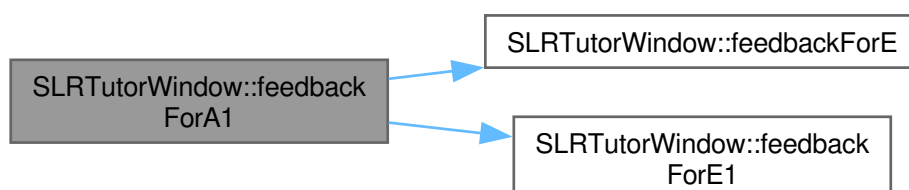
Here is the caller graph for this function:



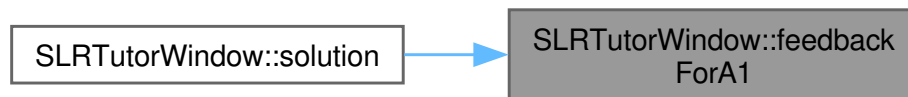
9.33.3.18 feedbackForA1()

```
QString SLRTutorWindow::feedbackForA1 ()
```

Here is the call graph for this function:



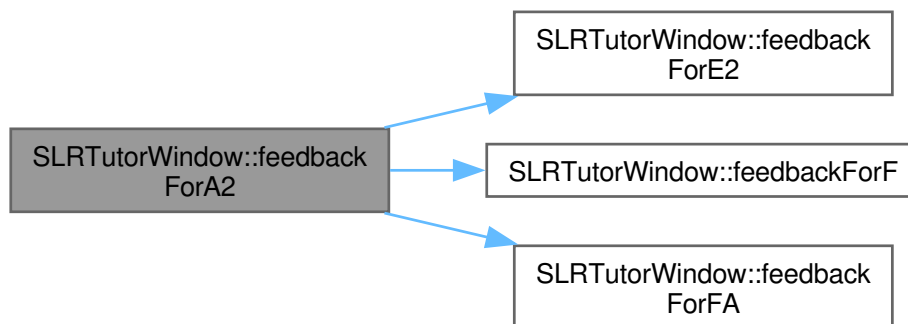
Here is the caller graph for this function:



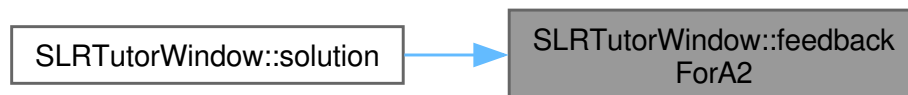
9.33.3.19 feedbackForA2()

```
QString SLRTutorWindow::feedbackForA2 ()
```

Here is the call graph for this function:



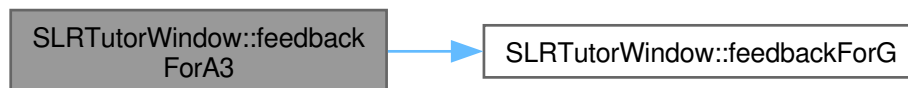
Here is the caller graph for this function:



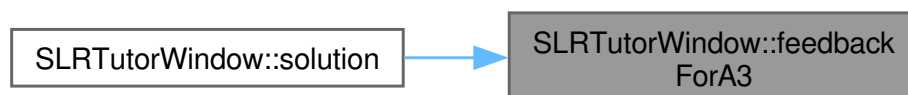
9.33.3.20 feedbackForA3()

```
QString SLRTutorWindow::feedbackForA3 ()
```

Here is the call graph for this function:



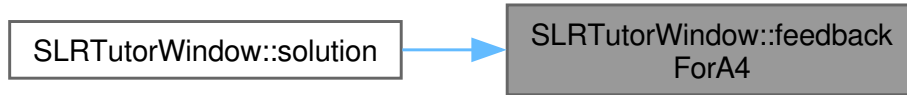
Here is the caller graph for this function:



9.33.3.21 feedbackForA4()

```
QString SLRTutorWindow::feedbackForA4 ()
```

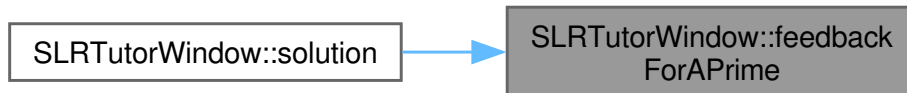
Here is the caller graph for this function:



9.33.3.22 feedbackForAPrime()

```
QString SLRTutorWindow::feedbackForAPrime ()
```

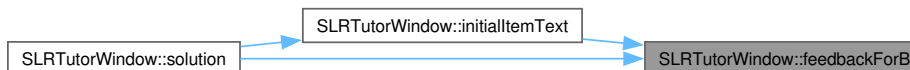
Here is the caller graph for this function:



9.33.3.23 feedbackForB()

```
QString SLRTutorWindow::feedbackForB ()
```

Here is the caller graph for this function:



9.33.3.24 feedbackForB1()

```
QString SLRTutorWindow::feedbackForB1 ()
```

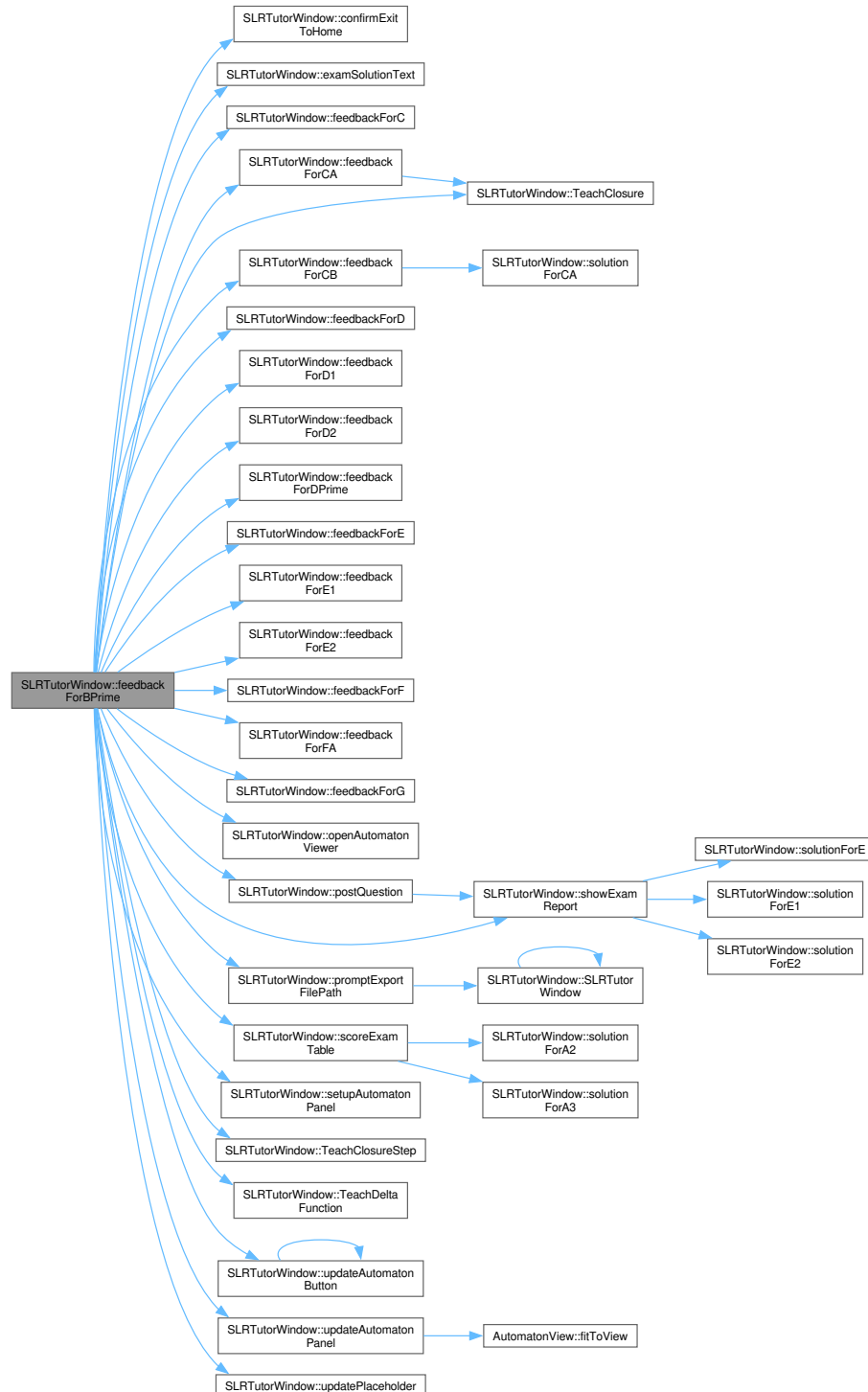
9.33.3.25 feedbackForB2()

```
QString SLRTutorWindow::feedbackForB2 ()
```

9.33.3.26 feedbackForBPrime()

QString SLRTutorWindow::feedbackForBPrime ()

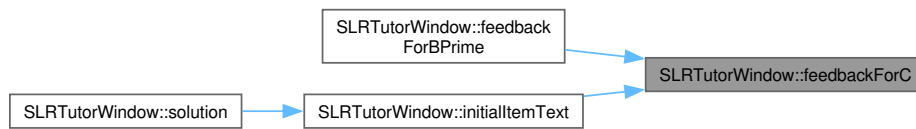
Here is the call graph for this function:



9.33.3.27 feedbackForC()

QString SLRTutorWindow::feedbackForC ()

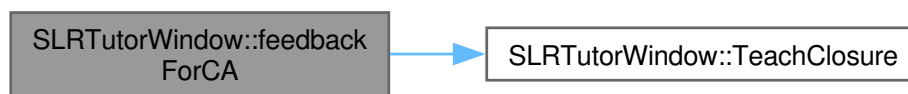
Here is the caller graph for this function:



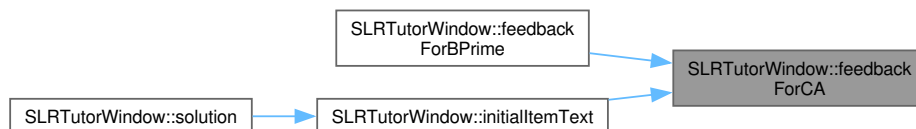
9.33.3.28 feedbackForCA()

```
QString SLRTutorWindow::feedbackForCA ()
```

Here is the call graph for this function:



Here is the caller graph for this function:



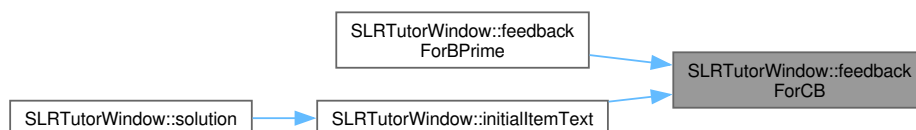
9.33.3.29 feedbackForCB()

```
QString SLRTutorWindow::feedbackForCB ()
```

Here is the call graph for this function:



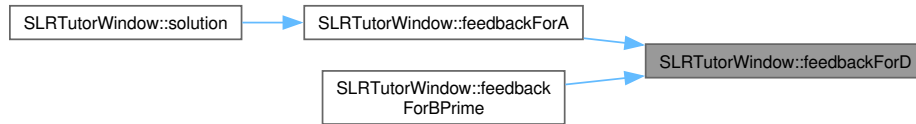
Here is the caller graph for this function:



9.33.3.30 feedbackForD()

```
QString SLRTutorWindow::feedbackForD ()
```

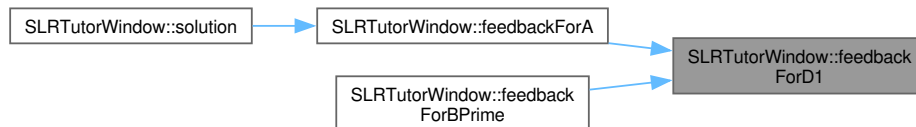
Here is the caller graph for this function:



9.33.3.31 feedbackForD1()

```
QString SLRTutorWindow::feedbackForD1 ()
```

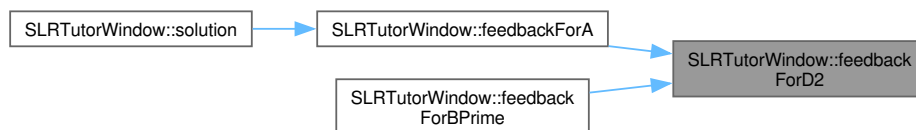
Here is the caller graph for this function:



9.33.3.32 feedbackForD2()

```
QString SLRTutorWindow::feedbackForD2 ()
```

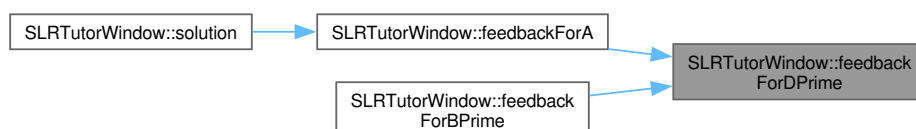
Here is the caller graph for this function:



9.33.3.33 feedbackForDPrime()

```
QString SLRTutorWindow::feedbackForDPrime ()
```

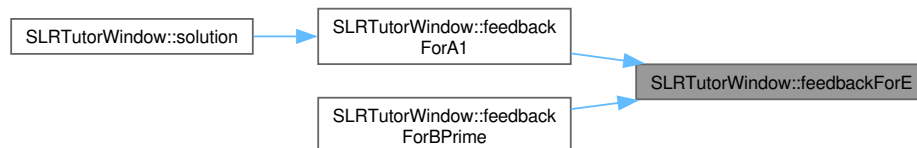
Here is the caller graph for this function:



9.33.3.34 feedbackForE()

```
QString SLRTutorWindow::feedbackForE ()
```

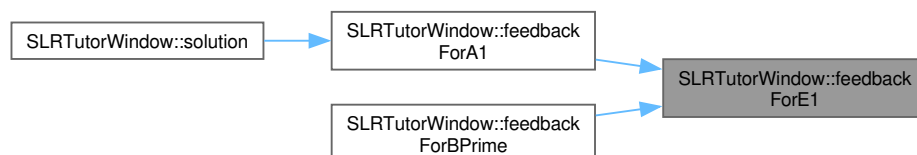
Here is the caller graph for this function:



9.33.3.35 feedbackForE1()

```
QString SLRTutorWindow::feedbackForE1 ()
```

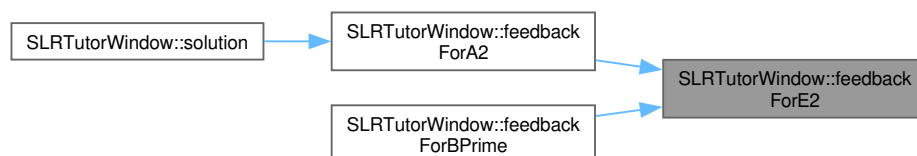
Here is the caller graph for this function:



9.33.3.36 feedbackForE2()

```
QString SLRTutorWindow::feedbackForE2 ()
```

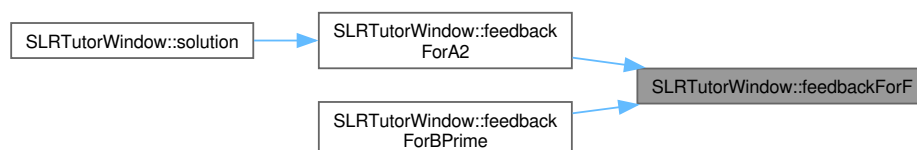
Here is the caller graph for this function:



9.33.3.37 feedbackForF()

```
QString SLRTutorWindow::feedbackForF ()
```

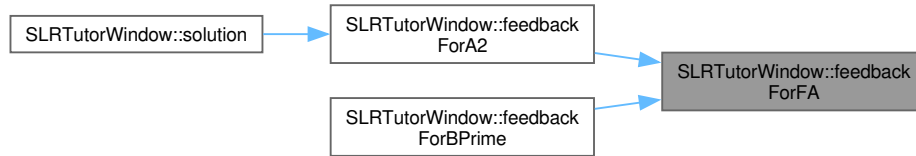
Here is the caller graph for this function:



9.33.3.38 feedbackForFA()

```
QString SLRTutorWindow::feedbackForFA ()
```

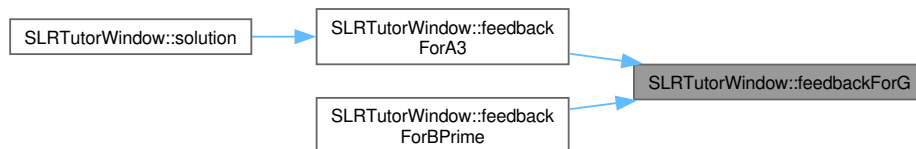
Here is the caller graph for this function:



9.33.3.39 feedbackForG()

```
QString SLRTutorWindow::feedbackForG ()
```

Here is the caller graph for this function:



9.33.3.40 fillSortedGrammar()

```
void SLRTutorWindow::fillSortedGrammar ()
```

9.33.3.41 FormatGrammar()

```
QString SLRTutorWindow::FormatGrammar (
    const Grammar & grammar)
```

Here is the call graph for this function:



9.33.3.42 generateQuestion()

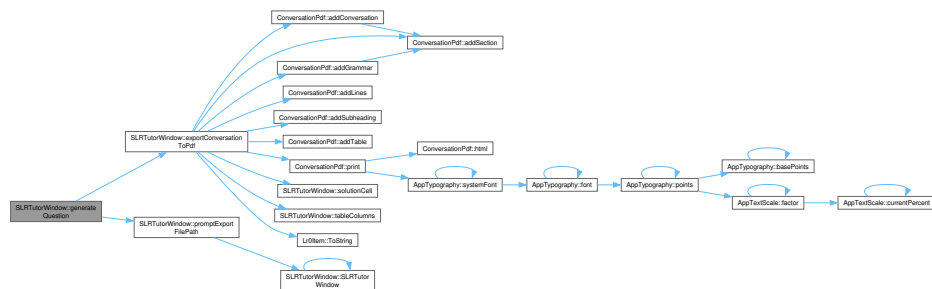
```
QString SLRTutorWindow::generateQuestion ()
```

Generates a new question for the current tutor state.

Returns

The formatted question string.

Here is the call graph for this function:

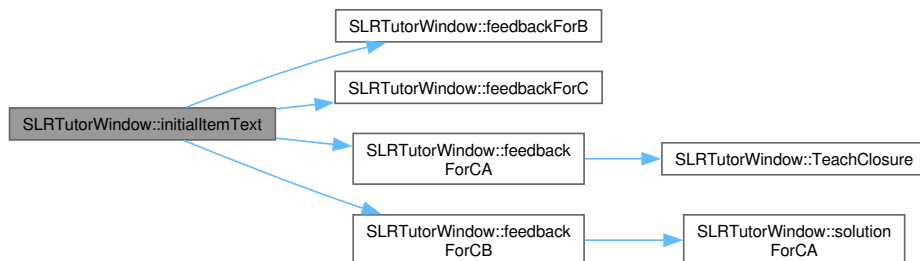


9.33.3.43 initialItemText()

```
QString SLRTutorWindow::initialItemText () const
```

The grammar's initial item, e.g. "S $\rightarrow$ $\cdot$ A \$".

Here is the call graph for this function:



Here is the caller graph for this function:

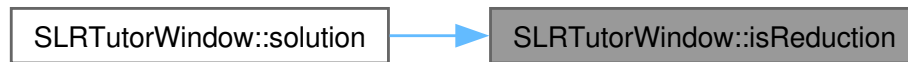


9.33.3.44 isReduction()

```
bool SLRTutorWindow::isReduction (
    const Lr0Item & item) const
```

Whether `item` reduces: complete, and not the axiom's.

The backend counts $S \rightarrow A \cdot \$$ as complete because that is where the input is accepted, but accepting is not a reduction. Here is the caller graph for this function:

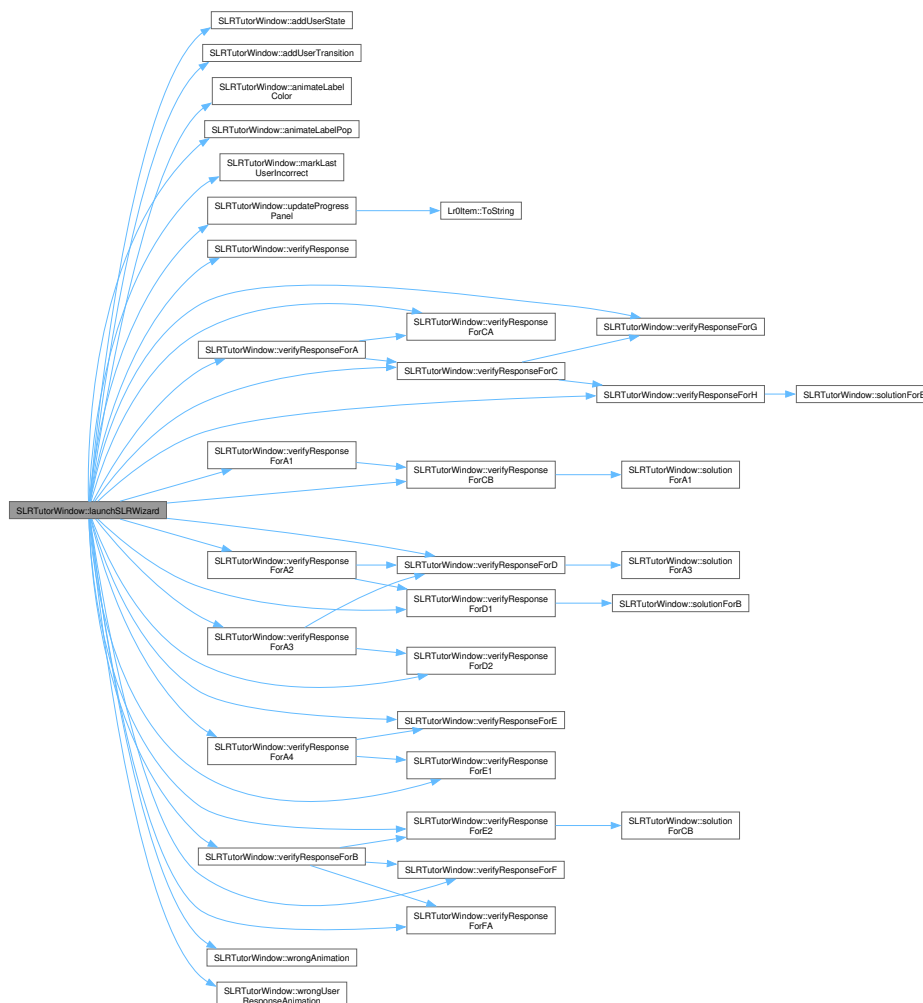


9.33.3.45 launchSLRWizard()

```
void SLRTutorWindow::launchSLRWizard ()
```

< Render SLR(1) table

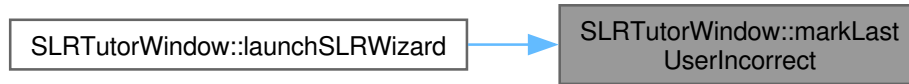
Here is the call graph for this function:



9.33.3.46 markLastUserIncorrect()

```
void SLRTutorWindow::markLastUserIncorrect ()
```

Here is the caller graph for this function:

**9.33.3.47 openAutomatonViewer()**

```
void SLRTutorWindow::openAutomatonViewer ()
```

Opens the floating automaton viewer, or raises it if already open.

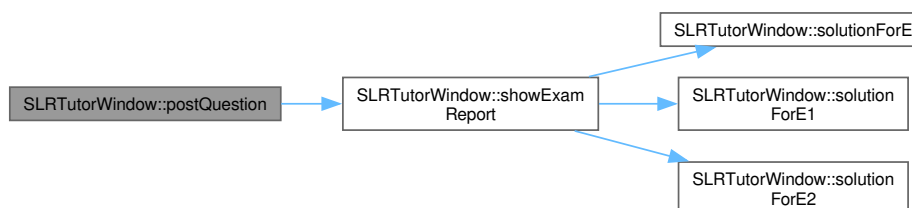
Here is the caller graph for this function:

**9.33.3.48 postQuestion()**

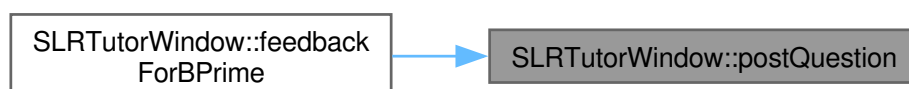
```
void SLRTutorWindow::postQuestion ()
```

Shows and remembers the next question.

Here is the call graph for this function:



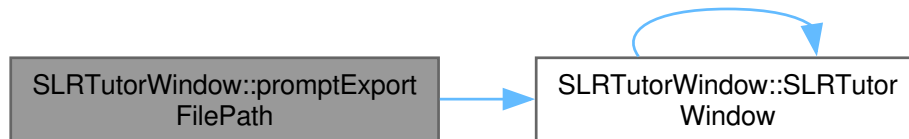
Here is the caller graph for this function:



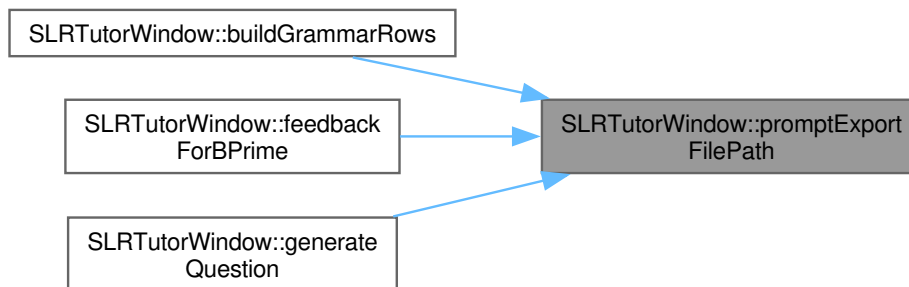
9.33.3.49 promptExportFilePath()

```
QString SLRTutorWindow::promptExportFilePath () const
```

Here is the call graph for this function:



Here is the caller graph for this function:

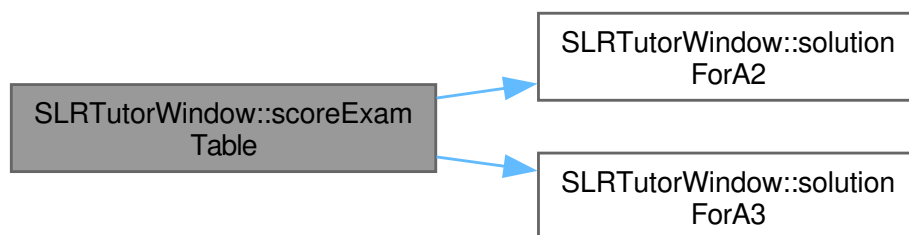


9.33.3.50 scoreExamTable()

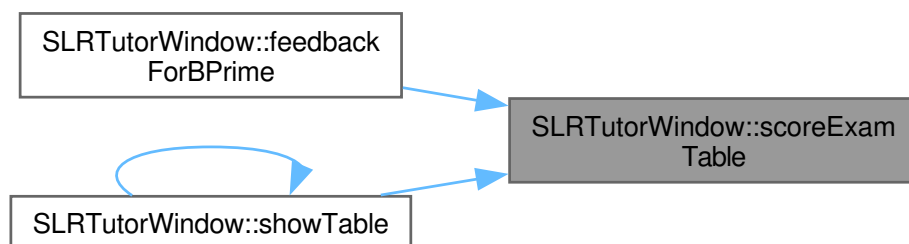
```
void SLRTutorWindow::scoreExamTable (
    const QStringList & colHeaders)
```

Grades the SLR table per cell.

Here is the call graph for this function:



Here is the caller graph for this function:



9.33.3.51 sessionFinished

```
void SLRTutorWindow::sessionFinished (
    int cntRight,
    int cntWrong) [signal]
```

Here is the caller graph for this function:

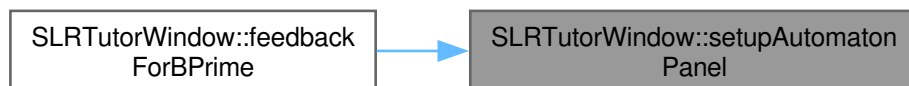


9.33.3.52 setupAutomatonPanel()

```
void SLRTutorWindow::setupAutomatonPanel ()
```

Builds the (hidden) automaton view and prepares the button.

Does nothing in exam mode: the automaton would reveal the collection. The view is not embedded in the main layout; it lives in a separate floating viewer opened on demand. Here is the caller graph for this function:

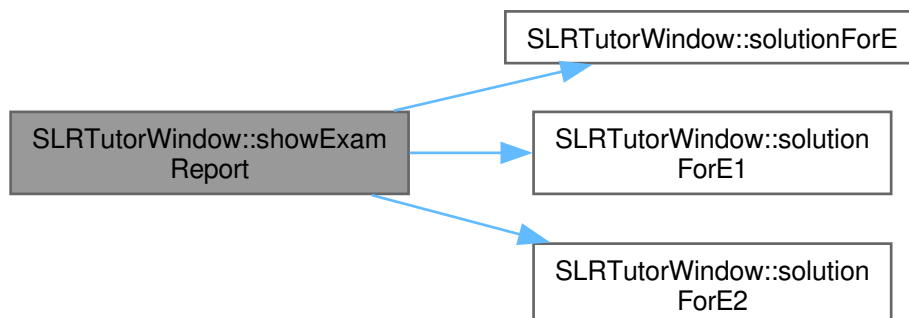


9.33.3.53 showExamReport()

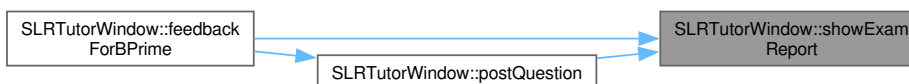
```
void SLRTutorWindow::showExamReport ()
```

Opens the end-of-exam report.

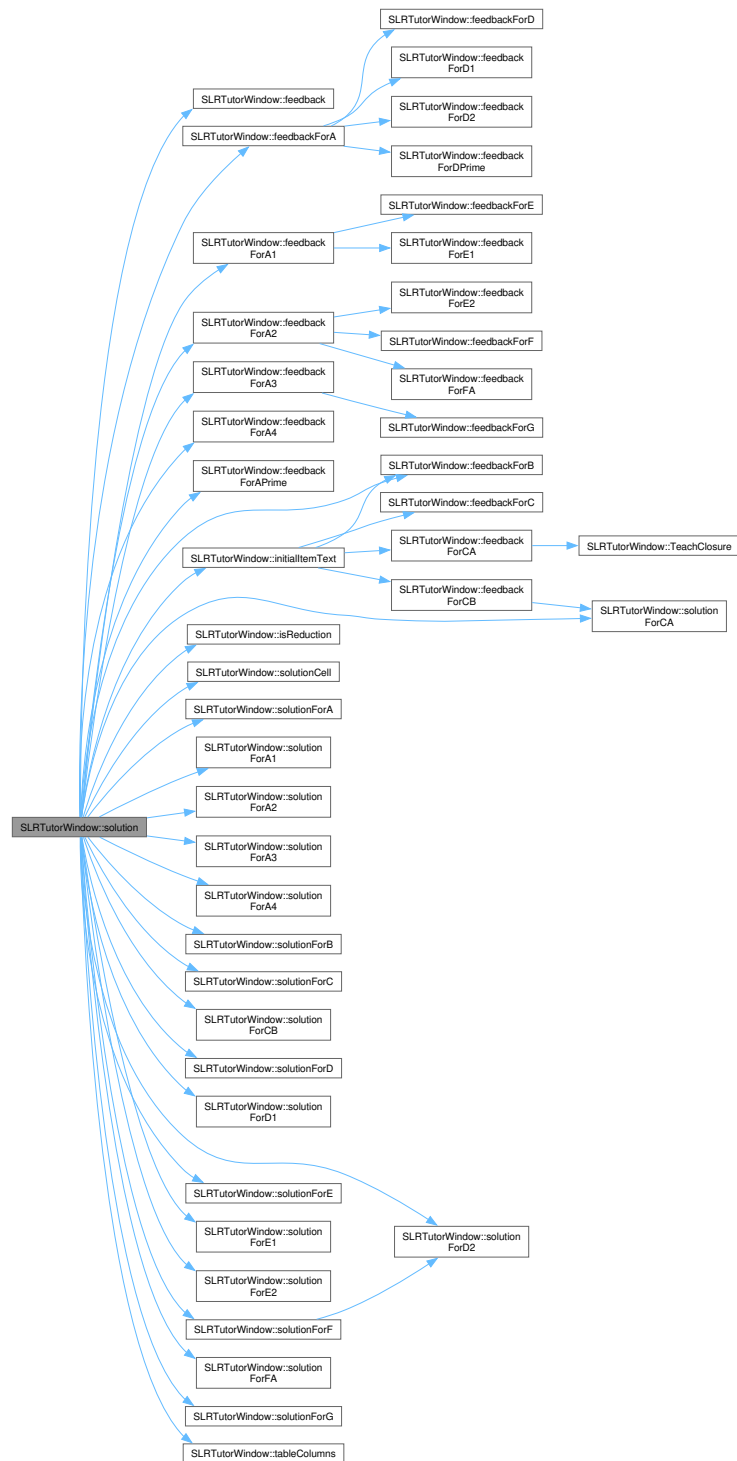
Here is the call graph for this function:



Here is the caller graph for this function:



Here is the call graph for this function:



9.33.3.56 solutionCell()

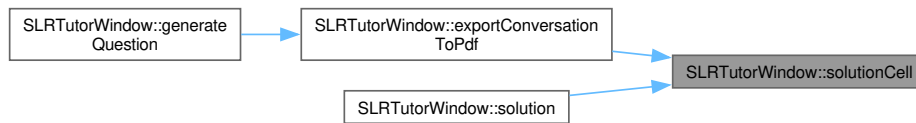
```

QString SLRTutorWindow::solutionCell (
    unsigned stateId,
    const QString & symbol)

```

The correct content of a table cell, as the student writes it (sN, rN, acc, or a state number for a non-terminal).

Here is the caller graph for this function:



9.33.3.57 solutionForA()

```
std::unordered_set< Lr0Item > SLRTutorWindow::solutionForA ()
```

Here is the caller graph for this function:



9.33.3.58 solutionForA1()

```
QString SLRTutorWindow::solutionForA1 ()
```

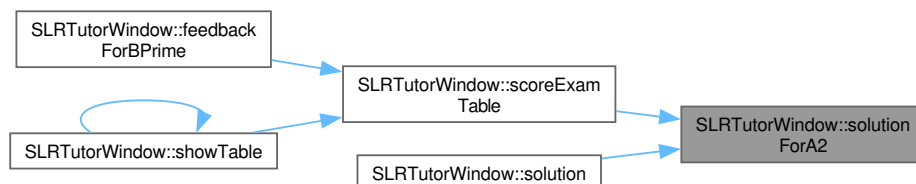
Here is the caller graph for this function:



9.33.3.59 solutionForA2()

```
QString SLRTutorWindow::solutionForA2 ()
```

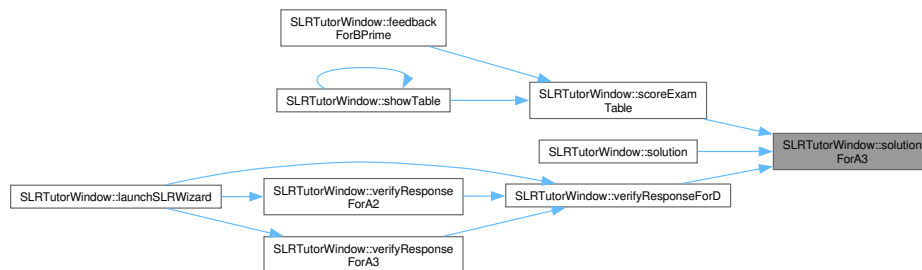
Here is the caller graph for this function:



9.33.3.60 solutionForA3()

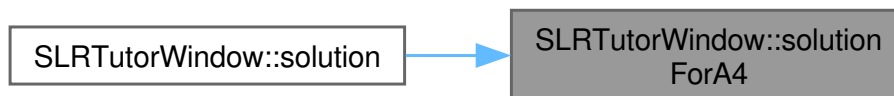
```
std::vector< std::pair< std::string, std::vector< std::string > > > SLRTutorWindow::solutionForA3 ()
```

Here is the caller graph for this function:

**9.33.3.61 solutionForA4()**

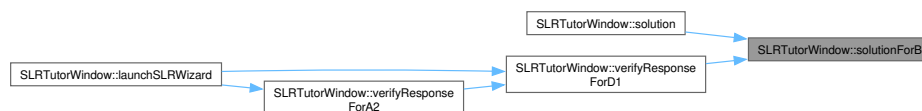
```
std::unordered_set< Lr0Item > SLRTutorWindow::solutionForA4 ()
```

Here is the caller graph for this function:

**9.33.3.62 solutionForB()**

```
unsigned SLRTutorWindow::solutionForB ()
```

Here is the caller graph for this function:

**9.33.3.63 solutionForC()**

```
unsigned SLRTutorWindow::solutionForC ()
```

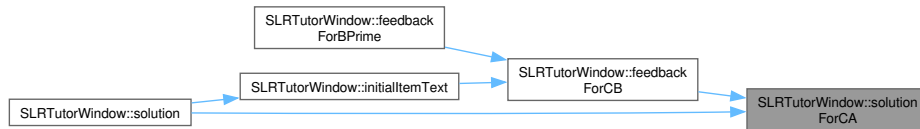
Here is the caller graph for this function:



9.33.3.64 solutionForCA()

```
QStringList SLRTutorWindow::solutionForCA ()
```

Here is the caller graph for this function:



9.33.3.65 solutionForCB()

```
std::unordered_set< Lr0Item > SLRTutorWindow::solutionForCB ()
```

Here is the caller graph for this function:



9.33.3.66 solutionForD()

```
QStringList SLRTutorWindow::solutionForD ()
```

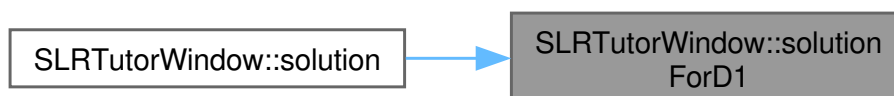
Here is the caller graph for this function:



9.33.3.67 solutionForD1()

```
QString SLRTutorWindow::solutionForD1 ()
```

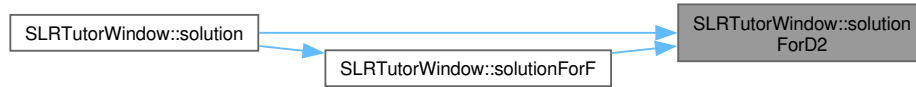
Here is the caller graph for this function:



9.33.3.68 solutionForD2()

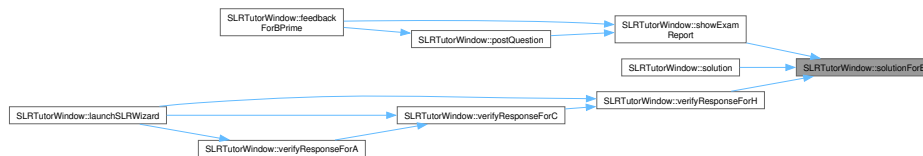
```
QString SLRTutorWindow::solutionForD2 ()
```

Here is the caller graph for this function:

**9.33.3.69 solutionForE()**

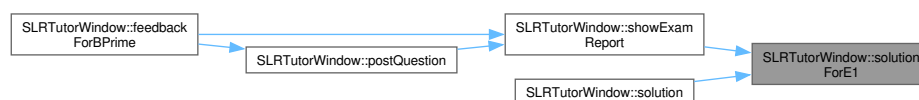
```
std::ptrdiff_t SLRTutorWindow::solutionForE ()
```

Here is the caller graph for this function:

**9.33.3.70 solutionForE1()**

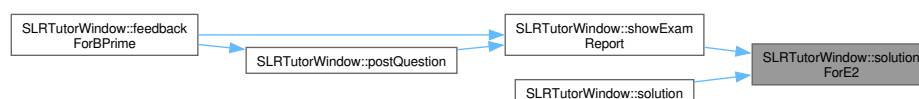
```
QSet< unsigned > SLRTutorWindow::solutionForE1 ()
```

Here is the caller graph for this function:

**9.33.3.71 solutionForE2()**

```
QMap< unsigned, unsigned > SLRTutorWindow::solutionForE2 ()
```

Here is the caller graph for this function:



9.33.3.72 solutionForF()

```
QSet< unsigned > SLRTutorWindow::solutionForF ()
```

Here is the call graph for this function:



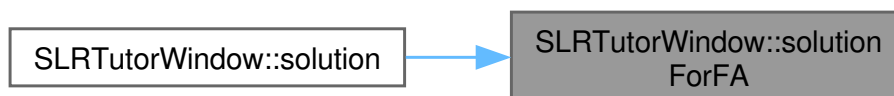
Here is the caller graph for this function:



9.33.3.73 solutionForFA()

```
QSet< QString > SLRTutorWindow::solutionForFA ()
```

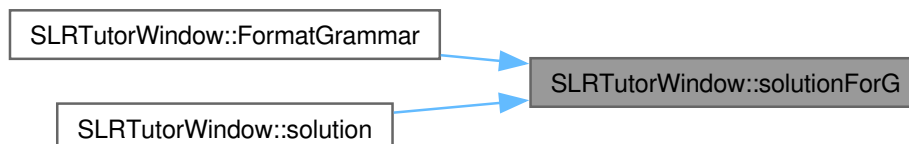
Here is the caller graph for this function:



9.33.3.74 solutionForG()

```
QSet< QString > SLRTutorWindow::solutionForG ()
```

Here is the caller graph for this function:

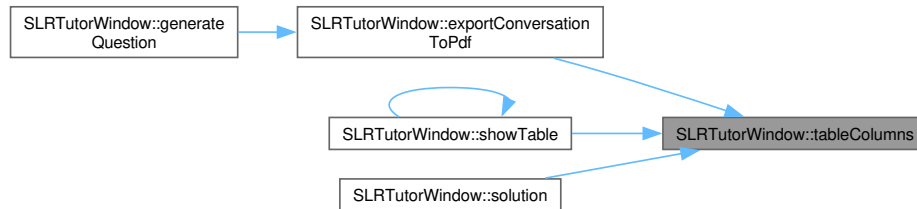


9.33.3.75 tableColumns()

```
QStringList SLRTutorWindow::tableColumns ()
```

Columns of the SLR(1) table: terminals, \$, non-terminals.

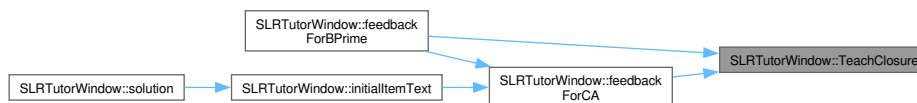
Here is the caller graph for this function:



9.33.3.76 TeachClosure()

```
QString SLRTutorWindow::TeachClosure (
    const std::unordered_set< Lr0Item > & initialItems)
```

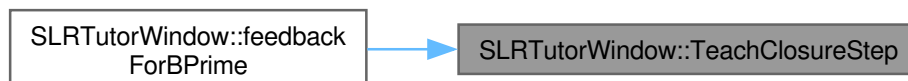
Here is the caller graph for this function:



9.33.3.77 TeachClosureStep()

```
void SLRTutorWindow::TeachClosureStep (
    std::unordered_set< Lr0Item > & items,
    unsigned int size,
    std::unordered_set< std::string > & visited,
    int depth,
    QString & output)
```

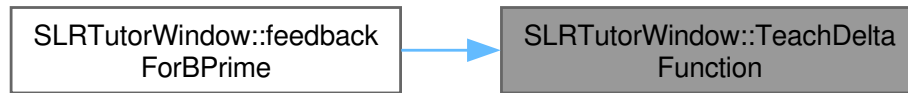
Here is the caller graph for this function:



9.33.3.78 TeachDeltaFunction()

```
QString SLRTutorWindow::TeachDeltaFunction (
    const std::unordered_set< Lr0Item > & items,
    const QString & symbol)
```

Here is the caller graph for this function:



9.33.3.79 updateAutomatonButton()

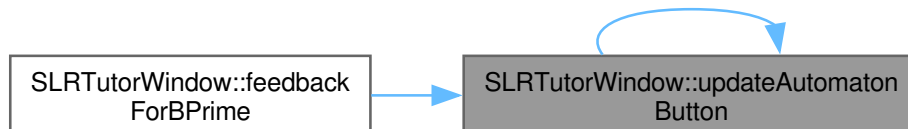
```
void SLRTutorWindow::updateAutomatonButton ()
```

Enables the viewer button once at least one state (l0) exists.

Here is the call graph for this function:



Here is the caller graph for this function:

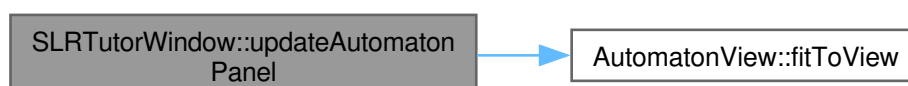


9.33.3.80 updateAutomatonPanel()

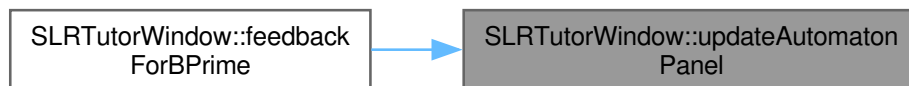
```
void SLRTutorWindow::updateAutomatonPanel ()
```

Syncs the automaton view with the current tutor state.

Highlights the state under analysis during the C/CA/CB loop, switches to full consultation mode from D onwards, and marks conflict (F/FA) and reduce (G) states. Updates apply whether or not the viewer is open. Here is the call graph for this function:



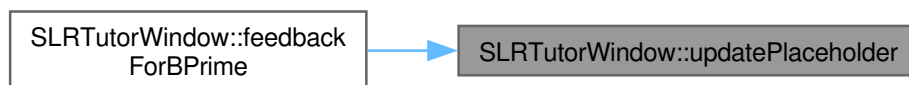
Here is the caller graph for this function:



9.33.3.81 updatePlaceholder()

```
void SLRTutorWindow::updatePlaceholder ()
```

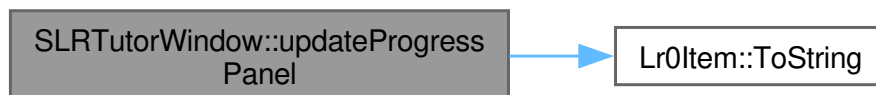
Here is the caller graph for this function:



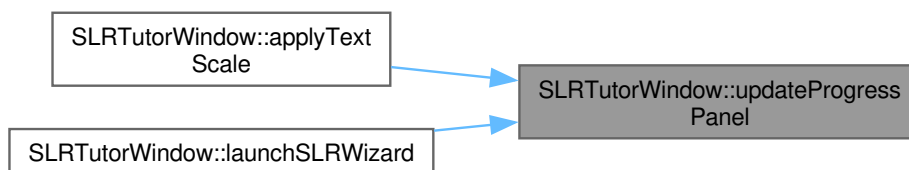
9.33.3.82 updateProgressPanel()

```
void SLRTutorWindow::updateProgressPanel ()
```

Here is the call graph for this function:



Here is the caller graph for this function:



9.33.3.83 updateState()

```
void SLRTutorWindow::updateState (
    bool isCorrect)
```

Updates tutor state based on whether the last answer was correct.

Parameters

<i>isCorrect</i>	Whether the user's answer was correct.
------------------	--

9.33.3.84 verifyResponse()

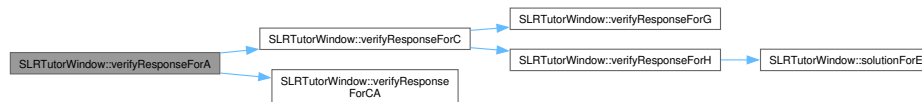
```
bool SLRTutorWindow::verifyResponse (
    const QString & userResponse)
```

Here is the caller graph for this function:

**9.33.3.85 verifyResponseForA()**

```
bool SLRTutorWindow::verifyResponseForA (
    const QString & userResponse)
```

Here is the call graph for this function:



Here is the caller graph for this function:

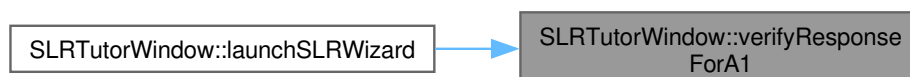
**9.33.3.86 verifyResponseForA1()**

```
bool SLRTutorWindow::verifyResponseForA1 (
    const QString & userResponse)
```

Here is the call graph for this function:



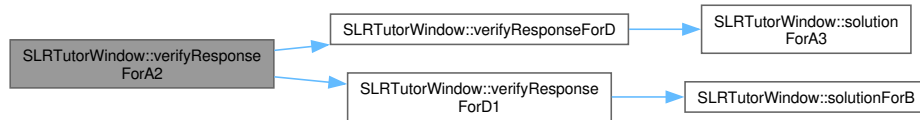
Here is the caller graph for this function:



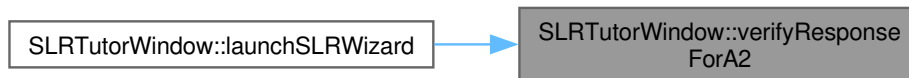
9.33.3.87 verifyResponseForA2()

```
bool SLRTutorWindow::verifyResponseForA2 (
    const QString & userResponse)
```

Here is the call graph for this function:

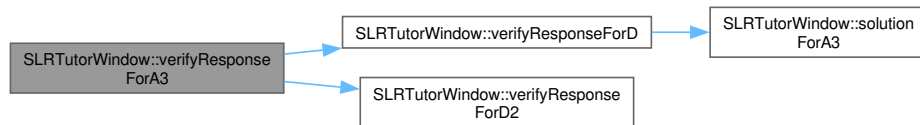


Here is the caller graph for this function:

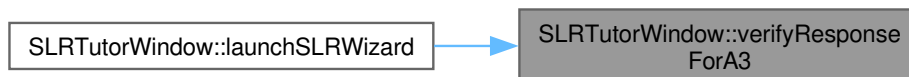
**9.33.3.88 verifyResponseForA3()**

```
bool SLRTutorWindow::verifyResponseForA3 (
    const QString & userResponse)
```

Here is the call graph for this function:

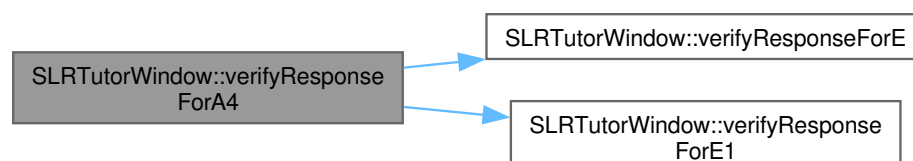


Here is the caller graph for this function:

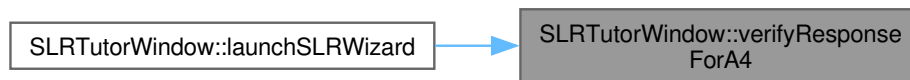
**9.33.3.89 verifyResponseForA4()**

```
bool SLRTutorWindow::verifyResponseForA4 (
    const QString & userResponse)
```

Here is the call graph for this function:



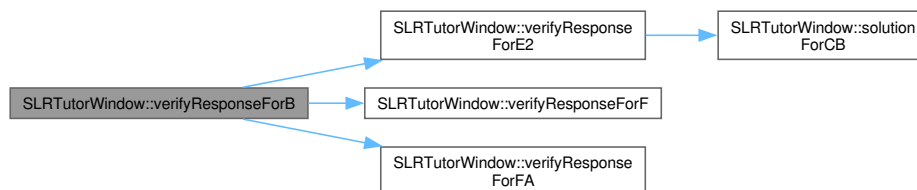
Here is the caller graph for this function:



9.33.3.90 verifyResponseForB()

```
bool SLRTutorWindow::verifyResponseForB (
    const QString & userResponse)
```

Here is the call graph for this function:



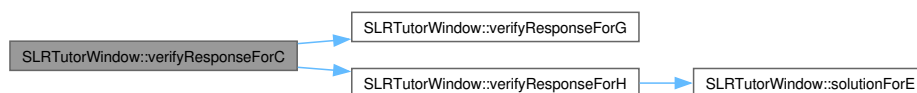
Here is the caller graph for this function:



9.33.3.91 verifyResponseForC()

```
bool SLRTutorWindow::verifyResponseForC (
    const QString & userResponse)
```

Here is the call graph for this function:



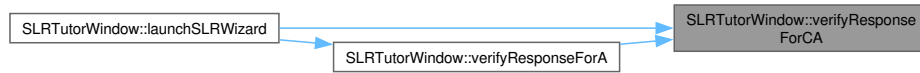
Here is the caller graph for this function:



9.33.3.92 verifyResponseForCA()

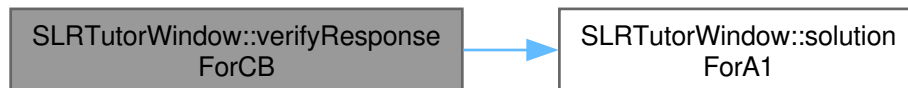
```
bool SLRTutorWindow::verifyResponseForCA (
    const QString & userResponse)
```

Here is the caller graph for this function:

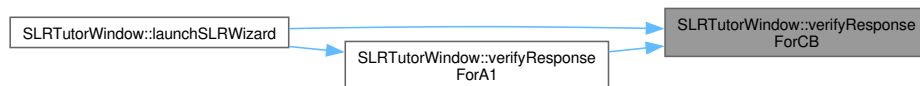
**9.33.3.93 verifyResponseForCB()**

```
bool SLRTutorWindow::verifyResponseForCB (
    const QString & userResponse)
```

Here is the call graph for this function:



Here is the caller graph for this function:

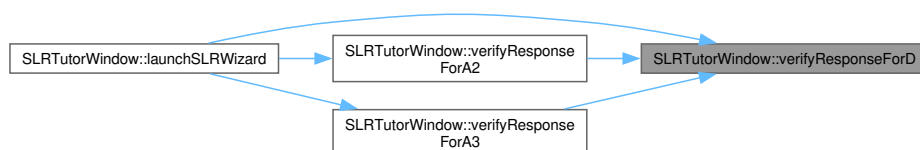
**9.33.3.94 verifyResponseForD()**

```
bool SLRTutorWindow::verifyResponseForD (
    const QString & userResponse)
```

Here is the call graph for this function:



Here is the caller graph for this function:



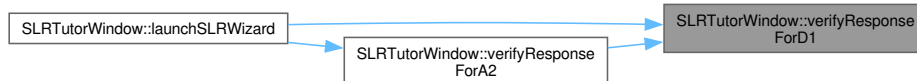
9.33.3.95 verifyResponseForD1()

```
bool SLRTutorWindow::verifyResponseForD1 (
    const QString & userResponse)
```

Here is the call graph for this function:



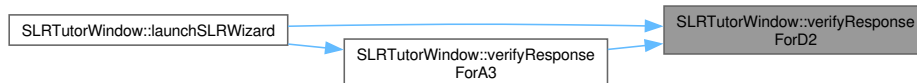
Here is the caller graph for this function:



9.33.3.96 verifyResponseForD2()

```
bool SLRTutorWindow::verifyResponseForD2 (
    const QString & userResponse)
```

Here is the caller graph for this function:



9.33.3.97 verifyResponseForE()

```
bool SLRTutorWindow::verifyResponseForE (
    const QString & userResponse)
```

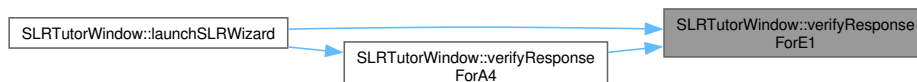
Here is the caller graph for this function:



9.33.3.98 verifyResponseForE1()

```
bool SLRTutorWindow::verifyResponseForE1 (
    const QString & userResponse)
```

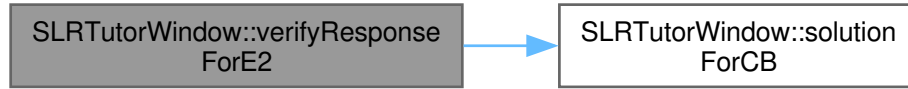
Here is the caller graph for this function:



9.33.3.99 verifyResponseForE2()

```
bool SLRTutorWindow::verifyResponseForE2 (
    const QString & userResponse)
```

Here is the call graph for this function:

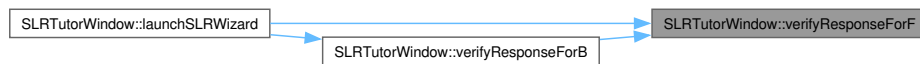


Here is the caller graph for this function:

**9.33.3.100 verifyResponseForF()**

```
bool SLRTutorWindow::verifyResponseForF (
    const QString & userResponse)
```

Here is the caller graph for this function:

**9.33.3.101 verifyResponseForFA()**

```
bool SLRTutorWindow::verifyResponseForFA (
    const QString & userResponse)
```

Here is the caller graph for this function:

**9.33.3.102 verifyResponseForG()**

```
bool SLRTutorWindow::verifyResponseForG (
    const QString & userResponse)
```

Here is the caller graph for this function:



9.33.3.103 verifyResponseForH()

```
bool SLRTutorWindow::verifyResponseForH ()
```

Here is the call graph for this function:



Here is the caller graph for this function:

**9.33.3.104 wrongAnimation()**

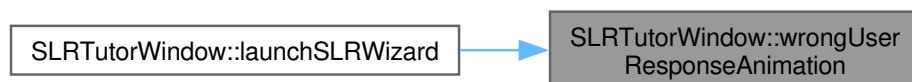
```
void SLRTutorWindow::wrongAnimation ()
```

Here is the caller graph for this function:

**9.33.3.105 wrongUserResponseAnimation()**

```
void SLRTutorWindow::wrongUserResponseAnimation ()
```

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

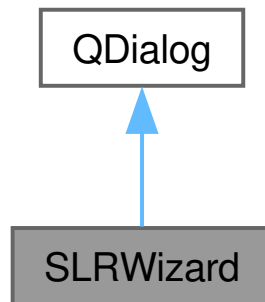
- [gui/slrtutorwindow.h](#)
- [gui/slrtutorwindow.cpp](#)

9.34 SLRWizard Class Reference

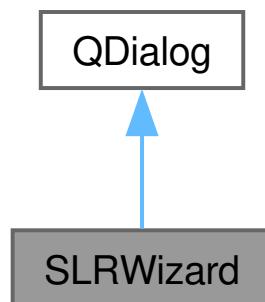
Interactive assistant that guides the student step-by-step through the SLR(1) parsing table.

```
#include <slrwizard.h>
```

Inheritance diagram for SLRWizard:



Collaboration diagram for SLRWizard:



Public Member Functions

- [SLRWizard](#) ([SLR1Parser](#) &parser, const QVector< QVector< QString > > &rawTable, const QString↵
List &colHeaders, const QVector< QPair< QString, QVector< QString > > > &sortedGrammar, QWidget
*parent=nullptr)
Constructs the SLR(1) guided dialog with all necessary parsing context.
- [SLRWizardPage](#) * [currentPage](#) () const
- QPushButton * [nextButton](#) () const
- QPushButton * [closeButton](#) () const
- bool [isOnLastPage](#) () const
- QVector< QString > [stdVectorToQVector](#) (const std::vector< std::string > &vec)
Converts a std::vector<std::string> to QVector<QString> for UI compatibility.

9.34.1 Detailed Description

Interactive assistant that guides the student step-by-step through the SLR(1) parsing table.

This dialog presents one cell of the SLR(1) parsing table at a time, asking the user to deduce the correct ACTION or GOTO entry based on the LR(0) automaton and FOLLOW sets. It intentionally avoids the native wizard widget to keep a consistent look across platforms.

9.34.2 Constructor & Destructor Documentation

9.34.2.1 SLRWizard()

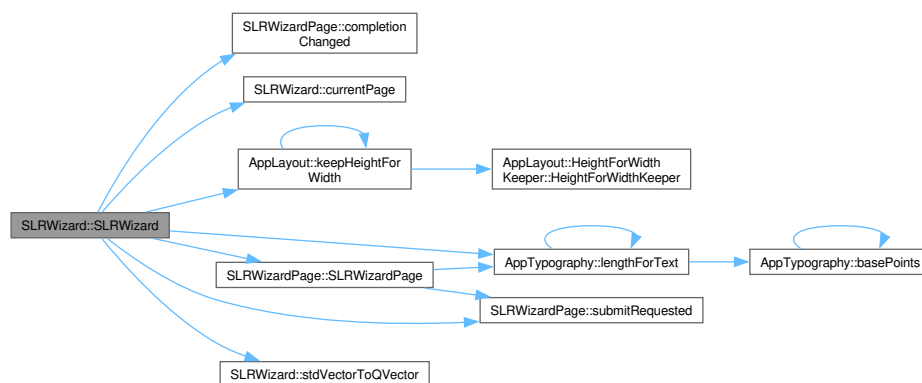
```
SLRWizard::SLRWizard (
    SLR1Parser & parser,
    const QVector< QVector< QString > > & rawTable,
    const QStringList & colHeaders,
    const QVector< QPair< QString, QVector< QString > > > & sortedGrammar,
    QWidget * parent = nullptr) [inline]
```

Constructs the SLR(1) guided dialog with all necessary parsing context.

Parameters

<i>parser</i>	The SLR(1) parser instance containing the LR(0) states and transitions.
<i>rawTable</i>	The target parsing table (student version or reference).
<i>colHeaders</i>	Header symbols (terminals and non-terminals).
<i>sortedGrammar</i>	Ordered list of grammar rules for reduce explanations.
<i>parent</i>	Parent widget.

Here is the call graph for this function:



Here is the caller graph for this function:



9.34.3 Member Function Documentation

9.34.3.1 closeButton()

```
QPushButton * SLRWizard::closeButton () const [inline]
```

9.34.3.2 currentPage()

```
SLRWizardPage * SLRWizard::currentPage () const [inline]
```

Here is the caller graph for this function:



9.34.3.3 isOnLastPage()

```
bool SLRWizard::isOnLastPage () const [inline]
```

9.34.3.4 nextButton()

```
QPushButton * SLRWizard::nextButton () const [inline]
```

9.34.3.5 stdVectorToQVector()

```
QVector< QString > SLRWizard::stdVectorToQVector (
    const std::vector< std::string > & vec) [inline]
```

Converts a `std::vector<std::string>` to `QVector<QString>` for UI compatibility.

Parameters

<code>vec</code>	The input vector of strings.
------------------	------------------------------

Returns

A `QVector` of `QString`s.

Here is the caller graph for this function:



The documentation for this class was generated from the following file:

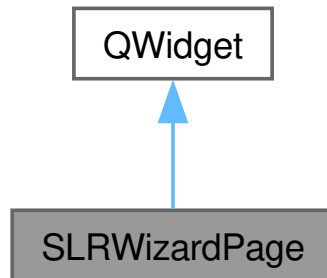
- [gui/slrwizard.h](#)

9.35 SLRWizardPage Class Reference

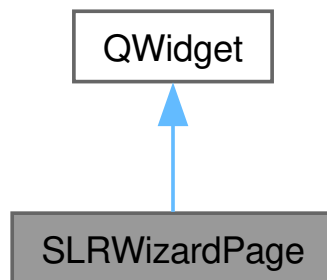
A single guided step for constructing the SLR(1) table.

```
#include <slrwizardpage.h>
```

Inheritance diagram for SLRWizardPage:



Collaboration diagram for SLRWizardPage:



Signals

- void [completionChanged](#) (bool complete)
- void [submitRequested](#) ()

Public Member Functions

- [SLRWizardPage](#) (int state, const QString &symbol, const QString &explanation, const QString &expected, QWidget *parent=nullptr)
Constructs a page for a specific cell in the SLR(1) table.
- QString [titleText](#) () const
- bool [isComplete](#) () const
- void [focusAnswerField](#) ()

Protected Member Functions

- void [resizeEvent](#) (QResizeEvent *event) override

9.35.1 Detailed Description

A single guided step for constructing the SLR(1) table.

This widget presents one (state, symbol) question at a time with a compact explanation card and an answer card. It validates the current input and emits a signal whenever the completion state changes.

9.35.2 Constructor & Destructor Documentation

9.35.2.1 SLRWizardPage()

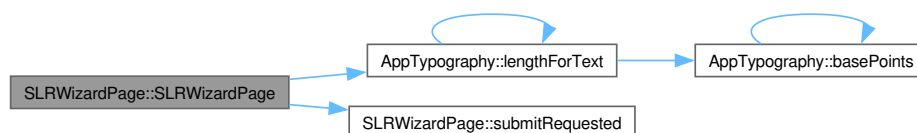
```
SLRWizardPage::SLRWizardPage (
    int state,
    const QString & symbol,
    const QString & explanation,
    const QString & expected,
    QWidget * parent = nullptr) [inline]
```

Constructs a page for a specific cell in the SLR(1) table.

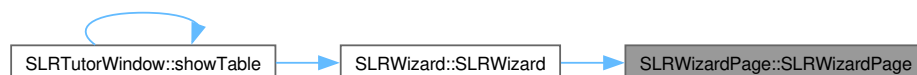
Parameters

<i>state</i>	The state ID (row index in the table).
<i>symbol</i>	The grammar symbol (column header).
<i>explanation</i>	A pedagogical explanation shown to the user.
<i>expected</i>	The expected answer (e.g., "s2", "r1", "acc", or a state number).
<i>parent</i>	The parent widget.

Here is the call graph for this function:



Here is the caller graph for this function:



9.35.3 Member Function Documentation

9.35.3.1 completionChanged

```
void SLRWizardPage::completionChanged (
    bool complete) [signal]
```

Here is the caller graph for this function:



9.35.3.2 focusAnswerField()

```
void SLRWizardPage::focusAnswerField () [inline]
```

9.35.3.3 isComplete()

```
bool SLRWizardPage::isComplete () const [inline]
```

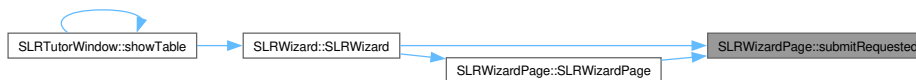
9.35.3.4 resizeEvent()

```
void SLRWizardPage::resizeEvent (
    QResizeEvent * event) [inline], [override], [protected]
```

9.35.3.5 submitRequested

```
void SLRWizardPage::submitRequested () [signal]
```

Here is the caller graph for this function:



9.35.3.6 titleText()

```
QString SLRWizardPage::titleText () const [inline]
```

The documentation for this class was generated from the following file:

- [gui/slrwizardpage.h](#)

9.36 state Struct Reference

Represents a state in the LR(0) automaton.

```
#include <state.hpp>
```

Public Member Functions

- bool `operator==` (const `state` &other) const
Equality operator for comparing states based on their items.

Public Attributes

- std::unordered_set< `Lr0Item` > `items_`
The set of LR(0) items that make up this state.
- unsigned int `id_`
Unique identifier of the state.

9.36.1 Detailed Description

Represents a state in the LR(0) automaton.

Each state consists of a unique identifier and a set of LR(0) items that define its core. States are used to build the SLR(1) parsing table.

9.36.2 Member Function Documentation

9.36.2.1 `operator==()`

```
bool state::operator== (
    const state & other) const [inline]
```

Equality operator for comparing states based on their items.

Parameters

<i>other</i>	The state to compare with.
--------------	----------------------------

Returns

true if both states have the same item set; false otherwise.

9.36.3 Member Data Documentation

9.36.3.1 `id_`

```
unsigned int state::id_
```

Unique identifier of the state.

9.36.3.2 items_

```
std::unordered_set<Lr0Item> state::items_
```

The set of LR(0) items that make up this state.

The documentation for this struct was generated from the following file:

- backend/[state.hpp](#)

9.37 SymbolTable Struct Reference

Stores and manages grammar symbols, including their classification and special markers.

```
#include <symbol_table.hpp>
```

Public Member Functions

- void [PutSymbol](#) (const std::string &identifier, bool isTerminal)
Adds a symbol to the symbol table.
- bool [In](#) (const std::string &s) const
Checks if a symbol exists in the symbol table.
- bool [IsTerminal](#) (const std::string &s) const
Checks if a symbol is a terminal.
- bool [IsTerminalWthoEol](#) (const std::string &s) const
Checks if a symbol is a terminal excluding EOL.
- bool [IsNonTerminal](#) (const std::string &s) const
Checks if a symbol is a non-terminal.
- bool [IsMeta](#) (const std::string &s) const
Checks if a symbol is a meta symbol (e.g., EPSILON).

Public Attributes

- std::string [EOL_](#) {"\$"}
End-of-line symbol used in parsing, initialized as "\$".
- std::string [EPSILON_](#) {"EPSILON"}
Epsilon symbol, representing empty transitions, initialized as "EPSILON".
- std::unordered_map< std::string, [symbol_type](#) > [st_](#)
Main symbol table, mapping identifiers to their symbol type.
- std::unordered_set< std::string > [terminals_](#) {[EOL_](#)}
Set of all terminal symbols (including EOL).
- std::unordered_set< std::string > [terminals_wtho_eol_](#) {}
Set of terminal symbols excluding the EOL symbol (\$).
- std::unordered_set< std::string > [non_terminals_](#)
Set of all non-terminal symbols.
- std::unordered_set< std::string > [meta_symbols_](#) {[EPSILON_](#)}
Set of meta symbols (e.g., EPSILON).

9.37.1 Detailed Description

Stores and manages grammar symbols, including their classification and special markers.

This structure holds information about all terminals and non-terminals used in a grammar, as well as special symbols such as EPSILON (meta symbol) and the end-of-line marker (\$). It supports symbol classification, membership checks, and filtered views such as terminals excluding \$.

9.37.2 Member Function Documentation

9.37.2.1 In()

```
bool SymbolTable::In (
    const std::string & s) const
```

Checks if a symbol exists in the symbol table.

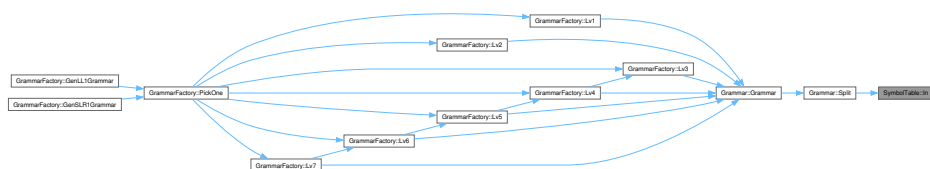
Parameters

s	Symbol identifier to search.
----------	------------------------------

Returns

true if the symbol is present, otherwise false.

Here is the caller graph for this function:



9.37.2.2 IsMeta()

```
bool SymbolTable::IsMeta (
    const std::string & s) const
```

Checks if a symbol is a meta symbol (e.g., EPSILON).

Parameters

s	Symbol identifier to check.
----------	-----------------------------

Returns

true if the symbol is meta, otherwise false.

9.37.2.3 IsNonTerminal()

```
bool SymbolTable::IsNonTerminal (
    const std::string & s) const
```

Checks if a symbol is a non-terminal.

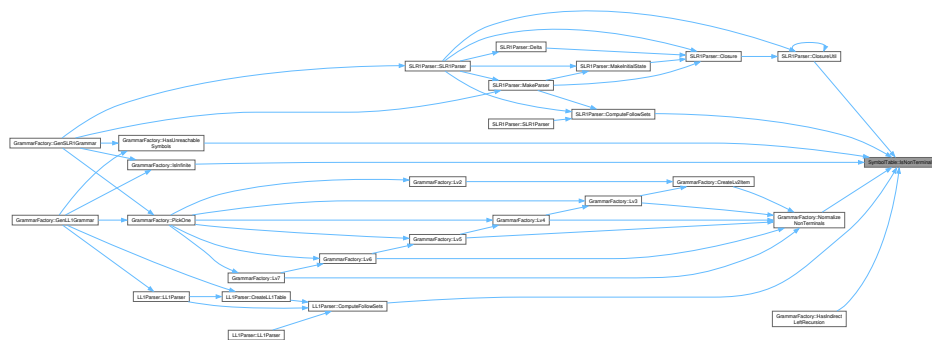
Parameters

s	Symbol identifier to check.
---	-----------------------------

Returns

true if the symbol is non-terminal, otherwise false.

Here is the caller graph for this function:



9.37.2.4 IsTerminal()

```
bool SymbolTable::IsTerminal (
    const std::string & s) const
```

Checks if a symbol is a terminal.

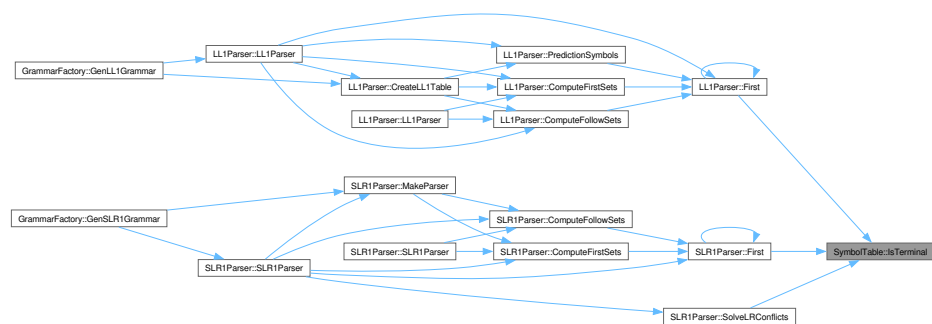
Parameters

s	Symbol identifier to check.
---	-----------------------------

Returns

true if the symbol is terminal, otherwise false.

Here is the caller graph for this function:



9.37.2.5 IsTerminalWthoEol()

```
bool SymbolTable::IsTerminalWthoEol (
    const std::string & s) const
```

Checks if a symbol is a terminal excluding EOL.

Parameters

s	Symbol identifier to check.
---	-----------------------------

Returns

true if the symbol is terminal, otherwise false.

9.37.2.6 PutSymbol()

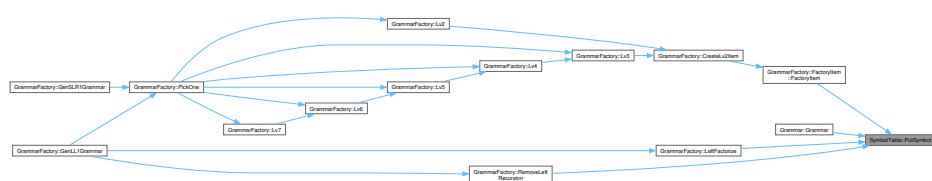
```
void SymbolTable::PutSymbol (
    const std::string & identifier,
    bool isTerminal)
```

Adds a symbol to the symbol table.

Parameters

<i>identifier</i>	Name of the symbol.
<i>isTerminal</i>	True if the identifier is a terminal symbol.

Here is the caller graph for this function:



9.37.3 Member Data Documentation

9.37.3.1 EOL_

```
std::string SymbolTable::EOL_ {"$"};
```

End-of-line symbol used in parsing, initialized as "\$".

9.37.3.2 EPSILON_

```
std::string SymbolTable::EPSILON_ {"EPSILON"}
```

Epsilon symbol, representing empty transitions, initialized as "EPSILON".

9.37.3.3 meta_symbols_

```
std::unordered_set<std::string> SymbolTable::meta_symbols_ {EPSILON_}
```

Set of meta symbols (e.g., EPSILON).

9.37.3.4 non_terminals_

```
std::unordered_set<std::string> SymbolTable::non_terminals_
```

Set of all non-terminal symbols.

9.37.3.5 st_

```
std::unordered_map<std::string, symbol_type> SymbolTable::st_
```

Initial value:

```
{  
    {EOL_, symbol_type::TERMINAL}, {EPSILON_, symbol_type::META}}  
}
```

Main symbol table, mapping identifiers to their symbol type.

9.37.3.6 terminals_

```
std::unordered_set<std::string> SymbolTable::terminals_ {EOL_}
```

Set of all terminal symbols (including EOL).

9.37.3.7 terminals_wtho_eol_

```
std::unordered_set<std::string> SymbolTable::terminals_wtho_eol_ {}
```

Set of terminal symbols excluding the EOL symbol (\$).

The documentation for this struct was generated from the following files:

- [backend/symbol_table.hpp](#)
- [backend/symbol_table.cpp](#)

9.38 AppTypography::Token Struct Reference

One entry of the type scale.

```
#include <apptypography.h>
```

Public Attributes

- [Role](#) `role`
- `QLatin1StringView` `name`
Name used in app.qss after \$font-.
- `qreal` `points`
Size at a 100% text scale.

9.38.1 Detailed Description

One entry of the type scale.

9.38.2 Member Data Documentation

9.38.2.1 `name`

```
QLatin1StringView AppTypography::Token::name
```

Name used in app.qss after \$font-.

9.38.2.2 `points`

```
qreal AppTypography::Token::points
```

Size at a 100% text scale.

9.38.2.3 `role`

```
Role AppTypography::Token::role
```

The documentation for this struct was generated from the following file:

- `widgets/apptypography.h`

9.39 LLTutorWindow::TreeNode Struct Reference

[TreeNode](#) structure used to build derivation trees.

```
#include <lltutorwindow.h>
```

Public Attributes

- QString [label](#)
- std::vector< std::unique_ptr< [TreeNode](#) > > [children](#)

9.39.1 Detailed Description

[TreeNode](#) structure used to build derivation trees.

9.39.2 Member Data Documentation

9.39.2.1 children

```
std::vector<std::unique_ptr<TreeNode> > LLTutorWindow::TreeNode::children
```

9.39.2.2 label

```
QString LLTutorWindow::TreeNode::label
```

The documentation for this struct was generated from the following file:

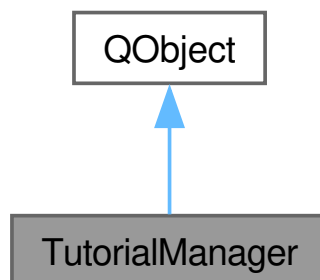
- [gui/lltutorwindow.h](#)

9.40 TutorialManager Class Reference

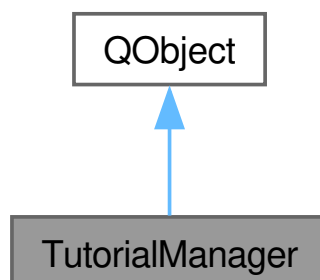
Manages interactive tutorials by highlighting UI elements and guiding the user.

```
#include <tutorialmanager.h>
```

Inheritance diagram for TutorialManager:



Collaboration diagram for TutorialManager:



Public Slots

- void `nextStep` ()
Advances to the next tutorial step.

Signals

- void `stepStarted` (int index)
Emitted when a new tutorial step starts.
- void `tutorialFinished` ()
Emitted when the full tutorial is finished.
- void `ll1Finished` ()
Emitted when the LL(1) tutorial ends.
- void `slr1Finished` ()
Emitted when the SLR(1) tutorial ends.

Public Member Functions

- `TutorialManager` (QWidget *rootWindow)
Constructs a `TutorialManager` for a given window.
- void `addStep` (QWidget *target, const QString &htmlText)
Adds a new step to the tutorial sequence.
- void `start` ()
Starts the tutorial from the beginning.
- void `setRootWindow` (QWidget *newRoot)
Sets the root window (used for repositioning the overlay).
- void `clearSteps` ()
Clears all steps in the tutorial.
- void `hideOverlay` ()
Hides the tutorial overlay immediately.
- void `finishLL1` ()
Ends the LL(1) tutorial sequence and emits its corresponding signal.
- void `finishSLR1` ()
Ends the SLR(1) tutorial sequence and emits its corresponding signal.

Protected Member Functions

- bool `eventFilter` (QObject *obj, QEvent *ev) override
Intercepts UI events to handle overlay behavior.

9.40.1 Detailed Description

Manages interactive tutorials by highlighting UI elements and guiding the user.

This class implements a step-by-step overlay system that visually highlights widgets and shows textual instructions to guide the user through the interface. It supports multiple tutorials (e.g., for LL(1) and SLR(1) modes), with custom steps and signals for tutorial completion.

9.40.2 Constructor & Destructor Documentation

9.40.2.1 TutorialManager()

```
TutorialManager::TutorialManager (
    QWidget * rootWindow)
```

Constructs a [TutorialManager](#) for a given window.

Parameters

<i>rootWindow</i>	The main application window used for relative positioning.
-------------------	--

9.40.3 Member Function Documentation

9.40.3.1 addStep()

```
void TutorialManager::addStep (
    QWidget * target,
    const QString & htmlText)
```

Adds a new step to the tutorial sequence.

Parameters

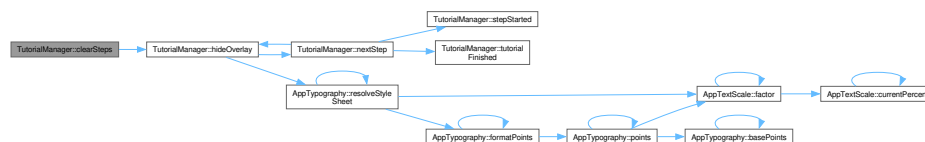
<i>target</i>	The widget to highlight during the step.
<i>htmlText</i>	The instructional HTML message for the step.

9.40.3.2 clearSteps()

```
void TutorialManager::clearSteps ()
```

Clears all steps in the tutorial.

Here is the call graph for this function:



9.40.3.3 eventFilter()

```
bool TutorialManager::eventFilter (
    QObject * obj,
    QEvent * ev) [override], [protected]
```

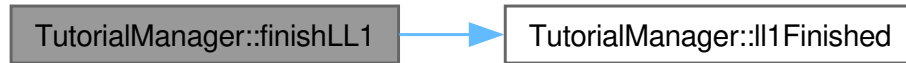
Intercepts UI events to handle overlay behavior.

9.40.3.4 finishLL1()

```
void TutorialManager::finishLL1 ()
```

Ends the LL(1) tutorial sequence and emits its corresponding signal.

Here is the call graph for this function:



9.40.3.5 finishSLR1()

```
void TutorialManager::finishSLR1 ()
```

Ends the SLR(1) tutorial sequence and emits its corresponding signal.

Here is the call graph for this function:

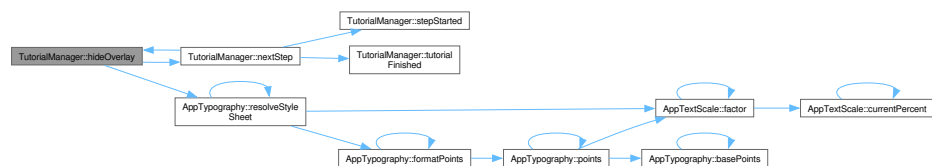


9.40.3.6 hideOverlay()

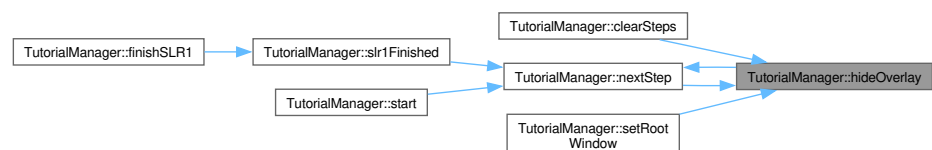
```
void TutorialManager::hideOverlay ()
```

Hides the tutorial overlay immediately.

Here is the call graph for this function:



Here is the caller graph for this function:

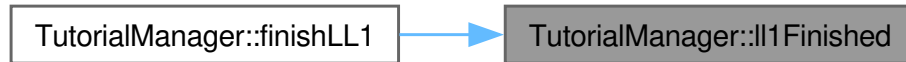


9.40.3.7 ll1Finished

```
void TutorialManager::ll1Finished () [signal]
```

Emitted when the LL(1) tutorial ends.

Here is the caller graph for this function:



9.40.3.8 nextStep

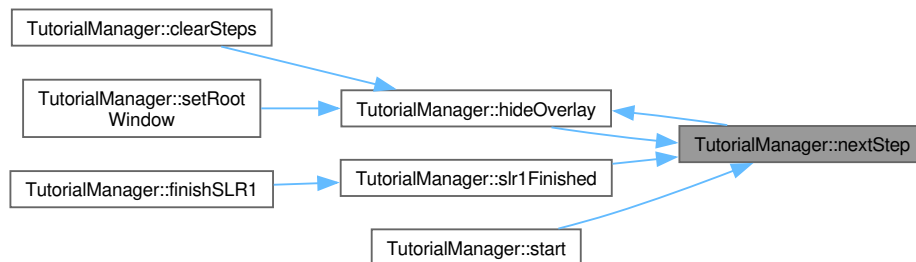
```
void TutorialManager::nextStep () [slot]
```

Advances to the next tutorial step.

Here is the call graph for this function:



Here is the caller graph for this function:



9.40.3.9 setRootWindow()

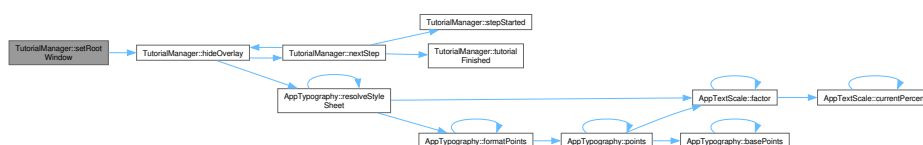
```
void TutorialManager::setRootWindow (
    QWidget * newRoot)
```

Sets the root window (used for repositioning the overlay).

Parameters

<i>newRoot</i>	The new main window to reference.
----------------	-----------------------------------

Here is the call graph for this function:

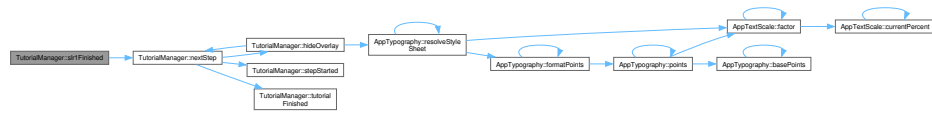


9.40.3.10 slr1Finished

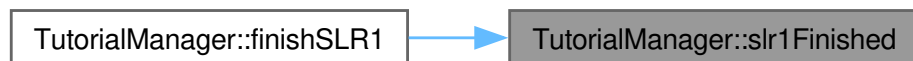
```
void TutorialManager::slr1Finished () [signal]
```

Emitted when the SLR(1) tutorial ends.

Here is the call graph for this function:



Here is the caller graph for this function:

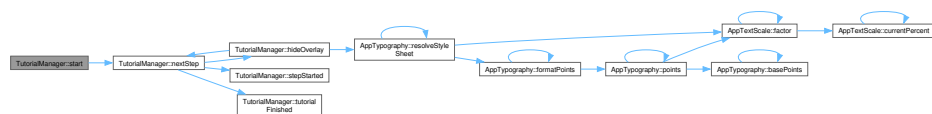


9.40.3.11 start()

```
void TutorialManager::start ()
```

Starts the tutorial from the beginning.

Here is the call graph for this function:



9.40.3.12 stepStarted

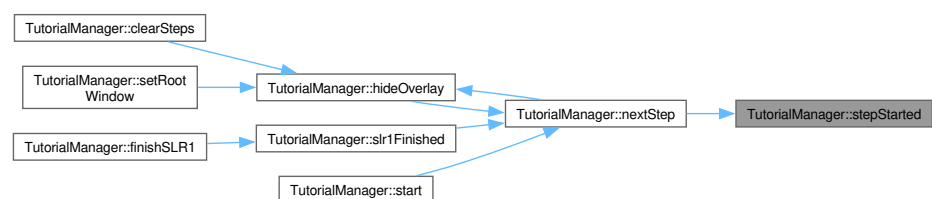
```
void TutorialManager::stepStarted (
    int index) [signal]
```

Emitted when a new tutorial step starts.

Parameters

<i>index</i>	Index of the current step.
--------------	----------------------------

Here is the caller graph for this function:

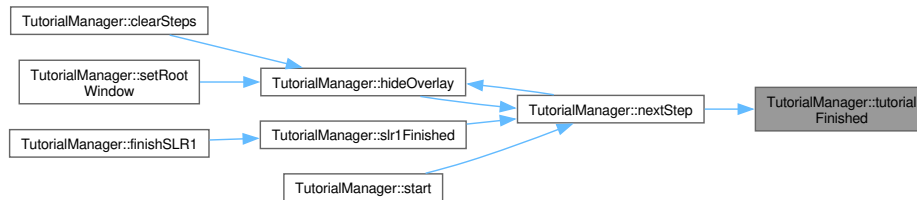


9.40.3.13 tutorialFinished

```
void TutorialManager::tutorialFinished () [signal]
```

Emitted when the full tutorial is finished.

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- widgets/[tutorialmanager.h](#)
- widgets/[tutorialmanager.cpp](#)

9.41 TutorialStep Struct Reference

Represents a single step in the tutorial sequence.

```
#include <tutorialmanager.h>
```

Public Attributes

- `QWidget *` [target](#)
Widget to highlight during the tutorial step.
- `QString` [htmlText](#)
HTML text to show as instruction or explanation.

9.41.1 Detailed Description

Represents a single step in the tutorial sequence.

Each step highlights a target widget and displays an associated HTML-formatted message.

9.41.2 Member Data Documentation

9.41.2.1 htmlText

```
QString TutorialStep::htmlText
```

HTML text to show as instruction or explanation.

9.41.2.2 target

```
QWidget* TutorialStep::target
```

Widget to highlight during the tutorial step.

The documentation for this struct was generated from the following file:

- widgets/[tutorialmanager.h](#)

9.42 UniqueQueue< T > Class Template Reference

A queue that ensures each element is inserted only once.

```
#include <UniqueQueue.h>
```

Public Member Functions

- void [push](#) (const T &value)
Pushes an element to the queue if it hasn't been inserted before.
- void [pop](#) ()
Removes the front element from the queue.
- const T & [front](#) () const
Accesses the front element of the queue.
- bool [empty](#) () const
Checks whether the queue is empty.
- void [clear](#) ()
Clears the queue and the set of seen elements.

9.42.1 Detailed Description

```
template<typename T>
class UniqueQueue< T >
```

A queue that ensures each element is inserted only once.

This data structure behaves like a standard FIFO queue but prevents duplicate insertions.

Internally, it uses a `std::queue` for ordering and a `std::unordered_set` to track seen elements.

Template Parameters

<i>T</i>	The type of elements stored in the queue. Must be hashable and comparable.
----------	--

9.42.2 Member Function Documentation

9.42.2.1 clear()

```
template<typename T>
void UniqueQueue< T >::clear () [inline]
```

Clears the queue and the set of seen elements.

9.42.2.2 empty()

```
template<typename T>
bool UniqueQueue< T >::empty () const [inline]
```

Checks whether the queue is empty.

Returns

true if the queue is empty; false otherwise.

9.42.2.3 front()

```
template<typename T>
const T & UniqueQueue< T >::front () const [inline]
```

Accesses the front element of the queue.

Returns

A reference to the front element.

9.42.2.4 pop()

```
template<typename T>
void UniqueQueue< T >::pop () [inline]
```

Removes the front element from the queue.

9.42.2.5 push()

```
template<typename T>
void UniqueQueue< T >::push (
    const T & value) [inline]
```

Pushes an element to the queue if it hasn't been inserted before.

Parameters

<i>value</i>	The element to insert.
--------------	------------------------

The documentation for this class was generated from the following file:

- [gui/UniqueQueue.h](#)

Chapter 10

File Documentation

10.1 /Users/jose/Documents/cpp/SyntaxTutor/README.md File Reference

10.2 app/main.cpp File Reference

```
#include "appversion.h"
#include "appicon.h"
#include "appsettings.h"
#include "apppalette.h"
#include "apptypography.h"
#include "mainwindow.h"
#include <QApplication>
#include <QDebug>
#include <QFontDatabase>
#include <QSettings>
#include <QStyleHints>
#include <QTranslator>
```

Include dependency graph for main.cpp:



Functions

- void [applyAppStyle](#) (QApplication &app)
- int [main](#) (int argc, char *argv[])

10.2.1 Function Documentation

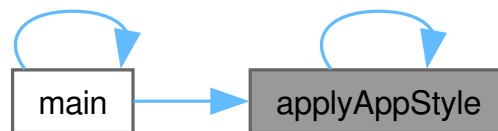
10.2.1.1 applyAppStyle()

```
void applyAppStyle (
    QApplication & app)
```

Here is the call graph for this function:



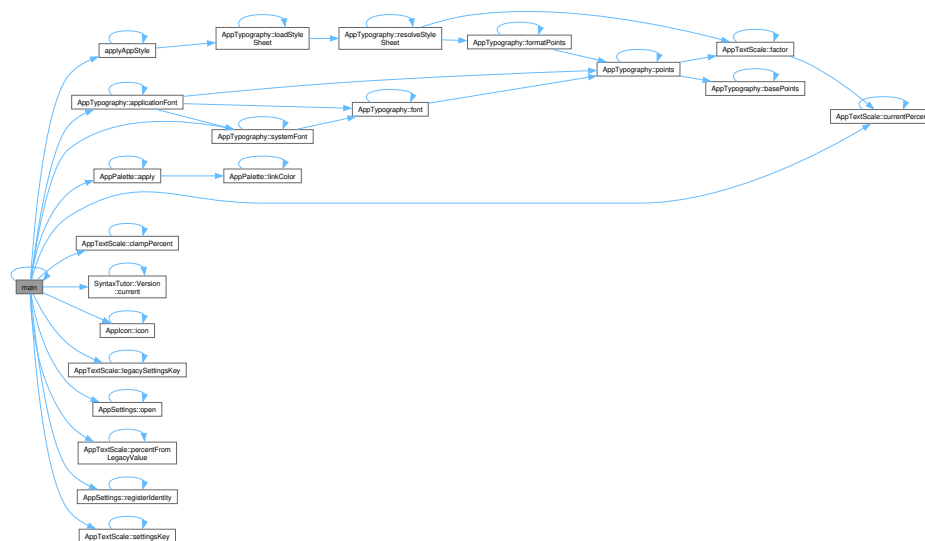
Here is the caller graph for this function:



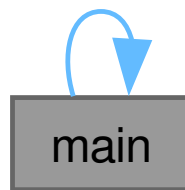
10.2.1.2 main()

```
int main (
    int argc,
    char * argv[])
```

Here is the call graph for this function:



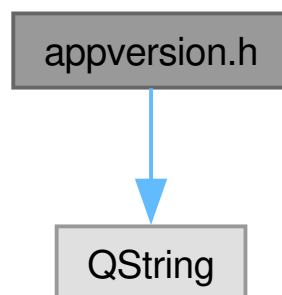
Here is the caller graph for this function:



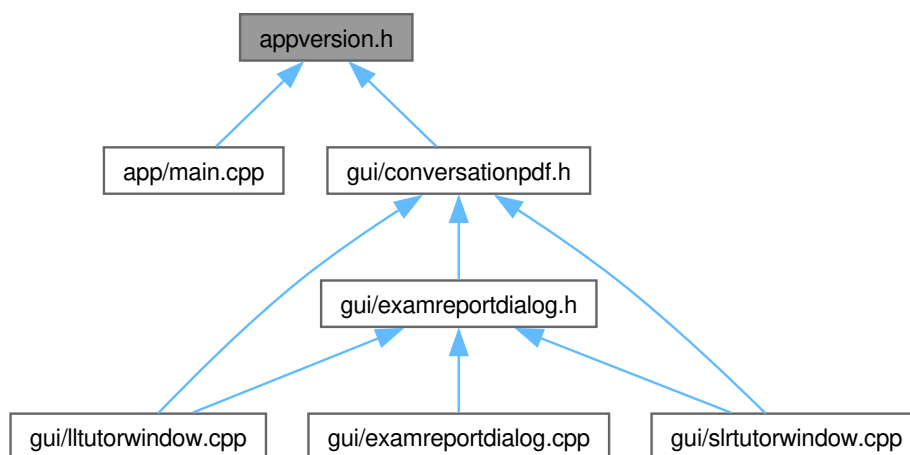
10.3 appversion.h File Reference

```
#include <QString>
```

Include dependency graph for appversion.h:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [SyntaxTutor](#)
- namespace [SyntaxTutor::Version](#)

Functions

- QString [SyntaxTutor::Version::current\(\)](#)
- const char * [SyntaxTutor::Version::raw\(\)](#)

10.4 appversion.h

[Go to the documentation of this file.](#)

```

00001 #ifndef APPVERSION_H
00002 #define APPVERSION_H
00003
00004 #include <QString>
00005
00006 namespace SyntaxTutor::Version {
00007     inline QString current() {
00008         return QStringLiteral("2.0.0");
00009     }
00010
00011     inline const char* raw() {
00012         return "2.0.0";
00013     }
00014 } // namespace SyntaxTutor::Version
00015 #endif // APPVERSION_H

```

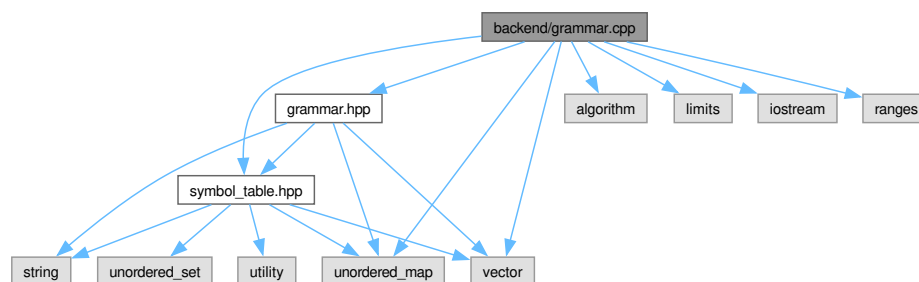
10.5 backend/grammar.cpp File Reference

```

#include "grammar.hpp"
#include "symbol_table.hpp"
#include <algorithm>
#include <limits>
#include <iostream>
#include <ranges>
#include <unordered_map>
#include <vector>

```

Include dependency graph for grammar.cpp:



10.6 backend/grammar.hpp File Reference

```

#include "symbol_table.hpp"
#include <string>
#include <unordered_map>

```


10.7 grammar.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018  #pragma once
00019  #include "symbol_table.hpp"
00020  #include <string>
00021  #include <unordered_map>
00022  #include <vector>
00023
00034  using production = std::vector<std::string>;
00035
00046  struct Grammar {
00047
00048      Grammar();
00049      explicit Grammar(
00050          const std::unordered_map<std::string, std::vector<production>&
00051              grammar);
00052
00065      int PresentationRank(const std::string& antecedent) const;
00066
00077      bool PresentationLess(const std::string& lhs,
00078                          const std::string& rhs) const;
00079
00083      std::vector<std::string> PresentationOrder() const;
00084
00093      void SetAxiom(const std::string& axiom);
00094
00105      bool HasEmptyProduction(const std::string& antecedent) const;
00106
00118      std::vector<std::pair<const std::string, production>
00119      FilterRulesByConsequent(const std::string& arg) const;
00120
00127      void Debug() const;
00128
00141      void AddProduction(const std::string& antecedent,
00142                      const std::vector<std::string>& consequent);
00143
00155      std::vector<std::string> Split(const std::string& s);
00156
00161      std::unordered_map<std::string, std::vector<production>> g_;
00162
00171      std::vector<std::string> lhs_order_;
00172
00176      std::string axiom_;
00177
00181      SymbolTable st_;
00182  };

```

10.8 backend/grammar_factory.cpp File Reference

```

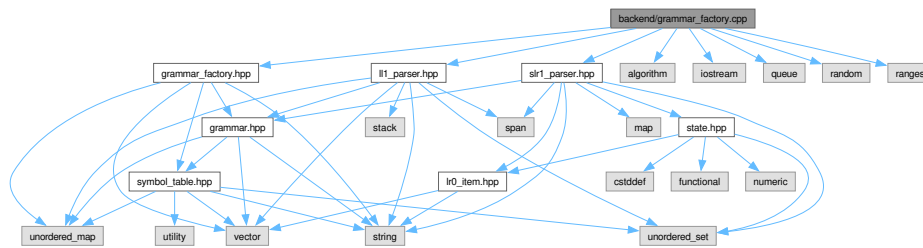
#include "grammar_factory.hpp"
#include "l1l1_parser.hpp"
#include "slr1_parser.hpp"
#include <algorithm>
#include <iostream>
#include <queue>
#include <random>

```



```
#include <ranges>
```

Include dependency graph for grammar_factory.cpp:



10.9 backend/grammar_factory.hpp File Reference

```
#include "grammar.hpp"
```

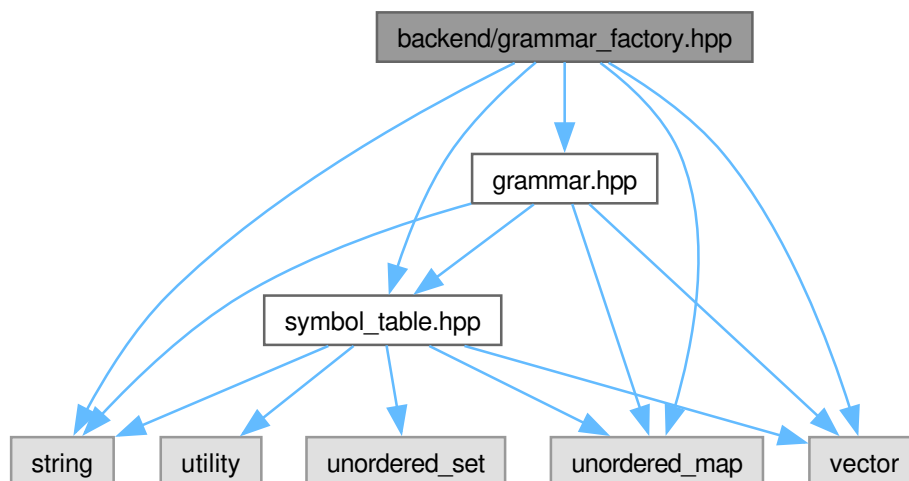
```
#include "symbol_table.hpp"
```

```
#include <string>
```

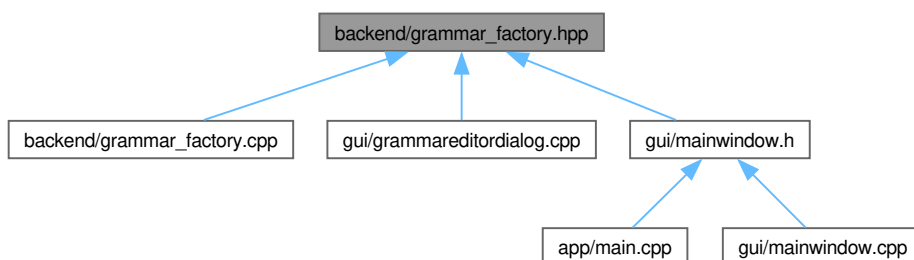
```
#include <unordered_map>
```

```
#include <vector>
```

Include dependency graph for grammar_factory.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [GrammarFactory](#)
Responsible for creating and managing grammar items and performing checks on grammars.
- struct [GrammarFactory::FactoryItem](#)
Represents an individual grammar item with its associated symbol table.

10.10 grammar_factory.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #pragma once
00020 #include "grammar.hpp"
00021 #include "symbol_table.hpp"
00022 #include <string>
00023 #include <unordered_map>
00024 #include <vector>
00025
00031 struct GrammarFactory {
00032
00038     struct FactoryItem {
00043         std::unordered_map<std::string, std::vector<production> g_;
00044
00048         SymbolTable st_;
00049
00055         explicit FactoryItem(
00056             const std::unordered_map<std::string, std::vector<production>&
00057             grammar);
00058     };
00059
00064     void Init();
00065
00072     Grammar PickOne(int level);
00073
00080     Grammar GenLL1Grammar(int level);
00087     Grammar GenSLR1Grammar(int level);
00088
00093     Grammar Lv1();
00094
00099     Grammar Lv2();
00100
00106     Grammar Lv3();
00107
00118     Grammar Lv4();
00119
00130     Grammar Lv5();
00131
00142     Grammar Lv6();
00143
00154     Grammar Lv7();
00155
00164     FactoryItem CreateLv2Item();
00165
00172     bool HasUnreachableSymbols(Grammar& grammar) const;
00173
00189     bool IsInfinite(Grammar& grammar) const;
00190
00197     bool HasDirectLeftRecursion(const Grammar& grammar) const;
00198
00204     bool HasIndirectLeftRecursion(const Grammar& grammar) const;

```

```

00205
00211     bool HasCycle(
00212         const std::unordered_map<std::string, std::unordered_set<std::string>&
00213         graph) const;
00214
00220     std::unordered_set<std::string>
00221     NullableSymbols(const Grammar& grammar) const;
00222
00242
00243     void RemoveLeftRecursion(Grammar& grammar);
00244
00266
00267     void LeftFactorize(Grammar& grammar);
00268
00280     std::vector<std::string>
00281     LongestCommonPrefix(const std::vector<production>& productions);
00282
00296     bool StartsWith(const production& prod,
00297         const std::vector<std::string>& prefix);
00298
00311     std::string GenerateNewNonTerminal(Grammar& grammar,
00312         const std::string& base);
00313
00326     void NormalizeNonTerminals(FactoryItem& item, const std::string& nt) const;
00327
00340     void AdjustTerminals(FactoryItem& base, const FactoryItem& cmb,
00341         const std::string& target_nt) const;
00342
00356     std::unordered_map<std::string, std::vector<production>>
00357     Merge(const FactoryItem& base, const FactoryItem& cmb) const;
00358
00363     std::vector<FactoryItem> items;
00364
00368     std::vector<std::string> terminal_alphabet_{ "a", "b", "c", "d", "e", "f",
00369         "g", "h", "i", "j", "k", "l" };
00370
00374     std::vector<std::string> non_terminal_alphabet_{ "A", "B", "C", "D",
00375         "E", "F", "G" };
00376 };

```

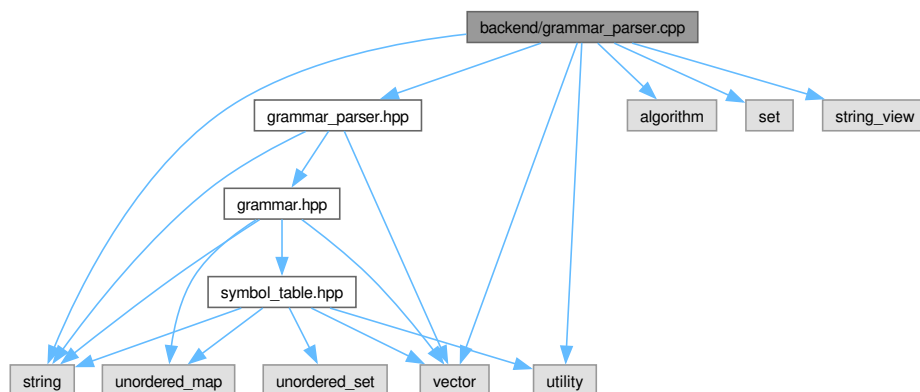
10.11 backend/grammar_parser.cpp File Reference

```

#include "grammar_parser.hpp"
#include <algorithm>
#include <set>
#include <string>
#include <string_view>
#include <utility>
#include <vector>

```

Include dependency graph for grammar_parser.cpp:



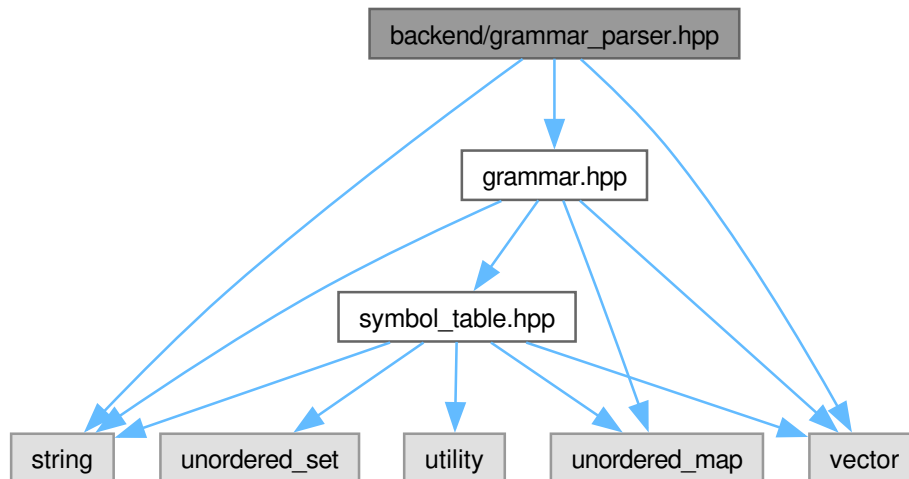
10.12 backend/grammar_parser.hpp File Reference

```
#include "grammar.hpp"
```

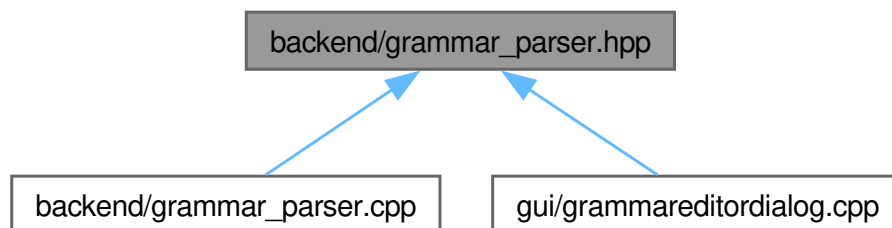
```
#include <string>
```

```
#include <vector>
```

Include dependency graph for grammar_parser.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [GrammarParseError](#)
Describes one problem found while parsing user-written grammar text.
- struct [GrammarParseResult](#)
Outcome of parsing user-written grammar text.
- struct [GrammarParser](#)
Parses user-written grammar text into a [Grammar](#) object.

10.13 grammar_parser.hpp

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it

```

```

00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018 #pragma once
00019
00020 #include "grammar.hpp"
00021 #include <string>
00022 #include <vector>
00023
00032 struct GrammarParseError {
00037     enum class Kind {
00039         EmptyGrammar,
00041         MissingArrow,
00043         MissingEndDot,
00045         EmptyLeftHandSide,
00047         MultipleLeftHandSide,
00049         ExtraArrow,
00051         ReservedSymbol,
00052     };
00053
00055     Kind kind;
00056
00058     int line;
00059
00061     std::string detail;
00062 };
00063
00073 struct GrammarParseResult {
00075     Grammar grammar;
00076
00078     std::vector<GrammarParseError> errors;
00079
00081     bool Ok() const { return errors.empty(); }
00082 };
00083
00103 struct GrammarParser {
00111     static GrammarParseResult Parse(const std::string& text);
00112
00123     static bool IsReservedToken(const std::string& token);
00124 };

```

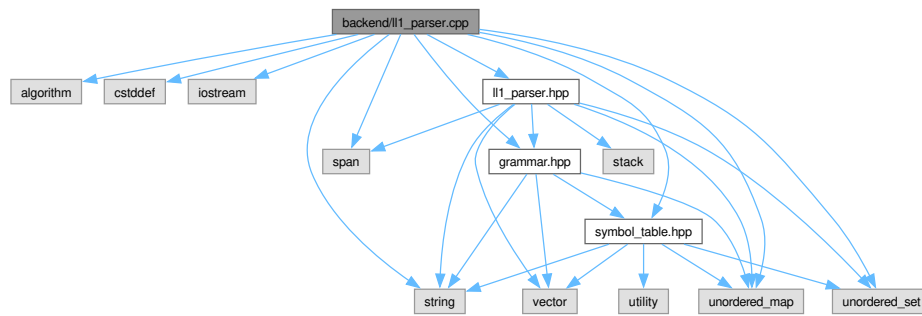
10.14 backend/l11_parser.cpp File Reference

```

#include <algorithm>
#include <cstdint>
#include <iostream>
#include <span>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include "grammar.hpp"
#include "l11_parser.hpp"
#include "symbol_table.hpp"

```

Include dependency graph for `ll1_parser.cpp`:



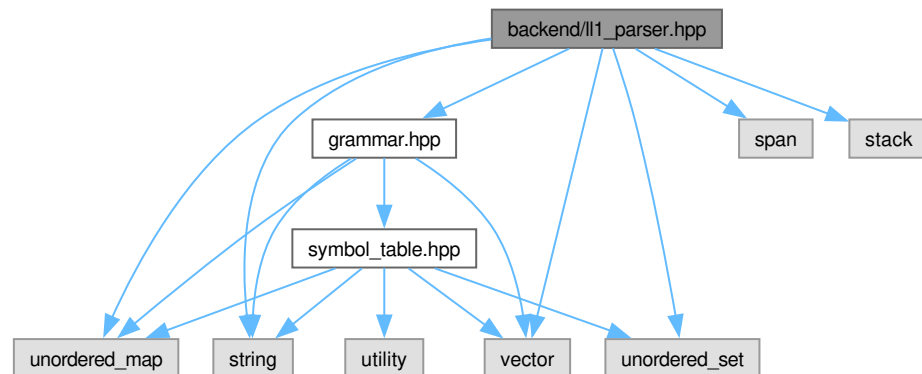
10.15 backend/ll1_parser.hpp File Reference

```

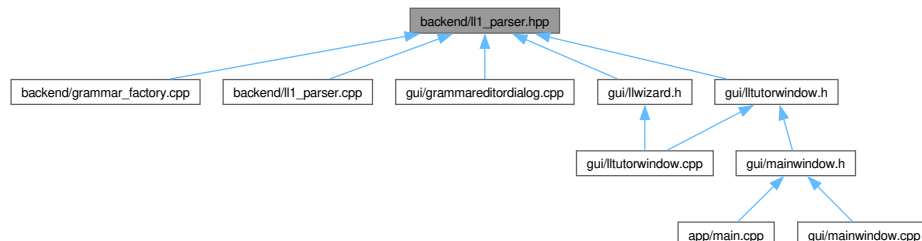
#include "grammar.hpp"
#include <span>
#include <stack>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <vector>

```

Include dependency graph for `ll1_parser.hpp`:



This graph shows which files directly or indirectly include this file:



Classes

- class [LL1Parser](#)

10.16 ll1_parser.hpp

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #pragma once
00020 #include "grammar.hpp"
00021 #include <span>
00022 #include <stack>
00023 #include <string>
00024 #include <unordered_map>
00025 #include <unordered_set>
00026 #include <vector>
00027
00028 class LL1Parser {
00029
00045     using ll1_table = std::unordered_map<
00046         std::string, std::unordered_map<std::string, std::vector<production>>>;
00047
00048 public:
00049     LL1Parser() = default;
00055     explicit LL1Parser(Grammar gr);
00056
00078     bool CreateLL1Table();
00079
00106     void First(std::span<const std::string> rule,
00107         std::unordered_set<std::string>& result);
00108
00119     void ComputeFirstSets();
00120
00146     void ComputeFollowSets();
00147
00162     std::unordered_set<std::string> Follow(const std::string& arg);
00163
00185     std::unordered_set<std::string>
00186     PredictionSymbols(const std::string& antecedent,
00187         const std::vector<std::string>& consequent);
00188
00191     ll1_table ll1_t_;
00192
00194     Grammar gr_;
00195
00197     std::unordered_map<std::string, std::unordered_set<std::string>
00198         first_sets_;
00199
00201     std::unordered_map<std::string, std::unordered_set<std::string>
00202         follow_sets_;
00203 };

```

10.17 backend/lr0_item.cpp File Reference

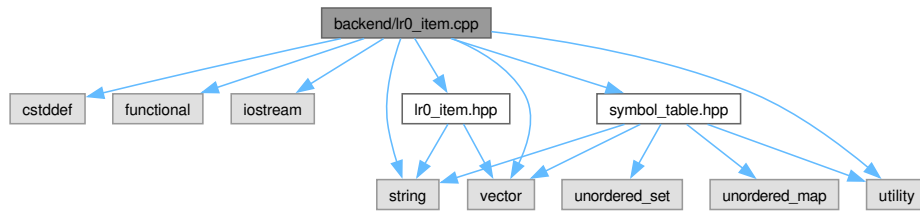
```

#include <cstddef>
#include <functional>
#include <iostream>
#include <string>
#include <utility>
#include <vector>
#include "lr0_item.hpp"

```

```
#include "symbol_table.hpp"
```

Include dependency graph for `lr0_item.cpp`:

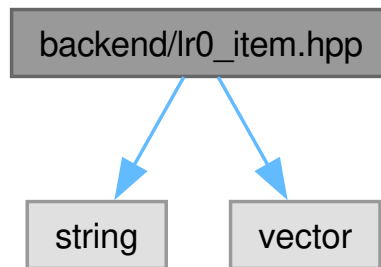


10.18 backend/lr0_item.hpp File Reference

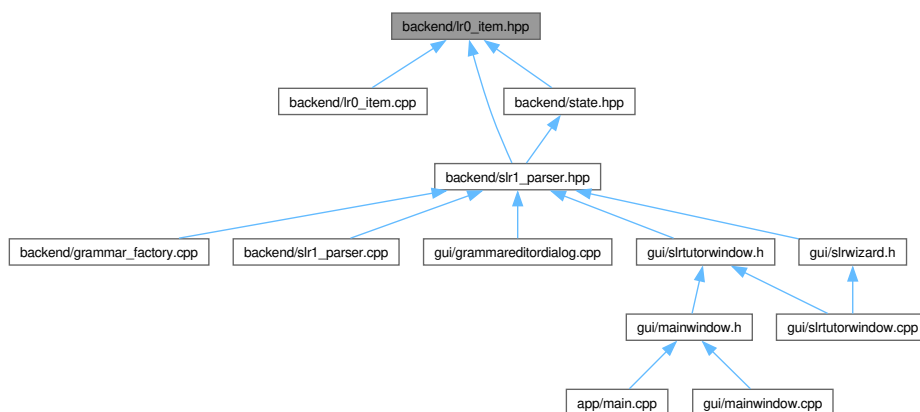
```
#include <string>
```

```
#include <vector>
```

Include dependency graph for `lr0_item.hpp`:



This graph shows which files directly or indirectly include this file:



Classes

- struct [Lr0Item](#)

Represents an LR(0) item used in LR automata construction.

10.19 lr0_item.hpp

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #pragma once
00020 #include <string>
00021 #include <vector>
00022
00034
00035 struct Lr0Item {
00039     std::string antecedent_;
00040
00044     std::vector<std::string> consequent_;
00045
00049     std::string epsilon_;
00050
00054     std::string eol_;
00055
00059     unsigned int dot_ = 0;
00060
00068     Lr0Item(std::string antecedent, std::vector<std::string> consequent,
00069             std::string epsilon, std::string eol);
00070
00079     Lr0Item(std::string antecedent, std::vector<std::string> consequent,
00080             unsigned int dot, std::string epsilon, std::string eol);
00081
00087     std::string NextToDot() const;
00088
00093     void PrintItem() const;
00094
00100     std::string ToString() const;
00101
00105     void AdvanceDot();
00106
00111     bool IsComplete() const;
00112
00118     bool operator==(const Lr0Item& other) const;
00119 };
00120
00121 namespace std {
00122     template <> struct hash<Lr0Item> {
00123         size_t operator()(const Lr0Item& item) const;
00124     };
00125 } // namespace std

```

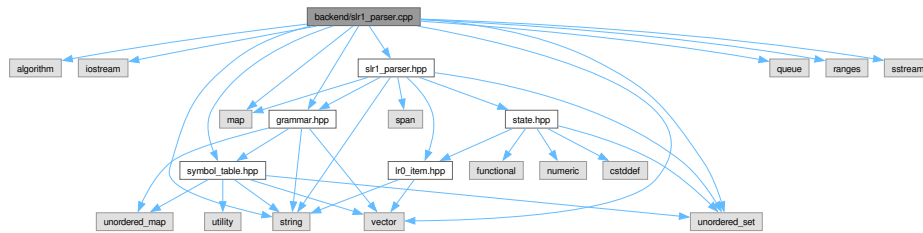
10.20 backend/slr1_parser.cpp File Reference

```

#include <algorithm>
#include <iostream>
#include <map>
#include <queue>
#include <ranges>
#include <sstream>
#include <string>
#include <unordered_set>
#include <vector>
#include "grammar.hpp"

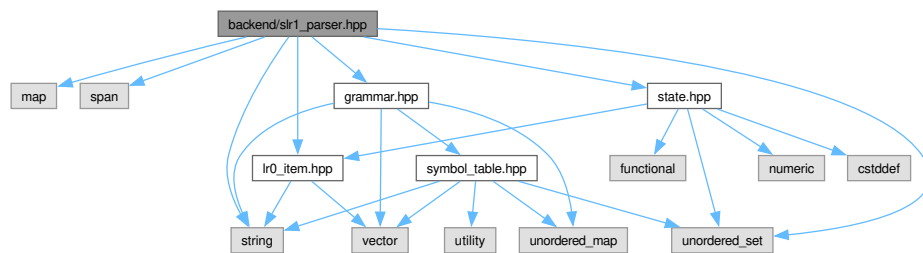
```

```
#include "slr1_parser.hpp"
#include "symbol_table.hpp"
Include dependency graph for slr1_parser.cpp:
```

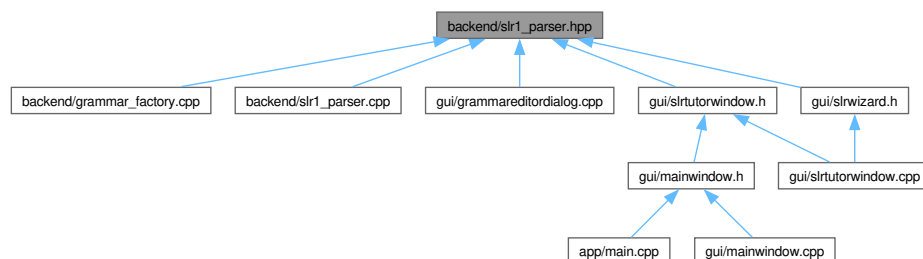


10.21 backend/slr1_parser.hpp File Reference

```
#include <map>
#include <span>
#include <string>
#include <unordered_set>
#include "grammar.hpp"
#include "lr0_item.hpp"
#include "state.hpp"
Include dependency graph for slr1_parser.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [SLR1Parser](#)
Implements an SLR(1) parser for context-free grammars.
- struct [SLR1Parser::s_action](#)

10.22 slr1_parser.hpp

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #pragma once
00020 #include <map>
00021 #include <span>
00022 #include <string>
00023 #include <unordered_set>
00024
00025 #include "grammar.hpp"
00026 #include "lr0_item.hpp"
00027 #include "state.hpp"
00028
00029 class SLR1Parser {
00030 public:
00031     enum class Action { Shift, Reduce, Accept, Empty };
00032
00033     struct s_action {
00034         const Lr0Item* item;
00035         Action action;
00036     };
00037
00038     using action_table =
00039         std::map<unsigned int, std::map<std::string, SLR1Parser::s_action>;
00040
00041     using transition_table =
00042         std::map<unsigned int, std::map<std::string, unsigned int>;
00043
00044     SLR1Parser() = default;
00045     explicit SLR1Parser(Grammar gr);
00046
00047     std::unordered_set<Lr0Item> AllItems() const;
00048
00049     void Closure(std::unordered_set<Lr0Item>& items);
00050
00051     void ClosureUtil(std::unordered_set<Lr0Item>& items, unsigned int size,
00052                     std::unordered_set<std::string>& visited);
00053
00054     std::unordered_set<Lr0Item> Delta(const std::unordered_set<Lr0Item>& items,
00055                                     const std::string& str);
00056
00057     bool SolveLRConflicts(const state& st);
00058
00059     void First(std::span<const std::string> rule,
00060               std::unordered_set<std::string>& result);
00061
00062     void ComputeFirstSets();
00063
00064     void ComputeFollowSets();
00065
00066     std::unordered_set<std::string> Follow(const std::string& arg);
00067
00068     void MakeInitialState();
00069
00070     bool MakeParser();
00071
00072     std::string PrintItems(const std::unordered_set<Lr0Item>& items) const;
00073
00074     Grammar gr_;
00075
00076     std::unordered_map<std::string, std::unordered_set<std::string>
00077         first_sets_;
00078
00079     std::unordered_map<std::string, std::unordered_set<std::string>
00080         follow_sets_;
00081
00082     action_table actions_;

```

```

00309
00312     transition_table transitions_;
00313
00315     std::unordered_set<state> states_;
00316 };

```

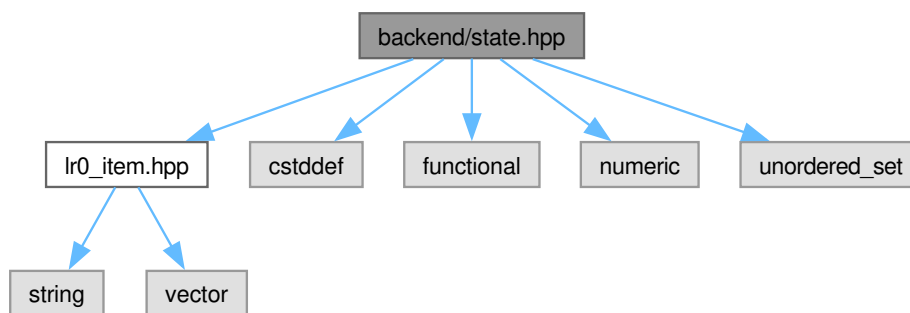
10.23 backend/state.hpp File Reference

```

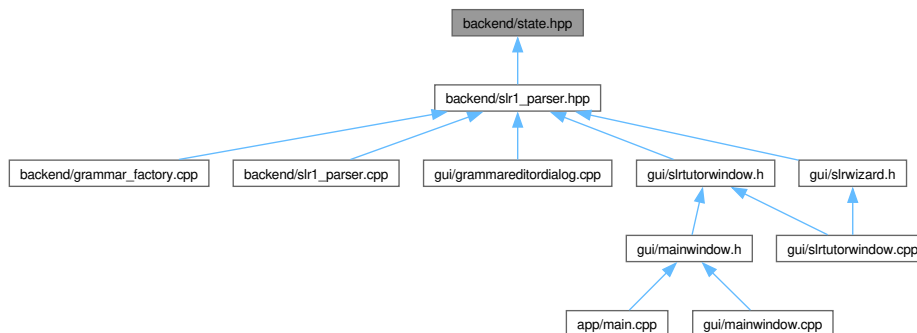
#include "lr0_item.hpp"
#include <cstddef>
#include <functional>
#include <numeric>
#include <unordered_set>

```

Include dependency graph for state.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [state](#)

Represents a state in the LR(0) automaton.

10.24 state.hpp

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)

```

```

00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #pragma once
00020 #include "lr0_item.hpp"
00021 #include <cstdint>
00022 #include <functional>
00023 #include <numeric>
00024 #include <unordered_set>
00025
00033 struct state {
00037     std::unordered_set<Lr0Item> items_;
00038
00042     unsigned int id_;
00043
00049     bool operator==(const state& other) const { return other.items_ == items_; }
00050 };
00051
00052 namespace std {
00053 template <> struct hash<state> {
00054     size_t operator()(const state& st) const {
00055         size_t seed =
00056             std::accumulate(st.items_.begin(), st.items_.end(), 0,
00057                 [](size_t acc, const Lr0Item& item) {
00058                     return acc ^ (std::hash<Lr0Item>()(item));
00059                 });
00060         return seed;
00061     }
00062 };
00063 } // namespace std

```

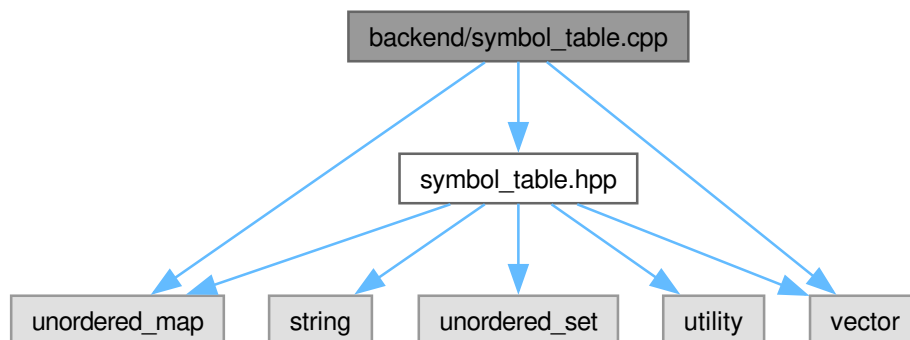
10.25 backend/symbol_table.cpp File Reference

```

#include "symbol_table.hpp"
#include <unordered_map>
#include <vector>

```

Include dependency graph for symbol_table.cpp:



10.26 backend/symbol_table.hpp File Reference

```

#include <string>
#include <unordered_map>

```


META	
------	--

10.27 symbol_table.hpp

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #pragma once
00020 #include <string>
00021 #include <unordered_map>
00022 #include <unordered_set>
00023 #include <utility>
00024 #include <vector>
00025
00033 enum class symbol_type { NO_TERMINAL, TERMINAL, META };
00034
00045 struct SymbolTable {
00047     std::string EOL_{"$"};
00048
00051     std::string EPSILON_{"EPSILON"};
00052
00054     std::unordered_map<std::string, symbol_type> st_{
00055         {EOL_, symbol_type::TERMINAL}, {EPSILON_, symbol_type::META}};
00056
00060     std::unordered_set<std::string> terminals_{EOL_};
00061
00065     std::unordered_set<std::string> terminals_wtho_eol_{};
00066
00070     std::unordered_set<std::string> non_terminals_;
00071
00075     std::unordered_set<std::string> meta_symbols_{EPSILON_};
00076
00083     void PutSymbol(const std::string& identifier, bool isTerminal);
00084
00091     bool In(const std::string& s) const;
00092
00099     bool IsTerminal(const std::string& s) const;
00100
00107     bool IsTerminalWthoEol(const std::string& s) const;
00108
00115     bool IsNonTerminal(const std::string& s) const;
00116
00123     bool IsMeta(const std::string& s) const;
00124 };

```

10.28 gui/automatonviewerdialog.cpp File Reference

```

#include "apptypography.h"
#include "applayout.h"
#include "automatonviewerdialog.h"
#include "appshortcuts.h"
#include "automatonview.h"
#include <QHBoxLayout>
#include <QLabel>
#include <QPushButton>

```


10.30 automatonviewerdialog.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef AUTOMATONVIEWERDIALOG_H
00020 #define AUTOMATONVIEWERDIALOG_H
00021
00022 #include <QDialog>
00023 #include <QPointer>
00024
00025 class AutomatonView;
00026
00027 class AutomatonViewerDialog : public QDialog {
00028     Q_OBJECT
00029 public:
00030     explicit AutomatonViewerDialog(AutomatonView* view,
00031                                     QWidget* parent = nullptr);
00032     ~AutomatonViewerDialog() override;
00033
00034 private:
00035     // QPointer so the dialog never dereferences the borrowed view if it is
00036     // destroyed first (e.g. by the tutor during teardown).
00037     QPointer<AutomatonView> view_;
00038 };
00039
00040 #endif // AUTOMATONVIEWERDIALOG_H

```

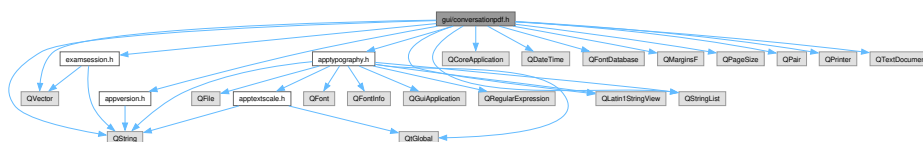
10.31 gui/conversationpdf.h File Reference

```

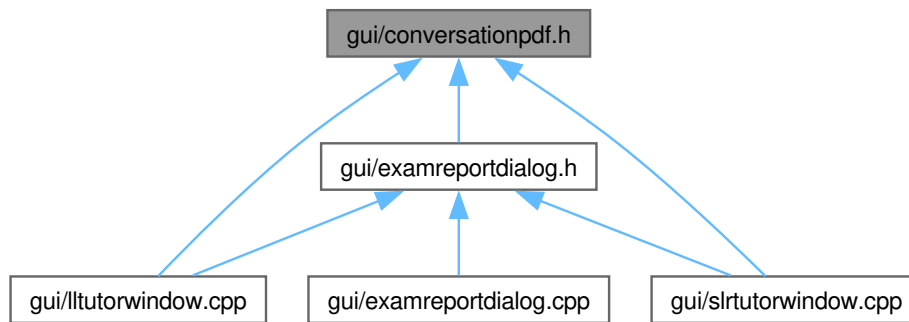
#include "apptypography.h"
#include "appversion.h"
#include "examsession.h"
#include <QCoreApplication>
#include <QDateTime>
#include <QFontDatabase>
#include <QLatin1StringView>
#include <QMarginsF>
#include <QPageSize>
#include <QPair>
#include <QPrinter>
#include <QString>
#include <QStringList>
#include <QTextDocument>
#include <QVector>

```

Include dependency graph for conversationpdf.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [ConversationPdf](#)
Builds the PDF of an LL(1) or SLR(1) session.
- struct [ConversationPdf::Message](#)
One message of the conversation.

10.32 conversationpdf.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef CONVERSATIONPDF_H
00020 #define CONVERSATIONPDF_H
00021
00022 #include "aptypography.h"
00023 #include "appversion.h"
00024 #include "examsession.h"
00025
00026 #include <QCoreApplication>
00027 #include <QDateTime>
00028 #include <QFontDatabase>
00029 #include <QLatin1StringView>
00030 #include <QMarginsF>
00031 #include <QPageSize>
00032 #include <QPair>
00033 #include <QPrinter>
00034 #include <QString>
00035 #include <QStringList>
00036 #include <QTextDocument>
00037 #include <QVector>
00038
00049 class ConversationPdf {
00050     Q_DECLARE_TR_FUNCTIONS(ConversationPdf)
00051
00052 public:
00054     struct Message {
00055         QString text;

```

```

00056         bool    fromStudent = false;
00057         bool    correct     = true;
00058     };
00059
00061     using Rule = QPair<QString, QVector<QString>;
00062
00064     static constexpr int kMaxTableColumns = 10;
00065
00067     explicit ConversationPdf(const QString& exercise) {
00068         html_ += QStringLiteral("<html><head><style>%1</style></head><body>")
00069             .arg(styleSheet());
00070         // The PNG, not the SVG: Qt SVG drops the icon's clip paths.
00071         html_ += QStringLiteral(
00072             "<table class='header' cellspacing='0' cellpadding='0'>"
00073             "<tr><td class='logo'><img src='%1' width='46' "
00074             "height='46'></td><td><p class='kicker'>SyntaxTutor</p>"
00075             "<h1>%2</h1><p class='meta'>%3</p></td></tr></table>")
00076             .arg(QStringLiteral(
00077                 ":/resources/icon/png/syntaxtutor-256.png"))
00078             .arg(exercise.toHtmlEscaped(),
00079                 tr("Generado el %1 con SyntaxTutor %2")
00080                 .arg(QDateTime::currentDateTime().toString(
00081                     QStringLiteral("dd/MM/yyyy HH:mm")),
00082                     SyntaxTutor::Version::current())
00083                 .toHtmlEscaped());
00084     }
00085
00087     void addGrammar(const QVector<Rule>& rules, bool numbered) {
00088         addSection(tr("Gramática"));
00089         html_ += QStringLiteral("<table class='grammar' cellspacing='0' "
00090             "cellpadding='1'>");
00091         for (int i = 0; i < rules.size(); ++i) {
00092             html_ += QStringLiteral("<tr>");
00093             if (numbered) {
00094                 html_ += QStringLiteral("<td class='num'>(%1)</td>").arg(i);
00095             }
00096             html_ += QStringLiteral("<td class='mono'>%1 → %2</td></tr>")
00097                 .arg(rules[i].first.toHtmlEscaped(),
00098                     QStringList(rules[i].second.begin(),
00099                         rules[i].second.end())
00100                     .join(QLatin1Char(' '))
00101                     .toHtmlEscaped());
00102         }
00103         html_ += QStringLiteral("</table>");
00104     }
00105
00107     void addConversation(const QVector<Message>& messages) {
00108         addSection(tr("Conversación"));
00109         html_ += QStringLiteral("<table class='chat' width='100%' "
00110             "cellspacing='0' cellpadding='6'>");
00111         for (const Message& message : messages) {
00112             const bool wrong = message.fromStudent && !message.correct;
00113             const QString rowClass =
00114                 wrong ? QStringLiteral("wrong")
00115                     : (message.fromStudent ? QStringLiteral("student")
00116                         : QStringLiteral("tutor"));
00117             QString who = message.fromStudent ? tr("Usuario") : tr("Tutor");
00118             if (wrong) {
00119                 who += QStringLiteral("<br><span class='flag'>%1</span>")
00120                     .arg(tr("incorrecta"));
00121             }
00122             html_ += QStringLiteral("<tr class='%1'><td class='who' "
00123                 "width='14%'>%2</td><td>%3</td></tr>")
00124                 .arg(rowClass, who, multiline(message.text));
00125         }
00126         html_ += QStringLiteral("</table>");
00127     }
00128
00135     void addResult(const QString& grade, bool passed,
00136         const QStringList& lines) {
00137         const QString tone =
00138             passed ? QStringLiteral("pass") : QStringLiteral("fail");
00139         html_ += QStringLiteral("<table class='result' cellspacing='0' "
00140             "cellpadding='0'><tr><td class='grade %1'>"
00141             "%2</td><td>")
00142             .arg(tone, grade.toHtmlEscaped());
00143         for (int i = 0; i < lines.size(); ++i) {
00144             const bool last = i == lines.size() - 1;
00145             html_ += QStringLiteral("<p class='%1'>%2</p>")
00146                 .arg(last ? QStringLiteral("verdict ") + tone
00147                     : QStringLiteral("summary"),
00148                     lines[i].toHtmlEscaped());
00149         }
00150         html_ += QStringLiteral("</td></tr></table>");
00151     }
00152
00154     void addReview(const QVector<ExamRecord>& records) {

```

```

00155         addSection(tr("Revisión"));
00156         html_ += QStringLiteral(
00157             "<table class='review' width='100%' cellpadding='0' "
00158             "cellpadding='5'><thead><tr><th width='40%'>%1</th>"
00159             "<th width='27%'>%2</th><th width='27%'>%3</th>"
00160             "<th width='6%'></th></tr></thead><tbody>"
00161             .arg(tr("Pregunta"), tr("Tu respuesta"),
00162                 tr("Respuesta correcta"));
00163         for (const ExamRecord& record : records) {
00164             const QString answer = record.userAnswer.trimmed().isEmpty()
00165                 ? tr("vacía")
00166                 : record.userAnswer;
00167             html_ += QStringLiteral(
00168                 "<tr class='%1'><td>%2</td><td class='mono'>%3</td>"
00169                 "<td class='mono'>%4</td><td class='mark'>%5</td>"
00170                 "</tr>");
00171             .arg(record.correct ? QStringLiteral("right")
00172                 : QStringLiteral("wrong"),
00173                 multiline(record.question), multiline(answer),
00174                 multiline(record.correctAnswer),
00175                 record.correct ? QStringLiteral("✓")
00176                 : QStringLiteral("x"));
00177         }
00178         html_ += QStringLiteral("</tbody></table>");
00179     }
00180
00182 void addSection(const QString& title, bool newPage = false) {
00183     html_ += QStringLiteral("<h2>%1</h2>");
00184     .arg(newPage ? QStringLiteral(" class='newpage'")
00185         : QString(),
00186         title.toHtmlEscaped());
00187 }
00188
00190 void addDefinitions(const QVector<QPair<QString, QString>& rows) {
00191     html_ += QStringLiteral("<table class='defs' cellpadding='0' "
00192         "cellpadding='1'>");
00193     for (const auto& [name, value] : rows) {
00194         html_ += QStringLiteral("<tr><td class='mono name'>%1</td>"
00195             "<td class='mono'>%2</td></tr>");
00196         .arg(name.toHtmlEscaped(), value.toHtmlEscaped());
00197     }
00198     html_ += QStringLiteral("</table>");
00199 }
00200
00202 void addSubheading(const QString& text) {
00203     html_ += QStringLiteral("<h3>%1</h3>").arg(text.toHtmlEscaped());
00204 }
00205
00207 void addLines(const QStringList& lines) {
00208     QStringList escaped;
00209     for (const QString& line : lines) {
00210         escaped « line.toHtmlEscaped();
00211     }
00212     html_ += QStringLiteral("<p class='mono lines'>%1</p>");
00213     .arg(escaped.join(QStringLiteral("<br>")));
00214 }
00215
00224 void addTable(const QStringList& headers,
00225     const QVector<QStringList>& rows) {
00226     const int dataColumns = static_cast<int>(headers.size()) - 1;
00227     for (int first = 0; first < qMax(dataColumns, 1);
00228         first += kMaxTableColumns) {
00229         const int last = qMin(first + kMaxTableColumns, dataColumns);
00230         if (dataColumns > kMaxTableColumns) {
00231             html_ += QStringLiteral("<p class='caption'>%1</p>");
00232             .arg(tr("Columnas %1 a %2 de %3")
00233                 .arg(first + 1)
00234                 .arg(last)
00235                 .arg(dataColumns));
00236         }
00237         html_ += QStringLiteral("<table class='grid' cellpadding='0' "
00238             "cellpadding='4'><thead><tr>");
00239         html_ += QStringLiteral("<th>%1</th>");
00240         .arg(headers.value(0).toHtmlEscaped());
00241         for (int c = first; c < last; ++c) {
00242             html_ += QStringLiteral("<th class='mono'>%1</th>");
00243             .arg(headers.value(c + 1).toHtmlEscaped());
00244         }
00245         html_ += QStringLiteral("</tr></thead><tbody>");
00246         for (int r = 0; r < rows.size(); ++r) {
00247             html_ += QStringLiteral("<tr class='%1'>");
00248             .arg(r % 2 ? QStringLiteral("odd")
00249                 : QStringLiteral("even"));
00250             html_ += QStringLiteral("<th class='mono'>%1</th>");
00251             .arg(rows[r].value(0).toHtmlEscaped());
00252             for (int c = first; c < last; ++c) {
00253                 html_ += QStringLiteral("<td class='mono'>%1</td>");

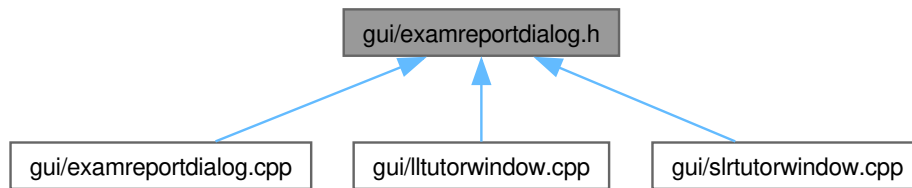
```

```

00254         .arg(rows[r].value(c + 1).toHtmlEscaped());
00255     }
00256     html_ += QStringLiteral("</tr>");
00257 }
00258 html_ += QStringLiteral("</tbody></table>");
00259 }
00260 }
00261
00263 QString html() const { return html_ + QStringLiteral("</body></html>"); }
00264
00266 void print(const QString& filePath) const {
00267     ensureMonospaceFont();
00268     QTextDocument doc;
00269     // A fixed size: paper does not follow the on-screen text scale.
00270     QFont font = AppTypography::systemFont();
00271     font.setPointSizeF(10.0);
00272     doc.setDefaultFont(font);
00273     doc.setHtml(html());
00274
00275     QPrinter printer(QPrinter::HighResolution);
00276     printer.setOutputFormat(QPrinter::PdfFormat);
00277     printer.setOutputFileName(filePath);
00278     printer.setPageSize(QPageSize(QPageSize::A4));
00279     printer.setPageMargins(QMarginsF(15, 15, 15, 15),
00280                             QPageLayout::Millimeter);
00281     doc.print(&printer);
00282 }
00283
00284 private:
00285     static QString multiline(const QString& text) {
00286         return text.toHtmlEscaped().replace(QLatin1Char('\n'),
00287                                             QStringLiteral("<br>"));
00288     }
00289
00290     static constexpr auto kMonospaceFamily = "JetBrains Mono";
00291
00292     // The PDF names its monospaced font in CSS, which needs a real family:
00293     // the platform's fixed font is an internal family on macOS that the
00294     // printer cannot resolve. The bundled JetBrains Mono is the same
00295     // everywhere; main() only loads it on Windows and Linux.
00296     static void ensureMonospaceFont() {
00297         if (!QFontDatabase::families().contains(
00298             QLatin1StringView(kMonospaceFamily))) {
00299             QFontDatabase::addApplicationFont(
00300                 QStringLiteral(":/resources/fonts/JetBrainsMono-Regular.ttf"));
00301             QFontDatabase::addApplicationFont(
00302                 QStringLiteral(":/resources/fonts/JetBrainsMono-Bold.ttf"));
00303         }
00304     }
00305
00306     static QString styleSheet() {
00307         // Print palette shared with the exam report.
00308         return QStringLiteral(
00309             "body { color: #1A1A1A; font-size: 10pt; }"
00310             "p.kicker { color: #007B8A; font-size: 9pt; "
00311             "font-weight: bold; margin: 0; }"
00312             "h1 { font-size: 18pt; margin: 2px 0 2px 0; }"
00313             "p.meta { color: #666666; font-size: 9pt; margin: 0; }"
00314             "table.header { margin-bottom: 12px; }"
00315             "td.logo { padding-right: 12px; vertical-align: middle; }"
00316             "h2 { color: #007B8A; font-size: 13pt; "
00317             "margin: 18px 0 6px 0; }"
00318             "h2.newpage { page-break-before: always; }"
00319             "h3 { font-size: 10.5pt; margin: 10px 0 2px 0; }"
00320             ".mono { font-family: '%1'; }"
00321             "p.lines { margin: 0 0 4px 12px; }"
00322             "p.caption { color: #666666; font-size: 8.5pt; "
00323             "margin: 10px 0 2px 0; }"
00324             "td.num { color: #666666; }"
00325             "td.name { padding-right: 12px; }"
00326             "table.chat { margin-top: 2px; }"
00327             "table.chat td { border-bottom: 1px solid #E0E0E0; }"
00328             "td.who { color: #007B8A; font-weight: bold; "
00329             "font-size: 9pt; }"
00330             "tr.student td.who { color: #555555; }"
00331             "tr.wrong td { background-color: #FBEAEA; }"
00332             "tr.wrong td.who { color: #B23A37; }"
00333             "span.flag { font-size: 8pt; font-weight: normal; }"
00334             "table.grid { border-collapse: collapse; font-size: 9pt; "
00335             "margin-bottom: 6px; }"
00336             "table.grid th, table.grid td { border: 1px solid "
00337             "#BBBBBB; text-align: center; }"
00338             "table.grid tthead th { background-color: #E8E8E8; }"
00339             "table.grid tbody th { background-color: #F3F3F3; }"
00340             "tr.odd td { background-color: #FAFAFA; }"
00341             "table.result { margin: 4px 0 6px 0; }"
00342             "td.grade { font-size: 30pt; font-weight: bold; }"

```


This graph shows which files directly or indirectly include this file:



Classes

- class [ExamReportDialog](#)

End-of-exam report: grade, stats and a question-by-question review.

10.35 examreportdialog.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef EXAMREPORTDIALOG_H
00020 #define EXAMREPORTDIALOG_H
00021
00022 #include "conversationpdf.h"
00023 #include "examsession.h"
00024
00025 #include <QDialog>
00026
00036 class ExamReportDialog : public QDialog {
00037     Q_OBJECT
00038     public:
00049     ExamReportDialog(const ExamSession& session, const QString& examTitle,
00050                     const QVector<ConversationPdf::Rule>& grammar,
00051                     bool numberedGrammar, QWidget* parent = nullptr);
00052
00054     void printReport(const QString& filePath) const {
00055         report_.print(filePath);
00056     }
00057
00058     signals:
00060     void exportRequested();
00061
00062     private:
00063     QString buildReviewHtml(const ExamSession& session) const;
00064
00065     ConversationPdf report_;
00066 };
00067
00068 #endif // EXAMREPORTDIALOG_H

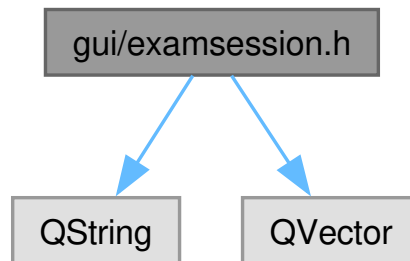
```

10.36 gui/examsession.h File Reference

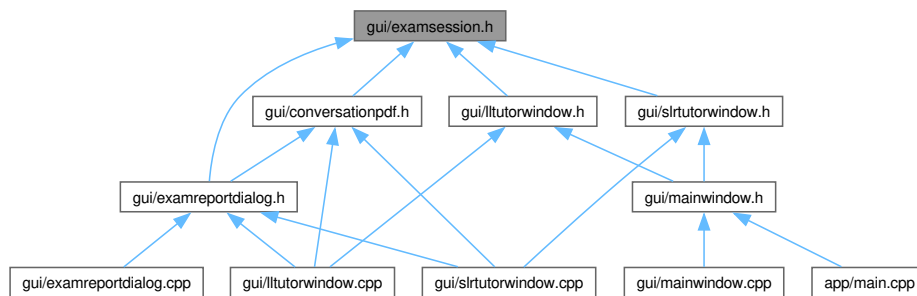
```
#include <QString>
```

```
#include <QVector>
```

Include dependency graph for examsession.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [ExamRecord](#)

One graded item of an exam: a question (or table cell), the user's answer, the expected answer, and whether it was correct.

- class [ExamSession](#)

Accumulates graded answers during an exam-mode exercise.

10.37 examsession.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
  
```



```

00017  */
00018
00019 #ifndef EXAMSESSION_H
00020 #define EXAMSESSION_H
00021
00022 #include <QString>
00023 #include <QVector>
00024
00030 struct ExamRecord {
00032     QString question;
00034     QString userAnswer;
00036     QString correctAnswer;
00038     bool correct;
00039 };
00040
00050 class ExamSession {
00051 public:
00060     void record(const QString& question, const QString& userAnswer,
00061               const QString& correctAnswer, bool correct) {
00062         records_.push_back({question, userAnswer, correctAnswer, correct});
00063     }
00064
00066     int total() const { return static_cast<int>(records_.size()); }
00067
00069     int right() const {
00070         int count = 0;
00071         for (const ExamRecord& record : records_) {
00072             if (record.correct) {
00073                 ++count;
00074             }
00075         }
00076         return count;
00077     }
00078
00084     double grade() const {
00085         if (records_.isEmpty()) {
00086             return 0.0;
00087         }
00088         return 10.0 * right() / total();
00089     }
00090
00092     const QVector<ExamRecord>& records() const { return records_; }
00093
00094 private:
00095     QVector<ExamRecord> records_;
00096 };
00097
00098 #endif // EXAMSESSION_H

```

10.38 gui/grammareditordialog.cpp File Reference

```

#include "grammareditordialog.h"
#include "appsettings.h"
#include "apptypography.h"
#include "applayout.h"
#include "appfonts.h"
#include "grammar_factory.hpp"
#include "grammar_parser.hpp"
#include "l1l_parser.hpp"
#include "slrl_parser.hpp"
#include <QFontDatabase>
#include <QFrame>
#include <QHBoxLayout>
#include <QLabel>
#include <QPlainTextEdit>
#include <QPushButton>
#include <QSettings>
#include <QShortcut>
#include <QStyle>
#include <QVBoxLayout>

```

```
#include <algorithm>
```

Include dependency graph for grammareditordialog.cpp:



10.39 gui/grammareditordialog.h File Reference

```
#include "grammar.hpp"
```

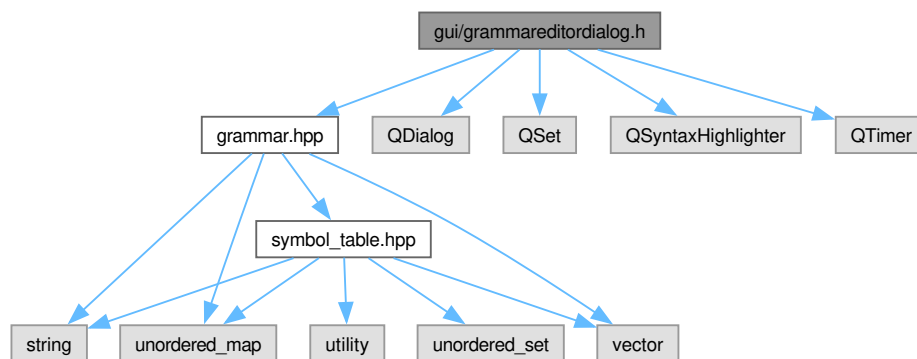
```
#include <QDialog>
```

```
#include <QSet>
```

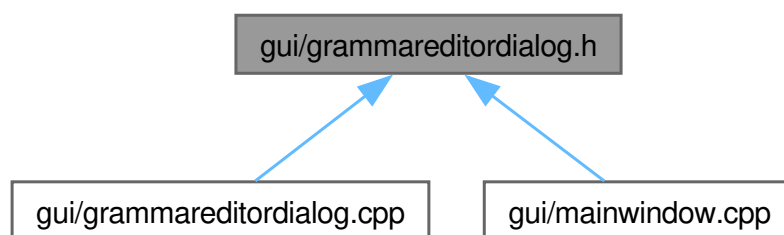
```
#include <QSyntaxHighlighter>
```

```
#include <QTimer>
```

Include dependency graph for grammareditordialog.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [GrammarSyntaxHighlighter](#)
Lightweight highlighter for user-written grammar text.
- class [GrammarEditorDialog](#)
Dialog where the user writes their own grammar for an exercise.

10.40 grammareditordialog.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef GRAMMAREDITORIALOG_H
00020 #define GRAMMAREDITORIALOG_H
00021
00022 #include "grammar.hpp"
00023 #include <QDialog>
00024 #include <QSet>
00025 #include <QSyntaxHighlighter>
00026 #include <QTimer>
00027
00028 class QLabel;
00029 class QPlainTextEdit;
00030 class QPushButton;
00031 struct GrammarParseResult;
00032
00042 class GrammarSyntaxHighlighter : public QSyntaxHighlighter {
00043     Q_OBJECT
00044     public:
00045         explicit GrammarSyntaxHighlighter(QTextDocument* parent);
00046
00054         void setNonTerminals(const QSet<QString>& nonTerminals);
00055
00056     protected:
00057         void highlightBlock(const QString& text) override;
00058
00059     private:
00060         QSet<QString> nonTerminals_;
00061         QTextCharFormat arrowFormat_;
00062         QTextCharFormat punctuationFormat_;
00063         QTextCharFormat nonTerminalFormat_;
00064 };
00065
00077 class GrammarEditorDialog : public QDialog {
00078     Q_OBJECT
00079     public:
00084         enum class Mode { LL1, SLR1 };
00085
00092         explicit GrammarEditorDialog(Mode mode, QWidget* parent = nullptr);
00093
00099         const Grammar& grammar() const { return grammar_; }
00100
00101 #ifndef SYNTAXTUTOR_TESTING
00103         void setGrammarTextForTest(const QString& text);
00105         bool isGrammarValidForTest() const { return valid_; }
00106 #endif
00107
00108     protected:
00109         void accept() override;
00110
00111     private slots:
00112         void scheduleValidation();
00113         void validateNow();
00114
00115     private:
00116         void buildUi();
00117         void updateSummary(const Grammar& grammar);
00118         void clearSummary();
00119         void setStatus(const QString& message, const QString& state);
00120         QString describeParseErrors(const GrammarParseResult& result);
00121
00122         Mode mode_;
00123         Grammar grammar_;
00124         bool valid_ = false;
00125         QTimer debounce_;
00126         QPlainTextEdit* input_ = nullptr;

```

```

00127     QLabel*           statusLabel_   = nullptr;
00128     QLabel*           axiomValue_    = nullptr;
00129     QLabel*           nonTermValue_  = nullptr;
00130     QLabel*           termValue_     = nullptr;
00131     QLabel*           rulesValue_    = nullptr;
00132     QPushButton*      startButton_   = nullptr;
00133     QPushButton*      cancelButton_  = nullptr;
00134     GrammarSyntaxHighlighter* highlighter_ = nullptr;
00135 };
00136
00137 #endif // GRAMMAREDITORDIALOG_H

```

10.41 gui/ltabledialog.cpp File Reference

```

#include "ltabledialog.h"
#include "applayout.h"
#include <QApplication>
#include <QFontDatabase>
#include <QStyledItemDelegate>

```

Include dependency graph for ltabledialog.cpp:



Classes

- class [CenterAlignDelegate](#)

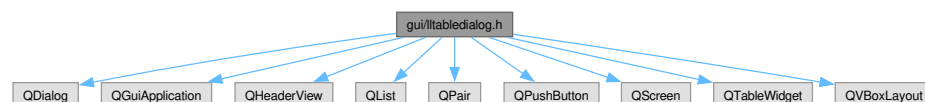
10.42 gui/ltabledialog.h File Reference

```

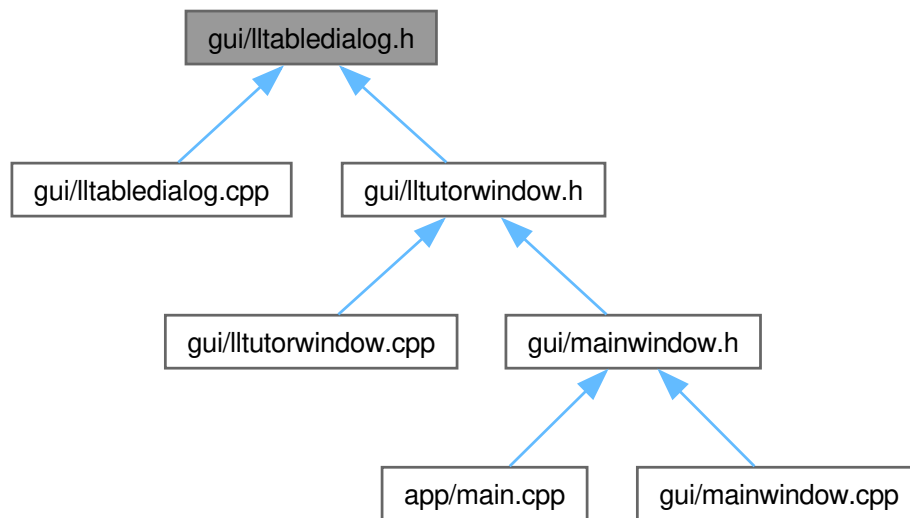
#include <QDialog>
#include <QGuiApplication>
#include <QHeaderView>
#include <QList>
#include <QPair>
#include <QPushButton>
#include <QScreen>
#include <QTableWidget>
#include <QVBoxLayout>

```

Include dependency graph for ltabledialog.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [LLTableDialog](#)
Dialog for filling and submitting an LL(1) parsing table.

10.43 lltabledialog.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef LLTABLEDIALOG_H
00020 #define LLTABLEDIALOG_H
00021
00022 #include <QDialog>
00023 #include <QGuiApplication>
00024 #include <QHeaderView>
00025 #include <QList>
00026 #include <QPair>
00027 #include <QPushButton>
00028 #include <QScreen>
00029 #include <QTableWidget>
00030 #include <QVBoxLayout>
00031
00041 class LLTableDialog : public QDialog {
00042     Q_OBJECT
00043     public:
00052     LLTableDialog(const QStringList& rowHeaders, const QStringList& colHeaders,
00053                  QWidget* parent,
00054                  QVector<QVector<QString>>* initialData = nullptr);
00055

```

```

00060     QVector<QVector<QString>> getTableData() const;
00061
00070     void setInitialData(const QVector<QVector<QString>>& data);
00071
00076     void highlightIncorrectCells(const QList<QPair<int, int>>& coords);
00077
00083     void setGuidedModeActive(bool active);
00084
00092     void setGuidedButtonVisible(bool visible);
00093
00094 signals:
00099     void submitted(const QVector<QVector<QString>>& data);
00100
00105     void guidedRequested(const QVector<QVector<QString>>& data);
00106
00107 private:
00108     void commitPendingEdit();
00109
00110     QTableWidget* table;
00111     QPushButton* submitButton;
00112     QPushButton* guidedButton;
00113 };
00114
00115 #endif // LLTABLEDIALOG_H

```

10.44 gui/lltutorwindow.cpp File Reference

```

#include "lltutorwindow.h"
#include "conversationpdf.h"
#include "apptypography.h"
#include "examreportdialog.h"
#include "grammarview.h"
#include "llwizard.h"
#include "tutorialmanager.h"
#include "ui_lltutorwindow.h"
#include <QAbstractButton>
#include <QApplication>
#include <QDateTime>
#include <QDir>
#include <QFileInfo>
#include <QFontDatabase>
#include <QHBoxLayout>
#include <QMessageBox>
#include <QRandomGenerator>
#include <QRegularExpression>
#include <QStandardPaths>
#include <QWheelEvent>

```

Include dependency graph for lltutorwindow.cpp:



10.45 gui/lltutorwindow.h File Reference

```

#include <QAbstractItemView>
#include <QDialog>
#include <QFileDialog>
#include <QGraphicsColorizeEffect>
#include <QGraphicsScene>

```

```

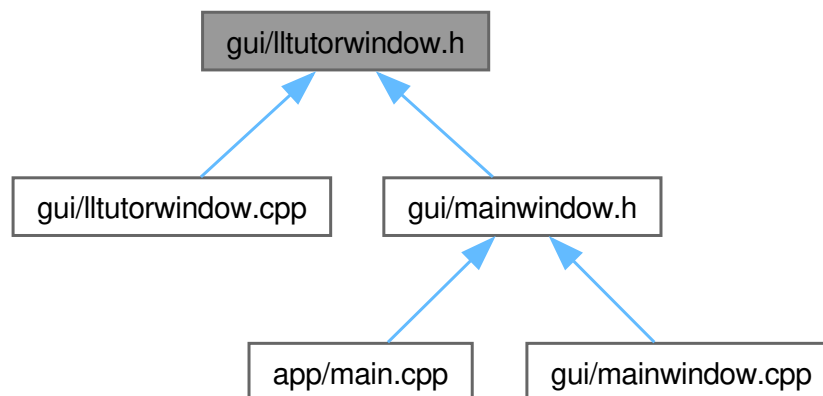
#include <QGraphicsTextItem>
#include <QGraphicsView>
#include <QListWidgetItem>
#include <QMessageBox>
#include <QPainter>
#include <QPropertyAnimation>
#include <QPushButton>
#include <QScrollBar>
#include <QShortcut>
#include <QTableWidget>
#include <QTextDocument>
#include <QTextEdit>
#include <QTime>
#include <QTimer>
#include <QTreeWidgetItem>
#include <QVBoxLayout>
#include <QWidget>
#include <QtPrintSupport/QtPrinter>
#include "examsession.h"
#include "grammar.hpp"
#include "grammarview.h"
#include "lll_parser.hpp"
#include "lltabledialog.h"

```

Include dependency graph for lltutorwindow.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [LLTutorWindow](#)
Main window for the LL(1) interactive tutoring mode in [SyntaxTutor](#).
- struct [LLTutorWindow::TreeNode](#)
[TreeNode](#) structure used to build derivation trees.

Enumerations

- enum class [State](#) {
[A](#) , [A1](#) , [A2](#) , [A_prime](#) ,
[B](#) , [B1](#) , [B2](#) , [B_prime](#) ,
[C](#) , [C_prime](#) , [fin](#) }

10.45.1 Enumeration Type Documentation

10.45.1.1 State

```
enum class State [strong]
```

Enumerator

A	
A1	
A2	
A_prime	
B	
B1	
B2	
B_prime	
C	
C_prime	
fin	

10.46 lltutorwindow.h

[Go to the documentation of this file.](#)

```
00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef LLTUTORWINDOW_H
00020 #define LLTUTORWINDOW_H
00021
00022 #include <QAbstractItemView>
00023 #include <QDialog>
00024 #include <QFileDialog>
00025 #include <QGraphicsColorizeEffect>
00026 #include <QGraphicsScene>
00027 #include <QGraphicsTextItem>
00028 #include <QGraphicsView>
```



```

00029 #include <QListWidgetItem>
00030 #include <QMessageBox>
00031 #include <QPainter>
00032 #include <QPropertyAnimation>
00033 #include <QPushButton>
00034 #include <QScrollBar>
00035 #include <QShortcut>
00036 #include <QTableWidget>
00037 #include <QTextDocument>
00038 #include <QTextEdit>
00039 #include <QTime>
00040 #include <QTimer>
00041 #include <QTreeWidget>
00042 #include <QVBoxLayout>
00043 #include <QWidget>
00044 #include <QtPrintSupport/QPrinter>
00045
00046 #include "examsession.h"
00047 #include "grammar.hpp"
00048 #include "grammarview.h"
00049 #include "lll_parser.hpp"
00050 #include "lltabledialog.h"
00051
00052 class TutorialManager;
00053
00054 namespace Ui {
00055 class LLTutorWindow;
00056 }
00057
00058 // ===== LL(1) Tutor States =====
00059 enum class State { A, A1, A2, A_prime, B, B1, B2, B_prime, C, C_prime, fin };
00060
00061 // ===== LL(1) Tutor Main Class =====
00081 class LLTutorWindow : public QWidget {
00082     Q_OBJECT
00083
00084 public:
00092     void applyTextScale();
00093
00094     // ===== Derivation Tree (used in TeachFirst) =====
00098     struct TreeNode {
00099         QString label;
00100         std::vector<std::unique_ptr<TreeNode>> children;
00101     };
00102
00103     // ===== Constructor / Destructor =====
00113     explicit LLTutorWindow(const Grammar& grammar, TutorialManager* tm = nullptr,
00114                             QWidget* parent = nullptr, bool examMode = false);
00115     ~LLTutorWindow();
00116
00117     // ===== State Machine & Question Logic =====
00122     QString generateQuestion();
00123
00128     void updateState(bool isCorrect);
00129
00135     QString FormatGrammar(const Grammar& grammar);
00136     QVector<GrammarView::Row> buildGrammarRows(const Grammar& grammar) const;
00137
00138     // ===== UI Interaction =====
00139     void addMessage(const QString& text,
00140                     bool isUser);
00141     void addGrammarMessage();
00142     void addWidgetMessage(QWidget* widget);
00143     void
00144     exportConversationToPdf(const QString& filePath);
00145     void showTable();
00146     void showTableForCPrime();
00147     void updateProgressPanel(); // Update progress panel
00148
00149     // ===== Visual Feedback / Animations =====
00150     void animateLabelPop(QLabel* label);
00151     void animateLabelColor(QLabel* label, const QColor& flashColor);
00152     void wrongAnimation();
00153     void
00154     wrongUserResponseAnimation();
00155     void markLastUserIncorrect();
00156
00157     // ===== Tree Generation (TeachFirst mode) =====
00158     void TeachFirstTree(const std::vector<std::string>& symbols,
00159                         std::unordered_set<std::string>& first_set, int depth,
00160                         std::unordered_set<std::string>& processing,
00161                         QTreeWidgetItem* parent);
00162
00163     std::unique_ptr<TreeNode>
00164     buildTreeNode(const std::vector<std::string>& symbols,
00165                  std::unordered_set<std::string>& first_set, int depth,
00166                  std::vector<std::pair<std::string, std::vector<std::string>>>&

```

```

00167         active_derivations);
00168
00169     int computeSubtreeWidth(const std::unique_ptr<TreeNode>& node,
00170                             int hSpacing);
00171     void drawTree(const std::unique_ptr<TreeNode>& root, QGraphicsScene* scene,
00172                   QPointF pos, int hSpacing, int vSpacing);
00173
00174     void showTreeGraphics(
00175         std::unique_ptr<TreeNode> root); // Display derivation tree visually
00176
00177     // ===== User Response Verification =====
00178     bool verifyResponse(const QString& userResponse); // Delegates to current
00179                                                         // state's verification
00180     bool verifyResponseForA(const QString& userResponse);
00181     bool verifyResponseForA1(const QString& userResponse);
00182     bool verifyResponseForA2(const QString& userResponse);
00183     bool verifyResponseForB(const QString& userResponse);
00184     bool verifyResponseForB1(const QString& userResponse);
00185     bool verifyResponseForB2(const QString& userResponse);
00186     bool verifyResponseForC(); // C is non-textual (checks internal table)
00187
00188     // ===== Expected Solutions (Auto-generated) =====
00189     QString solution(const std::string& state);
00190     QStringList solutionForA();
00191     QString solutionForA1();
00192     QString solutionForA2();
00193     QSet<QString> solutionForB();
00194     QSet<QString> solutionForB1();
00195     QSet<QString> solutionForB2();
00196
00197     // ===== Feedback (Corrective Explanations) =====
00198     QString feedback(); // Delegates by state
00199     QString feedbackForA();
00200     QString feedbackForA1();
00201     QString feedbackForA2();
00202     QString feedbackForAPrime();
00203     QString feedbackForB();
00204     QString feedbackForB1();
00205     QString feedbackForB2();
00206     QString feedbackForBPrime();
00207     QString feedbackForC();
00208     QString feedbackForCPrime();
00209     void feedbackForB1TreeWidget(); // TreeWidget of Teach (LL1 TeachFirst)
00210     void feedbackForB1TreeGraphics(); // Show derivation tree
00211     QString TeachFollow(const QString& nt);
00212     QString TeachPredictionSymbols(const QString& ant,
00213                                   const production& consequ);
00214     QString TeachLL1Table();
00215
00216     void handleTableSubmission(const QVector<QVector<QString>& raw,
00217                               const QStringList& colHeaders);
00218
00219     void connectGuidedMode(LLTableDialog* dialog,
00220                             const QStringList& colHeaders);
00221     void updatePlaceholder();
00222     bool confirmExitToHome();
00223     QString promptExportFilePath() const;
00224
00225     // ===== Exam Mode =====
00226     void postQuestion();
00227     QString examSolutionText();
00228     void scoreExamTable();
00229     void showExamReport();
00230 #ifndef SYNTAXTUTOR_TESTING
00231 public:
00232     QString currentStateForTest() const;
00233     QString currentRuleAntecedentForTest() const;
00234     QStringList currentRuleConsequentForTest() const;
00235     int rightCountForTest() const;
00236     int wrongCountForTest() const;
00237     void setAnswerForTest(const QString& text);
00238     void submitForTest();
00239     void setNextExportFilePathForTest(const QString& filePath);
00240     double examGradeForTest() const { return examSession.grade(); }
00241     int examTotalForTest() const { return examSession.total(); }
00242     int examRightForTest() const { return examSession.right(); }
00243 #endif
00244 private slots:
00245     void on_backButton_clicked();
00246     void on_confirmButton_clicked();
00247
00248 signals:
00249     void sessionFinished(int cntRight, int cntWrong);
00250     void exitRequested(bool applyResults, int cntRight, int cntWrong);
00251
00252 protected:
00253     void closeEvent(QCloseEvent* event) override {

```

```

00264         emit sessionFinished(cntRightAnswers, cntWrongAnswers);
00265         QWidget::closeEvent(event);
00266     }
00267
00268     bool eventFilter(QObject* obj, QEvent* event) override;
00269
00270 private:
00271     void relayChatMessages();
00272
00273     // ===== Core Objects =====
00274     Ui::LLTutorWindow* ui;
00275     Grammar            grammar;
00276     LLParser           llp;
00277
00278     // ===== State & Grammar Tracking =====
00279     State              currentState;
00280     size_t             currentRule = 0;
00281     const unsigned kMaxHighlightTries = 3;
00282     const unsigned kMaxTotalTries = 5;
00283     unsigned         lltries = 0;
00284     unsigned         cntRightAnswers = 0, cntWrongAnswers = 0;
00285
00286     // ===== Exam Mode =====
00287     bool examMode = false;
00288     ExamSession examSession;
00289     QString currentQuestionText;
00290
00291     using Cell = std::pair<QString, QString>;
00292     std::vector<Cell> lastWrongCells;
00293     LLTableDialog* currentDlg = nullptr;
00294
00295     QVector<QString> sortedNonTerminals;
00296     QVector<QPair<QString, QVector<QString>>> sortedGrammar;
00297     QString formattedGrammar;
00298     GrammarView* grammarView = nullptr;
00299
00300     QMap<QString, QMap<QString, QVector<QString>>> lltable;
00301     QVector<QVector<QString>> rawTable;
00302     QSet<QString> solutionSet;
00303
00304     // ===== Conversation Logging =====
00305     struct MessageLog {
00306         QString message;
00307         bool isUser;
00308         bool isCorrect = true;
00309         MessageLog(const QString& message, bool isUser)
00310             : message(message), isUser(isUser) {}
00311         void toggleIsCorrect() { isCorrect = false; }
00312     };
00313
00314     QVector<MessageLog> conversationLog;
00315     QWidget* lastUserMessage = nullptr;
00316     qsize_t lastUserMessageLogIdx = -1;
00317
00318     QMap<QString, QString> userCAB;
00319     QMap<QString, QString> userSIG;
00320     QMap<QString, QString> userSD;
00321
00322     // ===== Helper Conversions =====
00323     std::vector<std::string> QVectorToStdVector(const QVector<QString>& qvec);
00324     QVector<QString> stdVectorToQVector(const std::vector<std::string>& vec);
00325     QSet<QString>
00326     stdUnorderedSetToQSet(const std::unordered_set<std::string>& uset);
00327     std::unordered_set<std::string>
00328     qsetToStdUnorderedSet(const QSet<QString>& qset);
00329
00330     void setupTutorial();
00331     void requestExit(bool applyResults);
00332
00333     void
00334     fillSortedGrammar(); // Populate sortedGrammar from internal representation
00335
00336     QPropertyAnimation* m_shakeAnimation =
00337         nullptr; // For interrupting userResponse animation if they spam enter
00338                 // key
00339
00340     TutorialManager* tm = nullptr;
00341
00342 #ifdef SYNTAXTUTOR_TESTING
00343     mutable QString nextExportFilePathForTest;
00344 #endif
00345
00346     const QRegularExpression kRe{"^\\s+|\\s+$"};
00347     const QRegularExpression kWhitespace{"\\s+"};
00348 };
00349
00350 #endif // LLTUTORWINDOW_H

```

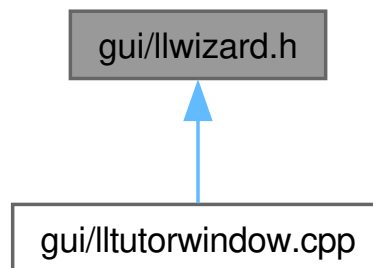
10.47 gui/llwizard.h File Reference

```
#include "ll1_parser.hpp"
#include "llwizardpage.h"
#include <QDialog>
#include <QFrame>
#include <QHBoxLayout>
#include <QLabel>
#include <QProgressBar>
#include <QPushButton>
#include "apptypography.h"
#include "applayout.h"
#include <QStackedWidget>
#include <QTimer>
#include <QVBoxLayout>
```

Include dependency graph for llwizard.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [LLWizard](#)

Interactive assistant that guides the student step-by-step through the LL(1) parsing table.

10.48 llwizard.h

[Go to the documentation of this file.](#)

```
00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
```

```

00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef LLWIZARD_H
00020 #define LLWIZARD_H
00021
00022 #include "lll_parser.hpp"
00023 #include "llwizardpage.h"
00024
00025 #include <QDialog>
00026 #include <QFrame>
00027 #include <QHBoxLayout>
00028 #include <QLabel>
00029 #include <QProgressBar>
00030 #include <QPushButton>
00031 #include "apptypography.h"
00032 #include "applayout.h"
00033 #include <QStackedWidget>
00034 #include <QTimer>
00035 #include <QVBoxLayout>
00036
00049 class LLWizard : public QDialog {
00050     Q_OBJECT
00051     public:
00060         LLWizard(LLlParser& parser, const QStringList& rowHeaders,
00061                 const QStringList& colHeaders, QWidget* parent = nullptr)
00062             : QDialog(parent) {
00063             setProperty("guidedDialog", "ll");
00064             setWindowTitle(tr("Completar tabla LL(1)"));
00065             setWindowFlag(Qt::WindowCloseButtonHint, true);
00066             setModal(true);
00067             resize(AppTypography::lengthForText(640),
00068                   AppTypography::lengthForText(440));
00069             AppLayout::keepHeightForWidth(this);
00070             setMinimumSize(AppTypography::lengthForText(560),
00071                           AppTypography::lengthForText(400));
00072
00073             auto* rootLayout = new QVBoxLayout(this);
00074             rootLayout->setContentsMargins(0, 0, 0, 0);
00075             rootLayout->setSpacing(0);
00076
00077             auto* panel = new QFrame(this);
00078             panel->setObjectName("llWizardPanel");
00079
00080             auto* panelLayout = new QVBoxLayout(panel);
00081             panelLayout->setContentsMargins(28, 24, 28, 24);
00082             panelLayout->setSpacing(20);
00083
00084             auto* headerLayout = new QHBoxLayout();
00085             headerLayout->setSpacing(12);
00086
00087             auto* titleColumn = new QVBoxLayout();
00088             titleColumn->setContentsMargins(0, 0, 0, 0);
00089             titleColumn->setSpacing(6);
00090
00091             m_titleLabel = new QLabel(panel);
00092             m_titleLabel->setObjectName("llWizardTitle");
00093             titleColumn->addWidget(m_titleLabel);
00094
00095             m_stepCounterLabel = new QLabel(panel);
00096             m_stepCounterLabel->setObjectName("llWizardStepCounter");
00097             titleColumn->addWidget(m_stepCounterLabel);
00098
00099             m_progressBar = new QProgressBar(panel);
00100             m_progressBar->setObjectName("llWizardProgress");
00101             m_progressBar->setTextVisible(false);
00102             m_progressBar->setRange(0, 100);
00103             m_progressBar->setFixedHeight(4);
00104             titleColumn->addWidget(m_progressBar);
00105
00106             headerLayout->addLayout(titleColumn, 1);
00107
00108             m_closeButton = new QPushButton(tr("Salir"), panel);
00109             m_closeButton->setObjectName("llWizardCloseButton");
00110             m_closeButton->setAutoDefault(false);
00111             m_closeButton->setDefault(false);
00112             m_closeButton->setCursor(Qt::PointingHandCursor);
00113             headerLayout->addWidget(m_closeButton, 0, Qt::AlignTop);
00114
00115             panelLayout->addLayout(headerLayout);
00116
00117             m_stack = new QStackedWidget(panel);
00118             m_stack->setObjectName("llWizardStack");
00119             panelLayout->addWidget(m_stack, 1);

```

```

00120
00121     auto* footerLayout = new QHBoxLayout();
00122     footerLayout->setContentsMargins(0, 0, 0, 0);
00123     footerLayout->setSpacing(12);
00124     footerLayout->addStretch(1);
00125
00126     m_nextButton = new QPushButton(panel);
00127     m_nextButton->setObjectName("llWizardNextButton");
00128     m_nextButton->setProperty("role", "primary");
00129     m_nextButton->setAutoDefault(false);
00130     m_nextButton->setDefault(false);
00131     footerLayout->addWidget(m_nextButton);
00132     panelLayout->addLayout(footerLayout);
00133
00134     rootLayout->addWidget(panel);
00135
00136     for (const QString& nonTerminal : rowHeaders) {
00137         const auto rowIt = parser.ll1_t_.find(nonTerminal.toStdString());
00138         if (rowIt == parser.ll1_t_.end()) {
00139             continue;
00140         }
00141         for (const QString& symbol : colHeaders) {
00142             const auto cellIt = rowIt->second.find(symbol.toStdString());
00143             if (cellIt == rowIt->second.end() || cellIt->second.empty() ||
00144                 cellIt->second[0].empty()) {
00145                 continue;
00146             }
00147
00148             QStringList production;
00149             for (const std::string& s : cellIt->second[0]) {
00150                 production.append(QString::fromStdString(s));
00151             }
00152             const QString expected = production.join(' ');
00153
00154             const QString explanation =
00155                 tr("La celda (%1, '%2') no está vacía: existe una "
00156                    "producción %1 →  $\alpha$  tal que '%2' ∈ SD(%1 →  $\alpha$ ).\n"
00157                    "¿Qué producción  $\alpha$  escribes en esta celda?")
00158                 .arg(nonTerminal, symbol);
00159
00160             auto* page = new LLWizardPage(nonTerminal, symbol, explanation,
00161                                           expected, m_stack);
00162             connect(page, &LLWizardPage::completionChanged, this,
00163                     &LLWizard::updateCurrentPage);
00164             connect(page, &LLWizardPage::submitRequested, this,
00165                     &LLWizard::advance);
00166             m_stack->addWidget(page);
00167         }
00168     }
00169
00170     connect(m_closeButton, &QPushButton::clicked, this, &QDialog::reject);
00171     connect(m_nextButton, &QPushButton::clicked, this, &LLWizard::advance);
00172
00173     updateCurrentPage();
00174     QTimer::singleShot(0, this, [this]() {
00175         if (auto* page = currentPage()) {
00176             page->focusAnswerField();
00177         }
00178     });
00179 }
00180
00181 LLWizardPage* currentPage() const {
00182     return qobject_cast<LLWizardPage*>(m_stack->currentWidget());
00183 }
00184
00185 QPushButton* nextButton() const { return m_nextButton; }
00186 QPushButton* closeButton() const { return m_closeButton; }
00187
00188 bool isOnLastPage() const {
00189     return m_stack->count() > 0 &&
00190         m_stack->currentIndex() == m_stack->count() - 1;
00191 }
00192
00193 private slots:
00194     void advance() {
00195         auto* page = currentPage();
00196         if (page == nullptr || !page->isComplete()) {
00197             return;
00198         }
00199
00200         if (isOnLastPage()) {
00201             accept();
00202             return;
00203         }
00204
00205         m_stack->setCurrentIndex(m_stack->currentIndex() + 1);
00206         updateCurrentPage();

```

```

00207         if (auto* nextPage = currentPage()) {
00208             nextPage->focusAnswerField();
00209         }
00210     }
00211
00212     void updateCurrentPage() {
00213         auto* page = currentPage();
00214         if (page == nullptr) {
00215             m_titleLabel->setText(tr("Modo guiado"));
00216             m_stepCounterLabel->clear();
00217             m_progressBar->setValue(0);
00218             m_nextButton->setEnabled(false);
00219             m_nextButton->setText(tr("Finalizar"));
00220             return;
00221         }
00222
00223         const int currentStep = m_stack->currentIndex() + 1;
00224         const int totalSteps = m_stack->count();
00225         m_titleLabel->setText(page->titleText());
00226         m_stepCounterLabel->setText(
00227             tr("Paso %1 de %2").arg(currentStep).arg(totalSteps));
00228         m_progressBar->setValue((100 * currentStep) / qMax(1, totalSteps));
00229         m_nextButton->setEnabled(page->isComplete());
00230         m_nextButton->setCursor(page->isComplete() ? Qt::PointingHandCursor
00231             : Qt::ArrowCursor);
00232         m_nextButton->setText(isOnLastPage() ? tr("Finalizar")
00233             : tr("Continuar"));
00234     }
00235
00236 private:
00237     QLabel*          m_titleLabel          = nullptr;
00238     QLabel*          m_stepCounterLabel    = nullptr;
00239     QProgressBar*     m_progressBar         = nullptr;
00240     QPushButton*     m_closeButton         = nullptr;
00241     QStackedWidget*  m_stack               = nullptr;
00242     QPushButton*     m_nextButton          = nullptr;
00243 };
00244
00245 #endif // LLWIZARD_H

```

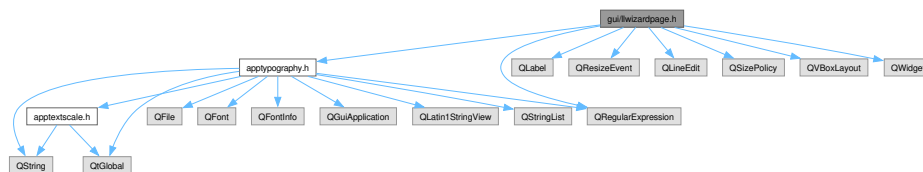
10.49 gui/llwizardpage.h File Reference

```

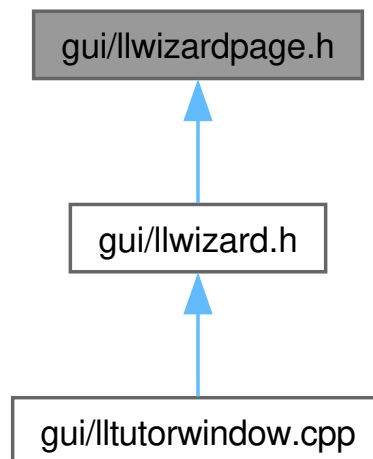
#include "apptypography.h"
#include <QLabel>
#include <QResizeEvent>
#include <QLineEdit>
#include <QRegularExpression>
#include <QSizePolicy>
#include <QVBoxLayout>
#include <QWidget>

```

Include dependency graph for llwizardpage.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [LLWizardPage](#)

A single guided step for constructing the LL(1) table.

10.50 llwizardpage.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef LLWIZARDPAGE_H
00020 #define LLWIZARDPAGE_H
00021
00022 #include "apptypography.h"
00023 #include <QLabel>
00024 #include <QResizeEvent>
00025 #include <QLineEdit>
00026 #include <QRegularExpression>
00027 #include <QSizePolicy>
00028 #include <QVBoxLayout>
00029 #include <QWidget>
00030
00041 class LLWizardPage : public QWidget {
00042     Q_OBJECT
00043     public:
00054         LLWizardPage(const QString& nonTerminal, const QString& symbol,
00055                     const QString& explanation, const QString& expected,
00056                     QWidget* parent = nullptr)
00057             : QWidget(parent),
00058               m_title(tr("Fila %1, columna '%2'").arg(nonTerminal).arg(symbol)),
00059               m_expected(expected) {
00060             auto* rootLayout = new QVBoxLayout(this);

```



```

00061     rootLayout->setContentsMargins(0, 0, 0, 0);
00062     rootLayout->setSpacing(22);
00063
00064     auto* explanationTitle = new QLabel(tr("Pista"), this);
00065     explanationTitle->setObjectName("llWizardSectionTitle");
00066     rootLayout->addWidget(explanationTitle);
00067
00068     auto* explanationLabel = new QLabel(explanation, this);
00069     explanationLabel->setObjectName("llWizardExplanationLabel");
00070     explanationLabel->setWordWrap(true);
00071     rootLayout->addWidget(explanationLabel);
00072     m_explanation = explanationLabel;
00073
00074     auto* answerTitle = new QLabel(tr("Tu respuesta"), this);
00075     answerTitle->setObjectName("llWizardSectionTitle");
00076     rootLayout->addWidget(answerTitle);
00077
00078     m_edit = new QLineEdit(this);
00079     m_edit->setObjectName("llWizardAnswerEdit");
00080     m_edit->setMinimumHeight(AppTypography::lengthForText(48));
00081     m_edit->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Fixed);
00082     m_edit->setPlaceholderText(
00083         tr("Escribe la producción (EPSILON si es vacía)"));
00084     rootLayout->addWidget(m_edit);
00085
00086     m_feedback = new QLabel(this);
00087     m_feedback->setObjectName("llWizardFeedbackLabel");
00088     m_feedback->setWordWrap(true);
00089     m_feedback->setMinimumHeight(AppTypography::lengthForText(40));
00090     m_feedback->setSizePolicy(QSizePolicy::Preferred, QSizePolicy::Fixed);
00091     rootLayout->addWidget(m_feedback);
00092
00093     rootLayout->addStretch(1);
00094
00095     connect(m_edit, &QLineEdit::textChanged, this,
00096             &LLWizardPage::onTextChanged);
00097     connect(m_edit, &QLineEdit::returnPressed, this,
00098             &LLWizardPage::submitRequested);
00099 }
00100
00101 protected:
00102     void resizeEvent(QResizeEvent* event) override {
00103         QWidget::resizeEvent(event);
00104         // These pages live in a QStackedWidget, which does not honour a
00105         // word-wrapped label's height-for-width: the hint ends up clipped to
00106         // whatever height was left over. Pin its minimum to what the text
00107         // actually needs at the width it got.
00108         if (m_explanation != nullptr && m_explanation->width() > 0) {
00109             m_explanation->setMinimumHeight(
00110                 m_explanation->heightForWidth(m_explanation->width()));
00111         }
00112     }
00113
00114 public:
00115     QString titleText() const { return m_title; }
00116     bool isComplete() const { return m_isComplete; }
00117     void focusAnswerField() { m_edit->setFocus(); }
00118
00119 #ifdef SYNTAXTUTOR_TESTING
00120     QString expectedForTest() const { return m_expected; }
00121 #endif
00122
00123 signals:
00124     void completionChanged(bool complete);
00125     void submitRequested();
00126
00127 private slots:
00128     void onTextChanged(const QString& text) {
00129         const QString trimmed = text.trimmed();
00130         const bool correct = (compact(trimmed) == compact(m_expected));
00131
00132         if (trimmed.isEmpty()) {
00133             m_feedback->clear();
00134         } else if (correct) {
00135             m_feedback->setText(
00136                 tr("✓ Respuesta correcta, pasa a la siguiente pregunta"));
00137         } else {
00138             m_feedback->setText(tr("X Incorrecto, repasa los símbolos "
00139                                     "directores (SD) en el panel del tutor."));
00140         }
00141
00142         if (m_isComplete == correct) {
00143             return;
00144         }
00145
00146         m_isComplete = correct;
00147         emit completionChanged(m_isComplete);
00148     }

```

```

00156     }
00157
00158     private:
00159         static QString compact(const QString& text) {
00160             QString result = text;
00161             result.remove(QRegularExpression("\\s+"));
00162             return result;
00163         }
00164
00165         QString    m_title;
00166         QString    m_expected;
00167         QLabel*    m_feedback;
00168         QLabel*    m_explanation = nullptr;
00169         QLineEdit* m_edit;
00170         bool       m_isComplete =
00171             false;
00172     };
00173
00174 #endif // LLWIZARDPAGE_H

```

10.51 gui/mainwindow.cpp File Reference

```

#include "mainwindow.h"
#include "grammareditorDialog.h"
#include "tutorialmanager.h"
#include "ui_mainwindow.h"
#include <QDialog>
#include <QHBoxLayout>
#include <QLabel>
#include <QMessageBox>
#include <QPixmap>
#include <QScrollArea>
#include <QStackedWidget>
#include "appicon.h"
#include "appsettings.h"
#include "apptypography.h"
#include "applayout.h"
#include <QActionGroup>
#include <QClipboard>
#include <QGuiApplication>
#include <QMenu>
#include <QScreen>
#include <QSlider>
#include <QSysInfo>
#include <QTextBrowser>
#include <QVBoxLayout>
#include <QProcess>

```

Include dependency graph for mainwindow.cpp:



10.52 gui/mainwindow.h File Reference

```

#include "grammar.hpp"
#include "grammar_factory.hpp"
#include "lltutorwindow.h"

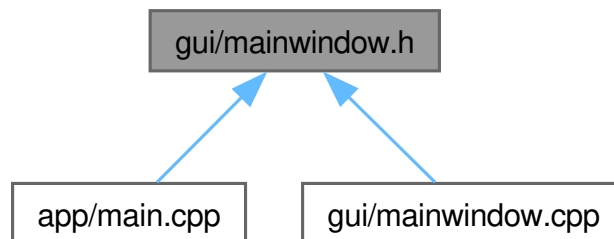
```

```
#include "slrtutorwindow.h"
#include "tutorialmanager.h"
#include <QMainWindow>
#include "apptypography.h"
#include <QSettings>
```

Include dependency graph for mainwindow.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [MainWindow](#)

Main application window of [SyntaxTutor](#), managing levels, exercises, and UI state.

10.53 mainwindow.h

[Go to the documentation of this file.](#)

```
00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef MAINWINDOW_H
00020 #define MAINWINDOW_H
00021
00022 #include "grammar.hpp"
00023 #include "grammar_factory.hpp"
00024 #include "lltutorwindow.h"
00025 #include "slrtutorwindow.h"
00026 #include "tutorialmanager.h"
00027 #include <QMainWindow>
00028 #include "apptypography.h"
00029 #include <QSettings>
00030
00031 class QStackedWidget;
00032 class QWidget;
```

```

00033
00034 static const QVector<QString> levelColors = {
00035     "#2C3E50", // 1: Navy oscuro
00036     "#2980B9", // 2: Azul brillante
00037     "#16A085", // 3: Teal
00038     "#27AE60", // 4: Verde esmeralda
00039     "#8E44AD", // 5: Púrpura medio
00040     "#9B59B6", // 6: Púrpura claro
00041     "#E67E22", // 7: Naranja
00042     "#D35400", // 8: Naranja oscuro
00043     "#CD7F32", // 9: Bronce
00044     "#FFD700" // 10: Oro puro
00045 };
00046
00047 QT_BEGIN_NAMESPACE
00048 namespace Ui {
00049 class MainWindow;
00050 }
00051 QT_END_NAMESPACE
00052
00063 class MainWindow : public QMainWindow {
00064     Q_OBJECT
00065     Q_PROPERTY(unsigned userLevel READ userLevel WRITE setUserLevel NOTIFY
00066                 userLevelChanged)
00067
00068 public:
00073     MainWindow(QWidget* parent = nullptr);
00074
00076     ~MainWindow();
00077
00083     unsigned thresholdFor(unsigned level) { return BASE_THRESHOLD * level; }
00084
00088     unsigned userLevel() const { return m_userLevel; };
00089
00094     void setUserLevel(unsigned lvl) {
00095         unsigned clamped = qMin(lvl, MAX_LEVEL);
00096         if (m_userLevel == clamped)
00097             return;
00098         m_userLevel = clamped;
00099         emit userLevelChanged(clamped);
00100     }
00101
00102 private slots:
00106     void on_lv1Button_clicked(bool checked);
00107     void on_lv2Button_clicked(bool checked);
00108     void on_lv3Button_clicked(bool checked);
00109
00116     void on_customGrammarCheck_toggled(bool checked);
00117
00121     void on_pushButton_clicked();
00122
00126     void on_pushButton_2_clicked();
00127
00131     void on_tutorial_clicked();
00132
00136     void on_actionSobre_la_aplicaci_n_triggered();
00137
00141     void on_actionReferencia_LL_1_triggered();
00142
00146     void on_actionReferencia_SLR_1_triggered();
00147
00151     void on_idiom_clicked();
00152
00153 signals:
00158     void userLevelChanged(unsigned lvl);
00159
00164     void userLevelUp(unsigned newLevel);
00165
00166 private:
00170     void setupTutorial();
00171
00176     void applyLevelStyling(unsigned lvl);
00177
00181     void restartTutorial();
00182
00183     void setNavigationEnabled(bool enabled);
00184
00191     void setDifficultySelectorEnabled(bool enabled);
00192
00199     void applyScaledMinimumSize();
00200
00202     void setupTextSizeMenu();
00203
00212     void setTextScale(int percent);
00213
00215     void promptCustomTextScale();
00216

```

```

00218     void syncTextSizeMenu();
00219     void showHomePage();
00220     void cleanupTutorPages();
00221     LLTutorWindow* startLLTutor(const Grammar& grammar, TutorialManager* tm,
00222                               bool examMode = false);
00223     SLRTutorWindow* startSLRTutor(const Grammar& grammar, TutorialManager* tm,
00224                                  bool examMode = false);
00225     void abortTutorialFlow();
00226
00232     void handleTutorFinished(int cntRight, int cntWrong);
00233
00237     void saveSettings();
00238
00242     void loadSettings();
00243
00244     Ui::MainWindow* ui;
00245     GrammarFactory  factory;
00246     int             level = 1;
00247     TutorialManager* tm   = nullptr;
00248     QStackedWidget* stack = nullptr;
00249     QWidget*        homePage = nullptr;
00250     QActionGroup*   textSizeGroup_ = nullptr;
00251     QAction*        customTextSizeAction_ = nullptr;
00252
00253     LLTutorWindow* llTutorPage = nullptr;
00254     SLRTutorWindow* slrTutorPage = nullptr;
00255     QString        defaultWindowTitle;
00256
00257     static constexpr unsigned MAX_LEVEL = 10;
00258     static constexpr unsigned MAX_SCORE = 999;
00259
00260     unsigned m_userLevel = 1;
00261     unsigned userScore   = 0;
00262     QSettings settings;
00263
00264     const unsigned BASE_THRESHOLD = 10;
00265 };
00266 #endif // MAINWINDOW_H

```

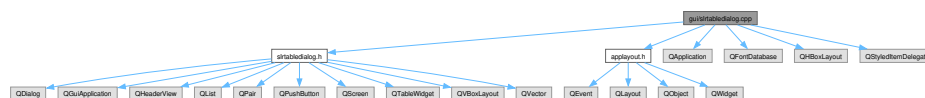
10.54 gui/slrtabledialog.cpp File Reference

```

#include "slrtabledialog.h"
#include "applayout.h"
#include <QApplication>
#include <QFontDatabase>
#include <QHBoxLayout>
#include <QStyledItemDelegate>

```

Include dependency graph for slrtabledialog.cpp:



Classes

- class [CenterAlignDelegate](#)

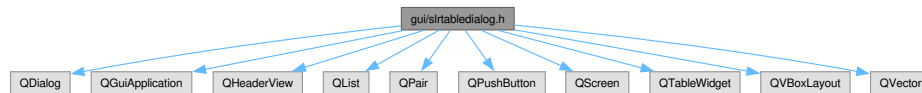
10.55 gui/slrtabledialog.h File Reference

```

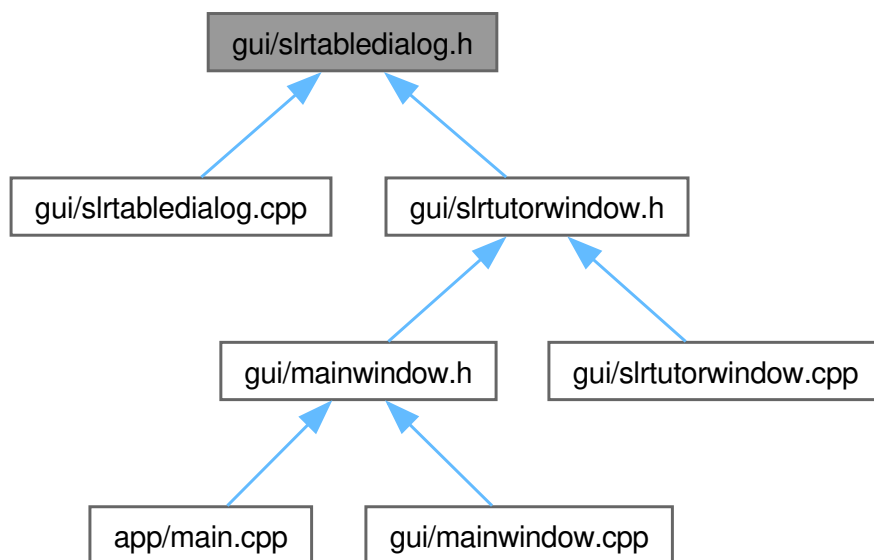
#include <QDialog>
#include <QGuiApplication>
#include <QHeaderView>
#include <QList>

```

```
#include <QPair>
#include <QPushButton>
#include <QScreen>
#include <QTableWidget>
#include <QVBoxLayout>
#include <QVector>
Include dependency graph for slrtabledialog.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [SLRTableDialog](#)

Dialog window for completing and submitting an SLR(1) parsing table.

10.56 slrtabledialog.h

[Go to the documentation of this file.](#)

```
00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
```

```

00016  * along with this program.  If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef SLRTABLEDIALOG_H
00020 #define SLRTABLEDIALOG_H
00021
00022 #include <QDialog>
00023 #include <QGuiApplication>
00024 #include <QHeaderView>
00025 #include <QList>
00026 #include <QPair>
00027 #include <QPushButton>
00028 #include <QScreen>
00029 #include <QTableWidget>
00030 #include <QVBoxLayout>
00031 #include <QVector>
00032
00042 class SLRTableDialog : public QDialog {
00043     Q_OBJECT
00044 public:
00054     SLRTableDialog(int rowCount, int colCount, const QStringList& colHeaders,
00055                   QWidget* parent = nullptr,
00056                   QVector<QVector<QString>>* initialData = nullptr);
00057
00062     QVector<QVector<QString>> getTableData() const;
00063
00072     void setInitialData(const QVector<QVector<QString>>& data);
00073     void setGuidedModeActive(bool active);
00074
00082     void setGuidedButtonVisible(bool visible);
00083
00084     void highlightIncorrectCells(const QList<QPair<int, int>>& coords);
00085     void highlightInvalidCells(const QList<QPair<int, int>>& coords);
00086
00087 private:
00088     void commitPendingEdit();
00089
00090     QTableWidget* table;
00091     QPushButton* submitButton;
00092     QPushButton* guidedButton;
00093
00094     signals:
00095     void submitted(const QVector<QVector<QString>>& data);
00096     void guidedRequested(const QVector<QVector<QString>>& data);
00097 };
00098
00099 #endif // SLRTABLEDIALOG_H

```

10.57 gui/slrtutorwindow.cpp File Reference

```

#include "slrtutorwindow.h"
#include "conversationpdf.h"
#include "apptypography.h"
#include "automatonviewerdialog.h"
#include "examreportdialog.h"
#include "grammarview.h"
#include "slrwizard.h"
#include "tutorialmanager.h"
#include "ui_slrtutorwindow.h"
#include <QApplication>
#include <QDateTime>
#include <QDir>
#include <QEasingCurve>
#include <QFileInfo>
#include <QFontDatabase>
#include <QHBoxLayout>
#include <QMessageBox>
#include <QRegularExpression>
#include <QStandardPaths>

```

```
#include <algorithm>
```

Include dependency graph for slrtutorwindow.cpp:



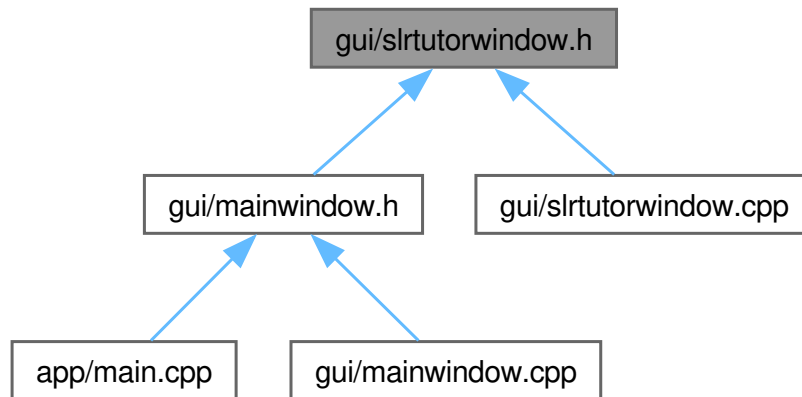
10.58 gui/slrtutorwindow.h File Reference

```
#include "UniqueQueue.h"
#include "automatonview.h"
#include "automatonviewerdialog.h"
#include "examsession.h"
#include "grammar.hpp"
#include "grammarview.h"
#include "slrl_parser.hpp"
#include "slrtabledialog.h"
#include <QAbstractItemView>
#include <QDialog>
#include <QFileDialog>
#include <QGraphicsColorizeEffect>
#include <QListWidgetItem>
#include <QMessageBox>
#include <QPropertyAnimation>
#include <QPushButton>
#include <QRegularExpression>
#include <QScrollBar>
#include <QShortcut>
#include <QTableWidget>
#include <QTextDocument>
#include <QTextEdit>
#include <QTime>
#include <QTimer>
#include <QVBoxLayout>
#include <QWidget>
#include <QtPrintSupport/QtPrinter>
```

Include dependency graph for slrtutorwindow.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [SLRTutorWindow](#)

Main window for the SLR(1) interactive tutoring mode in [SyntaxTutor](#).

Enumerations

- enum class [StateSlr](#) {
[A](#) , [A1](#) , [A2](#) , [A3](#) ,
[A4](#) , [A_prime](#) , [B](#) , [C](#) ,
[CA](#) , [CB](#) , [D](#) , [D1](#) ,
[D2](#) , [D_prime](#) , [E](#) , [E1](#) ,
[E2](#) , [F](#) , [FA](#) , [G](#) ,
[H](#) , [H_prime](#) , [fin](#) }

10.58.1 Enumeration Type Documentation

10.58.1.1 StateSlr

```
enum class StateSlr [strong]
```

Enumerator

A	
A1	
A2	
A3	
A4	
A_prime	
B	
C	

CA	
CB	
D	
D1	
D2	
D_prime	
E	
E1	
E2	
F	
FA	
G	
H	
H_prime	
fin	

10.59 slrtutorwindow.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef SLRTUTORWINDOW_H
00020 #define SLRTUTORWINDOW_H
00021
00022 #include "UniqueQueue.h"
00023 #include "automatonview.h"
00024 #include "automatonviewerdialog.h"
00025 #include "examsession.h"
00026 #include "grammar.hpp"
00027 #include "grammarview.h"
00028 #include "slrl_parser.hpp"
00029 #include "slrtabledialog.h"
00030 #include <QAbstractItemView>
00031 #include <QDialog>
00032 #include <QFileDialog>
00033 #include <QGraphicsColorizeEffect>
00034 #include <QListWidgetItem>
00035 #include <QMessageBox>
00036 #include <QPropertyAnimation>
00037 #include <QPushButton>
00038 #include <QRegularExpression>
00039 #include <QScrollBar>
00040 #include <QShortcut>
00041 #include <QTableWidget>
00042 #include <QTextDocument>
00043 #include <QTextEdit>
00044 #include <QTime>
00045 #include <QTimer>
00046 #include <QVBoxLayout>
00047 #include <QWidget>

```

```

00048 #include <QtPrintSupport/QPrinter>
00049
00050 namespace Ui {
00051 class SLRTutorWindow;
00052 }
00053
00054 // ===== SLR(1) Tutor States =====
00055 enum class StateSlr {
00056     A,
00057     A1,
00058     A2,
00059     A3,
00060     A4,
00061     A_prime,
00062     B,
00063     C,
00064     CA,
00065     CB,
00066     D,
00067     D1,
00068     D2,
00069     D_prime,
00070     E,
00071     E1,
00072     E2,
00073     F,
00074     FA,
00075     G,
00076     H,
00077     H_prime,
00078     fin
00079 };
00080
00081 class TutorialManager;
00082
00083 // ===== Main Tutor Class for SLR(1) =====
00084 class SLRTutorWindow : public QWidget {
00085     Q_OBJECT
00086
00087 public:
00088     void applyTextScale();
00089
00090     // ===== Constructor / Destructor =====
00091     explicit SLRTutorWindow(const Grammar& g, TutorialManager* tm = nullptr,
00092                             QWidget* parent = nullptr, bool examMode = false);
00093     ~SLRTutorWindow();
00094
00095     // ===== Core Flow Control =====
00096     QString generateQuestion();
00097
00098     void updateState(bool isCorrect);
00099     QString
00100     FormatGrammar(const Grammar& grammar);
00101     QVector<GrammarView::Row> buildGrammarRows(const Grammar& grammar) const;
00102     void fillSortedGrammar();
00103
00104     // ===== UI Interaction =====
00105     void addMessage(const QString& text, bool isUser);
00106     void addGrammarMessage();
00107     void addWidgetMessage(QWidget* widget);
00108     void exportConversationToPdf(
00109         const QString& filePath);
00110     void showTable();
00111     void launchSLRWizard();
00112     void updateProgressPanel();
00113     void addUserState(unsigned id);
00114     void addUserTransition(unsigned fromId, const std::string& symbol,
00115                             unsigned toId); // Register a user-created transition
00116
00117     // ===== Visual Feedback & Animations =====
00118     void animateLabelPop(QLabel* label);
00119     void animateLabelColor(QLabel* label, const QColor& flashColor);
00120     void wrongAnimation(); // Label animation for incorrect answer
00121     void wrongUserResponseAnimation(); // Message widget animation for incorrect
00122                                         // answer
00123     void markLastUserIncorrect();
00124
00125     // ===== Response Verification =====
00126     bool verifyResponse(const QString& userResponse);
00127     bool verifyResponseForA(const QString& userResponse);
00128     bool verifyResponseForA1(const QString& userResponse);
00129     bool verifyResponseForA2(const QString& userResponse);
00130     bool verifyResponseForA3(const QString& userResponse);
00131     bool verifyResponseForA4(const QString& userResponse);
00132     bool verifyResponseForB(const QString& userResponse);
00133     bool verifyResponseForC(const QString& userResponse);
00134     bool verifyResponseForCA(const QString& userResponse);

```

```

00173     bool verifyResponseForCB(const QString& userResponse);
00174     bool verifyResponseForD(const QString& userResponse);
00175     bool verifyResponseForD1(const QString& userResponse);
00176     bool verifyResponseForD2(const QString& userResponse);
00177     bool verifyResponseForE(const QString& userResponse);
00178     bool verifyResponseForE1(const QString& userResponse);
00179     bool verifyResponseForE2(const QString& userResponse);
00180     bool verifyResponseForF(const QString& userResponse);
00181     bool verifyResponseForFA(const QString& userResponse);
00182     bool verifyResponseForG(const QString& userResponse);
00183     bool verifyResponseForH();
00184
00185     // ===== Correct Solutions (Auto-generated) =====
00186     QString solution(const std::string& state);
00187     std::unordered_set<Lr0Item> solutionForA();
00188     QString solutionForA1();
00189     QString solutionForA2();
00190     std::vector<std::pair<std::string, std::vector<std::string>>
00191         solutionForA3();
00192     std::unordered_set<Lr0Item> solutionForA4();
00193     unsigned solutionForB();
00194     unsigned solutionForC();
00195     QStringList solutionForCA();
00196     std::unordered_set<Lr0Item> solutionForCB();
00197     QStringList solutionForD();
00198     QString solutionForD1();
00199     QString solutionForD2();
00200     std::ptrdiff_t solutionForE();
00201     QSet<unsigned> solutionForE1();
00202     QMap<unsigned, unsigned> solutionForE2();
00203     QSet<unsigned> solutionForF();
00204     QSet<QString> solutionForFA();
00205
00212     bool isReduction(const Lr0Item& item) const;
00213
00215     QStringList tableColumns();
00216
00219     QString solutionCell(unsigned stateId, const QString& symbol);
00220
00222     QString initialItemText() const;
00223     QSet<QString> solutionForG();
00224
00225     // ===== Pedagogical Feedback =====
00226     QString feedback(); // Delegates to appropriate feedback based on state
00227     QString feedbackForA();
00228     QString feedbackForA1();
00229     QString feedbackForA2();
00230     QString feedbackForA3();
00231     QString feedbackForA4();
00232     QString feedbackForAPrime();
00233     QString feedbackForB();
00234     QString feedbackForB1();
00235     QString feedbackForB2();
00236     QString feedbackForBPrime();
00237     QString feedbackForC();
00238     QString feedbackForCA();
00239     QString feedbackForCB();
00240     QString feedbackForD();
00241     QString feedbackForD1();
00242     QString feedbackForD2();
00243     QString feedbackForDPrime();
00244     QString feedbackForE();
00245     QString feedbackForE1();
00246     QString feedbackForE2();
00247     QString feedbackForF();
00248     QString feedbackForFA();
00249     QString feedbackForG();
00250     QString TeachDeltaFunction(const std::unordered_set<Lr0Item>& items,
00251         const QString& symbol);
00252     void TeachClosureStep(std::unordered_set<Lr0Item>& items, unsigned int size,
00253         std::unordered_set<std::string>& visited, int depth,
00254         QString& output);
00255     QString TeachClosure(const std::unordered_set<Lr0Item>& initialItems);
00256     void updatePlaceholder();
00257     bool confirmExitToHome();
00258     QString promptExportFilePath() const;
00259
00260     // ===== LR(0) Automaton Viewer =====
00268     void setupAutomatonPanel();
00269
00277     void updateAutomatonPanel();
00278
00283     void openAutomatonViewer();
00284
00288     void updateAutomatonButton();
00289
00290     // ===== Exam Mode =====

```

```

00291     void    postQuestion();
00292     QString examSolutionText();
00293     void    scoreExamTable(
00294         const QStringList& colHeaders);
00295     void    showExamReport();
00296 #ifndef SYNTAXTUTOR_TESTING
00297     public:
00298     QString    currentStateForTest() const;
00299     int        rightCountForTest() const;
00300     int        wrongCountForTest() const;
00301     void        setAnswerForTest(const QString& text);
00302     void        submitForTest();
00303     unsigned    currentStateIdForTest() const;
00304     QString    currentCbSymbolForTest() const;
00305     void        setNextExportFilePathForTest(const QString& filePath);
00306     double     examGradeForTest() const { return examSession.grade(); }
00307     int        examTotalForTest() const { return examSession.total(); }
00308     int        examRightForTest() const { return examSession.right(); }
00309 #endif
00310     private slots:
00311     void on_backButton_clicked();
00312     void on_confirmButton_clicked();
00313
00314     signals:
00315     void sessionFinished(int cntRight, int cntWrong);
00316     void exitRequested(bool applyResults, int cntRight, int cntWrong);
00317
00318     protected:
00319     void closeEvent(QCloseEvent* event) override {
00320         emit sessionFinished(cntRightAnswers, cntWrongAnswers);
00321         QWidget::closeEvent(event);
00322     }
00323
00324     bool eventFilter(QObject* obj, QEvent* event) override;
00325
00326     private:
00327     void relayayoutChatMessages();
00328
00329     // ===== Helper Functions =====
00330     std::vector<std::string> QVectorToStdVector(const QVector<QString>& qvec);
00331     QVector<QString> stdVectorToQVector(const std::vector<std::string>& vec);
00332     QSet<QString>
00333     stdUnorderedSetToQSet(const std::unordered_set<std::string>& uset);
00334     std::unordered_set<std::string>
00335     qsetToStdUnorderedSet(const QSet<QString>& qset);
00336     std::unordered_set<Lr0Item> ingestUserItems(const QString& userResponse);
00337     std::vector<std::pair<std::string, std::vector<std::string>>>
00338     ingestUserRules(const QString& userResponse);
00339     void setupTutorial();
00340     void requestExit(bool applyResults);
00341     // ===== Core Components =====
00342     Ui::SLRTutorWindow* ui;
00343     Grammar grammar;
00344     SLR1Parser slr1;
00345
00346     // ===== State and Grammar Tracking =====
00347     StateSlr currentState;
00348     QVector<QString> sortedNonTerminals;
00349     QVector<QPair<QString, QVector<QString>>> sortedGrammar;
00350     QString formattedGrammar;
00351     GrammarView* grammarView = nullptr;
00352
00353     unsigned cntRightAnswers = 0;
00354     unsigned cntWrongAnswers = 0;
00355
00356     // ===== Exam Mode =====
00357     bool examMode = false;
00358     ExamSession examSession;
00359     QString currentQuestionText;
00360
00361     // ===== State Machine Runtime Variables =====
00362     std::unordered_set<state> userMadeStates; // All states the user has created
00363     std::unordered_map<unsigned, std::unordered_map<std::string, unsigned>
00364         userMadeTransitions; // Transitions made by the user
00365     UniqueQueue<unsigned>
00366         statesIdQueue; // States to be processed in B-C-CA-CB loop
00367     unsigned currentStateId = 0;
00368     state currentSlrState;
00369
00370     QStringList followSymbols; // Used in CA-CB loop
00371     qsize_t currentFollowSymbolsIdx = 0;
00372     unsigned int nextStateId = 0;
00373
00374     QVector<const state*> statesWithLr0Conflict; // Populated in F
00375     std::queue<unsigned> conflictStatesIdQueue;
00376     unsigned currentConflictStateId = 0;
00377

```

```

00378 // ===== LR(0) Automaton Viewer =====
00379 AutomatonView* automatonView = nullptr; // null in exam mode
00380 AutomatonViewerDialog* automatonViewer = nullptr; // null when closed
00381 QSet<unsigned> reduceStateIds; // States highlighted in G
00382 state currentConflictState;
00383
00384 std::queue<unsigned>
00385     reduceStatesIdQueue; // States without conflicts but with reduce
00386 unsigned currentReduceStateId = 0;
00387 state currentReduceState;
00388
00389 struct ActionEntry {
00390     enum Type { Shift, Reduce, Accept, Goto } type;
00391     int target;
00392     static ActionEntry makeShift(int s) { return {Shift, s}; }
00393     static ActionEntry makeReduce(int r) { return {Reduce, r}; }
00394     static ActionEntry makeAccept() { return {Accept, 0}; }
00395     static ActionEntry makeGoto(int g) { return {Goto, g}; }
00396 };
00397
00398 QMap<int, QMap<QString, ActionEntry> > slrtable;
00399 QVector<QVector<QString> > rawTable;
00400
00401 // ===== Conversation Log =====
00402 struct MessageLog {
00403     QString message;
00404     bool isUser;
00405     bool isCorrect = true;
00406
00407     MessageLog(const QString& message, bool isUser)
00408         : message(message), isUser(isUser) {}
00409
00410     void toggleIsCorrect() { isCorrect = false; }
00411 };
00412
00413 QVector<MessageLog> conversationLog;
00414 QWidget* lastUserMessage = nullptr;
00415 qsize_t lastUserMessageLogIdx = -1;
00416
00417 QPropertyAnimation* m_shakeAnimation =
00418     nullptr; // For interrupting userResponse animation if they spam enter
00419             // key
00420
00421 TutorialManager* tm;
00422
00423 #ifndef SYNTAXTUTOR_TESTING
00424     mutable QString nextExportFilePathForTest;
00425 #endif
00426
00427 QRegularExpression re{"^\\s+|\\s+$"};
00428 const QRegularExpression kWhitespace{"\\s+"};
00429 };
00430
00431 #endif // SLRTUTORWINDOW_H

```

10.60 gui/slrwizard.h File Reference

```

#include "slrl_parser.hpp"
#include "slrwizardpage.h"
#include <QDialog>
#include <QFrame>
#include <QHBoxLayout>
#include <QLabel>
#include <QProgressBar>
#include <QPushButton>
#include "apptypography.h"
#include "applayout.h"
#include <QStackedWidget>
#include <QTimer>
#include <QVBoxLayout>

```



```

00048     Q_OBJECT
00049 public:
00062     SLRWizard(SLR1Parser& parser, const QVector<QVector<QString>& rawTable,
00063               const QStringList& colHeaders,
00064               const QVector<QPair<QString, QVector<QString>>& sortedGrammar,
00065               QWidget* parent = nullptr)
00066     : QDialog(parent) {
00067         setProperty("guidedDialog", "slr");
00068         setWindowTitle(tr("Completar tabla SLR"));
00069         setWindowFlag(Qt::WindowCloseButtonHint, true);
00070         setModal(true);
00071         resize(AppTypography::lengthForText(640),
00072               AppTypography::lengthForText(440));
00073         AppLayout::keepHeightForWidth(this);
00074         setMinimumSize(AppTypography::lengthForText(560),
00075               AppTypography::lengthForText(400));
00076
00077         auto* rootLayout = new QVBoxLayout(this);
00078         rootLayout->setContentsMargins(0, 0, 0, 0);
00079         rootLayout->setSpacing(0);
00080
00081         auto* panel = new QFrame(this);
00082         panel->setObjectName("slrWizardPanel");
00083
00084         auto* panelLayout = new QVBoxLayout(panel);
00085         panelLayout->setContentsMargins(28, 24, 28, 24);
00086         panelLayout->setSpacing(20);
00087
00088         auto* headerLayout = new QHBoxLayout();
00089         headerLayout->setSpacing(12);
00090
00091         auto* titleColumn = new QVBoxLayout();
00092         titleColumn->setContentsMargins(0, 0, 0, 0);
00093         titleColumn->setSpacing(6);
00094
00095         m_titleLabel = new QLabel(panel);
00096         m_titleLabel->setObjectName("slrWizardTitle");
00097         titleColumn->addWidget(m_titleLabel);
00098
00099         m_stepCounterLabel = new QLabel(panel);
00100         m_stepCounterLabel->setObjectName("slrWizardStepCounter");
00101         titleColumn->addWidget(m_stepCounterLabel);
00102
00103         m_progressBar = new QProgressBar(panel);
00104         m_progressBar->setObjectName("slrWizardProgress");
00105         m_progressBar->setTextVisible(false);
00106         m_progressBar->setRange(0, 100);
00107         m_progressBar->setFixedHeight(4);
00108         titleColumn->addWidget(m_progressBar);
00109
00110         headerLayout->addLayout(titleColumn, 1);
00111
00112         m_closeButton = new QPushButton(tr("Salir"), panel);
00113         m_closeButton->setObjectName("slrWizardCloseButton");
00114         m_closeButton->setAutoDefault(false);
00115         m_closeButton->setDefault(false);
00116         m_closeButton->setCursor(Qt::PointingHandCursor);
00117         headerLayout->addWidget(m_closeButton, 0, Qt::AlignTop);
00118
00119         panelLayout->addLayout(headerLayout);
00120
00121         m_stack = new QStackedWidget(panel);
00122         m_stack->setObjectName("slrWizardStack");
00123         panelLayout->addWidget(m_stack, 1);
00124
00125         auto* footerLayout = new QHBoxLayout();
00126         footerLayout->setContentsMargins(0, 0, 0, 0);
00127         footerLayout->setSpacing(12);
00128         footerLayout->addStretch(1);
00129
00130         m_nextButton = new QPushButton(panel);
00131         m_nextButton->setObjectName("slrWizardNextButton");
00132         m_nextButton->setProperty("role", "primary");
00133         m_nextButton->setAutoDefault(false);
00134         m_nextButton->setDefault(false);
00135         footerLayout->addWidget(m_nextButton);
00136         panelLayout->addLayout(footerLayout);
00137
00138         rootLayout->addWidget(panel);
00139
00140         const int nTerm = parser.gr_.st_.terminals_.size();
00141         for (int i = 0; i < rawTable.size(); ++i) {
00142             for (int j = 0; j < colHeaders.size(); ++j) {
00143                 const QString sym = colHeaders[j];
00144                 QString expected;
00145                 QString explanation;
00146             }

```



```

00147         if (j < nTerm) {
00148             auto itAct = parser.actions_.at(i).find(sym.toStdString());
00149             SLR1Parser::s_action act =
00150                 (itAct != parser.actions_.at(i).end()
00151                  ? itAct->second
00152                  : SLR1Parser::s_action{nullptr,
00153                                         SLR1Parser::Action::Empty});
00154
00155             switch (act.action) {
00156             case SLR1Parser::Action::Shift: {
00157                 const unsigned to =
00158                     parser.transitions_.at(i).at(sym.toStdString());
00159                 expected = QString("s%1").arg(to);
00160                 explanation = tr("Estado %1: existe transición  $\delta$ (%1, "
00161                                "%2'). ¿A qué estado harías shift?"
00162                                .arg(i)
00163                                .arg(sym);
00164                 break;
00165             }
00166             case SLR1Parser::Action::Reduce: {
00167                 int idx = -1;
00168                 for (int k = 0; k < sortedGrammar.size(); ++k) {
00169                     auto& rule = sortedGrammar[k];
00170                     if (rule.first.toStdString() ==
00171                         act.item->antecedent_ &&
00172                         std::vectorToQVector(act.item->consequent_) ==
00173                         rule.second) {
00174                         idx = k;
00175                         break;
00176                     }
00177                 }
00178                 expected = QString("r%1").arg(idx);
00179                 explanation = tr("Estado %1: contiene el ítem [%2 → "
00180                                "...] y '%3' ∈ SIG(%2). ¿Qué regla "
00181                                "usas para reducir (0, 1, ...)?")
00182                                .arg(i)
00183                                .arg(QString::fromStdString(
00184                                    act.item->antecedent_))
00185                                .arg(colHeaders[j]);
00186                 break;
00187             }
00188             case SLR1Parser::Action::Accept:
00189                 expected = "acc";
00190                 explanation = tr("Estado %1: contiene [S → A · $]. "
00191                                "¿Qué palabra clave usas para "
00192                                "aceptar?"
00193                                .arg(i);
00194                 break;
00195             case SLR1Parser::Action::Empty:
00196             default:
00197                 continue;
00198             }
00199         } else {
00200             const auto nonT = sym.toStdString();
00201             if (!parser.transitions_.contains(i)) {
00202                 continue;
00203             }
00204
00205             auto itGo = parser.transitions_.at(i).find(nonT);
00206             if (itGo == parser.transitions_.at(i).end()) {
00207                 continue;
00208             }
00209
00210             expected = QString::number(itGo->second);
00211             explanation = tr("Estado %1:  $\delta$ (%1, '%2') existe. ¿A qué "
00212                            "estado va la transición? (pon solo el "
00213                            "número)"
00214                            .arg(i)
00215                            .arg(sym);
00216         }
00217
00218         auto* page = new SLRWizardPage(i, sym, explanation, expected,
00219                                       m_stack);
00220         connect(page, &SLRWizardPage::completionChanged, this,
00221                &SLRWizard::updateCurrentPage);
00222         connect(page, &SLRWizardPage::submitRequested, this,
00223                &SLRWizard::advance);
00224         m_stack->addWidget(page);
00225     }
00226 }
00227
00228 connect(m_closeButton, &QPushButton::clicked, this,
00229        &QDialog::reject);
00230 connect(m_nextButton, &QPushButton::clicked, this,
00231        &SLRWizard::advance);
00232
00233 updateCurrentPage();

```

```

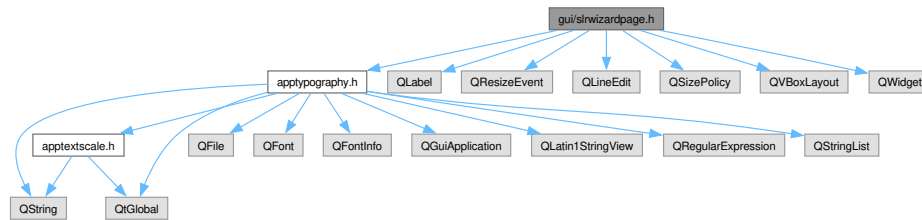
00234     QTimer::singleShot(0, this, [this]() {
00235         if (auto* page = currentPage()) {
00236             page->focusAnswerField();
00237         }
00238     });
00239 }
00240
00241 SLRWizardPage* currentPage() const {
00242     return qobject_cast<SLRWizardPage*>(m_stack->currentWidget());
00243 }
00244
00245 QPushButton* nextButton() const { return m_nextButton; }
00246 QPushButton* closeButton() const { return m_closeButton; }
00247
00248 bool isOnLastPage() const {
00249     return m_stack->count() > 0 &&
00250         m_stack->currentIndex() == m_stack->count() - 1;
00251 }
00252
00253 QVector<QString> stdVectorToQVector(const std::vector<std::string>& vec) {
00254     QVector<QString> result;
00255     result.reserve(vec.size());
00256     for (const auto& str : vec) {
00257         result.push_back(QString::fromStdString(str));
00258     }
00259     return result;
00260 }
00261
00262 private slots:
00263 void advance() {
00264     auto* page = currentPage();
00265     if (page == nullptr || !page->isComplete()) {
00266         return;
00267     }
00268     if (isOnLastPage()) {
00269         accept();
00270         return;
00271     }
00272     m_stack->setCurrentIndex(m_stack->currentIndex() + 1);
00273     updateCurrentPage();
00274     if (auto* nextPage = currentPage()) {
00275         nextPage->focusAnswerField();
00276     }
00277 }
00278
00279 void updateCurrentPage() {
00280     auto* page = currentPage();
00281     if (page == nullptr) {
00282         m_titleLabel->setText(tr("Modo guiado"));
00283         m_stepCounterLabel->clear();
00284         m_progressBar->setValue(0);
00285         m_nextButton->setEnabled(false);
00286         m_nextButton->setText(tr("Finalizar"));
00287         return;
00288     }
00289     const int currentStep = m_stack->currentIndex() + 1;
00290     const int totalSteps = m_stack->count();
00291     m_titleLabel->setText(page->titleText());
00292     m_stepCounterLabel->setText(
00293         tr("Paso %1 de %2").arg(currentStep).arg(totalSteps));
00294     m_progressBar->setValue((100 * currentStep) / qMax(1, totalSteps));
00295     m_nextButton->setEnabled(page->isComplete());
00296     m_nextButton->setCursor(page->isComplete() ? Qt::PointingHandCursor
00297         : Qt::ArrowCursor);
00298     m_nextButton->setText(isOnLastPage() ? tr("Finalizar")
00299         : tr("Continuar"));
00300 }
00301
00302 private:
00303 QLabel* m_titleLabel = nullptr;
00304 QLabel* m_stepCounterLabel = nullptr;
00305 QProgressBar* m_progressBar = nullptr;
00306 QPushButton* m_closeButton = nullptr;
00307 QStackedWidget* m_stack = nullptr;
00308 QPushButton* m_nextButton = nullptr;
00309 };
00310
00311 #endif // SLRWIZARD_H

```

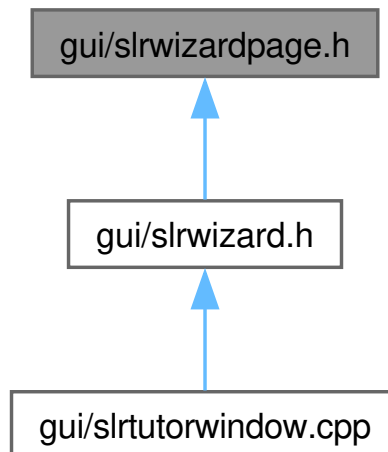
10.62 gui/slrwizardpage.h File Reference

```
#include "apptypography.h"
#include <QLabel>
#include <QResizeEvent>
#include <QLineEdit>
#include <QSizePolicy>
#include <QVBoxLayout>
#include <QWidget>
```

Include dependency graph for slrwizardpage.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [SLRWizardPage](#)
A single guided step for constructing the SLR(1) table.

10.63 slrwizardpage.h

[Go to the documentation of this file.](#)

```
00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
```

```

00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef SLRWIZARDPAGE_H
00020 #define SLRWIZARDPAGE_H
00021
00022 #include "apptypography.h"
00023 #include <QLabel>
00024 #include <QResizeEvent>
00025 #include <QLineEdit>
00026 #include <QSizePolicy>
00027 #include <QVBoxLayout>
00028 #include <QWidget>
00029
00030 class SLRWizardPage : public QWidget {
00031     Q_OBJECT
00032 public:
00033     SLRWizardPage(int state, const QString& symbol, const QString& explanation,
00034                   const QString& expected, QWidget* parent = nullptr)
00035         : QWidget(parent), m_title(tr("Estado %1, símbolo '%2'").arg(state).arg(symbol)),
00036           m_expected(expected) {
00037         auto* rootLayout = new QVBoxLayout(this);
00038         rootLayout->setContentsMargins(0, 0, 0, 0);
00039         rootLayout->setSpacing(22);
00040
00041         auto* explanationTitle = new QLabel(tr("Pista"), this);
00042         explanationTitle->setObjectName("slrWizardSectionTitle");
00043         rootLayout->addWidget(explanationTitle);
00044
00045         auto* explanationLabel = new QLabel(explanation, this);
00046         explanationLabel->setObjectName("slrWizardExplanationLabel");
00047         explanationLabel->setWordWrap(true);
00048         rootLayout->addWidget(explanationLabel);
00049         m_explanation = explanationLabel;
00050
00051         auto* answerTitle = new QLabel(tr("Tu respuesta"), this);
00052         answerTitle->setObjectName("slrWizardSectionTitle");
00053         rootLayout->addWidget(answerTitle);
00054
00055         m_edit = new QLineEdit(this);
00056         m_edit->setObjectName("slrWizardAnswerEdit");
00057         m_edit->setMinimumHeight(AppTypography::lengthForText(48));
00058         m_edit->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Fixed);
00059         m_edit->setPlaceholderText(
00060             tr("Escribe tu respuesta (p.ej. s3, r2, acc, 5)"));
00061         rootLayout->addWidget(m_edit);
00062
00063         m_feedback = new QLabel(this);
00064         m_feedback->setObjectName("slrWizardFeedbackLabel");
00065         m_feedback->setWordWrap(true);
00066         m_feedback->setMinimumHeight(AppTypography::lengthForText(40));
00067         m_feedback->setSizePolicy(QSizePolicy::Preferred, QSizePolicy::Fixed);
00068         rootLayout->addWidget(m_feedback);
00069
00070         rootLayout->addStretch(1);
00071
00072         connect(m_edit, &QLineEdit::textChanged, this,
00073             &SLRWizardPage::onTextChanged);
00074         connect(m_edit, &QLineEdit::returnPressed, this,
00075             &SLRWizardPage::submitRequested);
00076     }
00077
00078 protected:
00079     void resizeEvent(QResizeEvent* event) override {
00080         QWidget::resizeEvent(event);
00081         // These pages live in a QStackedWidget, which does not honour a
00082         // word-wrapped label's height-for-width: the hint ends up clipped to
00083         // whatever height was left over. Pin its minimum to what the text
00084         // actually needs at the width it got.
00085         if (m_explanation != nullptr && m_explanation->width() > 0) {
00086             m_explanation->setMinimumHeight(
00087                 m_explanation->heightForWidth(m_explanation->width()));
00088         }
00089     }
00090
00091 public:
00092     QString titleText() const { return m_title; }
00093     bool isComplete() const { return m_isComplete; }
00094     void focusAnswerField() { m_edit->setFocus(); }
00095
00096
00097
00098
00099
00100
00101
00102
00103
00104
00105
00106
00107
00108
00109
00110
00111
00112
00113

```

```

00114 #ifdef SYNTAXTUTOR_TESTING
00115     QString expectedForTest() const { return m_expected; }
00116 #endif
00117
00118     signals:
00119         void completionChanged(bool complete);
00120         void submitRequested();
00121
00122     private slots:
00127         void onTextChanged(const QString& text) {
00128             const QString trimmed = text.trimmed();
00129             const bool correct = (trimmed == m_expected);
00130
00131             if (trimmed.isEmpty()) {
00132                 m_feedback->clear();
00133             } else if (correct) {
00134                 m_feedback->setText(
00135                     tr("✓ Respuesta correcta, pasa a la siguiente pregunta"));
00136             } else {
00137                 m_feedback->setText(tr("X Incorrecto, revisa el enunciado. "
00138                                         "Consulta los estados que has "
00139                                         "construido.));
00140             }
00141
00142             if (m_isComplete == correct) {
00143                 return;
00144             }
00145
00146             m_isComplete = correct;
00147             emit completionChanged(m_isComplete);
00148         }
00149
00150     private:
00151         QString m_title;
00152         QString m_expected;
00153         QLabel* m_feedback;
00154         QLabel* m_explanation = nullptr;
00155         QLineEdit* m_edit;
00156         bool m_isComplete =
00157             false;
00158 };
00159
00160 #endif // SLRWIZARDPAGE_H

```

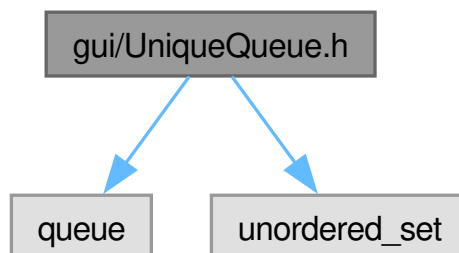
10.64 gui/UniqueQueue.h File Reference

```

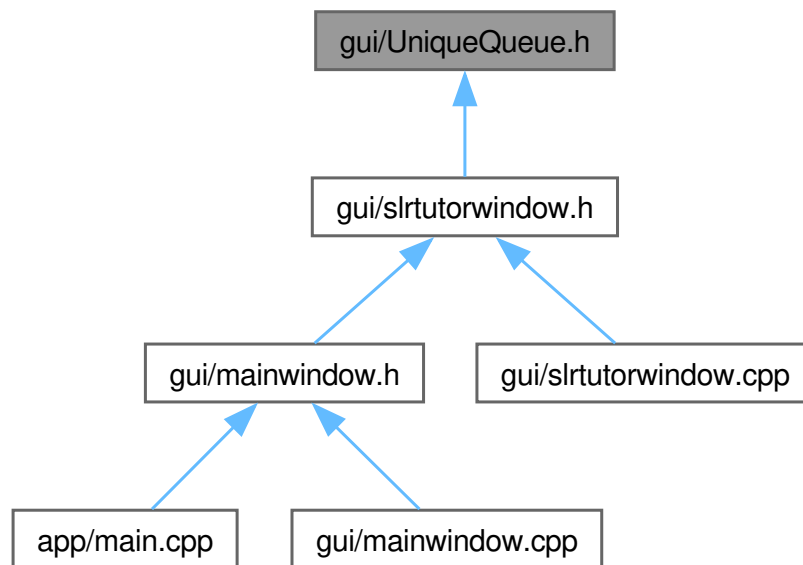
#include <queue>
#include <unordered_set>

```

Include dependency graph for UniqueQueue.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `UniqueQueue< T >`

A queue that ensures each element is inserted only once.

10.65 UniqueQueue.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef UNIQUEQUEUE_H
00020 #define UNIQUEQUEUE_H
00021 #include <queue>
00022 #include <unordered_set>
00023
00037 template <typename T> class UniqueQueue {
00038     public:
00043     void push(const T& value) {
00044         if (seen_.insert(value).second) {
00045             queue_.push(value);
00046         }
00047     }
00048
00052     void pop() {
00053         if (!queue_.empty()) {
00054             queue_.pop();
00055         }
00056     }

```

```

00057
00062     const T& front() const { return queue_.front(); }
00063
00068     bool empty() const { return queue_.empty(); }
00069
00073     void clear() {
00074         while (!queue_.empty())
00075             queue_.pop();
00076         seen_.clear();
00077     }
00078
00079     private:
00080         std::queue<T> queue_;
00081         std::unordered_set<T> seen_;
00082 };
00083 #endif // UNIQUEQUEUE_H

```

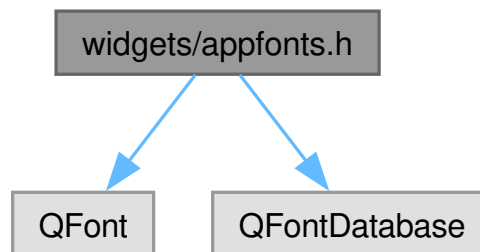
10.66 widgets/appfonts.h File Reference

```

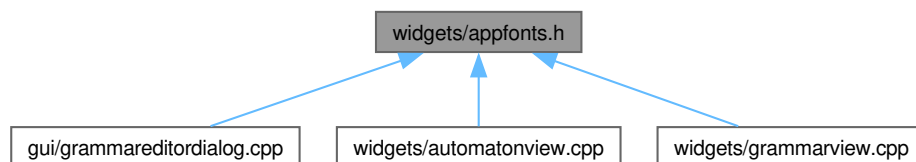
#include <QFont>
#include <QFontDatabase>

```

Include dependency graph for appfonts.h:



This graph shows which files directly or indirectly include this file:



Functions

- QFont [appMonospaceFont](#) ()
Application-wide monospace font.

10.66.1 Function Documentation

10.66.1.1 appMonospaceFont()

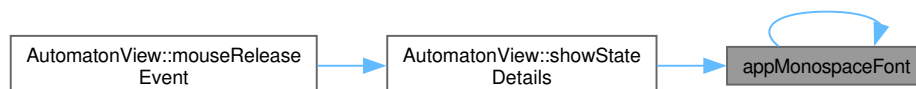
```
QFont appMonospaceFont () [inline]
```

Application-wide monospace font.

macOS keeps the native fixed font (Menlo); Windows and Linux use the bundled JetBrains Mono, loaded in [main\(\)](#), because their system fixed fonts (Courier New, distro-dependent) degrade the visuals. Here is the call graph for this function:



Here is the caller graph for this function:



10.67 appfonts.h

[Go to the documentation of this file.](#)

```

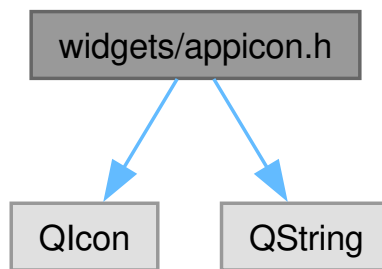
00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef APPFONTS_H
00020 #define APPFONTS_H
00021
00022 #include <QFont>
00023 #include <QFontDatabase>
00024
00025 inline QFont appMonospaceFont() {
00026     QFont font = QFontDatabase::systemFont(QFontDatabase::FixedFont);
00027     #ifndef Q_OS_MACOS
00028         font.setFamily(QStringLiteral("JetBrains Mono"));
00029         font.setStyleHint(QFont::Monospace);
00030     #endif
00031     return font;
00032 }
00033
00034 #endif // APPFONTS_H
  
```

10.68 widgets/appicon.h File Reference

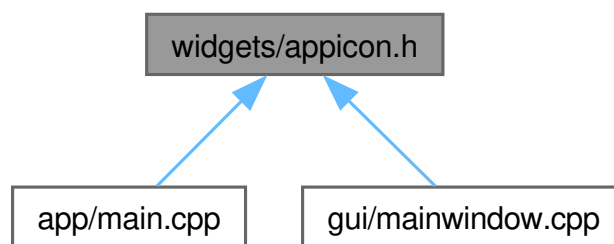
```

#include <QIcon>
#include <QString>
  
```


Include dependency graph for appicon.h:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [AppIcon](#)

Functions

- QIcon [AppIcon::icon](#) ()
The application icon, with a drawing for each size.

Variables

- constexpr int [AppIcon::kSizes](#) [] = {16, 24, 32, 48, 64, 128, 256, 512}
Sizes of the bitmap icon shipped in the resources, in pixels.

10.69 appicon.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
  
```

```

00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef APPICON_H
00020 #define APPICON_H
00021
00022 #include <QIcon>
00023 #include <QString>
00024
00025 namespace AppIcon {
00026
00028 constexpr int kSizes[] = {16, 24, 32, 48, 64, 128, 256, 512};
00029
00037 inline QIcon icon() {
00038     QIcon icon;
00039     for (const int size : kSizes) {
00040         icon.addFile(
00041             QStringLiteral(":/resources/icon/png/syntaxtutor-%1.png").arg(size),
00042             QSize(size, size));
00043     }
00044     return icon;
00045 }
00046
00047 } // namespace AppIcon
00048
00049 #endif // APPICON_H

```

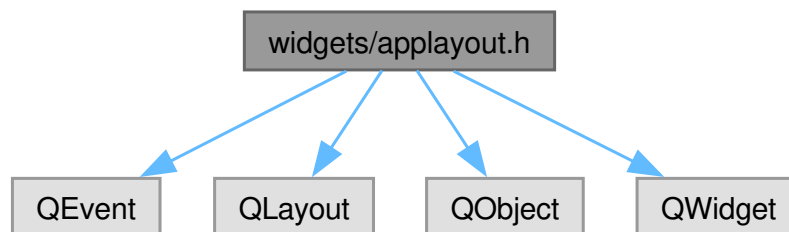
10.70 widgets/aplayout.h File Reference

```

#include <QEvent>
#include <QLayout>
#include <QObject>
#include <QWidget>

```

Include dependency graph for aplayout.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [AppLayout::HeightForWidthKeeper](#)
Event filter behind [keepHeightForWidth](#).

Namespaces

- namespace [AppLayout](#)

Layout helpers that keep text from being clipped.

Functions

- void [AppLayout::keepHeightForWidth](#) (QWidget *widget)

Keeps widget at least as tall as its layout needs at its current width.

10.71 applayout.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef APPLAYOUT_H
00020 #define APPLAYOUT_H
00021
00022 #include <QEvent>
00023 #include <QLayout>
00024 #include <QObject>
00025 #include <QWidget>
00026
00027 namespace AppLayout {
00028
00029 class HeightForWidthKeeper : public QObject {
00030 public:
00031     explicit HeightForWidthKeeper(QWidget* widget)
00032         : QObject(widget), widget_(widget) {
00033         widget_->installEventFilter(this);
00034     }
00035
00036 protected:
00037     bool eventFilter(QObject* watched, QEvent* event) override {
00038         if (watched == widget_ && (event->type() == QEvent::Resize ||
00039             event->type() == QEvent::LayoutRequest ||
00040             event->type() == QEvent::Show)) {
00041             update();
00042         }
00043         return false;
00044     }
00045
00046 private:
00047     void update() {
00048         QLayout* layout = widget_->layout();
00049         if (layout == nullptr || !layout->hasHeightForWidth()) {
00050             return;
00051         }
00052         // The minimum, not the preferred height: the preferred one also
00053         // counts what spacers would like to have, which over-constrains.
00054         const int needed =
00055             layout->totalMinimumHeightForWidth(widget_->width());
00056         if (needed <= 0) {
00057             return;
00058         }
00059         if (widget_->minimumHeight() != needed) {
00060             widget_->setMinimumHeight(needed);
00061         }
00062     }
00063
00064     QWidget* widget_;
00065 };
00066
00067 }
00068
00069 #endif

```

```

00073         // A window is not resized by its minimum alone once it is shown.
00074         if (widget_>isWindow() && widget_>height() < needed) {
00075             widget_>resize(widget_>width(), needed);
00076         }
00077     }
00078
00079     QWidget* widget_;
00080 };
00081
00090 inline void keepHeightForWidth(QWidget* widget) {
00091     new HeightForWidthKeeper(widget);
00092 }
00093
00094 } // namespace AppLayout
00095
00096 #endif // APPLAYOUT_H

```

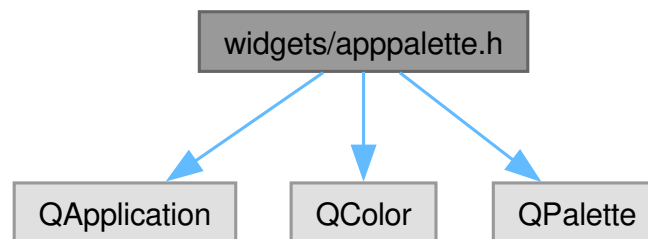
10.72 widgets/apppalette.h File Reference

```

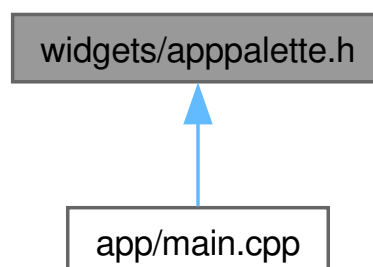
#include <QApplication>
#include <QColor>
#include <QPalette>

```

Include dependency graph for apppalette.h:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [AppPalette](#)

Functions

- QColor [AppPalette::linkColor](#) ()
Colour of links in rich text: the teal accent of the theme.
- void [AppPalette::apply](#) ()
Palette entries the theme needs that app.qss cannot set.

10.73 apppalette.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef APPPALETTE_H
00020 #define APPPALETTE_H
00021
00022 #include <QApplication>
00023 #include <QColor>
00024 #include <QPalette>
00025
00026 namespace AppPalette {
00027
00028     inline QColor linkColor() { return QColor(0x36, 0xC5, 0xCC); }
00029
00030     inline void apply() {
00031         QPalette palette = QApplication::palette();
00032         palette.setColor(QPalette::Link, linkColor());
00033         palette.setColor(QPalette::LinkVisited, linkColor());
00034         QApplication::setPalette(palette);
00035     }
00036 } // namespace AppPalette
00037
00038 #endif // APPPALETTE_H

```

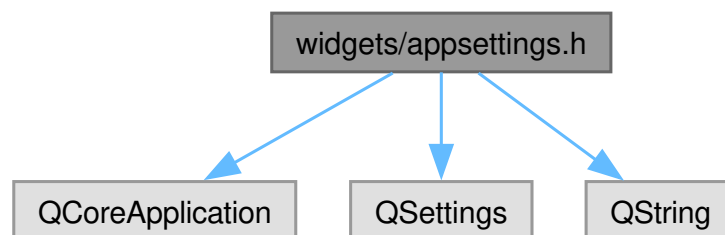
10.74 widgets/appsettings.h File Reference

```

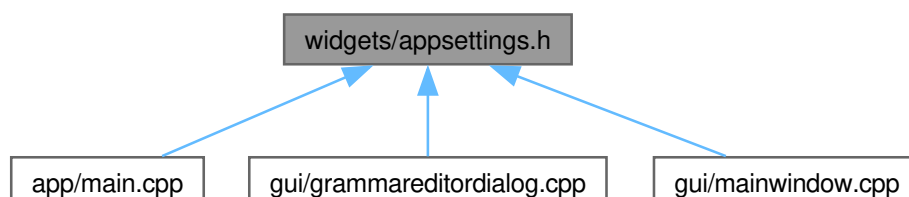
#include <QCoreApplication>
#include <QSettings>
#include <QString>

```

Include dependency graph for appsettings.h:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [AppSettings](#)
Where the app keeps its settings.

Functions

- void [AppSettings::registerIdentity](#) ()
Registers the organisation, domain and application name.
- QSettings [AppSettings::open](#) ()
The app's settings.

10.75 appsettings.h

[Go to the documentation of this file.](#)

```

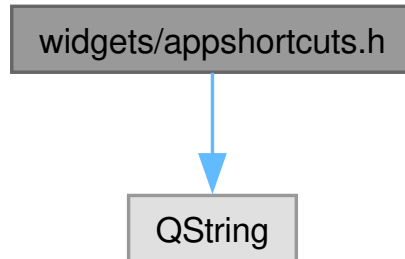
00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef APPSETTINGS_H
00020 #define APPSETTINGS_H
00021
00022 #include <QCoreApplication>
00023 #include <QSettings>
00024 #include <QString>
00025
00026 namespace AppSettings {
00027
00028 inline void registerIdentity() {
00029     QCoreApplication::setOrganizationName(QStringLiteral("SyntaxTutor"));
00030     QCoreApplication::setOrganizationDomain(QStringLiteral("jram.me"));
00031     QCoreApplication::setApplicationName(QStringLiteral("SyntaxTutor"));
00032 }
00033
00034 inline QSettings open() {
00035     #ifdef SYNTAXTUTOR_TESTING
00036         return QSettings(QStringLiteral("SyntaxTutor-Test"),
00037                         QStringLiteral("SyntaxTutor-Test"));
00038     #else
00039         return QSettings();
00040     #endif
00041 }
00042
00043 } // namespace AppSettings
00044
00045 #endif // APPSETTINGS_H

```

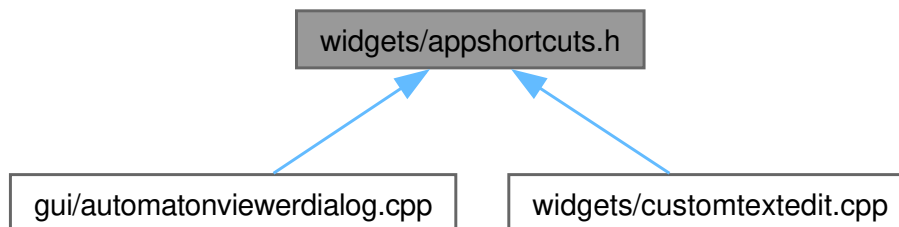
10.76 widgets/appshortcuts.h File Reference

```
#include <QString>
```

Include dependency graph for appshortcuts.h:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [AppShortcuts](#)

Functions

- `QString AppShortcuts::primaryModifierName ()`
Name of the key behind Qt::ControlModifier on this platform.

10.77 appshortcuts.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
  
```


- *Multiplier currently in effect.*
- QString `AppTextScale::settingsKey ()`
Settings key holding the selected size, in percent.
- QString `AppTextScale::legacySettingsKey ()`
Key used by the first version of this setting, which named steps.
- int `AppTextScale::percentFromLegacyValue (const QString &value)`
Percentage matching one of the original named steps.

Variables

- constexpr int `AppTextScale::kMinPercent = 80`
Smallest text size the user can pick, in percent.
- constexpr int `AppTextScale::kMaxPercent = 200`
Largest text size the user can pick, in percent.
- constexpr int `AppTextScale::kDefaultPercent = 100`
Size used until the user picks one, in percent.
- constexpr int `AppTextScale::kPresetPercents [] = {90, 100, 115, 130, 150, 175, 200}`
Sizes offered directly in the menu, in percent.

10.79 apptextscale.h

[Go to the documentation of this file.](#)

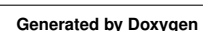
```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef APPTEXTSCALE_H
00020 #define APPTEXTSCALE_H
00021
00022 #include <QString>
00023 #include <QtGlobal>
00024
00025 namespace AppTextScale {
00026
00027     constexpr int kMinPercent = 80;
00028     constexpr int kMaxPercent = 200;
00029     constexpr int kDefaultPercent = 100;
00030     constexpr int kPresetPercents[] = {90, 100, 115, 130, 150, 175, 200};
00031
00032     inline int clampPercent(int percent) {
00033         return qBound(kMinPercent, percent, kMaxPercent);
00034     }
00035
00036     inline int& currentPercent() {
00037         static int percent = kDefaultPercent;
00038         return percent;
00039     }
00040
00041     inline double factor() { return currentPercent() / 100.0; }
00042
00043     inline QString settingsKey() { return QStringLiteral("ui/textScalePercent"); }
00044
00045     inline QString legacySettingsKey() { return QStringLiteral("ui/textScale"); }
00046
00047 }

```

10.80 widgets/apptypography.h File Reference

Include dependency graph for apptypography.h:



Enumerations

- enum class `AppTypography::Role` {
`AppTypography::Micro` , `AppTypography::Caption` , `AppTypography::Label` , `AppTypography::Body` ,
`AppTypography::BodyLarge` , `AppTypography::Reading` , `AppTypography::Action` , `AppTypography::Headline`
, `AppTypography::TitleSmall` , `AppTypography::Title` , `AppTypography::Hero` , `AppTypography::Display` }

What a piece of text is for, which fixes its size.

Functions

- qreal `AppTypography::basePoints` (`Role` role)
Size of `role` at a 100% text scale, in points.
- qreal `AppTypography::points` (`Role` role)
Size of `role` at the current text scale, in points.
- QString `AppTypography::formatPoints` (qreal points)
A point size written the way CSS and QSS expect it, e.g. 14.5pt.
- QString `AppTypography::cssSize` (`Role` role)
Current size of `role` as a CSS length, for rich text and QSS.
- QFont & `AppTypography::systemFont` ()
The system font as it was before the app touched it.
- QFont `AppTypography::applicationFont` ()
The application font: the system family at the body size.
- QFont `AppTypography::font` (`Role` role, QFont base=QGuiApplication::font())
base resized to `role`, keeping its family and style.
- QString `AppTypography::resolveStyleSheet` (const QString &styleSheet, QStringList *unresolved=nullptr)
Replaces every `$font-<name>` in `styleSheet` with its size.
- QString `AppTypography::loadStyleSheet` ()
The application style sheet with its sizes resolved.
- int `AppTypography::lengthForText` (int designPx)
A length that has to hold text, grown to how big text renders here.

Variables

- constexpr `Token AppTypography::kTokens` []
The type scale.

10.81 apptypography.h

[Go to the documentation of this file.](#)

```
00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
```

```

00015  * You should have received a copy of the GNU General Public License
00016  * along with this program.  If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef APPTTYPOGRAPHY_H
00020 #define APPTTYPOGRAPHY_H
00021
00022 #include "apptextscale.h"
00023
00024 #include <QFile>
00025 #include <QFont>
00026 #include <QFontInfo>
00027 #include <QGuiApplication>
00028 #include <QLatin1StringView>
00029 #include <QRegularExpression>
00030 #include <QString>
00031 #include <QStringList>
00032 #include <QtGlobal>
00033
00049 namespace AppTypography {
00050
00052 enum class Role {
00053     Micro,
00054     Caption,
00055     Label,
00056     Body,
00057     BodyLarge,
00058     Reading,
00059     Action,
00060     Headline,
00061     TitleSmall,
00062     Title,
00063     Hero,
00064     Display,
00065 };
00066
00068 struct Token {
00069     Role          role;
00070     QLatin1StringView name;
00071     qreal         points;
00072 };
00073
00082 inline constexpr Token kTokens[] = {
00083     {Role::Micro, QLatin1StringView("micro"), 10.0},
00084     {Role::Caption, QLatin1StringView("caption"), 11.0},
00085     {Role::Label, QLatin1StringView("label"), 12.0},
00086     {Role::Body, QLatin1StringView("body"), 13.0},
00087     {Role::BodyLarge, QLatin1StringView("body-large"), 14.0},
00088     {Role::Reading, QLatin1StringView("reading"), 15.0},
00089     {Role::Action, QLatin1StringView("action"), 16.0},
00090     {Role::Headline, QLatin1StringView("headline"), 20.0},
00091     {Role::TitleSmall, QLatin1StringView("title-small"), 24.0},
00092     {Role::Title, QLatin1StringView("title"), 28.0},
00093     {Role::Hero, QLatin1StringView("hero"), 30.0},
00094     {Role::Display, QLatin1StringView("display"), 56.0},
00095 };
00096
00098 inline qreal basePoints(Role role) {
00099     for (const Token& token : kTokens) {
00100         if (token.role == role) {
00101             return token.points;
00102         }
00103     }
00104     Q_UNREACHABLE_RETURN(13.0);
00105 }
00106
00108 inline qreal points(Role role) {
00109     return basePoints(role) * AppTextScale::factor();
00110 }
00111
00113 inline QString formatPoints(qreal points) {
00114     return QString::number(points, 'g', 6) + QStringLiteral("pt");
00115 }
00116
00118 inline QString cssSize(Role role) { return formatPoints(points(role)); }
00119
00126 inline QFont& systemFont() {
00127     static QFont font;
00128     return font;
00129 }
00130
00132 inline QFont applicationFont() {
00133     QFont font = systemFont();
00134     font.setPointSizeF(points(Role::Body));
00135     return font;
00136 }
00137

```

```

00139 inline QFont font(Role role, QFont base = QApplication::font()) {
00140     base.setPointSizeF(points(role));
00141     return base;
00142 }
00143
00150 inline QString resolveStyleSheet(const QString& styleSheet,
00151                                 QStringList* unresolved = nullptr) {
00152     static const QRegularExpression tokenRe(
00153         QStringLiteral("\\$font-([a-z]+(?:-[a-z]+)*)"));
00154
00155     QString out;
00156     qsize_t last = 0;
00157     auto it = tokenRe.globalMatch(styleSheet);
00158     while (it.hasNext()) {
00159         const QRegularExpressionMatch match = it.next();
00160         out += styleSheet.mid(last, match.capturedStart() - last);
00161         last = match.capturedEnd();
00162
00163         const QString name = match.captured(1);
00164         bool found = false;
00165         for (const Token& token : kTokens) {
00166             if (name == token.name) {
00167                 out += formatPoints(token.points * AppTextScale::factor());
00168                 found = true;
00169                 break;
00170             }
00171         }
00172         if (!found) {
00173             out += match.captured(0);
00174             if (unresolved != nullptr) {
00175                 unresolved->append(name);
00176             }
00177         }
00178     }
00179     out += styleSheet.mid(last);
00180     return out;
00181 }
00182
00184 inline QString loadStyleSheet() {
00185     QFile file(QStringLiteral(":/resources/styles/app.qss"));
00186     if (!file.open(QFile::ReadOnly | QFile::Text)) {
00187         return {};
00188     }
00189     return resolveStyleSheet(QString::fromUtf8(file.readAll()));
00190 }
00191
00202 inline int lengthForText(int designPx) {
00203     const QFontInfo rendered(QApplication::font());
00204     const qreal textPixels = rendered.pixelSize() > 0
00205         ? rendered.pixelSize()
00206         : basePoints(Role::Body);
00207     return qRound(designPx * textPixels / basePoints(Role::Body));
00208 }
00209
00210 } // namespace AppTypography
00211
00212 #endif // APPTYPGRAPHY_H

```

10.82 widgets/automatonview.cpp File Reference

```

#include "automatonview.h"
#include "appfonts.h"
#include "apptypography.h"
#include <QApplication>
#include <QFontDatabase>
#include <QGraphicsEllipseItem>
#include <QGraphicsPathItem>
#include <QGraphicsPolygonItem>
#include <QGraphicsScene>
#include <QGraphicsSimpleTextItem>
#include <QGraphicsTextItem>
#include <QKeyEvent>
#include <QMouseEvent>
#include <QQueue>

```



```

00131     int shownStateId() const { return shownId_; }
00132
00133     void showStateDetails(unsigned id);
00134
00135     void hideStateDetails();
00136
00137 protected:
00138     void wheelEvent(QWheelEvent* event) override;
00139     void mousePressEvent(QMouseEvent* event) override;
00140     void mouseReleaseEvent(QMouseEvent* event) override;
00141     void keyPressEvent(QKeyEvent* event) override;
00142
00143 private:
00144     struct Node {
00145         unsigned          id = 0;
00146         QPointF           center;
00147         QGraphicsEllipseItem* circle = nullptr;
00148         QGraphicsSimpleTextItem* label = nullptr;
00149         QString           itemsText;
00150         bool              revealed = false;
00151     };
00152
00153     struct Edge {
00154         unsigned          from = 0;
00155         unsigned          to = 0;
00156         QStringList       symbols;
00157         QGraphicsPathItem* path = nullptr;
00158         QGraphicsPolygonItem* arrow = nullptr;
00159         QGraphicsSimpleTextItem* label = nullptr;
00160         bool              revealed = false;
00161     };
00162
00163     QHash<unsigned, int>
00164     computeLevels(const QVector<AutomatonTransitionInfo>& transitions) const;
00165     void buildEdgeGeometry(Edge& edge);
00166
00167     void applyNodeStyle(Node& node);
00168     void setNodeVisible(Node& node, bool visible);
00169     void setEdgeVisible(Edge& edge, bool visible);
00170     void updatePlaceholder();
00171     int nodeAt(const QPoint& viewPos) const;
00172
00173     QGraphicsScene* scene_ = nullptr;
00174     QGraphicsTextItem* placeholder_ = nullptr;
00175     QHash<unsigned, Node> nodes_;
00176     QList<Edge> edges_;
00177     QSet<unsigned> conflictIds_;
00178     QSet<unsigned> reduceIds_;
00179     int currentId_ = -1;
00180     bool fullyRevealed_ = false;
00181     double zoom_ = 1.0;
00182
00183     // Items of the clicked state, drawn in the scene next to its node.
00184     QGraphicsPathItem* details_ = nullptr;
00185     int shownId_ = -1;
00186     QPoint pressPos_;
00187 };
00188
00189 #endif // AUTOMATONVIEW_H

```

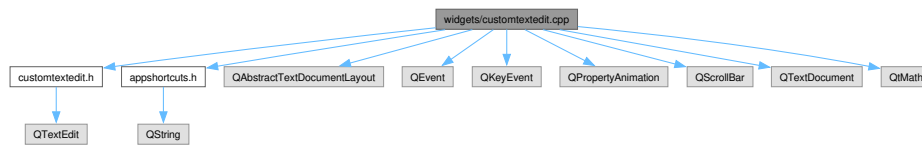
10.85 widgets/customtextedit.cpp File Reference

```

#include "customtextedit.h"
#include "appshortcuts.h"
#include <QAbstractTextDocumentLayout>
#include <QEvent>
#include <QKeyEvent>
#include <QPropertyAnimation>
#include <QScrollBar>
#include <QTextDocument>
#include <QtMath>

```

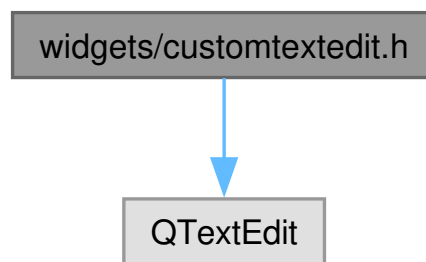

Include dependency graph for customtextedit.cpp:



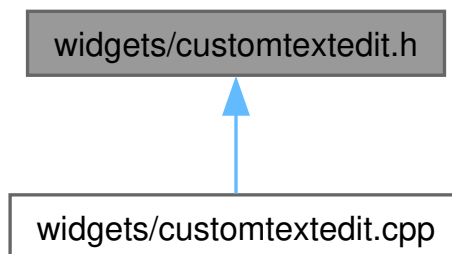
10.86 widgets/customtextedit.h File Reference

```
#include <QTextEdit>
```

Include dependency graph for customtextedit.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [CustomTextEdit](#)

10.87 customtextedit.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,

```

```

00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef CUSTOMTEXTEDIT_H
00020 #define CUSTOMTEXTEDIT_H
00021
00022 #include <QTextEdit>
00023
00024 class QPropertyAnimation;
00025
00026 class CustomTextEdit : public QTextEdit {
00027     Q_OBJECT
00028     Q_PROPERTY(int growHeight READ growHeight WRITE setGrowHeight)
00029 public:
00030     explicit CustomTextEdit(QWidget* parent = nullptr);
00031
00032     static QString newlineShortcutText();
00033
00034     void setAutoGrowLines(int minLines, int maxLines);
00035
00036     int minimumGrowHeight() const { return minGrowHeight_; }
00037
00038 signals:
00039     void sendRequested();
00040
00041     void minimumGrowHeightChanged(int height);
00042
00043 protected:
00044     void showEvent(QShowEvent* event) override;
00045     void changeEvent(QEvent* event) override;
00046     void keyPressEvent(QKeyEvent* event) override;
00047
00048 private:
00049     int growHeight() const;
00050     void setGrowHeight(int height);
00051     void updateGrowHeight();
00052     void refreshGrowBounds();
00053     int heightForLines(int lines) const;
00054
00055     int chrome_ = 0;
00056     int minGrowLines_ = 0;
00057     int maxGrowLines_ = 0;
00058     int minGrowHeight_ = 0;
00059     int maxGrowHeight_ = 0;
00060     QPropertyAnimation* growAnimation_ = nullptr;
00061 };
00062 #endif // CUSTOMTEXTEDIT_H

```

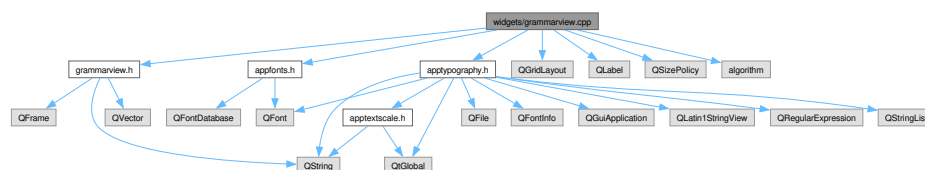
10.88 widgets/grammarview.cpp File Reference

```

#include "grammarview.h"
#include "appfonts.h"
#include "apptypography.h"
#include <QGridLayout>
#include <QLabel>
#include <QSizePolicy>
#include <algorithm>

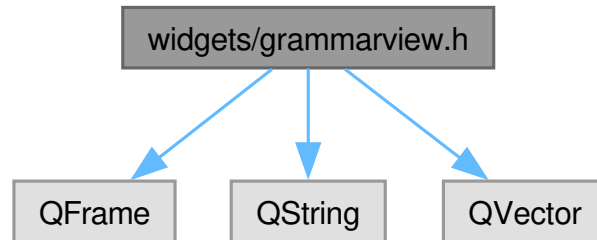
```

Include dependency graph for grammarview.cpp:

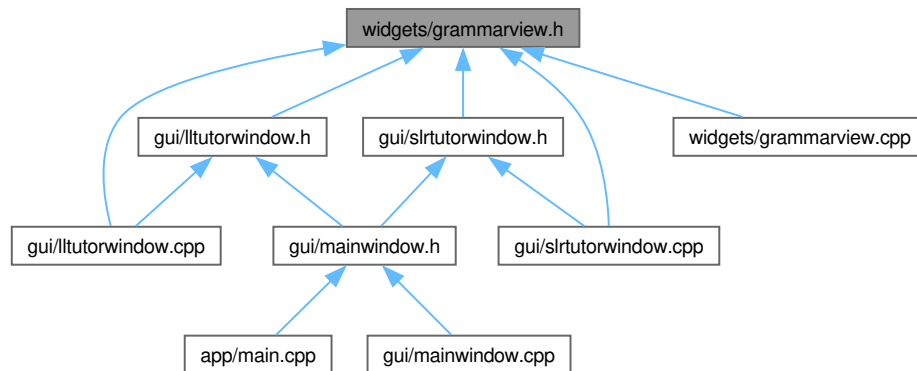


10.89 widgets/grammarview.h File Reference

```
#include <QFrame>
#include <QString>
#include <QVector>
Include dependency graph for grammarview.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [GrammarView](#)
- struct [GrammarView::Row](#)

10.90 grammarview.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00003 #include <QFrame>
00004 #include <QString>
00005 #include <QVector>
00006
00007 class QGridLayout;
00008
00009 class GrammarView : public QFrame {
00010     Q_OBJECT
00011 public:
00012     struct Row {
00013         QString index;
00014         QString lhs;
00015         QString marker;
00016         QString rhs;
00017     };
```


Classes

- struct [TutorialStep](#)
Represents a single step in the tutorial sequence.
- class [TutorialManager](#)
Manages interactive tutorials by highlighting UI elements and guiding the user.

10.93 tutorialmanager.h

[Go to the documentation of this file.](#)

```

00001  /*
00002  * SyntaxTutor - Interactive Tutorial About Syntax Analyzers
00003  * Copyright (C) 2025 Jose R. (jose-rzm)
00004  *
00005  * This program is free software: you can redistribute it and/or modify it
00006  * under the terms of the GNU General Public License as published by
00007  * the Free Software Foundation, either version 3 of the License, or
00008  * (at your option) any later version.
00009  *
00010  * This program is distributed in the hope that it will be useful,
00011  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00012  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00013  * GNU General Public License for more details.
00014  *
00015  * You should have received a copy of the GNU General Public License
00016  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00017  */
00018
00019 #ifndef TUTORIALMANAGER_H
00020 #define TUTORIALMANAGER_H
00021
00022 #include <QEvent>
00023 #include <QFrame>
00024 #include <QObject>
00025 #include <QPushButton>
00026 #include <QString>
00027 #include <QTextBrowser>
00028 #include <QVector>
00029 #include <QWidget>
00030
00038 struct TutorialStep {
00039     QWidget* target;
00040     QString htmlText;
00041 };
00042
00053 class TutorialManager : public QObject {
00054     Q_OBJECT
00055 public:
00061     TutorialManager(QWidget* rootWindow);
00062
00068     void addStep(QWidget* target, const QString& htmlText);
00069
00073     void start();
00074
00079     void setRootWindow(QWidget* newRoot);
00080
00084     void clearSteps();
00085
00089     void hideOverlay();
00090
00095     void finishLL1();
00096
00101     void finishSLR1();
00102
00103 protected:
00107     bool eventFilter(QObject* obj, QEvent* ev) override;
00108
00109 signals:
00114     void stepStarted(int index);
00115
00119     void tutorialFinished();
00120
00124     void ll1Finished();
00125
00129     void slr1Finished();
00130
00131 public slots:

```

```
00135     void nextStep();
00136
00137 private:
00141     void showOverlay();
00142
00146     void repositionOverlay();
00147
00148     QWidget*          m_root;
00149     QVector<TutorialStep> m_steps;
00150     int               m_index = -1;
00151
00152     QWidget* m_overlay = nullptr;
00153     QFrame*  m_highlight =
00154         nullptr;
00155     QTextBrowser* m_textBox = nullptr;
00156     QPushButton* m_nextBtn = nullptr;
00157 };
00158
00159 #endif // TUTORIALMANAGER_H
```

Index

/Users/jose/Documents/cpp/SyntaxTutor/README.md, [245](#)

~AutomatonViewerDialog
AutomatonViewerDialog, [50](#)

~LLTutorWindow
LLTutorWindow, [126](#)

~MainWindow
MainWindow, [160](#)

~SLRTutorWindow
SLRTutorWindow, [187](#)

A

lltutorwindow.h, [282](#)
slrtutorwindow.h, [299](#)

A1

lltutorwindow.h, [282](#)
slrtutorwindow.h, [299](#)

A2

lltutorwindow.h, [282](#)
slrtutorwindow.h, [299](#)

A3

slrtutorwindow.h, [299](#)

A4

slrtutorwindow.h, [299](#)

A_prime

lltutorwindow.h, [282](#)
slrtutorwindow.h, [299](#)

Accept

SLR1Parser, [169](#)

accept

GrammarEditorDialog, [79](#)

Action

AppTypography, [30](#)
SLR1Parser, [169](#)

action

SLR1Parser::s_action, [165](#)

action_table

SLR1Parser, [168](#)

actions_

SLR1Parser, [178](#)

addConversation

ConversationPdf, [54](#)

addDefinitions

ConversationPdf, [54](#)

addGrammar

ConversationPdf, [54](#)

addGrammarMessage

LLTutorWindow, [127](#)
SLRTutorWindow, [187](#)

addLines

ConversationPdf, [54](#)

addMessage

LLTutorWindow, [127](#)
SLRTutorWindow, [187](#)

AddProduction

Grammar, [72](#)

addResult

ConversationPdf, [55](#)

addReview

ConversationPdf, [55](#)

addSection

ConversationPdf, [55](#)

addStep

TutorialManager, [236](#)

addSubheading

ConversationPdf, [55](#)

addTable

ConversationPdf, [56](#)

addUserState

SLRTutorWindow, [187](#)

addUserTransition

SLRTutorWindow, [187](#)

addWidgetMessage

LLTutorWindow, [127](#)
SLRTutorWindow, [188](#)

AdjustTerminals

GrammarFactory, [81](#)

AdvanceDot

Lr0Item, [156](#)

AllItems

SLR1Parser, [170](#)

animateLabelColor

LLTutorWindow, [127](#)
SLRTutorWindow, [188](#)

animateLabelPop

LLTutorWindow, [127](#)
SLRTutorWindow, [188](#)

antecedent_

Lr0Item, [158](#)

app Directory Reference, [15](#)

app/main.cpp, [245](#)

appfonts.h

appMonospaceFont, [313](#)

AppIcon, [19](#)

icon, [19](#)

kSizes, [20](#)

AppLayout, [20](#)

keepHeightForWidth, [20](#)

AppLayout::HeightForWidthKeeper, [109](#)

- eventFilter, [111](#)
 - HeightForWidthKeeper, [110](#)
- applicationFont
 - AppTypography, [30](#)
- apply
 - AppPalette, [21](#)
- applyAppStyle
 - main.cpp, [246](#)
- applyTextScale
 - LLTutorWindow, [128](#)
 - SLRTutorWindow, [188](#)
- appMonospaceFont
 - appfonts.h, [313](#)
- AppPalette, [21](#)
 - apply, [21](#)
 - linkColor, [21](#)
- AppSettings, [22](#)
 - open, [23](#)
 - registerIdentity, [23](#)
- AppShortcuts, [24](#)
 - primaryModifierName, [24](#)
- AppTextScale, [24](#)
 - clampPercent, [25](#)
 - currentPercent, [25](#)
 - factor, [26](#)
 - kDefaultPercent, [28](#)
 - kMaxPercent, [28](#)
 - kMinPercent, [28](#)
 - kPresetPercents, [28](#)
 - legacySettingsKey, [26](#)
 - percentFromLegacyValue, [27](#)
 - settingsKey, [27](#)
- AppTypography, [29](#)
 - Action, [30](#)
 - applicationFont, [30](#)
 - basePoints, [31](#)
 - Body, [30](#)
 - BodyLarge, [30](#)
 - Caption, [30](#)
 - cssSize, [31](#)
 - Display, [30](#)
 - font, [32](#)
 - formatPoints, [32](#)
 - Headline, [30](#)
 - Hero, [30](#)
 - kTokens, [36](#)
 - Label, [30](#)
 - lengthForText, [33](#)
 - loadStyleSheet, [33](#)
 - Micro, [30](#)
 - points, [34](#)
 - Reading, [30](#)
 - resolveStyleSheet, [34](#)
 - Role, [30](#)
 - systemFont, [35](#)
 - Title, [30](#)
 - TitleSmall, [30](#)
- AppTypography::Token, [233](#)
- name, [233](#)
 - points, [233](#)
 - role, [233](#)
- appversion.h, [247](#)
- AutomatonStateInfo, [39](#)
 - id, [39](#)
 - itemsText, [39](#)
- AutomatonTransitionInfo, [40](#)
 - from, [40](#)
 - symbol, [40](#)
 - to, [40](#)
- AutomatonView, [41](#)
 - AutomatonView, [42](#)
 - centerOnCurrentState, [43](#)
 - clearStateMarks, [43](#)
 - conflictStates, [43](#)
 - currentStateId, [43](#)
 - fitToView, [43](#)
 - hideStateDetails, [43](#)
 - isFullyRevealed, [44](#)
 - isStateVisible, [44](#)
 - keyPressEvent, [44](#)
 - mousePressEvent, [44](#)
 - mouseReleaseEvent, [45](#)
 - reduceStates, [45](#)
 - revealAll, [45](#)
 - revealState, [45](#)
 - revealTransition, [45](#)
 - setAutomaton, [46](#)
 - setConflictStates, [46](#)
 - setCurrentState, [46](#)
 - setReduceStates, [46](#)
 - shownStateId, [47](#)
 - showStateDetails, [47](#)
 - totalStateCount, [47](#)
 - visibleStateCount, [47](#)
 - visibleTransitionCount, [48](#)
 - wheelEvent, [48](#)
 - zoomIn, [48](#)
 - zoomOut, [48](#)
- AutomatonViewerDialog, [49](#)
 - ~AutomatonViewerDialog, [50](#)
 - AutomatonViewerDialog, [50](#)
- axiom_
 - Grammar, [76](#)
- B
 - lltutorwindow.h, [282](#)
 - slrtutorwindow.h, [299](#)
- B1
 - lltutorwindow.h, [282](#)
- B2
 - lltutorwindow.h, [282](#)
- B_prime
 - lltutorwindow.h, [282](#)
- backend Directory Reference, [15](#)
- backend/grammar.cpp, [248](#)
- backend/grammar.hpp, [248](#), [250](#)
- backend/grammar_factory.cpp, [250](#)

- backend/grammar_factory.hpp, 251, 252
- backend/grammar_parser.cpp, 253
- backend/grammar_parser.hpp, 254
- backend/ll1_parser.cpp, 255
- backend/ll1_parser.hpp, 256, 257
- backend/lr0_item.cpp, 257
- backend/lr0_item.hpp, 258, 259
- backend/slr1_parser.cpp, 259
- backend/slr1_parser.hpp, 260, 261
- backend/state.hpp, 262
- backend/symbol_table.cpp, 263
- backend/symbol_table.hpp, 263, 265
- basePoints
 - AppTypography, 31
- Body
 - AppTypography, 30
- BodyLarge
 - AppTypography, 30
- buildGrammarRows
 - LLTutorWindow, 128
 - SLRTutorWindow, 189
- buildTreeNode
 - LLTutorWindow, 128
- C
 - lltutorwindow.h, 282
 - slrtutorwindow.h, 299
- C_prime
 - lltutorwindow.h, 282
- CA
 - slrtutorwindow.h, 300
- Caption
 - AppTypography, 30
- CB
 - slrtutorwindow.h, 300
- CenterAlignDelegate, 51
 - initStyleOption, 51
- centerOnCurrentState
 - AutomatonView, 43
- changeEvent
 - CustomTextEdit, 59
- children
 - LLTutorWindow::TreeNode, 234
- clampPercent
 - AppTextScale, 25
- clear
 - UniqueQueue< T >, 242
- clearStateMarks
 - AutomatonView, 43
- clearSteps
 - TutorialManager, 236
- closeButton
 - LLWizard, 150
 - SLRWizard, 222
- closeEvent
 - LLTutorWindow, 128
 - SLRTutorWindow, 189
- Closure
 - SLR1Parser, 170
- ClosureUtil
 - SLR1Parser, 171
- completionChanged
 - LLWizardPage, 153
 - SLRWizardPage, 226
- ComputeFirstSets
 - LL1Parser, 113
 - SLR1Parser, 171
- ComputeFollowSets
 - LL1Parser, 113
 - SLR1Parser, 172
- computeSubtreeWidth
 - LLTutorWindow, 129
- confirmExitToHome
 - LLTutorWindow, 129
 - SLRTutorWindow, 189
- conflictStates
 - AutomatonView, 43
- connectGuidedMode
 - LLTutorWindow, 129
- consequent_
 - Lr0Item, 158
- ConversationPdf, 52
 - addConversation, 54
 - addDefinitions, 54
 - addGrammar, 54
 - addLines, 54
 - addResult, 55
 - addReview, 55
 - addSection, 55
 - addSubheading, 55
 - addTable, 56
 - ConversationPdf, 53
 - html, 56
 - kMaxTableColumns, 57
 - print, 56
 - Rule, 53
- ConversationPdf::Message, 163
 - correct, 164
 - fromStudent, 164
 - text, 164
- correct
 - ConversationPdf::Message, 164
 - ExamRecord, 62
- correctAnswer
 - ExamRecord, 62
- CreateLL1Table
 - LL1Parser, 114
- CreateLv2Item
 - GrammarFactory, 82
- cssSize
 - AppTypography, 31
- current
 - SyntaxTutor::Version, 36
- currentPage
 - LLWizard, 150
 - SLRWizard, 222
- currentPercent

- AppTextScale, 25
- currentStateId
 - AutomatonView, 43
- CustomTextEdit, 57
 - changeEvent, 59
 - CustomTextEdit, 58
 - growHeight, 61
 - keyPressEvent, 59
 - minimumGrowHeight, 59
 - minimumGrowHeightChanged, 59
 - newlineShortcutText, 60
 - sendRequested, 60
 - setAutoGrowLines, 60
 - showEvent, 61
- D
 - slrtutorwindow.h, 300
- D1
 - slrtutorwindow.h, 300
- D2
 - slrtutorwindow.h, 300
- D_prime
 - slrtutorwindow.h, 300
- Debug
 - Grammar, 72
- Delta
 - SLR1Parser, 173
- detail
 - GrammarParseError, 102
- Display
 - AppTypography, 30
- dot_
 - Lr0Item, 158
- drawTree
 - LLTutorWindow, 130
- E
 - slrtutorwindow.h, 300
- E1
 - slrtutorwindow.h, 300
- E2
 - slrtutorwindow.h, 300
- Empty
 - SLR1Parser, 169
- empty
 - UniqueQueue< T >, 242
- EmptyGrammar
 - GrammarParseError, 102
- EmptyLeftHandSide
 - GrammarParseError, 102
- EOL_
 - SymbolTable, 231
- eol_
 - Lr0Item, 158
- EPSILON_
 - SymbolTable, 231
- epsilon_
 - Lr0Item, 158
- errors
 - GrammarParseResult, 105
- eventFilter
 - AppLayout::HeightForWidthKeeper, 111
 - LLTutorWindow, 130
 - SLRTutorWindow, 189
 - TutorialManager, 236
- ExamRecord, 61
 - correct, 62
 - correctAnswer, 62
 - question, 62
 - userAnswer, 62
- ExamReportDialog, 63
 - ExamReportDialog, 64
 - exportRequested, 64
 - printReport, 64
- ExamSession, 65
 - grade, 66
 - record, 66
 - records, 66
 - right, 67
 - total, 67
- examSolutionText
 - LLTutorWindow, 130
 - SLRTutorWindow, 190
- exitRequested
 - LLTutorWindow, 130
 - SLRTutorWindow, 190
- exportConversationToPdf
 - LLTutorWindow, 131
 - SLRTutorWindow, 190
- exportRequested
 - ExamReportDialog, 64
- ExtraArrow
 - GrammarParseError, 102
- F
 - slrtutorwindow.h, 300
- FA
 - slrtutorwindow.h, 300
- factor
 - AppTextScale, 26
- FactoryItem
 - GrammarFactory::FactoryItem, 68
- feedback
 - LLTutorWindow, 131
 - SLRTutorWindow, 190
- feedbackForA
 - LLTutorWindow, 131
 - SLRTutorWindow, 191
- feedbackForA1
 - LLTutorWindow, 132
 - SLRTutorWindow, 191
- feedbackForA2
 - LLTutorWindow, 132
 - SLRTutorWindow, 192
- feedbackForA3
 - SLRTutorWindow, 192
- feedbackForA4
 - SLRTutorWindow, 192

- feedbackForAPrime
 - LLTutorWindow, [133](#)
 - SLRTutorWindow, [193](#)
- feedbackForB
 - LLTutorWindow, [133](#)
 - SLRTutorWindow, [193](#)
- feedbackForB1
 - LLTutorWindow, [133](#)
 - SLRTutorWindow, [193](#)
- feedbackForB1TreeGraphics
 - LLTutorWindow, [133](#)
- feedbackForB1TreeWidget
 - LLTutorWindow, [134](#)
- feedbackForB2
 - LLTutorWindow, [134](#)
 - SLRTutorWindow, [193](#)
- feedbackForBPrime
 - LLTutorWindow, [134](#)
 - SLRTutorWindow, [193](#)
- feedbackForC
 - LLTutorWindow, [135](#)
 - SLRTutorWindow, [194](#)
- feedbackForCA
 - SLRTutorWindow, [195](#)
- feedbackForCB
 - SLRTutorWindow, [195](#)
- feedbackForCPrime
 - LLTutorWindow, [135](#)
- feedbackForD
 - SLRTutorWindow, [195](#)
- feedbackForD1
 - SLRTutorWindow, [196](#)
- feedbackForD2
 - SLRTutorWindow, [196](#)
- feedbackForDPrime
 - SLRTutorWindow, [196](#)
- feedbackForE
 - SLRTutorWindow, [196](#)
- feedbackForE1
 - SLRTutorWindow, [197](#)
- feedbackForE2
 - SLRTutorWindow, [197](#)
- feedbackForF
 - SLRTutorWindow, [197](#)
- feedbackForFA
 - SLRTutorWindow, [197](#)
- feedbackForG
 - SLRTutorWindow, [198](#)
- fillSortedGrammar
 - SLRTutorWindow, [198](#)
- FilterRulesByConsequent
 - Grammar, [72](#)
- fin
 - lltutorwindow.h, [282](#)
 - slrtutorwindow.h, [300](#)
- finishLL1
 - TutorialManager, [236](#)
- finishSLR1
 - TutorialManager, [237](#)
- First
 - LL1Parser, [115](#)
 - SLR1Parser, [173](#)
- first_sets_
 - LL1Parser, [117](#)
 - SLR1Parser, [178](#)
- fitToView
 - AutomatonView, [43](#)
- focusAnswerField
 - LLWizardPage, [153](#)
 - SLRWizardPage, [226](#)
- Follow
 - LL1Parser, [116](#)
 - SLR1Parser, [174](#)
- follow_sets_
 - LL1Parser, [117](#)
 - SLR1Parser, [178](#)
- font
 - AppTypography, [32](#)
- FormatGrammar
 - LLTutorWindow, [136](#)
 - SLRTutorWindow, [198](#)
- formatPoints
 - AppTypography, [32](#)
- from
 - AutomatonTransitionInfo, [40](#)
- fromStudent
 - ConversationPdf::Message, [164](#)
- front
 - UniqueQueue< T >, [242](#)
- G
 - slrtutorwindow.h, [300](#)
- g_
 - Grammar, [76](#)
 - GrammarFactory::FactoryItem, [69](#)
- GenerateNewNonTerminal
 - GrammarFactory, [82](#)
- generateQuestion
 - LLTutorWindow, [136](#)
 - SLRTutorWindow, [198](#)
- GenLL1Grammar
 - GrammarFactory, [83](#)
- GenSLR1Grammar
 - GrammarFactory, [84](#)
- getTableData
 - LLTableDialog, [120](#)
 - SLRTableDialog, [181](#)
- gr_
 - LL1Parser, [118](#)
 - SLR1Parser, [178](#)
- grade
 - ExamSession, [66](#)
- Grammar, [70](#)
 - AddProduction, [72](#)
 - axiom_, [76](#)
 - Debug, [72](#)
 - FilterRulesByConsequent, [72](#)

- g_, 76
- Grammar, 71
- HasEmptyProduction, 73
- lhs_order_, 76
- PresentationLess, 73
- PresentationOrder, 74
- PresentationRank, 74
- SetAxiom, 75
- Split, 75
- st_, 77
- grammar
 - GrammarEditorDialog, 79
 - GrammarParseResult, 105
- grammar.hpp
 - production, 249
- GrammarEditorDialog, 77
 - accept, 79
 - grammar, 79
 - GrammarEditorDialog, 79
 - LL1, 78
 - Mode, 78
 - SLR1, 78
- GrammarFactory, 80
 - AdjustTerminals, 81
 - CreateLv2Item, 82
 - GenerateNewNonTerminal, 82
 - GenLL1Grammar, 83
 - GenSLR1Grammar, 84
 - HasCycle, 85
 - HasDirectLeftRecursion, 85
 - HasIndirectLeftRecursion, 86
 - HasUnreachableSymbols, 86
 - Init, 87
 - IsInfinite, 87
 - items, 100
 - LeftFactorize, 88
 - LongestCommonPrefix, 89
 - Lv1, 89
 - Lv2, 90
 - Lv3, 91
 - Lv4, 92
 - Lv5, 93
 - Lv6, 94
 - Lv7, 95
 - Merge, 96
 - non_terminal_alphabet_, 100
 - NormalizeNonTerminals, 97
 - NullableSymbols, 97
 - PickOne, 98
 - RemoveLeftRecursion, 99
 - StartsWith, 99
 - terminal_alphabet_, 100
- GrammarFactory::FactoryItem, 68
 - FactoryItem, 68
 - g_, 69
 - st_, 69
- GrammarParseError, 101
 - detail, 102
 - EmptyGrammar, 102
 - EmptyLeftHandSide, 102
 - ExtraArrow, 102
 - Kind, 102
 - kind, 102
 - line, 102
 - MissingArrow, 102
 - MissingEndDot, 102
 - MultipleLeftHandSide, 102
 - ReservedSymbol, 102
- GrammarParser, 102
 - IsReservedToken, 103
 - Parse, 103
- GrammarParseResult, 104
 - errors, 105
 - grammar, 105
 - Ok, 105
- GrammarSyntaxHighlighter, 106
 - GrammarSyntaxHighlighter, 107
 - highlightBlock, 107
 - setNonTerminals, 107
- GrammarView, 107
 - GrammarView, 108
 - naturalWidth, 108
 - refresh, 108
 - setRows, 109
- GrammarView::Row, 164
 - index, 164
 - lhs, 164
 - marker, 164
 - rhs, 165
- growHeight
 - CustomTextEdit, 61
- gui Directory Reference, 16
- gui/automatonviewerdialog.cpp, 265
- gui/automatonviewerdialog.h, 266, 267
- gui/conversationpdf.h, 267, 268
- gui/examreportdialog.cpp, 272
- gui/examreportdialog.h, 272, 273
- gui/examsession.h, 274
- gui/grammareditorialdialog.cpp, 275
- gui/grammareditorialdialog.h, 276, 277
- gui/ltabledialog.cpp, 278
- gui/ltabledialog.h, 278, 279
- gui/ltutorwindow.cpp, 280
- gui/ltutorwindow.h, 280, 282
- gui/lwizard.h, 286
- gui/lwizardpage.h, 289, 290
- gui/mainwindow.cpp, 292
- gui/mainwindow.h, 292, 293
- gui/slrtabledialog.cpp, 295
- gui/slrtabledialog.h, 295, 296
- gui/slrtutorwindow.cpp, 297
- gui/slrtutorwindow.h, 298, 300
- gui/slrlizard.h, 304, 305
- gui/slrlizardpage.h, 309
- gui/UniqueQueue.h, 311, 312
- guidedRequested

- LLTableDialog, [120](#)
- SLRTableDialog, [181](#)
- H
 - slrtutorwindow.h, [300](#)
- H_prime
 - slrtutorwindow.h, [300](#)
- handleTableSubmission
 - LLTutorWindow, [137](#)
- HasCycle
 - GrammarFactory, [85](#)
- HasDirectLeftRecursion
 - GrammarFactory, [85](#)
- HasEmptyProduction
 - Grammar, [73](#)
- HasIndirectLeftRecursion
 - GrammarFactory, [86](#)
- HasUnreachableSymbols
 - GrammarFactory, [86](#)
- Headline
 - AppTypography, [30](#)
- HeightForWidthKeeper
 - AppLayout::HeightForWidthKeeper, [110](#)
- Hero
 - AppTypography, [30](#)
- hideOverlay
 - TutorialManager, [237](#)
- hideStateDetails
 - AutomatonView, [43](#)
- highlightBlock
 - GrammarSyntaxHighlighter, [107](#)
- highlightIncorrectCells
 - LLTableDialog, [120](#)
 - SLRTableDialog, [181](#)
- highlightInvalidCells
 - SLRTableDialog, [181](#)
- html
 - ConversationPdf, [56](#)
- htmlText
 - TutorialStep, [240](#)
- icon
 - Applcon, [19](#)
- id
 - AutomatonStateInfo, [39](#)
- id_
 - state, [227](#)
- In
 - SymbolTable, [229](#)
- index
 - GrammarView::Row, [164](#)
- Init
 - GrammarFactory, [87](#)
- initialItemText
 - SLRTutorWindow, [199](#)
- initStyleOption
 - CenterAlignDelegate, [51](#)
- IsComplete
 - LrOItem, [156](#)
- isComplete
 - LLWizardPage, [153](#)
 - SLRWizardPage, [226](#)
- isFullyRevealed
 - AutomatonView, [44](#)
- IsInfinite
 - GrammarFactory, [87](#)
- IsMeta
 - SymbolTable, [229](#)
- IsNonTerminal
 - SymbolTable, [229](#)
- isOnLastPage
 - LLWizard, [151](#)
 - SLRWizard, [223](#)
- isReduction
 - SLRTutorWindow, [199](#)
- IsReservedToken
 - GrammarParser, [103](#)
- isStateVisible
 - AutomatonView, [44](#)
- IsTerminal
 - SymbolTable, [230](#)
- IsTerminalWthoEol
 - SymbolTable, [230](#)
- item
 - SLR1Parser::s_action, [165](#)
- items
 - GrammarFactory, [100](#)
- items_
 - state, [227](#)
- itemsText
 - AutomatonStateInfo, [39](#)
- kDefaultPercent
 - AppTextScale, [28](#)
- keepHeightForWidth
 - AppLayout, [20](#)
- keyPressEvent
 - AutomatonView, [44](#)
 - CustomTextEdit, [59](#)
- Kind
 - GrammarParseError, [102](#)
- kind
 - GrammarParseError, [102](#)
- kMaxPercent
 - AppTextScale, [28](#)
- kMaxTableColumns
 - ConversationPdf, [57](#)
- kMinPercent
 - AppTextScale, [28](#)
- kPresetPercents
 - AppTextScale, [28](#)
- kSizes
 - Applcon, [20](#)
- kTokens
 - AppTypography, [36](#)
- Label
 - AppTypography, [30](#)

- label
 - LLTutorWindow::TreeNode, [234](#)
- launchSLRWizard
 - SLRTutorWindow, [200](#)
- LeftFactorize
 - GrammarFactory, [88](#)
- legacySettingsKey
 - AppTextScale, [26](#)
- lengthForText
 - AppTypography, [33](#)
- lhs
 - GrammarView::Row, [164](#)
- lhs_order_
 - Grammar, [76](#)
- line
 - GrammarParseError, [102](#)
- linkColor
 - AppPalette, [21](#)
- LL1
 - GrammarEditorDialog, [78](#)
- ll1_t_
 - LL1Parser, [118](#)
- ll1Finished
 - TutorialManager, [237](#)
- LL1Parser, [111](#)
 - ComputeFirstSets, [113](#)
 - ComputeFollowSets, [113](#)
 - CreateLL1Table, [114](#)
 - First, [115](#)
 - first_sets_, [117](#)
 - Follow, [116](#)
 - follow_sets_, [117](#)
 - gr_, [118](#)
 - ll1_t_, [118](#)
 - LL1Parser, [112](#)
 - PredictionSymbols, [116](#)
- LLTableDialog, [118](#)
 - getTableData, [120](#)
 - guidedRequested, [120](#)
 - highlightIncorrectCells, [120](#)
 - LLTableDialog, [119](#)
 - setGuidedButtonVisible, [121](#)
 - setGuidedModeActive, [121](#)
 - setInitialData, [121](#)
 - submitted, [122](#)
- LLTutorWindow, [122](#)
 - ~LLTutorWindow, [126](#)
 - addGrammarMessage, [127](#)
 - addMessage, [127](#)
 - addWidgetMessage, [127](#)
 - animateLabelColor, [127](#)
 - animateLabelPop, [127](#)
 - applyTextScale, [128](#)
 - buildGrammarRows, [128](#)
 - buildTreeNode, [128](#)
 - closeEvent, [128](#)
 - computeSubtreeWidth, [129](#)
 - confirmExitToHome, [129](#)
 - connectGuidedMode, [129](#)
 - drawTree, [130](#)
 - eventFilter, [130](#)
 - examSolutionText, [130](#)
 - exitRequested, [130](#)
 - exportConversationToPdf, [131](#)
 - feedback, [131](#)
 - feedbackForA, [131](#)
 - feedbackForA1, [132](#)
 - feedbackForA2, [132](#)
 - feedbackForAPrime, [133](#)
 - feedbackForB, [133](#)
 - feedbackForB1, [133](#)
 - feedbackForB1TreeGraphics, [133](#)
 - feedbackForB1TreeWidget, [134](#)
 - feedbackForB2, [134](#)
 - feedbackForBPrime, [134](#)
 - feedbackForC, [135](#)
 - feedbackForCPrime, [135](#)
 - FormatGrammar, [136](#)
 - generateQuestion, [136](#)
 - handleTableSubmission, [137](#)
 - LLTutorWindow, [125](#)
 - markLastUserIncorrect, [137](#)
 - postQuestion, [137](#)
 - promptExportFilePath, [138](#)
 - scoreExamTable, [139](#)
 - sessionFinished, [139](#)
 - showExamReport, [140](#)
 - showTable, [140](#)
 - showTableForCPrime, [140](#)
 - showTreeGraphics, [140](#)
 - solution, [141](#)
 - solutionForA, [141](#)
 - solutionForA1, [142](#)
 - solutionForA2, [142](#)
 - solutionForB, [142](#)
 - solutionForB1, [142](#)
 - solutionForB2, [143](#)
 - TeachFirstTree, [143](#)
 - TeachFollow, [144](#)
 - TeachLL1Table, [144](#)
 - TeachPredictionSymbols, [144](#)
 - updatePlaceholder, [144](#)
 - updateProgressPanel, [145](#)
 - updateState, [145](#)
 - verifyResponse, [146](#)
 - verifyResponseForA, [146](#)
 - verifyResponseForA1, [146](#)
 - verifyResponseForA2, [147](#)
 - verifyResponseForB, [147](#)
 - verifyResponseForB1, [147](#)
 - verifyResponseForB2, [148](#)
 - verifyResponseForC, [148](#)
 - wrongAnimation, [148](#)
 - wrongUserResponseAnimation, [148](#)
- lltutorwindow.h
 - A, [282](#)

- A1, [282](#)
- A2, [282](#)
- A_prime, [282](#)
- B, [282](#)
- B1, [282](#)
- B2, [282](#)
- B_prime, [282](#)
- C, [282](#)
- C_prime, [282](#)
- fin, [282](#)
- State, [282](#)
- LLTutorWindow::TreeNode, [233](#)
 - children, [234](#)
 - label, [234](#)
- LLWizard, [149](#)
 - closeButton, [150](#)
 - currentPage, [150](#)
 - isOnLastPage, [151](#)
 - LLWizard, [150](#)
 - nextButton, [151](#)
- LLWizardPage, [151](#)
 - completionChanged, [153](#)
 - focusAnswerField, [153](#)
 - isComplete, [153](#)
 - LLWizardPage, [152](#)
 - resizeEvent, [153](#)
 - submitRequested, [153](#)
 - titleText, [154](#)
- loadStyleSheet
 - AppTypography, [33](#)
- LongestCommonPrefix
 - GrammarFactory, [89](#)
- Lr0Item, [154](#)
 - AdvanceDot, [156](#)
 - antecedent_, [158](#)
 - consequent_, [158](#)
 - dot_, [158](#)
 - eol_, [158](#)
 - epsilon_, [158](#)
 - IsComplete, [156](#)
 - Lr0Item, [155](#)
 - NextToDot, [156](#)
 - operator==, [157](#)
 - PrintItem, [157](#)
 - ToString, [157](#)
- Lv1
 - GrammarFactory, [89](#)
- Lv2
 - GrammarFactory, [90](#)
- Lv3
 - GrammarFactory, [91](#)
- Lv4
 - GrammarFactory, [92](#)
- Lv5
 - GrammarFactory, [93](#)
- Lv6
 - GrammarFactory, [94](#)
- Lv7
 - GrammarFactory, [95](#)
- main
 - main.cpp, [246](#)
- main.cpp
 - applyAppStyle, [246](#)
 - main, [246](#)
- MainWindow, [159](#)
 - ~MainWindow, [160](#)
 - MainWindow, [160](#)
 - setUserLevel, [161](#)
 - thresholdFor, [161](#)
 - userLevel, [162](#), [163](#)
 - userLevelChanged, [162](#)
 - userLevelUp, [162](#)
- MakeInitialState
 - SLR1Parser, [175](#)
- MakeParser
 - SLR1Parser, [175](#)
- marker
 - GrammarView::Row, [164](#)
- markLastUserIncorrect
 - LLTutorWindow, [137](#)
 - SLRTutorWindow, [200](#)
- Merge
 - GrammarFactory, [96](#)
- META
 - symbol_table.hpp, [265](#)
- meta_symbols_
 - SymbolTable, [232](#)
- Micro
 - AppTypography, [30](#)
- minimumGrowHeight
 - CustomTextEdit, [59](#)
- minimumGrowHeightChanged
 - CustomTextEdit, [59](#)
- MissingArrow
 - GrammarParseError, [102](#)
- MissingEndDot
 - GrammarParseError, [102](#)
- Mode
 - GrammarEditorDialog, [78](#)
- mousePressEvent
 - AutomatonView, [44](#)
- mouseReleaseEvent
 - AutomatonView, [45](#)
- MultipleLeftHandSide
 - GrammarParseError, [102](#)
- name
 - AppTypography::Token, [233](#)
- naturalWidth
 - GrammarView, [108](#)
- newlineShortcutText
 - CustomTextEdit, [60](#)
- nextButton
 - LLWizard, [151](#)
 - SLRWizard, [223](#)
- nextStep

- TutorialManager, 238
- NextToDot
 - Lr0Item, 156
- NO_TERMINAL
 - symbol_table.hpp, 264
- non_terminal_alphabet_
 - GrammarFactory, 100
- non_terminals_
 - SymbolTable, 232
- NormalizeNonTerminals
 - GrammarFactory, 97
- NullableSymbols
 - GrammarFactory, 97
- Ok
 - GrammarParseResult, 105
- open
 - AppSettings, 23
- openAutomatonViewer
 - SLRTutorWindow, 201
- operator==
 - Lr0Item, 157
 - state, 227
- Parse
 - GrammarParser, 103
- percentFromLegacyValue
 - AppTextScale, 27
- PickOne
 - GrammarFactory, 98
- points
 - AppTypography, 34
 - AppTypography::Token, 233
- pop
 - UniqueQueue< T >, 242
- postQuestion
 - LLTutorWindow, 137
 - SLRTutorWindow, 201
- PredictionSymbols
 - LL1Parser, 116
- PresentationLess
 - Grammar, 73
- PresentationOrder
 - Grammar, 74
- PresentationRank
 - Grammar, 74
- primaryModifierName
 - AppShortcuts, 24
- print
 - ConversationPdf, 56
- PrintItem
 - Lr0Item, 157
- PrintItems
 - SLR1Parser, 176
- printReport
 - ExamReportDialog, 64
- production
 - grammar.hpp, 249
- promptExportFilePath
 - LLTutorWindow, 138
- SLRTutorWindow, 201
- push
 - UniqueQueue< T >, 242
- PutSymbol
 - SymbolTable, 231
- question
 - ExamRecord, 62
- raw
 - SyntaxTutor::Version, 37
- Reading
 - AppTypography, 30
- record
 - ExamSession, 66
- records
 - ExamSession, 66
- Reduce
 - SLR1Parser, 169
- reduceStates
 - AutomatonView, 45
- refresh
 - GrammarView, 108
- registerIdentity
 - AppSettings, 23
- RemoveLeftRecursion
 - GrammarFactory, 99
- ReservedSymbol
 - GrammarParseError, 102
- resizeEvent
 - LLWizardPage, 153
 - SLRWizardPage, 226
- resolveStyleSheet
 - AppTypography, 34
- revealAll
 - AutomatonView, 45
- revealState
 - AutomatonView, 45
- revealTransition
 - AutomatonView, 45
- rhs
 - GrammarView::Row, 165
- right
 - ExamSession, 67
- Role
 - AppTypography, 30
- role
 - AppTypography::Token, 233
- Rule
 - ConversationPdf, 53
- scoreExamTable
 - LLTutorWindow, 139
 - SLRTutorWindow, 202
- sendRequested
 - CustomTextEdit, 60
- sessionFinished
 - LLTutorWindow, 139

- SLRTutorWindow, 202
- setAutoGrowLines
 - CustomTextEdit, 60
- setAutomaton
 - AutomatonView, 46
- SetAxiom
 - Grammar, 75
- setConflictStates
 - AutomatonView, 46
- setCurrentState
 - AutomatonView, 46
- setGuidedButtonVisible
 - LLTableDialog, 121
 - SLRTableDialog, 181
- setGuidedModeActive
 - LLTableDialog, 121
 - SLRTableDialog, 182
- setInitialData
 - LLTableDialog, 121
 - SLRTableDialog, 182
- setNonTerminals
 - GrammarSyntaxHighlighter, 107
- setReduceStates
 - AutomatonView, 46
- setRootWindow
 - TutorialManager, 238
- setRows
 - GrammarView, 109
- settingsKey
 - AppTextScale, 27
- setUpAutomatonPanel
 - SLRTutorWindow, 203
- setUserLevel
 - MainWindow, 161
- Shift
 - SLR1Parser, 169
- showEvent
 - CustomTextEdit, 61
- showExamReport
 - LLTutorWindow, 140
 - SLRTutorWindow, 203
- shownStateId
 - AutomatonView, 47
- showStateDetails
 - AutomatonView, 47
- showTable
 - LLTutorWindow, 140
 - SLRTutorWindow, 203
- showTableForCPrime
 - LLTutorWindow, 140
- showTreeGraphics
 - LLTutorWindow, 140
- SLR1
 - GrammarEditorDialog, 78
- slr1Finished
 - TutorialManager, 238
- SLR1Parser, 166
 - Accept, 169
 - Action, 169
 - action_table, 168
 - actions_, 178
 - AllItems, 170
 - Closure, 170
 - ClosureUtil, 171
 - ComputeFirstSets, 171
 - ComputeFollowSets, 172
 - Delta, 173
 - Empty, 169
 - First, 173
 - first_sets_, 178
 - Follow, 174
 - follow_sets_, 178
 - gr_, 178
 - MakeInitialState, 175
 - MakeParser, 175
 - PrintItems, 176
 - Reduce, 169
 - Shift, 169
 - SLR1Parser, 169
 - SolveLRConflicts, 177
 - states_, 178
 - transition_table, 168
 - transitions_, 178
- SLR1Parser::s_action, 165
 - action, 165
 - item, 165
- SLRTableDialog, 179
 - getTableData, 181
 - guidedRequested, 181
 - highlightIncorrectCells, 181
 - highlightInvalidCells, 181
 - setGuidedButtonVisible, 181
 - setGuidedModeActive, 182
 - setInitialData, 182
 - SLRTableDialog, 180
 - submitted, 182
- SLRTutorWindow, 183
 - ~SLRTutorWindow, 187
 - addGrammarMessage, 187
 - addMessage, 187
 - addUserState, 187
 - addUserTransition, 187
 - addWidgetMessage, 188
 - animateLabelColor, 188
 - animateLabelPop, 188
 - applyTextScale, 188
 - buildGrammarRows, 189
 - closeEvent, 189
 - confirmExitToHome, 189
 - eventFilter, 189
 - examSolutionText, 190
 - exitRequested, 190
 - exportConversationToPdf, 190
 - feedback, 190
 - feedbackForA, 191
 - feedbackForA1, 191

- feedbackForA2, 192
- feedbackForA3, 192
- feedbackForA4, 192
- feedbackForAPrime, 193
- feedbackForB, 193
- feedbackForB1, 193
- feedbackForB2, 193
- feedbackForBPrime, 193
- feedbackForC, 194
- feedbackForCA, 195
- feedbackForCB, 195
- feedbackForD, 195
- feedbackForD1, 196
- feedbackForD2, 196
- feedbackForDPrime, 196
- feedbackForE, 196
- feedbackForE1, 197
- feedbackForE2, 197
- feedbackForF, 197
- feedbackForFA, 197
- feedbackForG, 198
- fillSortedGrammar, 198
- FormatGrammar, 198
- generateQuestion, 198
- initialItemText, 199
- isReduction, 199
- launchSLRWizard, 200
- markLastUserIncorrect, 200
- openAutomatonViewer, 201
- postQuestion, 201
- promptExportFilePath, 201
- scoreExamTable, 202
- sessionFinished, 202
- setupAutomatonPanel, 203
- showExamReport, 203
- showTable, 203
- SLRTutorWindow, 186
- solution, 204
- solutionCell, 205
- solutionForA, 206
- solutionForA1, 206
- solutionForA2, 206
- solutionForA3, 206
- solutionForA4, 207
- solutionForB, 207
- solutionForC, 207
- solutionForCA, 207
- solutionForCB, 208
- solutionForD, 208
- solutionForD1, 208
- solutionForD2, 208
- solutionForE, 209
- solutionForE1, 209
- solutionForE2, 209
- solutionForF, 209
- solutionForFA, 210
- solutionForG, 210
- tableColumns, 210
- TeachClosure, 211
- TeachClosureStep, 211
- TeachDeltaFunction, 211
- updateAutomatonButton, 212
- updateAutomatonPanel, 212
- updatePlaceholder, 213
- updateProgressPanel, 213
- updateState, 213
- verifyResponse, 214
- verifyResponseForA, 214
- verifyResponseForA1, 214
- verifyResponseForA2, 214
- verifyResponseForA3, 215
- verifyResponseForA4, 215
- verifyResponseForB, 216
- verifyResponseForC, 216
- verifyResponseForCA, 216
- verifyResponseForCB, 217
- verifyResponseForD, 217
- verifyResponseForD1, 217
- verifyResponseForD2, 218
- verifyResponseForE, 218
- verifyResponseForE1, 218
- verifyResponseForE2, 218
- verifyResponseForF, 219
- verifyResponseForFA, 219
- verifyResponseForG, 219
- verifyResponseForH, 219
- wrongAnimation, 220
- wrongUserResponseAnimation, 220
- slrtutorwindow.h
 - A, 299
 - A1, 299
 - A2, 299
 - A3, 299
 - A4, 299
 - A_prime, 299
 - B, 299
 - C, 299
 - CA, 300
 - CB, 300
 - D, 300
 - D1, 300
 - D2, 300
 - D_prime, 300
 - E, 300
 - E1, 300
 - E2, 300
 - F, 300
 - FA, 300
 - fin, 300
 - G, 300
 - H, 300
 - H_prime, 300
 - StateSlr, 299
- SLRWizard, 221
 - closeButton, 222
 - currentPage, 222

- isOnLastPage, [223](#)
- nextButton, [223](#)
- SLRWizard, [222](#)
- stdVectorToQVector, [223](#)
- SLRWizardPage, [224](#)
 - completionChanged, [226](#)
 - focusAnswerField, [226](#)
 - isComplete, [226](#)
 - resizeEvent, [226](#)
 - SLRWizardPage, [225](#)
 - submitRequested, [226](#)
 - titleText, [226](#)
- solution
 - LLTutorWindow, [141](#)
 - SLRTutorWindow, [204](#)
- solutionCell
 - SLRTutorWindow, [205](#)
- solutionForA
 - LLTutorWindow, [141](#)
 - SLRTutorWindow, [206](#)
- solutionForA1
 - LLTutorWindow, [142](#)
 - SLRTutorWindow, [206](#)
- solutionForA2
 - LLTutorWindow, [142](#)
 - SLRTutorWindow, [206](#)
- solutionForA3
 - SLRTutorWindow, [206](#)
- solutionForA4
 - SLRTutorWindow, [207](#)
- solutionForB
 - LLTutorWindow, [142](#)
 - SLRTutorWindow, [207](#)
- solutionForB1
 - LLTutorWindow, [142](#)
- solutionForB2
 - LLTutorWindow, [143](#)
- solutionForC
 - SLRTutorWindow, [207](#)
- solutionForCA
 - SLRTutorWindow, [207](#)
- solutionForCB
 - SLRTutorWindow, [208](#)
- solutionForD
 - SLRTutorWindow, [208](#)
- solutionForD1
 - SLRTutorWindow, [208](#)
- solutionForD2
 - SLRTutorWindow, [208](#)
- solutionForE
 - SLRTutorWindow, [209](#)
- solutionForE1
 - SLRTutorWindow, [209](#)
- solutionForE2
 - SLRTutorWindow, [209](#)
- solutionForF
 - SLRTutorWindow, [209](#)
- solutionForFA
 - SLRTutorWindow, [210](#)
- solutionForG
 - SLRTutorWindow, [210](#)
- SolveLRConflicts
 - SLR1Parser, [177](#)
- Split
 - Grammar, [75](#)
- st_
 - Grammar, [77](#)
 - GrammarFactory::FactoryItem, [69](#)
 - SymbolTable, [232](#)
- start
 - TutorialManager, [239](#)
- StartsWith
 - GrammarFactory, [99](#)
- State
 - lltutorwindow.h, [282](#)
- state, [227](#)
 - id_, [227](#)
 - items_, [227](#)
 - operator==, [227](#)
- states_
 - SLR1Parser, [178](#)
- StateSlr
 - slrtutorwindow.h, [299](#)
- stdVectorToQVector
 - SLRWizard, [223](#)
- stepStarted
 - TutorialManager, [239](#)
- submitRequested
 - LLWizardPage, [153](#)
 - SLRWizardPage, [226](#)
- submitted
 - LLTableDialog, [122](#)
 - SLRTableDialog, [182](#)
- symbol
 - AutomatonTransitionInfo, [40](#)
- symbol_table.hpp
 - META, [265](#)
 - NO_TERMINAL, [264](#)
 - symbol_type, [264](#)
 - TERMINAL, [264](#)
- symbol_type
 - symbol_table.hpp, [264](#)
- SymbolTable, [228](#)
 - EOL_, [231](#)
 - EPSILON_, [231](#)
 - In, [229](#)
 - IsMeta, [229](#)
 - IsNonTerminal, [229](#)
 - IsTerminal, [230](#)
 - IsTerminalWthoEol, [230](#)
 - meta_symbols_, [232](#)
 - non_terminals_, [232](#)
 - PutSymbol, [231](#)
 - st_, [232](#)
 - terminals_, [232](#)
 - terminals_wtho_eol_, [232](#)

- SyntaxTutor, [1](#), [36](#)
- SyntaxTutor::Version, [36](#)
 - current, [36](#)
 - raw, [37](#)
- systemFont
 - AppTypography, [35](#)
- tableColumns
 - SLRTutorWindow, [210](#)
- target
 - TutorialStep, [240](#)
- TeachClosure
 - SLRTutorWindow, [211](#)
- TeachClosureStep
 - SLRTutorWindow, [211](#)
- TeachDeltaFunction
 - SLRTutorWindow, [211](#)
- TeachFirstTree
 - LLTutorWindow, [143](#)
- TeachFollow
 - LLTutorWindow, [144](#)
- TeachLL1Table
 - LLTutorWindow, [144](#)
- TeachPredictionSymbols
 - LLTutorWindow, [144](#)
- TERMINAL
 - symbol_table.hpp, [264](#)
- terminal_alphabet_
 - GrammarFactory, [100](#)
- terminals_
 - SymbolTable, [232](#)
- terminals_wtho_eol_
 - SymbolTable, [232](#)
- text
 - ConversationPdf::Message, [164](#)
- thresholdFor
 - MainWindow, [161](#)
- Title
 - AppTypography, [30](#)
- TitleSmall
 - AppTypography, [30](#)
- titleText
 - LLWizardPage, [154](#)
 - SLRWizardPage, [226](#)
- to
 - AutomatonTransitionInfo, [40](#)
- ToString
 - Lr0Item, [157](#)
- total
 - ExamSession, [67](#)
- totalStateCount
 - AutomatonView, [47](#)
- transition_table
 - SLR1Parser, [168](#)
- transitions_
 - SLR1Parser, [178](#)
- tutorialFinished
 - TutorialManager, [239](#)
- TutorialManager, [234](#)
 - addStep, [236](#)
 - clearSteps, [236](#)
 - eventFilter, [236](#)
 - finishLL1, [236](#)
 - finishSLR1, [237](#)
 - hideOverlay, [237](#)
 - ll1Finished, [237](#)
 - nextStep, [238](#)
 - setRootWindow, [238](#)
 - slr1Finished, [238](#)
 - start, [239](#)
 - stepStarted, [239](#)
 - tutorialFinished, [239](#)
 - TutorialManager, [236](#)
- TutorialStep, [240](#)
 - htmlText, [240](#)
 - target, [240](#)
- UniqueQueue< T >, [241](#)
 - clear, [242](#)
 - empty, [242](#)
 - front, [242](#)
 - pop, [242](#)
 - push, [242](#)
- updateAutomatonButton
 - SLRTutorWindow, [212](#)
- updateAutomatonPanel
 - SLRTutorWindow, [212](#)
- updatePlaceholder
 - LLTutorWindow, [144](#)
 - SLRTutorWindow, [213](#)
- updateProgressPanel
 - LLTutorWindow, [145](#)
 - SLRTutorWindow, [213](#)
- updateState
 - LLTutorWindow, [145](#)
 - SLRTutorWindow, [213](#)
- userAnswer
 - ExamRecord, [62](#)
- userLevel
 - MainWindow, [162](#), [163](#)
- userLevelChanged
 - MainWindow, [162](#)
- userLevelUp
 - MainWindow, [162](#)
- verifyResponse
 - LLTutorWindow, [146](#)
 - SLRTutorWindow, [214](#)
- verifyResponseForA
 - LLTutorWindow, [146](#)
 - SLRTutorWindow, [214](#)
- verifyResponseForA1
 - LLTutorWindow, [146](#)
 - SLRTutorWindow, [214](#)
- verifyResponseForA2
 - LLTutorWindow, [147](#)
 - SLRTutorWindow, [214](#)
- verifyResponseForA3

- SLRTutorWindow, [215](#)
- verifyResponseForA4
 - SLRTutorWindow, [215](#)
- verifyResponseForB
 - LLTutorWindow, [147](#)
 - SLRTutorWindow, [216](#)
- verifyResponseForB1
 - LLTutorWindow, [147](#)
- verifyResponseForB2
 - LLTutorWindow, [148](#)
- verifyResponseForC
 - LLTutorWindow, [148](#)
 - SLRTutorWindow, [216](#)
- verifyResponseForCA
 - SLRTutorWindow, [216](#)
- verifyResponseForCB
 - SLRTutorWindow, [217](#)
- verifyResponseForD
 - SLRTutorWindow, [217](#)
- verifyResponseForD1
 - SLRTutorWindow, [217](#)
- verifyResponseForD2
 - SLRTutorWindow, [218](#)
- verifyResponseForE
 - SLRTutorWindow, [218](#)
- verifyResponseForE1
 - SLRTutorWindow, [218](#)
- verifyResponseForE2
 - SLRTutorWindow, [218](#)
- verifyResponseForF
 - SLRTutorWindow, [219](#)
- verifyResponseForFA
 - SLRTutorWindow, [219](#)
- verifyResponseForG
 - SLRTutorWindow, [219](#)
- verifyResponseForH
 - SLRTutorWindow, [219](#)
- visibleStateCount
 - AutomatonView, [47](#)
- visibleTransitionCount
 - AutomatonView, [48](#)
- wheelEvent
 - AutomatonView, [48](#)
- widgets Directory Reference, [16](#)
- widgets/appfonts.h, [313](#), [314](#)
- widgets/appicon.h, [314](#), [315](#)
- widgets/applayout.h, [316](#), [317](#)
- widgets/apppalette.h, [318](#), [319](#)
- widgets/appsettings.h, [319](#), [320](#)
- widgets/appshortcuts.h, [321](#)
- widgets/apptextscale.h, [322](#), [323](#)
- widgets/apptypography.h, [324](#), [325](#)
- widgets/automatonview.cpp, [327](#)
- widgets/automatonview.h, [328](#), [329](#)
- widgets/customtextedit.cpp, [330](#)
- widgets/customtextedit.h, [331](#)
- widgets/grammarview.cpp, [332](#)
- widgets/grammarview.h, [333](#)
- widgets/tutorialmanager.cpp, [334](#)
- widgets/tutorialmanager.h, [334](#), [335](#)
- wrongAnimation
 - LLTutorWindow, [148](#)
 - SLRTutorWindow, [220](#)
- wrongUserResponseAnimation
 - LLTutorWindow, [148](#)
 - SLRTutorWindow, [220](#)
- zoomIn
 - AutomatonView, [48](#)
- zoomOut
 - AutomatonView, [48](#)